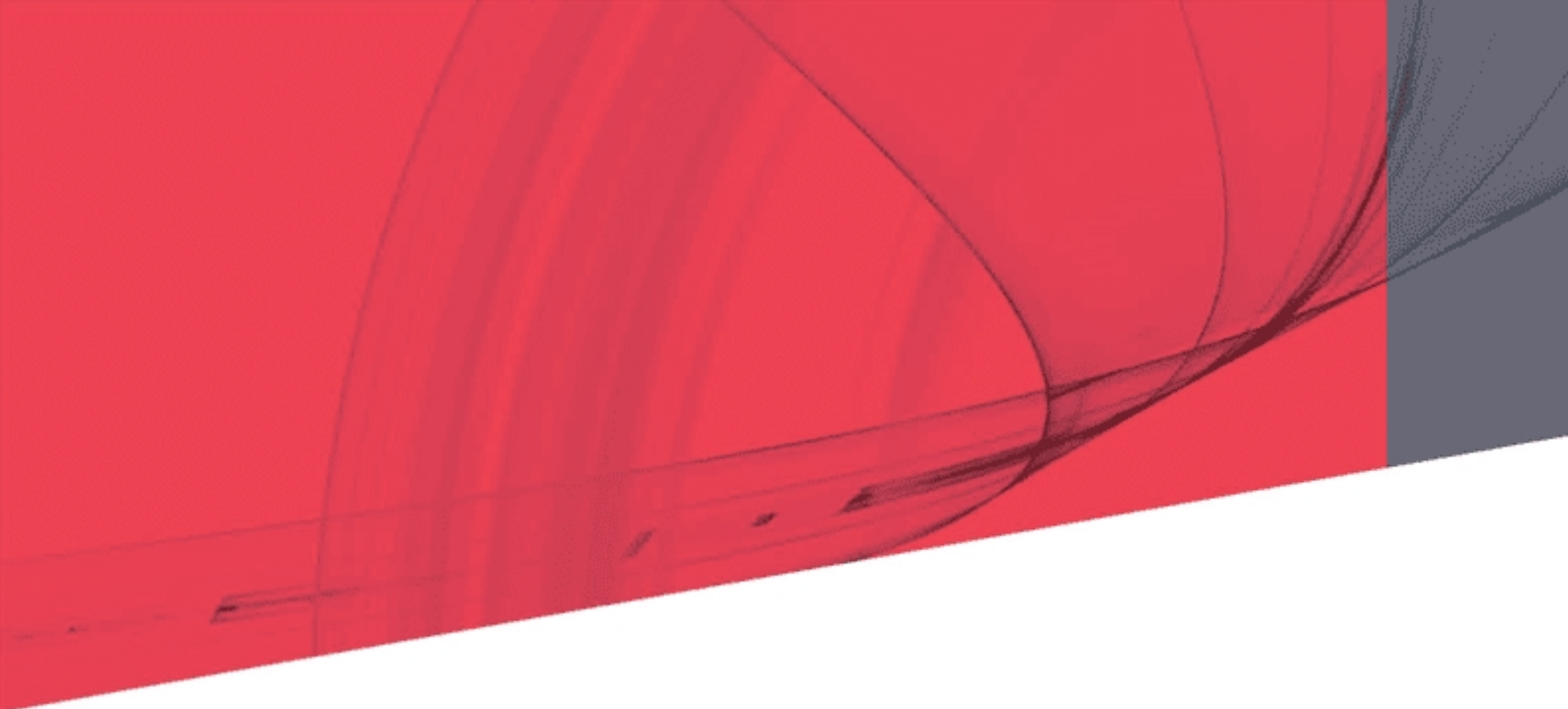




SPiiPlus COM Library

Programmer's Guide

May 2022

Document Revision: 3.12

SPiiPlus COM Library

Release Date: May 2022

COPYRIGHT

© ACS Motion Control Ltd., 2021. All rights reserved.

Changes are periodically made to the information in this document. Changes are published as release notes and later incorporated into revisions of this document.

No part of this document may be reproduced in any form without prior written permission from ACS Motion Control.

TRADEMARKS

Windows and Intellisense are trademarks of Microsoft Corporation.

EtherCAT® is registered trademark and patented technology, licensed by Beckhoff Automation GmbH, Germany.

Any other companies and product names mentioned herein may be the trademarks of their respective owners.

PATENTS

Israel Patent No. 235022

US Patent Application No. 14/532,023

Europe Patent application No.15187586.1

Japan Patent Application No.: 2015-193179

Chinese Patent Application No.: 201510639732.X

Taiwan(R.O.C.) Patent Application No. 104132118

Korean Patent Application No. 10-2015-0137612

www.acsmotioncontrol.com

support@acsmotioncontrol.com

sales@acsmotioncontrol.com

NOTICE

The information in this document is deemed to be correct at the time of publishing. ACS Motion Control reserves the right to change specifications without notice. ACS Motion Control is not responsible for incidental, consequential, or special damages of any kind in connection with using this document.

Revision History

Date	Revision	Description
May 2022	3.12	New Release
December 2021	3.11.02	Note minimum version supporting feature
September 2021	3.11.01	New Release
July 2020	3.01	Add note about segmented motion function
April 2019	2.70	Updated to match functionality of .NET library
June 2018	2.60	Replaced "X" in programming examplew with an axis number, as necessary.
June 2017	2.40	Updated for SPiiPlus ADK Suite v2.30: Added new bits for ECST
August 2016	2.30	Updated for SPiiPlus ADK Suite v2.30: Removed functions: acsc_OpenCommDirect Added functions: OpenCommSimulator and _CloseSimulator Corrected syntax from ASCS_COMM_AUTORECOVER_HW_ERROR to ACSC_COMM_AUTORECOVER_HW_ERROR
September 2014	01	First Release

Conventions Used in this Guide

Text Formats

Format	Description
Bold	Names of GUI objects or commands
BOLD + UPPERCASE	ACSPL+ variables and commands
<code>Monospace + grey background</code>	Code example
<i>Italic</i>	Names of other documents
Blue	Hyperlink
[]	In commands indicates optional item(s)
	In commands indicates either/or items

Flagged Text

	Note - includes additional information or programming tips.
	Caution - describes a condition that may result in damage to equipment.
	Warning - describes a condition that may result in serious bodily injury or death.
	Model - highlights a specification, procedure, condition, or statement that depends on the product model
	Advanced - indicates a topic for advanced users.

Related Documents

Documents listed in the following table provide additional information related to this document.

The most updated version of the documents can be downloaded by authorized users from www.acsmotioncontrol.com/downloads.

Online versions for all ACS software manuals and SPiiPlus ADK suite Release Notes are available to authorized users at [ACS Motion Control Knowledge Center](http://www.acsmotioncontrol.com/knowledge-center).

Document	Description
<i>SPiiPlus Setup Guide</i>	Communication, configuration and adjustment procedures for SPiiPlus motion control products.
<i>SPiiPlus Command & Variable Reference Guide</i>	Command and variables of high level language for programming SPiiPlus controllers.
<i>ACSPL+ Programmer's Guide</i>	Guide for using ACSPL+ command set for programming SPiiPlus controllers.

Table of Contents

1. Introduction	20
1.1 Scope	20
2. General Information	21
2.1 General	21
2.2 Operating Environment	21
2.3 Communication Channels	21
2.4 Controller Simulation	21
2.5 Supported Programming Languages	21
2.6 Installation and Supplied Components	22
2.7 Asynchronous Calls Support	22
2.8 Standard (SPiiPlus C Library) Features	22
2.9 Specific COM Library Features	23
2.10 Communication Scenarios	24
3. Using the SPiiPlus COM Library	26
3.1 Redistribution	26
3.2 Visual Basic	26
3.3 LabView	26
3.4 VBScript (HTML) Example	27
3.5 Visual Studio .NET	28
4. COM Methods	29
4.1 Communication Methods	30
4.1.1 CancelOperation	31
4.1.2 Command	32
4.1.3 CloseComm	32
4.1.4 FlushLogFile	33
4.1.5 GetBinary	34
4.1.6 GetConnectionsList	35
4.1.7 GetEthernetCards	36
4.1.8 GetPCICards	37
4.1.9 GetString	38
4.1.10 OpenCommSimulator	38
4.1.11 CloseSimulator	39

4.1.12	OpenCommEthernetTCP	40
4.1.13	OpenCommEthernetUDP	40
4.1.14	OpenCommPCI	41
4.1.15	OpenCommSerial	42
4.1.16	GetConnectionInfo	43
4.1.16.1	CONNECTION_TYPE	44
4.1.16.2	CONNECTION_INFO	44
4.1.17	SetServerExtLogin	45
4.1.18	TerminateConnection	46
4.1.19	Transaction	47
4.2	EtherCAT® Methods	47
4.2.1	GetEtherCATState	48
4.2.2	GetEtherCATError	50
4.2.3	MapEtherCATInput	51
4.2.4	MapEtherCATOutput	52
4.2.5	UnmapEtherCATInputsOutputs	53
4.2.6	GetEtherCATSlaveIndex	54
4.2.7	GetEtherCATSlaveOffset	55
4.2.8	GetEtherCATSlaveVendorID	56
4.2.9	GetEtherCATSlaveProductID	57
4.2.10	GetEtherCATSlaveRevision	58
4.2.11	GetEtherCATSlaveType	59
4.2.12	GetEtherCATSlaveState	60
4.3	Service Communication Methods	62
4.3.1	GetACSCHandle	62
4.3.2	GetCOMLibraryVersion	63
4.3.3	GetCommOptions	64
4.3.4	GetDefaultTimeout	64
4.3.5	GetErrorString	65
4.3.6	GetLibraryVersion	66
4.3.7	GetTimeout	67
4.3.8	SetIterations	67
4.3.9	SetCommOptions	68
4.3.10	SetTimeout	69

4.4 ACSPL+ Program Management Methods	70
4.4.1 AppendBuffer	70
4.4.2 ClearBuffer	71
4.4.3 CompileBuffer	72
4.4.4 LoadBuffer	73
4.4.5 LoadBuffersFromFile	74
4.4.6 RunBuffer	75
4.4.7 StopBuffer	76
4.4.8 SuspendBuffer	77
4.4.9 UploadBuffer	78
4.5 Read and Write Variables Methods	79
4.5.1 ReadVariable	79
4.5.2 ReadVariableAsScalar	83
4.5.3 ReadVariableAsVector	85
4.5.4 ReadVariableAsMatrix	86
4.5.5 WriteVariable	90
4.6 History Buffer Management Methods	95
4.6.1 OpenHistoryBuffer	95
4.6.2 CloseHistoryBuffer	95
4.6.3 GetHistory	96
4.7 Unsolicited Messages Buffer Management Methods	97
4.7.1 OpenMessageBuffer	97
4.7.2 CloseMessageBuffer	98
4.7.3 GetSingleMessage	99
4.8 Log File Management Methods	100
4.8.1 OpenLogFile	100
4.8.2 CloseLogFile	101
4.8.3 WriteLogFile	101
4.8.4 GetLogData	102
4.9 SPiiPlusSC Log File Management Methods	103
4.9.1 OpenSCLogFile	104
4.9.2 CloseSCLogFile	104
4.9.3 WriteSCLogFile	105
4.9.4 FlushSCLogFile	106

4.10 Shared Memory Methods	106
4.10.1 GetSharedMemoryAddress	107
4.10.2 ReadSharedMemoryInteger	108
4.10.3 WriteSharedMemoryInteger	108
4.10.4 ReadSharedMemoryReal	109
4.10.5 WriteSharedMemoryReal	109
4.10.6 Shared Memory Program Example	110
4.11 System Configuration Methods	110
4.11.1 SetConf	111
4.11.2 GetConf	112
4.11.3 GetVolatileMemoryUsage	112
4.11.4 GetVolatileMemoryTotal	113
4.11.5 GetVolatileMemoryFree	114
4.11.6 GetNonVolatileMemoryUsage	115
4.11.7 GetNonVolatileMemoryTotal	116
4.11.8 GetNonVolatileMemoryFree	117
4.12 Setting and Reading Motion Parameters Methods	118
4.12.1 SetVelocity	119
4.12.2 SetVelocityImm	120
4.12.3 GetVelocity	121
4.12.4 SetAcceleration	122
4.12.5 SetAccelerationImm	123
4.12.6 GetAcceleration	124
4.12.7 SetDeceleration	125
4.12.8 SetDecelerationImm	126
4.12.9 GetDeceleration	127
4.12.10 SetJerk	127
4.12.11 SetJerkImm	128
4.12.12 GetJerk	129
4.12.13 SetKillDeceleration	130
4.12.14 SetKillDecelerationImm	131
4.12.15 GetKillDeceleration	132
4.12.16 SetFPosition	133
4.12.17 GetFPosition	134

4.12.18 SetRPosition	135
4.12.19 GetRPosition	136
4.12.20 GetFVelocity	137
4.12.21 GetRVelocity	138
4.13 Axis/Motor Management Methods	138
4.13.1 CommutExt	139
4.13.2 Enable	140
4.13.3 EnableM	141
4.13.4 Disable	141
4.13.5 DisableAll	142
4.13.6 DisableExt	143
4.13.7 DisableM	144
4.13.8 Group	145
4.13.9 Split	145
4.13.10 SplitAll	146
4.14 Motion Management Methods	147
4.14.1 Go	148
4.14.2 GoM	149
4.14.3 Halt	150
4.14.4 HaltM	151
4.14.5 Kill	152
4.14.6 KillAll	153
4.14.7 KillM	154
4.14.8 KillExt	155
4.14.9 Break	156
4.14.10 BreakM	158
4.15 Point-to-Point Motion Methods	160
4.15.1 ToPoint	161
4.15.2 ToPointM	162
4.15.3 ExtToPoint	164
4.15.4 ExtToPointM	165
4.16 Track Motion Control Method	167
4.16.1 Track	168
4.17 Jog Methods	169

4.17.1 Jog	169
4.17.2 JogM	170
4.18 Slaved Motion Methods	172
4.18.1 SetMaster	172
4.18.2 Slave	174
4.18.3 SlaveStalled	175
4.19 Multi-Point Motion Methods	177
4.19.1 MultiPoint	177
4.19.2 MultiPointM	179
4.20 Arbitrary Path Motion Methods	181
4.20.1 Spline	182
4.20.2 SplineM	184
4.21 PVT Methods	186
4.21.1 AddPVPoint	186
4.21.2 AddPVPointM	188
4.21.3 AddPVTPoint	189
4.21.4 AddPVTPointM	190
4.22 Segmented Motion Methods	192
4.22.1 ExtendedSegmentedMotionExt	194
4.22.2 SegmentLineExt	201
4.22.3 ExtendedSegmentArc1	204
4.22.4 ExtendedSegmentArc2	208
4.22.5 BlendedSegmentMotion	212
4.22.6 BlendedLine	214
4.22.7 BlendedArc1	217
4.22.8 BlendedArc2	220
4.22.9 Stopper	222
4.22.10 Projection	224
4.23 Points and Segments Manipulation Methods	226
4.23.1 AddPoint	227
4.23.2 AddPointM	228
4.23.3 ExtAddPoint	229
4.23.4 ExtAddPointM	230
4.23.5 EndSequence	232

4.23.6 EndSequenceM	233
4.24 Data Collection Methods	234
4.24.1 DataCollectionExt	234
4.24.2 StopCollect	236
4.25 Status Report Methods	237
4.25.1 GetMotorState	238
4.25.2 GetAxisState	239
4.25.3 GetIndexState	239
4.25.4 ResetIndexState	240
4.25.5 GetProgramState	241
4.26 Inputs/Outputs Access Methods	242
4.26.1 GetInput	243
4.26.2 GetInputPort	244
4.26.3 GetOutput	245
4.26.4 GetOutputPort	245
4.26.5 SetOutput	246
4.26.6 SetOutputPort	247
4.26.7 GetAnalogInputNT	248
4.26.8 GetAnalogOutputNT	249
4.26.9 SetAnalogOutputNT	250
4.26.10 GetExtInput	251
4.26.11 GetExtInputPort	252
4.26.12 GetExtOutput	253
4.26.13 GetExtOutputPort	253
4.26.14 SetExtOutput	254
4.26.15 SetExtOutputPort	255
4.27 Safety Control Methods	256
4.27.1 GetFault	257
4.27.2 SetFaultMask	258
4.27.3 GetFaultMask	259
4.27.4 EnableFault	260
4.27.5 DisableFault	261
4.27.6 SetResponseMask	262
4.27.7 GetResponseMask	263

4.27.8 EnableResponse	264
4.27.9 DisableResponse	264
4.27.10 GetSafetyInput	265
4.27.11 GetSafetyInputPort	266
4.27.12 GetSafetyInputPortInv	267
4.27.13 SetSafetyInputPortInv	269
4.27.14 FaultClear	270
4.27.15 FaultClearM	270
4.28 Wait-for-Condition Methods	271
4.28.1 WaitMotionEnd	272
4.28.2 WaitLogicalMotionEnd	272
4.28.3 WaitCollectEndExt	273
4.28.4 WaitProgramEnd	274
4.28.5 WaitMotorCommutated	275
4.28.6 WaitMotorEnabled	276
4.28.7 WaitInput	277
4.29 Event and Interrupt Handling Methods	278
4.29.1 EnableEvent	278
4.29.2 DisableEvent	279
4.29.3 SetEventMask	280
4.29.4 GetEventMask	282
4.30 Variables Management Methods	284
4.30.1 DeclareVariable	284
4.30.2 ClearVariables	285
4.31 Service Methods	286
4.31.1 GetFirmwareVersion	286
4.31.2 GetSerialNumber	287
4.31.3 GetSysInfo	287
4.31.4 GetBuffersCount	289
4.31.5 GetAxesCount	290
4.31.6 GetDBufferIndex	291
4.31.7 GetUMDVersion	291
4.32 Error Diagnosis Methods	292
4.32.1 GetMotorError	293

4.32.2	GetMotionError	293
4.32.3	GetProgramError	294
4.32.4	ParseCOMErrorCode	295
4.33	Dual Port RAM (DPRAM) Access Methods	296
4.33.1	ReadDPRAMInteger	296
4.33.2	WriteDPRAMInteger	297
4.33.3	ReadDPRAMReal	298
4.33.4	WriteDPRAMReal	298
4.34	Position Event Generation (PEG) Methods	299
4.34.1	PegInc	300
4.34.2	PegRandom	301
4.34.3	AssignPins	303
4.34.4	StopPeg	304
4.34.5	AssignPegNT	305
4.34.6	AssignPegOutputsNT	306
4.34.7	AssignFastInputsNT	307
4.34.8	PegIncNT	308
4.34.9	PegRandomNT	310
4.34.10	WaitPegReadyNT	311
4.34.11	StartPegNT	312
4.34.12	StopPegNT	313
4.35	Application Save/Load Methods	314
4.35.1	Structures	314
4.35.1.1	ApplicationFileInfo	314
4.35.1.2	Attribute	315
4.35.1.3	Section	315
4.35.2	Enum	315
4.35.2.1	FileType	315
4.35.3	AnalyzeApplication	316
4.35.4	LoadApplication	316
4.35.5	SaveApplication	317
4.35.6	FreeApplication	317
4.36	Load/Upload Data To/From Controller Methods	318
4.36.1	LoadDataToController	318

4.36.2 UploadDataFromController	320
4.37 Emergency Stop Methods	321
4.37.1 RegisterEmergencyStop	322
4.37.2 UnregisterEmergencyStop	323
4.38 Reboot Methods	323
4.38.1 ControllerReboot	324
4.38.2 ControllerFactoryDefault	324
4.39 Host-Controller File Operations	325
4.39.1 CopyFileToController	325
4.39.2 DeleteFileFromController	326
4.40 Save to Flash Functions	327
4.40.1 ControllerSaveToFlash	327
4.41 SPiiPlusSC Management	329
4.41.1 StartSPiiPlusSC	329
4.41.2 StopSPiiPlusSC	329
5. Properties	331
5.1 General Properties	331
5.2 General Communication Options	331
5.3 Ethernet Communication Options	332
5.4 Axis Definitions	332
5.5 Buffer Definitions	332
5.6 Servo Processor (SP) Definitions	332
5.7 Motion Flags	333
5.8 Data Collection Flags	335
5.9 Motor State Flags	335
5.10 Axis State Flags	335
5.11 Index and Mark State Flags	336
5.12 Program State Flags	336
5.13 Safety Control Masks	337
5.14 Interrupt Types	338
5.15 Callback Interrupt Masks	340
5.16 Configuration Keys	340
6. Events	342
7. Error Handling	343

7.1 Error Handling in Visual Basic	343
7.2 Error Handling Example in LabVIEW 7	344
7.3 Error Handling Example in C#	345
7.4 Error Codes	345

List Of Figures

Figure 3-1. LabView Automation Open	27
Figure 4-1. Emergency Stop Button	322
Figure 7-1. Communication not Established Message - Visual Basic	343
Figure 7-2. Error Handling Example in LabView 7	344
Figure 7-3. LabVIEW Error Message	344
Figure 7-4. C# Error Message	345

List of Tables

Table 4-1. Communication Methods	30
Table 4-2. EtherCAT Methods	48
Table 4-3. EtherCAT States	49
Table 4-4. EtherCAT Error Codes	50
Table 4-5. Service Communication Methods	62
Table 4-6. ACSPL+ Program Management Methods	70
Table 4-7. Read and Write Variables Methods	79
Table 4-8. Value Dimension Indices	92
Table 4-9. History Buffer Management Methods	95
Table 4-10. Unsolicited Messages Buffer Management Methods	97
Table 4-11. Log File Management Methods	100
Table 4-12. SPiiPlusSC Log File Management Methods	104
Table 4-13. Shared Memory Methods	107
Table 4-14. System Configuration Methods	110
Table 4-15. Setting and Reading Motion Parameters Methods	118
Table 4-16. Axis/Motor Management Methods	138
Table 4-17. Motion Management Methods	147
Table 4-18. Point-to-Point Motion Methods	160
Table 4-19. Track Motion Control Method	167
Table 4-20. Jog Methods	169
Table 4-21. Slaved Motion Methods	172
Table 4-22. Multi-Point Motion Methods	177
Table 4-23. Arbitrary Path Motion Methods	181
Table 4-24. PVT Methods	186
Table 4-25. Segmented Motion Methods	192
Table 4-26. Points and Segments Manipulation Methods	226
Table 4-27. Data Collection Methods	234
Table 4-28. Status Report Methods	238
Table 4-29. Inputs/Outputs Access Methods	243
Table 4-30. Safety Control Methods	256
Table 4-31. Wait-for-Condition Methods	271

Table 4-32. Event and Interrupt Handling Methods	278
Table 4-33. Variables Management Methods	284
Table 4-34. Service Methods	286
Table 4-35. System Information Keys	288
Table 4-36. Error Diagnosis Methods	292
Table 4-37. Dual Port RAM (DPRAM) Access Methods	296
Table 4-38. Non-NT Position Event Generation (PEG) Methods	299
Table 4-39. NT Position Event Generation (PEG) Methods	299
Table 4-40. Application Save/Load Methods	314
Table 4-41. Load/Upload Data To/From Controller Methods	318
Table 4-42. Emergency Stop Methods	322
Table 4-43. Reboot Methods	323
Table 4-44. Host-Controller File Copying Methods	325
Table 4-45. Host-Controller File Copying Methods	327
Table 4-46. Host-Controller File Copying Methods	329
Table 5-1. General Properties	331
Table 5-2. General Communication Options	331
Table 5-3. Ethernet Communication Options	332
Table 5-4. Motion Flags	333
Table 5-5. Data Collection Flags	335
Table 5-6. Motor State Flags	335
Table 5-7. Axis State Flags	335
Table 5-8. Index and Mark State Flags	336
Table 5-9. Program State Flags	336
Table 5-10. Safety Control Masks	337
Table 5-11. Interrupt Types	338
Table 5-12. Callback Interrupt Masks	340
Table 5-13. Configuration Keys	340
Table 6-1. Events and Interrupts	342
Table 7-1. COM Library Error Codes	346

1. Introduction

1.1 Scope

The SPiiPlus COM Library supports the creation of a user application that operates in a PC host computer and communicates with SPiiPlus motion controllers. The SPiiPlus COM Library implements a rich set of controller operations and conceals from the application the complexity of low-level communication and synchronization with the controller.

This document is intended for the use of software engineers.

2. General Information

2.1 General

The COM (Component Object Model) has become the most widely used component software model in the world. COM provides a software architecture that enables components from different programming languages and platforms to be combined into a variety of applications.

The SPiiPlus COM Library provides the easiest way to incorporate SPiiPlus motion control functionality.

2.2 Operating Environment

The SPiiPlus COM Library supports Microsoft® Windows® environments:

- > Windows 7 SP1 (x86 and x64)
- > Windows 8 (x86 and x64)
- > Windows 8.1 (x86 and x64)
- > Windows 10
- > Windows Server 2008 R2 SP1 (x64)
- > Windows Server 2012 R2 (x64)
- > Windows Server 2016 R2 (x64)
- > Windows Server 2019 (x64)



Windows XP and Windows Server 2003 are no longer supported.

2.3 Communication Channels

The SPiiPlus COM Library supports all communication channels provided by SPiiPlus motion controllers:

- > Serial (RS-232)
- > Ethernet (point-to-point and network)
- > PCI Bus

2.4 Controller Simulation

The SPiiPlus COM Library supports communication with a **SPiiPlus Controller Simulator** running on the same PC. The **SPiiPlus Controller Simulator** emulates a controller, enabling program debugging and demonstrations without a connection to a physical controller.

2.5 Supported Programming Languages

The SPiiPlus COM Library supports C#, C++, Visual Basic, VBScript, LabView and other programming languages that support the use of COM components. The library also supports Delphi, Power Builder, Microsoft Office, Internet Explorer and other development environments.

The SPiiPlus COM Library has been tested with Visual C++, Visual Basic, VBScript, LabView and .NET applications.

2.6 Installation and Supplied Components

The SPiiPlus COM Library is supplied as a standard COM component (DLL module) that must be registered in Windows before using. This is done automatically during the installation of the **SPiiPlus Software Tools**. See [Using the SPiiPlus COM Library](#) for details. Samples for Visual Basic 6, LabVIEW 7, VDScrip (HTML), .NET are also installed.

2.7 Asynchronous Calls Support

The *SPiiPlus COM Library Version NT 2.28* provides additional *IAsyncChannel* interface. This interface provides the ability to call methods asynchronously. Each method that supports asynchronous mode has two additional arguments:

ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.



The methods can also be called synchronously through the *IAsyncChannel* interface. The *IChannel* interface is also supported but its methods can only be called synchronously

2.8 Standard (SPiiPlus C Library) Features

The SPiiPlus COM Library provides the same standard features as the SPiiPlus C Library (see *SPiiPlus C Library Reference*), including:

- > Unified support for all communication channels (Serial, Ethernet, PCI Bus)
All SPiiPlus COM Library methods except the **OpenCommxxx** methods are identical for all communication channels. A (host computer) user application doesn't require any other modification to work with different communication channels.
- > Controller simulator communication channel
All SPiiPlus COM Library methods support the **SPiiPlus Controller Simulator**. The user client application activates the simulator by opening a special communication channel. The user is not required to change his application in order to communicate with the simulator.
- > Concurrent support for up to 10 communication channels in one application

One user client application can open up to ten communication channels simultaneously. Different communication channels are usually connected to different controllers. However, two or more communication channels can be connected to the same controller. For example, an application can communicate with a controller through both the controller's Ethernet channel and its serial channels.

> Acknowledgement for each command sent to the controller

The library automatically checks the status of each command sent by the user application to the controller. The user application can check the status to confirm that the command was received successfully.

> Communication history

The SPiiPlus COM Library enables storage in a memory buffer of all messages sent to, and received from, the controller. The application can retrieve data from the buffer and can clear the buffer.

> Separate processing of unsolicited messages

Most messages sent from the controller to the host are responses to host commands. However, the controller can send unsolicited messages, for example, as output from a **disp** command. The library separates the unsolicited messages from the overall message flow and provides a special method for handling unsolicited messages.

> Rich functionality

The COM library supports setting and reading parameters, advanced motion control, program management, I/O, safety, and more.

> Debug tools

The SPiiPlus COM Library provides tools that facilitate debugging of the user application. The simulator and the communication history mentioned above are the primary debugging tools. The user can also open a log file that stores all communications between the application and the controller.

2.9 Specific COM Library Features

The SPiiPlus COM Library also supports features based on COM technology:

> Generating COM events for predefined controller events

The SPiiPlus COM Library generates COM events for the user client application if one of the following events occurs:

PEG, MARK, or Emergency Stop signal has been generated

Motion has ended

Motion has been interrupted due to a fault

Motor has been disabled due to a fault

ACSPL+ program in the controller has completed

Digital input has been changed

Other events

> Error handling

The SPiiPlus COM Library supports a COM error object to provide rich error information to a user client application such as error code, error description and help context.

> Automation support

The SPiiPlus COM Library supports scripting languages such as VBScript used in Microsoft Office, Internet Explorer, and other environments.

2.10 Communication Scenarios

One user application can open up to 10 communication channels simultaneously through the SPiiPlus COM Library. Usually the application opens different communication channels to work with different controller at the same time.

The following communication scenarios are typical for application development.

> Application works with one controller through one communication channel

Application creates communication channel object, opens communication and then uses any SPiiPlus COM Library methods. Before closing the application closes communication and then destroys the communication channel object.

```
Dim Ch As Channel
Set Ch =new Channel           'create communication channel object
Call Ch.OpenComxxx(...)      'open communication through any channel
...
FPosition = Ch.GetFPosition(Ch.ACSC_AXIS_0)
                               'get motor feedback position
...
Ch.CloseCommunication         'close communication
                               'in VB there is no need to destroy
                               'communication channel
```

> Application works with several controllers by turns. Only one communication channel is open at the same time

Application creates communication channel object, opens communication with first controller and then uses any SPiiPlus COM Library methods. When the application needs to connect to the second controller, it closes the previous communication and then opens new a communication. In this case the application uses the same communication channel object for each controller.

Before closing the application closes communication, then destroys communication channel.

```
Dim Ch As Channel
Set Ch =new Channel           'create communication channel
Call Ch.OpenCommPCI(Ch.ACSC_NONE) 'open communication with 1st controller
...
Fposition1 = Ch.GetFPosition(Ch.ACSC_AXIS_0)
                               'get motor feedback position from 1st
                               'controller
...
Ch.CloseCommunication         'close communication
```



```

Call Ch.OpenCommSerial(1, 115200)      'open communication with 2nd
                                         'controller
...
Fposition2 = Ch.GetFPosition(Ch.ACSC_AXIS_0)
                                         'get motor feedback position from
...
Ch.CloseCommunication                  'close communication
Call Ch.OpenCommSerial(2, 115200)      'open communication with 3rd
                                         'controller
...
Fposition3 = Ch.GetFPosition(Ch.ACSC_AXIS_0)
                                         'get motor feedback position from
...
Ch.CloseCommunication                  'close communication
                                         'in VB there is no need to destroy
                                         'communication channel

```

> Application works with several controllers at the same time

In this case the application needs to open several communication channel objects, one for each controller. Then for each communication channel it opens communication with different controllers. Before closing the application closes all communications, then destroys all communication channels.

```

Dim Ch1 As Channel
Dim Ch2 As Channel
Dim Ch3 As Channel
Set Ch1 =new Channel                  'create communication channel
Set Ch2 =new Channel                  'create communication channel
Set Ch3 =new Channel                  'create communication channel
Call Ch1.OpenCommPCI(Ch.ACSC_NONE)   'open communication with 1st
                                         'controller
Call Ch2.OpenCommSerial(1, 115200)   'open communication with 2nd
                                         'controller
Call Ch3.OpenCommSerial(2, 115200)   'open communication with 3rd
                                         'controller
...
                                         'get motor feedback position:
Fposition1 = Ch1.GetFPosition(Ch1.ACSC_AXIS_0)
                                         'from 1st controller
Fposition2 = Ch2.GetFPosition(Ch2.ACSC_AXIS_0)
                                         'from 2nd controller
Fposition3 = Ch3.GetFPosition(Ch3.ACSC_AXIS_0)
                                         'from 3rd controller
...
Ch1.CloseCommunication                'close all communications
Ch2.CloseCommunication
Ch3.CloseCommunication
                                         'in VB there is no need to destroy
                                         'communication channel

```

3. Using the *SPiiPlus COM Library*

This chapter describes how to register the SPiiPlus COM Library and provides examples of how to call SPiiPlus COM methods from Visual Basic, .NET (Visual Basic and C#), LabVIEW, and VBScript (HTML).

See also demo projects for Visual Basic, .NET (Visual Basic and C#), LabVIEW, and VBScript in the SPiiPlus COM installation folder.

3.1 Redistribution



SPiiPlus COM Library is registered automatically by the SPiiPlus installation. However, when you deploy an application that uses the SPiiPlus COM Library to other computers, you will have to register the library.

Follow these steps to register the SPiiPlus COM Library using the **REGSVR32.EXE** utility that is supplied with the Microsoft Windows OS:

1. Click the Windows **Start** button and choose **Run**.
2. Enter regsvr32 and then SPiiPlusCOM660_x86.dll (for 32-bit environments) or SPiiPlusCOM660_x64.dll (for 64-bit environments) with the full path. The path and DLL name should be in quotation marks. For example:

```
regsvr32 "C:\MyProjects\SPiiPlusCOM660_x86.dll"
```

The SPiiPlus COM Library requires the SpiiPlus C Library and drivers. See the *SPiiPlus C Library Programmer's Guide* for details of how to redistribute the SPiiPlus C Library.

3.2 Visual Basic

1. In the **Project** menu click **References**. The dialog box opens.
2. In the dialog box, select **SPiiPlusCOM 6.60 Type Library**
3. In your project source file declare:

```
Public Ch As Channel
```

4. In the method that is called when your form is loaded, include

```
Set Ch = New Channel
```

5. Now you can call SPiiPlus COM Library methods. For example,

```
Call ch.OpenCommPCI(-1)
```

3.3 LabView

This procedure applies for LabView version 7.

1. On the **Controls** palette select the **Refnum → Automation Refnum** control.

2. On the control, click the right mouse button and select **Select ActiveX Class → Browse**. A list is displayed
3. Select **SPiiPlus COM 6.60 Type Library Version 1.0** from the **Type Library** combo box. In the **Objects** list, **Channel** should be selected.
4. Right click the **SPiiPlusCOMLib660** control and select **Find Terminal**.
5. On your diagram, open the **Function** palette and select **Communication → ActiveX → Automation Open**.
6. Connect the **SPiiPlusCOMLib660.IChannel** control to the **Automation Open** control, as shown.

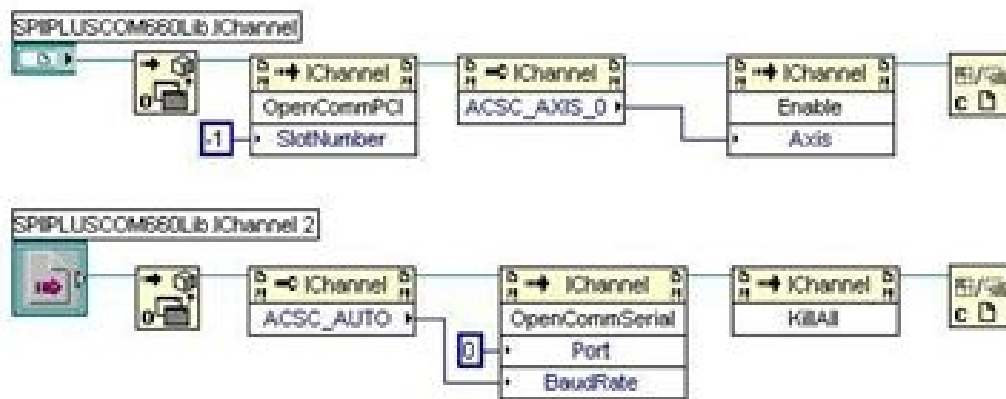


Figure 3-1. LabView Automation Open

3.4 VBScript (HTML) Example

Add the following declaration to your HTML file after <BODY>:

```
<OBJECT classid=clsid: 73C1FDD7-7A9F-45cb-AC12-73AC50F9A3C8 height=1
id=Ch
VIEWASTEXT></OBJECT>
```

For example:

```
<HTML>
<HEAD>
<META NAME="GENERATOR" Content="Microsoft Visual Studio 6.0">
<TITLE></TITLE>
<SCRIPT LANGUAGE="VBScript">
<!--
Sub button1_onclick
Ch.OpenCommPCI (-1)
End Sub
/-->
</SCRIPT>
</HEAD>
<BODY>
```

```
<OBJECT classid=clsid: 73C1FDD7-7A9F-45cb-AC12-73AC50F9A3C8 height=1
id=Ch
VIEWASTEXT></OBJECT>
<INPUT id=button1 name=button1 style="HEIGHT: 28px; WIDTH: 127px"
type=button value=Button>&nbsp;
</BODY>
</HTML>
```

3.5 Visual Studio .NET

1. In the "Solution Explorer" window right click on **Reference** and select "**Add Reference...**" In the **Project** menu click **References**. The dialog box opens.
2. In the dialog box click on "COM" panel, select **SPiiPlusCOM 6.60 Type Library**, click **Select**, then **OK**.
3. Declare in your project:

Visual basic:

```
Public Ch As SPIIPLUSCOM660Lib.Channel
```

C#:

```
Public SPIIPLUSCOM660Lib.Channel Ch;
```

In the method that is called when your form is loaded, include:

```
Ch = New SPIIPLUSCOM660Lib.Channel
```

Now you can call SPiiPlus COM Library methods. For example,

```
Ch.OpenCommPCI(-1)
```

4. COM Methods

This chapter details the methods available in the SPiiPlus COM Library. For each method there is a:

- > Brief description
- > Syntax in the form of: **object.method_name(argument, argument, ...)**, where **object** is a placeholder for a valid object name, and each argument is of the form: **type argument_name**.
- > Description of the arguments and their function.
- > Return Value
- > Remarks
- > Example - A short Visual Basic code example.

The methods are broken down into the following categories:

- > Communication Methods
- > EtherCAT® Methods
- > Service Communication Methods
- > ACSPL+ Program Management Methods
- > Read and Write Variables Methods
- > History Buffer Management Methods
- > Unsolicited Messages Buffer Management Methods
- > Log File Management Methods
- > SPiiPlusSC Log File Management Methods
- > Shared Memory Methods
- > System Configuration Methods
- > Setting and Reading Motion Parameters Methods
- > Axis/Motor Management Methods
- > Motion Management Methods
- > Point-to-Point Motion Methods
- > Track Motion Control Method
- > Jog Methods
- > Slaved Motion Methods
- > Multi-Point Motion Methods
- > Arbitrary Path Motion Methods
- > PVT Methods
- > Segmented Motion Methods
- > Points and Segments Manipulation Methods

- > Data Collection Methods
- > Status Report Methods
- > Inputs/Outputs Access Methods
- > Safety Control Methods
- > Wait-for-Condition Methods
- > Event and Interrupt Handling Methods
- > Variables Management Methods
- > Service Methods
- > Error Diagnosis Methods
- > Dual Port RAM (DPRAM) Access Methods
- > Position Event Generation (PEG) Methods
- > Application Save/Load Methods
- > Load/Upload Data To/From Controller Methods
- > Emergency Stop Methods
- > Reboot Methods

4.1 Communication Methods

SPiiPlus COM Communication methods are as follows:

Table 4-1. Communication Methods

CancelOperation	Cancels all operations.
Command	Sends a command to the controller and analyzes the controller response.
CloseComm	Closes communication (for all kinds of communication).
FlushLogFile	Allows flushing the User-Mode Driver (UMD) internal binary buffer to a specified text file, from a user application.
GetBinary	Retrieves binary data from an asynchronous call.
GetConnectionInfo	Retrieves the details of opened communication channel .
GetConnectionsList	Retrieves all currently opened on active server connections and its details.
GetEthernetCards	Retrieves all SPiiPlus controller IP addresses within a local domain.
GetPCICards	Retrieves information about the installed SPiiPlus PCI cards.
GetString	Retrieves string data from an asynchronous call.

OpenCommSimulator	Starts up the Simulator and opens communication with it.
CloseSimulator	Closes the Simulator.
OpenCommEthernetTCP	Opens communication via Ethernet using the TCP protocol.
OpenCommEthernetUDP	Opens communication via Ethernet using the UDP protocol.
OpenCommPCI	Opens communication with the SPiiPlus PCI via PCI Bus.
OpenCommSerial	Opens communication via serial port.
SetServerExtLogin	Sets the remote User-Mode Driver (UMD) with a specified IP address, port number, and user login.
TerminateConnection	Terminates specified communication channel (connection) on active server.
Transaction	Executes one transaction with the controller, i.e., sends the request and receives the controller response.

4.1.1 *CancelOperation*

Description

Cancels all operations.

Syntax

object.CancelOperation()

Arguments

None.

Return Value

None.

Remarks

The method waits for the controller response.

If the method fails, the Error object is filled with the error description.

Example

```
Sub CancelOperation_Sample()
On Error GoTo except
    'The method cancels all the operations
    Call Ch.CancelOperation
Exit Sub
except:
MyErrorMsg    'See Error Handling in Visual Basic
End Sub
```

4.1.2 Command

Description

The method sends a command to the controller and analyzes the controller response.

Syntax

object.Command(string CommandName)

Arguments

CommandName	The command to be sent.
-------------	-------------------------

Return Value

None.

Remarks

The method sends a string containing one or more ACSPL+ commands to the controller. The method verifies that the controller receives the command but does not return the controller response. The method is intended for commands where the controller response does not include any information except the prompt.



For commands where the controller response includes some information, use [Transaction](#).

If the method fails, the Error object is filled with the error description.

Example

```
Sub Command_Sample()  
On Error GoTo except  
  
    'Executed controller's command  
    'Call Ch.Command("enable 0")  
  
Exit Sub  
except:  
    MyErrorMsg    'See Error Handling in Visual Basic  
End Sub
```

4.1.3 CloseComm

Description

The method closes communication via the specified communication channel.

Syntax

object.CloseComm()

Arguments

None

Return Value

None.

Remarks

The method closes the communication channel and releases all system resources related to the channel. If the method closes communication with the Simulator, it also terminates the Simulator.

Each **OpenComm***** (for example, **OpenCommSerial**) call in the application must be followed at some time with a **CloseComm** call in order to return the resources to the system.

If the method fails, the Error object is filled with the error description.

Example

```
Sub CloseComm_Sample()
On Error GoTo except

    Call Ch.CloseComm
Exit Sub
except:
    MyErrorMsg
End Sub
```

'Closes the communication channel

'See [Error Handling in Visual Basic](#)

4.1.4 FlushLogFile**Description**

The method allows flushing the User-Mode Driver (UMD) internal binary buffer to a specified text file, from a user application.

Syntax

object.FlushLogFile(string FileName)

Arguments

FileName

String specifying the text file into which the buffer is to be flushed

Return Value

None.

Remarks

If Continuous Log is active, the function will fail.

Example

```
Sub FlushLogFile()
Dim
On Error GoTo except
except:
    MyErrorMsg
End Sub
```

'See [Error Handling in Visual Basic](#)

4.1.5 GetBinary

Description

The method waits for completion of an asynchronous call and retrieves a binary data.

Syntax

object.GetBinary(long Timeout, object pWait)

Arguments

Timeout	Maximum waiting time in milliseconds. If Timeout is INFINITE, the method's time-out interval never elapses.
pWait	The same pointer that was specified for asynchronous call of a SPiiPlus COM method (see Asynchronous Calls Support).

Return Value

Variant - The type of return value is real or integer, depending on the type of controller variable. The return value can be scalar, vector (one-dimensional array) or matrix (two-dimensional array) depending on data that the controller retrieves.

Remarks

The method waits for completion of asynchronous call, corresponding to **pWait**, and retrieves the controller response as VARIANT. **pWait** must be the same pointer passed to the SPiiPlus COM method when the asynchronous call was initiated.

If the call of SPiiPlus COM method was successful, the function retrieves controller response as VARIANT.

If the SPiiPlus COM method has not been completed in **Timeout** milliseconds or has been aborted by the [CancelOperation](#), the **Error** object is filled with the error description.

Example

```
Sub GetBinary_Sample()
    Dim Var
    Dim pWait
    On Error GoTo except
        'Reading whole variable, if the variable is scalar
        'Parameters: "FPOS" - controller variable name.
        ' Ch.ACSC_NONE - ACSPL+ variable
        Call Ch.ReadVariableAsVector("FPOS(0)", Ch.ACSC_NONE, Ch.ACSC_
ASYNCHRONOUS, pWait)
        'Wait for binary result during 10 sec
        Var = Ch.GetBinary(10000, pWait)
        'Now Var is a vector of size 1
        'Reading whole variable, if the variable is one-dimensional array
        'Parameters: "VEL" - controller variable name.
        ' Ch.ACSC_NONE - ACSPL+ variable
        Call Ch.ReadVariableAsVector ("VEL", Ch.ACSC_NONE, Ch.ACSC_
ASYNCHRONOUS, pWait)
        'Wait for binary result during 10 sec
```

```

        Var = Ch.GetBinary(10000, pWait)
        'Now Var is a vector of size 8
Exit Sub
except:
    MyErrorMsg                                     'See Error Handling in Visual Basic
End Sub

```

4.1.6 GetConnectionsList

Description

This method retrieves all currently opened on active server connections and its details.

Syntax

object.GetConnectionsList(variant HandlesList, variant AppNamesList, variant ProcessIdList)

Arguments

HandlesList	Array of numbers with unique connection handles.
AppNamesList	Array of strings with application names.
ProcessIdList	Array of numbers with Process IDs.

Return Value

Long.

The method returns the number of connections found or zero if there are no connections on active server.

Remarks

Each connection is described by its handle, application name and process ID, therefore, all arguments will have the same size and order, after the method returns.

This method can be used to check if there are some unused connections that remain from applications that did not close the communication channel or were not gracefully closed (terminated or killed).

Each returned connection can be terminated **only** by the [TerminateConnection](#) method.

Using any method of the **SetServer** family makes previously returned connections irrelevant because it changes the active server.

Example

```

{
    object handlesList = null;
    object appNamesList = null;
        object processIdList = null;
    long numOfConnections;
    string name = "neededApplication.exe";
    numOfConnections=channel.GetConnectionsList(out handlesList,out
    appNamesList,out processIdList);
}

```

```

handlesList=(object[])handlesList;
appNamesList=(object[])appNamesList;
processIdList =(object[])processIdList;
for (int index = 0; index < numOfConnections; index++)
{
    if (name.CompareTo((string)appNamesList[index])==0)
        channel.TerminateConnection((long)handlesList[index],
        (string)appNamesList[index]);
    }
}
catch (Exception ex)
{
    ShowException(ex);
}

```

4.1.7 GetEthernetCards

Description

The method retrieves all SPiiPlus controller IP addresses within a local domain through a standard Ethernet communication protocol.

Syntax

object.GetEthernetCards(out variant arrIPAddress, [string BroadcastAddress])

Arguments

arrIPAddress	Array of string with the returned IP addresses of the controllers
BroadcastAddress	IP address for broadcasting. Normally has to be ACSC_NONE .

Return Value

Long.

The method returns the number of cards found or zero if no cards are detected.

Remarks

If more than one controller card is detected, the method initializes **arrIPAddress** as an array, where each element is a string that specifies the IP address returned by a controller card. If no controller card is detected, the method returns zero and does not assign values to the **arrIPAddress** parameters.

Example

```

Sub GetEthernetCards()
    object ipAddresses = null;
    Channel.GetEthernetCards(out ipAddresses, "255.255.255.255");
    foreach (object ipAddress in (object[])ipAddresses) {
        string address = ipAddress.ToString();
    }
On Error GoTo except
except:

```

```
MyErrorMsg 'See Error Handling in Visual Basic
End Sub
```

4.1.8 GetPCICards

Description

This method retrieves information about the controller cards inserted in the computer PCI Bus.

Syntax

object.GetPCICards(variant arrBusNumber, variant arrSlotNumber, variant arrFunction)

Arguments

arrBusNumber	Array of bus numbers that is filled in by the method.
arrSlotNumber	Array of slot numbers of the controller card.
arrFunction	Array of function of the controller card.

Return Value

Long.

The method returns the number of cards found.

Remarks

If no controller cards are detected, the method returns zero and does not assign values to the parameters.

The method also fills the parameters with information about the detected cards.

Parameters **arrSlotNumber** can be used in the [OpenCommPCI](#) call to open communication with a specific card. Other parameters have no use in the SPiiPlus COM Library.

For example, if there are two cards, the method will fill in two rows in **arrSlotNumber**, **arrBusNumber**, **arrFunction**.

If the method fails, the Error object is filled with the error description.

Example

```
Sub GetPCICards_Sample()
    Dim BusNumber As Variant
    Dim SlotNumber As Variant
    Dim MethodV As Variant
    Dim CardsNumber As Long
    On Error GoTo except
        'Retrieves information about the controller
        'cards
        CardsNumber = Ch.GetPCICards(BusNumber, SlotNumber, MethodV)
Exit Sub
except:
    MyErrorMsg 'See Error Handling in Visual Basic
End Sub
```

4.1.9 GetString

Description

The method waits for completion of asynchronous call and retrieves a string data.

Syntax

object.GetString(long Timeout, object pWait)

Arguments

Timeout	Maximum waiting time in milliseconds. If Timeout is INFINITE, the method's time-out interval never elapses.
pWait	The same pointer that was specified for asynchronous call of a SPiiPlus COM method (see Asynchronous Calls Support).

Return Value

String.

Remarks

This method waits for completion of an asynchronous call, corresponding to **pWait**, and retrieves the controller response as String. **pWait** must be the same pointer passed to the SPiiPlus COM method when asynchronous call was initiated.

If the call of the SPiiPlus COM method was successful, function retrieves controller response as String.

If SPiiPlus COM method has not been completed in **Timeout** milliseconds or has been aborted by the [CancelOperation](#), the **Error** object is filled with the error description.

Example

```
Sub GetString_Sample()
    Dim Str As String
    Dim pWait
    On Error GoTo except
    Call Ch.UploadBuffer(0, Ch.ACSC_ASYNCHRONOUS, pWait)
    'Wait for uploading buffer ends during 10 sec
    Str = Ch.GetString(10000, pWait)
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.1.10 OpenCommSimulator

Description

The method executes the SPiiPlus stand-alone simulator via SPiiPlus User Mode Driver in the case that it is not running.

The function connects to the simulator via TCP/IP protocol.

Syntax

object.OpenCommSimulator()

Error codes (in case of failure)

Code 133 – if the ports set for simulator in SPiiPlus User Mode Driver are taken by another application.

Code 136 – in case a simulator execution attempt was made without setting a simulator executable and default simulator executable was not found.

Example

```
Sub OpenCommSimulator_Sample()
On Error GoTo except

                                'Open Ethernet with TCP protocol
                                'communication with the SPiiPlus

simulator

    Call Ch.OpenCommSimulator()
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.1.11 CloseSimulator**Description**

The method stops the SPiiPlus simulator via UMD in the case it is running.

Syntax

object.CloseSimulator()

Error codes (in case of failure)

Code 131 - in case an attempt to stop simulator was made without it running in the first place.

Example

```
Sub CloseSimulator_Sample()
On Error GoTo except

                                'Open Ethernet with TCP protocol
                                'communication with the SPiiPlus simulator

    Call Ch.CloseSimulator()
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.1.12 *OpenCommEthernetTCP*

Description

The method opens communication with the controller via Ethernet using the TCP protocol.

Syntax

object.OpenCommEthernetTCP(string Address, long Port)

Arguments

Address	Pointer to a null-terminated character string that contains the network address of the controller in symbolic or TCP/IP dotted form.
Port	Service port.

Return Value

None.

Remarks

After a channel is open, any SPiiPlus COM method works with the channel irrespective of the physical nature of the channel.

Use TCP protocol if the host computer is connected to the controller through the multi-node Ethernet network.

If the method fails, the Error object is filled with the error description.

Example

```
Sub OpenCommEthernetTCP_Sample()
On Error GoTo except
        'Open Ethernet with TCP protocol
        'communication with the controller
        'TCP/IP address: 10.0.0.100
Call Ch.OpenCommEthernet("10.0.0.100", Ch.ACSC_SOCKET_STREAM_PORT)
Exit Sub
except:
    MyErrorMsg                'See Error Handling in Visual Basic
End Sub
```

4.1.13 *OpenCommEthernetUDP*

Description

The method opens communication with the controller via Ethernet using the UDP protocol.

Syntax

object.OpenCommEthernetUDP(string Address, long Port)

Arguments

Address	Pointer to a null-terminated character string that contains the network address of the controller in symbolic or TCP/IP dotted form.
Port	Service port.

Return Value

None.

Remarks

After a channel is open, any SPiiPlus COM method works with the channel irrespective of the physical nature of the channel.

Use UDP protocol if the host computer has a point-to-point Ethernet connection to the controller.

If the method fails, the Error object is filled with the error description.

Example

```
Sub OpenCommEthernetUDP_Sample()
On Error GoTo except
    'Open Ethernet with UDP protocol
    'communication with the controller
    'TCP/IP address: 10.0.0.100
Call Ch.OpenCommEthernetUDP("10.0.0.100", Ch.ACSC_SOCKET_DGRAM_PORT)
Exit Sub
except:
    MyErrorMsg    'See Error Handling in Visual Basic
End Sub
```

4.1.14 OpenCommPCI**Description**

The method opens a communication channel with the controller via PCI Bus.

Up to 4-communication channels can be open simultaneously with the same SPiiPlus card through the PCI Bus.

Syntax

object.OpenCommPCI(long SlotNumber)

Arguments

SlotNumber	Number of the slot of the controller card. If SlotNumber is ACSC_NONE , the method opens communication with the first found controller card.
------------	---

Return Value

None.

Remarks

To open PCI communication the host PC, one or more controller cards must be inserted into the computer PCI Bus.

After a channel is open, any SPiiPlus COM method works with the channel irrespective of the physical nature of the channel.

If the method fails, the Error object is filled with the error description.

Example

```
Sub OpenCommPCI_Sample()
On Error GoTo except
    'Open communication with the first found
    'controller card
    Call Ch.OpenCommPCI(Ch.ACSC_NONE)
Exit Sub
except:
    MyErrorMsg    'See Error Handling in Visual Basic
End Sub
```

4.1.15 OpenCommSerial**Description**

The method opens communication with the controller via a serial port.

Syntax

object.OpenCommSerial(long Channel, long Rate)

Arguments

Channel	Communication channel: 1 corresponds to COM1, 2 – to COM2, etc.
Rate	Communication rate in bits per second (baud). This parameter must be equal to the controller variable IOBAUD for the successful link with the controller. If ACSC_AUTO property is passed, the method will automatically determine the baud rate.

Return Value

None.

Remarks

After a channel is open, any SPiiPlus COM method works with the channel irrespective of the physical nature of the channel.

If the method fails, the Error object is filled with the error description.

Example

```

Sub OpenCommSerial_Sample()
On Error GoTo except
                                'Open serial communication through port COM1
                                'with baud rate 115200
    Call Ch.OpenCommSerial(1, 115200)
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub

```

4.1.16 GetConnectionInfo**Description**

The method is used to retrieve the details of opened communication channel .

Syntax

object. GetConnectionInfo(CONNECTION_INFO ConnectionInfo);

Arguments

ConnectionInfo	Details of the referenced communication channel.
----------------	--

Return Value

None.

Example

```

SPIIPLUSCOM660Lib.CONNECTION_INFO connectionInfo;
try
{
    Ch.GetConnectionInfo(out connectionInfo);
}
catch (COMException Ex)
{
    ErorMsg(Ex);
}
this.txtLog.AppendText("IP: "+connectionInfo.EthernetIP.ToString());
this.txtLog.AppendText("Ethernet Port: "+connectionInfo.EthernetPort.ToString());
this.txtLog.AppendText("Ethernet Protocol: "+connectionInfo.EthernetProtocol.ToString());
this.txtLog.AppendText("PCI Slot: "+connectionInfo.PCISlot.ToString());
this.txtLog.AppendText("Serial Baud Rate: "+connectionInfo.SerialBaudRate.ToString());
this.txtLog.AppendText("Serial Port: "+connectionInfo.SerialPort.ToString());

```

4.1.16.1 CONNECTION_TYPE

Description

This enum is used for setting communication type. Used in the **GetConnectionInfo** method

Format

```
[
  uuid(4422883A-7E14-4F86-A68C-2112D7D6FF0B)
]
enum CONNECTION_TYPE
{
  NOT_CONNECTED,                //value 0: not connected
  SERIAL,                       //value 1: Serial Communication
  PCI,                          //value 2: PCI Communication
  ETHERNET,                     //value 3: EtherNet Communication
  DIRECT                        //value 4: Direct (Simulator) Communication
};
```

4.1.16.2 CONNECTION_INFO

Description

The structure provides information about specified controller connection for an application. Used in the **GetConnectionInfo** method.

Format

```
[
  uuid(7BD37EE9-6A8D-459E-ABF7-032273E29802)
]
struct CONNECTION_INFO
{
  CONNECTION_TYPE Type;
  int SerialPort;
  int SerialBaudRate;
  int PCISlot;
  int EthernetProtocol;
  BSTR EthernetIP;
  int EthernetPort;
};
```

Members

Type	Connection Type
SerialPort	Communication channel of serial communication: 1 corresponds to COM1, 2 – to COM2, etc.
SerialBaudRate	Communication rate of serial communication in bits per second (baud).
PCISlot	Number of the PCI slot of the controller card.
EthernetProtocol	Ethernet protocol
EthernetIP	Contains the network address of the controller in symbolic or TCP/IP dotted form.
EthernetPort	Service port.

4.1.17 SetServerExtLogin**Description**

The method sets the remote User-Mode Driver (UMD) with a specified IP address, port number, and user login.

Syntax

object.SetServerExtLogin(string IPAddress, string User, string Password, string Domain)

Arguments

IPAddress	IP address of the remote UMD. This address should be the same address displayed in the remote UMD dialog.
User	User name on the remote host.
Password	Password on the remote host.
Domain	Domain name on the remote host.

Return Value

None.

Remarks

The method sets the remote UMD with the specified IP address, port number, and user login information. User login information is required if the remote UMD does not run in the current domain, or the remote computer was not previously logged in to the current domain.

Example

```

Sub SetServerExtLogin()
Dim
On Error GoTo except
except:
    MyErrorMsg                                     'See Error Handling in Visual Basic
End Sub

```

4.1.18 TerminateConnection**Description**

This method terminates specified communication channel (connection) on active server.

Syntax

object.TerminateConnection(long Handle, string AppName)

Arguments

Handle	The value specifies required connection handle.
AppName	The value specifies application name of required connection.

Return Value

None

Remarks

This method can be used to terminate connections that remain from applications that did not close the communication channel or were not gracefully closed (terminated or killed). Both arguments should be passed as these were retrieved by [GetConnectionsList](#) method.

Using any method of the **SetServer** family makes previously returned connections list irrelevant because it changes active server, and obligates new call of **GetConnectionsList**.

Example

```

try
{
    object handlesList = null;
    object appNamesList = null;
    long numOfConnections;
    string name = "neededApplication.exe";
    numOfConnections=channel.GetConnectionsList(out handlesList, out
    appNamesList);
    handlesList=(object[])handlesList;
    appNamesList=(object[])appNamesList;
    for (int index = 0; index < numOfConnections; index++)
    {
        if (name.CompareTo((string)appNamesList[index])==0)
            channel.TerminateConnection((long)handlesList[index],

```

```

(string) appNamesList[index]);
    }
}
catch (Exception ex)
{
    ShowException(ex);
}

```

4.1.19 Transaction

Description

The method executes one transaction with the controller, i.e., it sends a command and receives a controller response.

Syntax

object.Transaction (string Command)

Arguments

Command

Command to be sent.

Return Value

String.

Remarks

The full operation of transaction includes the following steps:

1. Send **Command** to the controller.
2. Wait until the controller response is received or the timeout occurs.
3. Return a controller response.
4. If the method fails, the Error object is filled with the error description.

Example

```

Sub Transaction_Sample()
Dim Cmd As String
Cmd = "?SN"           'get controller serial number
On Error GoTo except
                        'The method executes one transaction with the controller
    reply = Ch.Transaction(Cmd)
Exit Sub
except:
    MyErrorMsg        'See Error Handling in Visual Basic
End Sub

```

4.2 EtherCAT[®] Methods

The EtherCAT methods are:

Table 4-2. EtherCAT Methods

GetEtherCATState	Retrieves the EtherCAT state.
GetEtherCATError	Retrieves the last EtherCAT error code.
MapEtherCATInput	Maps network input variables.
MapEtherCATOutput	Maps network output variables.
UnmapEtherCATInputsOutputs	Cancels mapping of input or output variables.
GetEtherCATSlaveIndex	Retrieves index of specified EtherCAT slave.
GetEtherCATSlaveOffset	Retrieves offset of specified EtherCAT slave.
GetEtherCATSlaveVendorID	Retrieves Vendor ID of specified EtherCAT slave.
GetEtherCATSlaveProductID	Retrieves Product ID of specified EtherCAT slave.
GetEtherCATSlaveRevision	Retrieves revision of specified EtherCAT slave.
GetEtherCATSlaveType	Retrieves type of specified EtherCAT slave.
GetEtherCATSlaveState	Retrieves EtherCAT state of specified EtherCAT slave.

4.2.1 *GetEtherCATState*

Description

The method is used to retrieve the EtherCAT state.

Syntax

object. GetEtherCATState (int* State, ULONGLONG CallType, VARIANT *pWait);

Arguments

State	Pointer to a variable that receives EtherCAT State.
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None.

Remarks

The EtherCAT State contained in the variable designated by the **State** argument is defined by the following bits:

Table 4-3. EtherCAT States

Bit	Name	Description
0	#SCAN	The scan process was performed successfully.
1	#CONFIG	There is no deviation between XML and actual setup.
2	#INITOK	All bus devices are successfully set to INIT state.
3	#CONNECTED	Indicates valid Ethernet cable connection to the master.
4	#INSYNC	If DCM is used, indicates synchronization between master and the bus.
5	#OP	The EtherCAT bus is operational.
6	#DCSYNC	Distributed clocks are synchronized.
7	#RINGMODE	Ring Topology mode status
8	#RINGCOMM	Ring Communication active status
9	#EXTCONN	External clock is connected
10	#DCXSYNC	External clock/slaves are synchronized

All bits (except #INSYNC in some cases) should be true for proper bus functioning. When monitoring the bus state, checking bit #OP is enough. Any bus error will reset the #OP bit.

Return Value

None

Example

```
int Value = new int();
try
{
    Ch.GetEtherCATState(out Value);
}
catch (COMException Ex)
{
    ErorMsg(Ex);
}
```

4.2.2 GetEtherCATError

Description

The method is used to retrieve the last EtherCAT error code.

Syntax

object. GetEtherCATError (int* Error, ULONGLONG CallType, VARIANT *pWait);

Arguments

Error	Pointer to a variable that receives the last EtherCAT error that has occurred.
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Remarks

The EtherCAT error codes are:

Table 4-4. EtherCAT Error Codes

Error	Description
6000	General EtherCAT Error
6001	EtherCAT cable not connected
6002	EtherCAT master is in incorrect State
6003	Not all EtherCAT frames can be processed
6004	EtherCAT Slave Error
6005	EtherCAT initialization failure
6006	EtherCAT can't complete the operation
6007	EtherCAT work count error
6008	Not all EtherCAT slaves are in the OP State
6009	EtherCAT protocol timeout

Error	Description
6010	Slave initialization failed
6011	Bus configuration mismatch

Return Value

None.

Example

```
int Value = new int();
try
{
    Ch.GetEtherCATError(out Value);
}
catch (COMException Ex)
{
    ErorMsg(Ex);
}
```

4.2.3 MapEtherCATInput**Description**

The method is used for raw mapping of network input variables of any size. Once the method is called successfully, the firmware copies the value of the network input variable at the corresponding EtherCAT offset into the ACSPL+ variable every controller cycle.



The method call is legal only when the EtherCAT State is OP.

Syntax

object.MapEtherCATInput (int Flags, int Offset, BSTR VariableName, ULONGLONG CallType, VARIANT *pWait);

Arguments

Flags	Bit-mapped parameter. Currently should be 0.
Offset	Internal EtherCAT offset of network input variable. Should be taken from the response to the #ETHERCAT command in the SPiiPlus MMI Application Studio Communication Terminal or can be seen via EtherCAT Configurator .
VariableName	Valid name of ACSPL+ variable, global or standard

ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None.

Example

In the following example, network variable of EtherCAT node 0 at offset 43 is being mapped to the global variable: **IO**.

```
try
{
Ch.MapEtherCATInput(0, 43, "IO");
}
catch (COMException Ex)
{
    ErrorMsg(Ex);
}
```

4.2.4 MapEtherCATOutput**Description**

The method is used for raw mapping of network output variables of any size. Once the method is called successfully, the firmware copies the value of specified ACSPL+ variable into the network output variable at the corresponding EtherCAT offset, during every controller cycle.



The method call is legal only when the EtherCAT State is OP.

Syntax

object. MapEtherCATOutput (int Flags, int Offset, BSTR VariableName, ULONGLONG CallType, VARIANT *pWait);

Arguments

Flags	Bit-mapped parameter. Currently should be 0.
Offset	Internal EtherCAT offset of network output variable. Should be taken from the response to the #ETHERCAT command in the SPiiPlus MMI Application Studio Communication Terminal or can be seen via EtherCAT Configurator .
VariableName	Valid name of ACSPL+ variable, global or standard
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None.

Example

In the following example, network variable of EtherCAT node 0 at offset 26 is being mapped to global variable: **I1**.

```
try
{
    Ch.MapEtherCATOutput(0, 26, "I1");
}
catch (COMException Ex)
{
    ErrorMsg(Ex);
}
```

4.2.5 UnmapEtherCATInputsOutputs**Description**

The method resets all previous mapping defined by [MapEtherCATInput](#) and [MapEtherCATOutput](#) methods.



The method call is legal only when the EtherCAT State is OP.

Syntax

object. UnmapEtherCATInputsOutputs (ULONGLONG CallType, VARIANT *pWait);

Arguments

ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None.

Example

```
try
{
    Ch.UnmapEtherCATInputsOutputs();
}
catch (COMException Ex)
{
    ErrorMsg(Ex);
}
```

4.2.6 GetEtherCATSlaveIndex

Description

The method retrieves the index of an EtherCAT slave according to provided parameters.

Syntax

object. GetEtherCATSlaveIndex (int VendorID, int ProductID, int Count, double* SlaveIndex, ULONGLONG CallType, VARIANT *pWait);

Arguments

VendorID	EtherCAT Vendor ID of the desired slave.
ProductID	EtherCAT Product ID of the desired slave.
Count	Internal count of devices with the same Product and Vendor IDs.
SlaveIndex	Pointer to a variable that receives the index of EtherCAT slave.
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None.

Example

```

Double Value = new Double();
try
{
    Ch.GetEtherCATSlaveIndex(0x2, 0x10243052, 0, out Value);
}
catch (COMException Ex)
{
    ErorMsg(Ex);
}

```

4.2.7 GetEtherCATSlaveOffset**Description**

The method retrieves the offset of a network variable of a specified EtherCAT slave in EtherCAT telegram.

Syntax

object. GetEtherCATSlaveOffset (BSTR VariableName, int SlaveIndex, double* SlaveOffset, ULONGLONG CallType, VARIANT *pWait);

Arguments

VariableName	Name of the EtherCAT network variable.
SlaveIndex	Index of the required EtherCAT slave, which can be determined by using the GetEtherCATSlaveIndex method.
SlaveOffset	Pointer to a variable that receives the offset of the desired network variable of the specified EtherCAT slave.
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None.

Example

```

Double Value = new Double();
try
{
    Ch.GetEtherCATSlaveOffset("Output", 3, out Value);
}
catch (COMException Ex)
{
    ErorMsg(Ex);
}

```

4.2.8 GetEtherCATSlaveVendorID**Description**

The method retrieves the Vendor ID of a specified EtherCAT slave.

Syntax

object. GetEtherCATSlaveVendorID (int SlaveIndex, double* VendorID, ULONGLONG CallType, VARIANT *pWait);

Arguments

SlaveIndex	Index of the desired EtherCAT slave, which can be determined by using the GetEtherCATSlaveIndex method.
VendorID	Pointer to a variable that receives the vendor ID of specified EtherCAT slave.
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None.

Example

```
Double Value = new Double();
try
{
    Ch.GetEtherCATSlaveVendorID(0, out Value);
}
catch (COMException Ex)
{
    ErorMsg(Ex);
}
```

4.2.9 GetEtherCATSlaveProductID**Description**

The method retrieves the Product ID of a specified EtherCAT slave.

Syntax

object. GetEtherCATSlaveProductID (int SlaveIndex, double* ProductID, ULONGLONG CallType, VARIANT *pWait);

Arguments

SlaveIndex	Index of the desired EtherCAT slave, which can be determined by using the GetEtherCATSlaveIndex method.
ProductID	Pointer to a variable that receives the Product ID of the specified EtherCAT slave.
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None.

Example

```
Double Value = new Double();
try
{
    Ch.GetEtherCATSlaveProductID(0, out Value);
}
catch (COMException Ex)
{
    ErorMsg(Ex);
}
```

4.2.10 GetEtherCATSlaveRevision**Description**

The method retrieves the revision of a specified EtherCAT slave.

Syntax

object. GetEtherCATSlaveRevision (int SlaveIndex, double* Revision, ULONGLONG CallType, VARIANT *pWait);

Arguments

SlaveIndex	Index of the desired EtherCAT slave, which can be determined by using the GetEtherCATSlaveIndex method.
Revision	Pointer to a variable that receives the revision of the specified EtherCAT slave.
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None.

Example

```
Double Value = new Double();
try
{
    Ch.GetEtherCATSlaveRevision(0, out Value);
}
catch (COMException Ex)
{
    ErorMsg(Ex);
}
```

4.2.11 GetEtherCATSlaveType**Description**

The method retrieves the type of a specified EtherCAT slave.

Syntax

object. GetEtherCATSlaveType (int VendorID, int ProductID, double* SlaveType, ULONGLONG CallType, VARIANT *pWait);

Arguments

VendorID	Vendor ID of the EtherCAT slave.
ProductID	Product ID of the EtherCAT slave.
SlaveType	Pointer to a variable that receives the type of specified EtherCAT slave. The value that this variable contains can be one of the following: 0 – ACS device 1 – Non-ACS Servo 2 – Non-ACS Stepper 3 – Non-ACS I/O -1 – Device not found at slave index
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None.

Example

```
Double Value = new Double();
try
{
    Ch.GetEtherCATSlaveType(0x2, 0x044c2c52, out Value);
}
catch (COMException Ex)
{
    ErrorMsg(Ex);
}
```

4.2.12 GetEtherCATSlaveState**Description**

The method retrieves the EtherCAT state of a specified EtherCAT slave.

Syntax

object.GetEtherCATSlaveState (int SlaveIndex, double* SlaveState, ULONGLONG CallType, VARIANT *pWait);

Arguments

SlaveIndex	Index of the desired EtherCAT slave, which can be determined by using the GetEtherCATSlaveIndex method.
SlaveState	Pointer to a variable that receives the the state of specified EtherCAT slave. The value that this variable contains can be one of the following: 1 – INIT 2 – PREOP 4 – SAFEOP 8 – OP -1 – Device not found at slave index
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None.

Example

```
Double Value = new Double();
try
{
    Ch.GetEtherCATSlaveState(0, out Value);
}
catch (COMException Ex)
{
    ErrorMsg(Ex);
}
```

4.3 Service Communication Methods

The Service Communication methods are:

Table 4-5. Service Communication Methods

GetACSCHandle	Retrieves the SPiiPlus C Library communication handle
GetCOMLibraryVersion	Retrieves the SPiiPlus COM Library version number.
GetCommOptions	Retrieves the communication options.
GetDefaultTimeout	Retrieves default communication timeout.
GetErrorString	Retrieves the explanation of an error code.
GetLibraryVersion	Retrieves the legacy SPiiPlus C Library version number.
GetTimeout	Retrieves communication timeout.
SetIterations	Sets the number of iterations of one transaction.
SetCommOptions	Sets the communication options.
SetTimeout	Sets communication timeout.

4.3.1 *GetACSCHandle*

Description

The method retrieves the SPiiPlus C Library communication handle.

Syntax

object.GetACSCHandle()

Arguments

None.

Return Value

Long.

Remarks

The method retrieves the SPiiPlus C Library communication handle.

If the method fails, the Error object is filled with the error description.

Example

```

Sub GetACSCHandle_Sample()
Dim Handle
On Error GoTo except

    Handle = Ch.GetACSCHandle()
Exit Sub
except:
    MyErrorMsg
End Sub

```

'Retrieves communication handle

'See [Error Handling in Visual Basic](#)

4.3.2 GetCOMLibraryVersion**Description**

The method retrieves the SPiiPlus COM Library version number.

Syntax

object.GetCOMLibraryVersion()

Arguments

None.

Return Value

Long.

The method retrieves the SPiiPlus COM Library version number.

Remarks

The SPiiPlus COM Library version consists of four (or less) numbers separated by points: #.#.#.#. The binary version number is represented by 32-bit unsigned integer value. Each byte of this value specifies one number in the following order: high byte of high word – first number, low byte of high word – second number, high byte of low word – third number and low byte of low word – forth number. For example the version “2.10” has the following binary representation (hexadecimal format): 0x020A0000.

The first two numbers in the string form are obligatory. Any release version of the library consists of two numbers. The third and fourth numbers specify an alpha or beta version, special or private build, etc.

If the method fails, the Error object is filled with the error description.

Example

```

Sub GetCOMLibraryVersion_Sample()
Dim LVersion As Double
On Error GoTo except

    LVersion = Ch.GetCOMLibraryVersion
Exit Sub
except:

```

'Retrieves the SPiiPlus COM Library version number.

```
MyErrorMsg                                     'See Error Handling in Visual Basic
End Sub
```

4.3.3 GetCommOptions

Description

The method retrieves the communication options.

Syntax

object.GetCommOptions()

Arguments

None.

Return Value

Long.

The method retrieves the current communication options.

Remarks

To set the communication options call [SetCommOptions](#).

The return value is bit-mapped to represent the current communication options. Currently only the following option flag is supported:

ACSC_COMM_USE_CHECKSUM - communication mode when each command sends to the controller with checksum and the controller responds with checksum.

If the method fails, the Error object is filled with the error description.

Example

```
Sub GetCommOptions_Sample()
Dim Options As Double
On Error GoTo except
                                'Retrieves the current communication options
Options = Ch.GetCommOptions
Options = Options Or Ch.ACSC_COMM_USE_CHECKSUM
Exit Sub
except:
MyErrorMsg                                     'See Error Handling in Visual Basic
End Sub
```

4.3.4 GetDefaultTimeout

Description

The method retrieves default communication timeout.

Syntax

object.GetDefaultTimeout()

Arguments

None.

Return Value

Long.

Remarks

The value of the default timeout depends on the type of the established communication channel. Timeout depends also on the baud rate value for serial communication.

If the method fails, the Error object is filled with the error description.

Example

```
Sub GetDefaultTimeout_Sample()
Dim TimeOut As Double
On Error GoTo except
    'Retrieves default communication timeout
    TimeOut = Ch.GetDefaultTimeout
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.3.5 GetErrorString**Description**

The method retrieves the explanation of an error code.

Syntax

object.GetErrorString(long ErrorCode)

Arguments

ErrorCode	An error code returned by the following methods: ParseCOMErrorCode GetMotorError GetMotionError GetProgramError
------------------	---

Return Value

String.

The method retrieves the string that contains the text explanation of the error code returned by the [ParseCOMErrorCode](#), [GetMotorError](#), [GetMotionError](#), and [GetProgramError](#) methods.

Remarks

If the error relates to SPiiPlus COM Library, the method returns immediately with the text explanation. If the error relates to the controller, the method receives the text explanation from the controller.

If the method fails, the Error object is filled with the error description.

Example

```

Sub GetErrorString_Sample()
Dim Str As String
On Error GoTo except

        'The method retrieves the explanation of
        'an error code 3260.

    Str = Ch.GetErrorString(3260)
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub

```

4.3.6 GetLibraryVersion**Description**

The method retrieves the legacy SPiiPlus C Library version number.

Syntax

object.GetLibraryVersion()

Arguments

None.

Return Value

Long.

The method retrieves the legacy SPiiPlus C Library version number.

Remarks

The SPiiPlus C Library version consists of four (or less) numbers separated by points: #.#.#.#. The binary version number is represented by 32-bit unsigned integer value. Each byte of this value specifies one number in the following order: high byte of high word – first number, low byte of high word – second number, high byte of low word – third number and low byte of low word – forth number. For example the version "2.10" has the following binary representation (hexadecimal format): 0x020A0000.

The first two numbers in the string form are obligatory. Any release version of the library consists of two numbers. The third and fourth numbers specify an alpha or beta version, special or private build, etc.

If the method fails, the Error object is filled with the error description.

Example

```

Sub GetLibraryVersion_Sample()
Dim LVersion As Double
On Error GoTo except
    'Retrieves the SPiiPlus C Library version number.
    LVersion = Ch.GetLibraryVersion
Exit Sub
except:

```

```
MyErrorMsg                                     'See Error Handling in Visual Basic
End Sub
```

4.3.7 *GetTimeout*

Description

The method retrieves communication timeout.

Syntax

object.GetTimeout()

Arguments

None.

Return Value

Long.

The method retrieves communication timeout.

Remarks

If the method succeeds, the return value is the current timeout value in milliseconds.

If the method fails, the Error object is filled with the error description.

Example

```
Sub GetTimeout_Sample()
Dim TimeO As Long
On Error GoTo except
                                'Retrieves communication timeout
    TimeO = Ch.GetTimeout
Exit Sub
except:
    MyErrorMsg                                     'See Error Handling in Visual Basic
End Sub
```

4.3.8 *SetIterations*

Description

The method sets the number of iterations in one transaction.

Syntax

object.SetIterations(long Iterations)

Arguments

Iterations	Number of iterations

Return Value

None.

Remarks

If, after the transmission of command to the controller, the controller response is not received during the predefined time, the library repeats the command transmission. The number of those iterations is defined by **Iterations** parameter for each communication channel independently.

Most of the SPiiPlus COM methods perform communication with the controller by transactions (i.e. they send commands and wait for responses) that are based on the [Transaction](#) method. Therefore, the changing of the number of iterations can have an influence on the behavior of the user application.

The default number of iterations for all communication channels is 2.

If the method fails, the Error object is filled with the error description.

Example

```
Sub SetIterations_Sample()
On Error GoTo except

    'Sets the number of iterations in one
    'transaction number of iterations for all
    'communication channels is 2.

    Call Ch.SetIterations(2)
Exit Sub
except:
    MyErrorMsg          'See Error Handling in Visual Basic
End Sub
```

4.3.9 SetCommOptions**Description**

The method sets the communication options.

Syntax

object.SetCommOptions(long Options)

Arguments

Options	Communication options to be set. Bit-mapped parameter that can include one of the following flags: ACSC_COMM_USE_CHECKSUM: the communication mode used when each command is sent to the controller with checksum and the controller also responds with checksum. ACSC_COMM_AUTORECOVER_HW_ERROR: when a hardware error is detected in the communication channel and this bit is set, the library automatically repeats the transaction, without counting iterations.
----------------	--

Return Value

None.

Remarks

The method sets the communication options. To get current communication option, call [GetCommOptions](#).

To add some communication options to the current configuration, modify **Options** that has been filled in by a call to [GetCommOptions](#). This ensures that the other communication options will have same values.

If the method fails, the Error object is filled with the error description.

Example

```
Sub SetCommOptions_Sample()
    'The example sets the mode with checksum
    Dim Options As Long
    On Error GoTo except
        Options = Ch.GetCommOptions
        Options = Options Or Ch.ACSC_COMM_USE_CHECKSUM
        Call Ch.SetCommOptions(Options)
    Exit Sub
except:
    MyErrorMsg 'See Error Handling in Visual Basic
End Sub
```

4.3.10 SetTimeout**Description**

The method sets the communication timeout.

Syntax

object.SetTimeout(long Timeout)

Arguments

Timeout	Maximum waiting time in milliseconds.

Return Value

None.

Remarks

The method sets the communication timeout.

All of the subsequent waiting calls of the methods will wait for the controller response **Timeout** in milliseconds. If the controller does not respond to the sent command during this time, SPiiPlus COM methods return with zero value. In this case, the the Error object is assigned **ACSC_TIMEOUT**.

If the method fails, the Error object is filled with the error description.

Example

```

Sub SetTimeout_Sample()
Dim TimeO As Double
On Error GoTo except
    'Sets the communication timeout of 2000
    Call Ch.SetTimeout(2000)
    TimeO = Ch.GetTimeout
Exit Sub
except:
    MyErrorMsg 'See Error Handling in Visual Basic
End Sub

```

4.4 ACSPL+ Program Management Methods

The ACSPL+ Program Management methods are:

Table 4-6. ACSPL+ Program Management Methods

AppendBuffer	Appends one or more ACSPL+ lines to the program in the specified program buffer.
ClearBuffer	Deletes the specified ACSPL+ program lines in the specified program buffer.
CompileBuffer	Compiles ACSPL+ program in the specified program buffer(s).
LoadBuffer	Clears the specified program buffer and then loads ACSPL+ program to this buffer.
LoadBuffersFromFile	Opens a file that contains one or more ACSPL+ programs allocated to several buffers and download the programs to the corresponding buffers.
RunBuffer	Starts up ACSPL+ program in the specified program buffer.
StopBuffer	Stops ACSPL+ program in the specified program buffer(s).
SuspendBuffer	Suspends ACSPL+ program in the specified program buffer(s).
UploadBuffer	Uploads ACSPL+ program from the specified program buffer.

4.4.1 AppendBuffer**Description**

The method appends one or more ACSPL+ lines to the program in the specified buffer.

Syntax

object.AppendBuffer(long Buffer, string Program, long Count)

Arguments

Buffer	Number of a program buffer in the controller..
Program	Pointer to the buffer containing an ACSPL+ program(s).
Count	Number of ACSPL+ program lines in the buffer.

Return Value

None.

Remarks

The method appends one or more ACSPL+ lines to the program in the specified buffer. If the buffer already contains any program, the new text is appended to the end of the existing program.

No compilation or syntax check is provided during downloading. In fact, any text, not only a correct program, can be inserted into a buffer. In order to compile the program and check its accuracy, the compile command must be executed after downloading.

If the method fails, the Error object is filled with the error description.

Example

```
Sub AppendBuffer_Sample()
Dim StringBuf As String
StringBuf = "enable 0;jog 0;stop" & Chr$(10)
On Error GoTo except
                                'The method appends one ACSPL+ line to
                                'buffer 0
    Call Ch.AppendBuffer(0, StringBuf(1))
Exit Sub
except:
    MyErrorMsg                    'See Error Handling in Visual BasicEnd Sub
```

4.4.2 ClearBuffer**Description**

Deletes the specified ACSPL+ program lines in the specified program buffer.

Syntax

object.ClearBuffer(long Buffer, long FromLine, long ToLine)

Arguments

Buffer	Buffer number, from 0 to 9.
FromLine, ToLine	These arguments specify a range of lines to be deleted. FromLine starts from 1. If ToLine is larger then the total number of lines in the specified program buffer, the range includes the last program line.

Return Value

None.

Remarks

The method deletes the specified ACSPL+ program lines in the specified program buffer.

If the method fails, the Error object is filled with the error description.

Example

```
Sub ClearBuffer_Sample()
Dim StringBuf(1) As String
StringBuf(0) = "enable 0;jog 0;stop" & Chr$(10)
On Error GoTo except
        'Appends the line "enable 0;jog 0;stop" to
        'buffer 0
    Call Ch.AppendBuffer(0, StringBuf(1))
        'Delete buffer 0 from line 1 to 1000
    Call Ch.ClearBuffer(0, 1, 1000)
    StringBuf(1) = Ch.UploadBuffer(0)
Exit Sub
except:
    MyErrorMsg          'See Error Handling in Visual BasicEnd Sub
```

4.4.3 CompileBuffer**Description**

The method compiles the ACSPL+ program in the specified program buffer(s).

Syntax

object.CompileBuffer(long Buffer)

Arguments

Buffer

Buffer number, from 0 to 9.

You can use **ACSC_NONE** instead of the buffer number to compile all programs in all buffers.

Return Value

None.

Remarks

The method compiles an ACSPL+ program in the specified program buffer or all programs in all buffers if **Buffer** is **ACSC_NONE**.



If attempting to compile the D-Buffer, all other buffers will be stopped and put in a non-compiled state.

The method succeeds if the compile command was transmitted successfully to the controller, i.e., the communication channel is OK and the specified buffer was not running. However, this does not mean that the compile operation was completed successfully.

In order to get information about the results of the compile operation, use [ReadVariable](#) to read **PERR**, which contains the most recent error that occurred in each buffer. If **PERR** is zero, the buffer was compiled successfully.

Otherwise, **PERR** contains the error that occurred during the compilation.

The method waits for the controller response.

If the method fails, the Error object is filled with the error description.

Example

```
Sub CompileBuffer_Sample()
Dim StringBuf(1) As String
StringBuf(1) = "!Compiled buffer;stop" & Chr$(10)
On Error GoTo except
        'Append a line to buffer 0
        Call Ch.AppendBuffer(0, StringBuf(1))
        'The method compiles ACSPL+ program in
        'buffer 0
        Call Ch.CompileBuffer(0)
Exit Sub
except:
    MyErrorMsg          'See Error Handling in Visual BasicEnd Sub
```

4.4.4 LoadBuffer

Description

The method clears the specified program buffer and then loads ACSPL+ program to this buffer.

Syntax

object.LoadBuffer(long Buffer, string Program, long Count)

Arguments

Buffer	Number of a program buffer in the controller.
Program	Pointer to the buffer containing an ACSPL+ program(s).
Count	Number of ACSPL+ program lines in the buffer.

Return Value

None.

Remarks

The method clears the specified program buffer and then loads ACSPL+ program to this buffer.

No compilation or syntax check is provided during downloading. Any text, not only a correct program, can be inserted into a buffer. In order to compile the program and check its accuracy, the compile command must be executed after downloading.

If the method fails, the Error object is filled with the error description.

Example

```
Sub LoadBuffer_Sample()
Dim Str As String
Str = "!This is a test ACSPL+ program;Stop" & Chr$(10)
On Error GoTo except
    'Load a line to buffer 0
    Call Ch.LoadBuffer(0, Str)
Exit Sub
except:
MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.4.5 LoadBuffersFromFile

Description

The method opens a file that contains one or more ACSPL+ programs allocated to several buffers and download the programs to the corresponding buffers.



Before calling the function LoadBuffersFromFile, the buffers must be reset or cleared.

- > To reset the buffers use the command: Api.Transaction("##SR").
- > To clear the buffer use the function: Api.(long Buffer, long FromLine, long ToLine).

Syntax

object.LoadBuffersFromFile(string Filename)

Arguments

Filename

Name and path of the file to be loaded.

Return Value

None.

Remarks

The method analyzes the file, determines which program buffers should be loaded, clears them and then loads ACSPL+ programs to those buffers.



The method can be called only synchronously.

SPiiPlus software tools save ACSPL+ programs in the following format:

```
# Header: Date, Firmware version, etc.
#Buf1 (buffer number)
ACSPL+ program of Buf1
#Buf2 (buffer number)
ACSPL+ program of Buf2
#Buf3 (buffer number)
ACSPL+ program of Buf3 etc.
```

The number of buffers in file may change from 1 to 10, without any default order.

No compilation or syntax check is provided during downloading. Any text, not only a correct program, can be inserted into a buffer. In order to compile the program and check its accuracy, the compile command must be executed after downloading.

If the method fails, the Error object is filled with the error description.

Example

```
Sub LoadBuffersFromFile_Sample()
Dim FileP As String
FileP = "C:\\ACSPLFile.txt"
On Error GoTo except
                                'Opens a file to load
    Call Ch.LoadBuffersFromFile(FileP)
Exit Sub
except:
MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.4.6 RunBuffer

Description

The method starts up ACSPL+ program in the specified buffer.

Syntax

object.RunBuffer(long Buffer, [string Label])

Arguments

Buffer	Number of a program buffer in the controller.
Label	Label in the program from which the execution starts. If NULL ("" - the default) is specified instead of a pointer, the execution starts from the first line.

Return Value

None.

Remarks

The method starts up ACSPL+ program in the specified buffer. The execution starts from the specified label, or from the first line if the label is not specified.

If the program was not compiled before, the method first compiles the program and then starts it. If an error was encountered during compilation, the program does not start.

If the program was suspended by the [SuspendBuffer](#) method, the method resumes the program execution from the powhere the program was suspended.

The method waits for the controller response.

The controller response indicates that the program in the specified buffer was started successfully. The method does not wait for the program end. To wait for the program end, use the [WaitProgramEnd](#) method.

If the method fails, the Error object is filled with the error description.

Example

```
Sub RunBuffer_Sample()
Dim Index As Long
Dim PState As Long

                                'Appends and runs buffers 0 to 8
On Error GoTo except
  For Index = 0 To 8
    Call Ch.AppendBuffer(Index, "ST:")
    Call Ch.AppendBuffer(Index, "!an empty loop")
    Call Ch.AppendBuffer(Index, "goto ST")
    Call Ch.RunBuffer(Index)
  Next Index
Exit Sub
except:
  MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.4.7 StopBuffer**Description**

The method stops the execution of ACSPL+ program in the specified buffer(s).

Syntax

object.StopBuffer(long Buffer)

Arguments

Buffer	Buffer number, from 0 to 9. To stop the execution of all buffers, use ACSC_NONE instead of the buffer number.
---------------	---

Return Value

None.

Remarks

The method stops ACSPL+ program execution in the specified buffer or in all buffers if **Buffer** is **ACSC_NONE**.

The method has no effect if the program in the specified buffer is not running.

If the method fails, the Error object is filled with the error description.

Example

```
Sub StopBuffer_Sample()
Dim Index As Long
On Error GoTo except
  For Index = 0 To 8
    'Appends a line to all buffers
    Call Ch.AppendBuffer(Index, "ST:;! Empty buffer;goto ST")
    'Run all buffers
    Call Ch.RunBuffer(Index)
    'Stop running all the buffers
    Call Ch.StopBuffer(Index)
  Next Index
Exit Sub
except:
  MyErrorMsg      'See Error Handling in Visual Basic
End Sub
```

4.4.8 SuspendBuffer**Description**

The method suspends the execution of ACSPL+ program in the specified program buffer(s).

Syntax

object.SuspendBuffer(long Buffer)

Arguments

Buffer	Buffer number, from 0 to 9. To suspend the execution of all buffers, use ACSC_NONE instead of the buffer number.
---------------	--

Return Value

None.

Remarks

The method suspends ACSPL+ program execution in the specified buffer or in all buffers if **Buffer** is **ACSC_NONE**. The method has no effect if the program in the specified buffer is not running.

To resume execution of the program in the specified buffer, call the [RunBuffer](#) method.

If the method fails, the Error object is filled with the error description.

Example

```

Sub SuspendBuffer_Sample()
Dim Index As Long
On Error GoTo except

    For Index = 0 To 8

        'Appends all buffers
        Call Ch.AppendBuffer(Index, "ST;!! Empty buffer;goto ST")
        'Run all buffers
        Call Ch.RunBuffer(Index)
        'Suspends all buffers
        Call Ch.SuspendBuffer(Index)
    Next Index
Exit Sub
except:
    MyErrorMsg      'See Error Handling in Visual Basic
End Sub

```

4.4.9 UploadBuffer**Description**

The method uploads ACSPL+ program from the specified program buffer.

Syntax

object.UploadBuffer(long Buffer)

Arguments

Buffer	Number of a program buffer in the controller.
---------------	---

Return Value

String.

Remarks

The method uploads ACSPL+ program from the specified program buffer.

If the method fails, the Error object is filled with the error description.

Example

```

Sub UploadBuffer_Sample()
Dim Str(1) As String
Str(1) = "!This is an empty buffer" & Chr$(10)
On Error GoTo except

    'Appends buffer 0 with Str(1)
    Call Ch.AppendBuffer(0, Str(1))
    'Uploading buffer 0 from line 0
    Str(0) = Ch.UploadBuffer(0)
Exit Sub
except:

```

```
MyErrorMsg      'See Error Handling in Visual Basic
End Sub
```

4.5 Read and Write Variables Methods

The Read and Write Variable methods are:

Table 4-7. Read and Write Variables Methods

ReadVariable	Reads variable from a controller and returns it in the form of VARIANT
ReadVariableAsScalar	Reads variable from a controller and returns it as scalar VARIANT
ReadVariableAsVector	Reads variable from a controller and returns it in the form of VARIANT that contains one-dimension array
ReadVariableAsMatrix	Reads variable from a controller and returns it in the form of VARIANT that contains two-dimensions array
WriteVariable	Writes to controller variable(s)

4.5.1 ReadVariable

Description

The method reads a variable from the controller.

Syntax

object.ReadVariable(string Variable, long NBuf, [long From1], [long To1], [long From2], [long To2])

Arguments

Variable	Name of the controller variable
NBuf	Number of program buffer for local variable or ACSC_NONE for global and ACSPL+ variables.
From1, To1	Index range (first dimension) starting from zero. Default value: ACSC_NONE .
From2, To2	Index range (second dimension) starting from zero. Default value: ACSC_NONE .

Return Value

VARIANT.

The type of return value is real or integer, depending on the type of controller variable..

The return value can be scalar, vector (one-dimensional array) or matrix (two-dimensional array) depending on data that the controller retrieves.

Remarks

The method reads scalar variables, vectors, rows of matrix, columns of matrix or matrices from a controller and retrieves data as VARIANT. The resultant data are initialized depending on what data is received from the controller: scalar, vector or matrix.

If you want the return value to be in some specified type, use [ReadVariableAsScalar](#), [ReadVariableAsVector](#) or [ReadVariableAsMatrix](#) methods.

The variable can be a ACSPL+ controller variable, a user global variable, or a user local variable.

ACSPL+ and user global variables have global scope; therefore **NBuf** must be **ACSC_NONE** (-1) for these classes of variables.

User local variable exists only within a buffer. The buffer number must be specified for user local variable.

The index parameters can be specified as follows:

- > To read the whole variable
- > To read one element of the variable
- > To read several elements of the variable

The following explains how to specify indexes in several typical cases.

1. Reading whole variable

The **From1**, **To1**, **From2** and **To2** indexes should be omitted or specified as **ACSC_NONE**. The method returns all elements of **Variable** as VARIANT initialized as:

Scalar, if the Variable is scalar

Vector, if the Variable is a one-dimensional array

Matrix, if the Variable is a two-dimensional array

Reading one element from vector (one-dimensional array)

The **From1** and **To1** should be equal and specify the index of the element. **From2** and **To2** should be omitted or specified as **ACSC_NONE**. The method returns the element value as VARIANT, initialized as scalar.

2. Reading one element from matrix (two-dimensional array)

From1 should equal **To1** and specify row number. The **From2** and **To2** should be equal and specify column number. The method returns the element value as VARIANT, initialized as scalar method

3. Reading a sub-vector (more than one vector element)

From1 and **To1** should specify the sub-vector index range, where **From1** should be less than **To1**. **From2** and **To2** should be omitted or specified as **ACSC_NONE**. The method returns data as VARIANT, initialized as a vector with the number of elements equal to **To1-From1+1**.

4. Reading a row or part of a row from a matrix

From1 should equal **To1** and specify the row number. The **From2** and **To2** should specify the element range within the specified row, where **From2** should be less than **To2**. The method returns data as VARIANT, initialized as a vector with the number of elements equal **To2-From2+1**.

5. Reading a column or part of a column from a matrix

From1 and **To1** should specify the range of rows within the specified column, where **From1** should be less than **To1**. **From2** and **To2** should be equal and specify the column number. The method returns data as VARIANT, initialized as a vector with the number of elements equal to **To1-From1+1**.

6. Reading sub-matrix from matrix

From1 and **To1** should specify the range of rows, where **From1** should be less than **To1**. **From2** and **To2** should specify columns range, where **From2** should be less than **To2**. The method returns data as VARIANT, initialized as a matrix with the number of rows equal to **To1-From1+1** and the number of columns equal to **To2-From2+1**.

If the method fails, the Error object is filled with the error description.

Example

```
Sub ReadVariable_Sample()
Dim Var
On Error GoTo except
'Reading whole variable, if the variable is scalar
'Parameters: "BAUD " - controller variable name
'           Ch.ACSC_NONE - ACSPL+ variable
Var = Ch.ReadVariable ("BAUD ", Ch.ACSC_NONE)
'Now Var is a scalar value
'Reading whole variable, if the variable is one-dimensional array
'Parameters: "VEL" - controller variable name
'           Ch.ACSC_NONE - ACSPL+ variable
Var = Ch.ReadVariable("VEL", Ch.ACSC_NONE)
'Now Var is a vector of size 8
'Reading whole Variable, if the Variable is two-dimensional array
'Assumed that MyMatrix variable is declared as global MyMatrix(3) (4)
in the
'controller
'Parameters: "MyMatrix - user variable name
'           Ch.ACSC_NONE - global variable
Var = Ch.ReadVariable ("MyMatrix", Ch.ACSC_NONE)
'Now Var is a matrix of size 3x4
'Reading one element from vector (one-dimensional array)
'Reading one element with index 3
'Parameters: "VEL" - controller variable name
'           Ch.ACSC_NONE - ACSPL+ variable
'           3 - From1
'           3 - To1
Var = Ch.ReadVariable("VEL", Ch.ACSC_NONE, 3,3)
'Now Var is a scalar value

'Reading one element from matrix (two-dimensional array)
'Reading one element of matrix from row 1,column 2)
'Assumed that MyMatrix variable is declared as global MyMatrix(3) (4)
in the
'controller
```

```

'Parameters: "MyMatrix - user variable name
'           Ch.ACSC_NONE - global variable
'           1 - From1
'           1 - To1
'           2 - From2
'           2 - To2
Var = Ch.ReadVariable ("MyMatrix ", Ch.ACSC_NONE, 1, 1, 2, 2)
'Now Var is a scalar value
'Reading sub-vector (more then one element) from vector
'Reading sub-vector with indexes from 1 to 5
'Parameters: "VEL" - controller variable name
'           Ch.ACSC_NONE - ACSPL+ variable
'           1 - From1
'           5 - To1
Var = Ch.ReadVariable("VEL", Ch.ACSC_NONE,1,5)
'Now Var is a vector of size 5
'Reading row or part of row from matrix
'Reading a part of row 1 from element with index 0 till element with
' index 2
'Assumed that MyMatrix variable is declared as global MyMatrix(3)(4)
in the
'controller
'Parameters: "MyMatrix - user variable name
'           Ch.ACSC_NONE - global variable
'           1 - From1
'           1 - To1
'           0 - From2
'           2 - To2
Var = Ch.ReadVariable ("MyMatrix", Ch.ACSC_NONE, 1,1,0,2)
'Now Var is a vector of size 3
'Reading column or part of column from matrix
'Reading a part of column 1 from element with index 0 till element
with
' index 1
'Assumed that MyMatrix variable is declared as global MyMatrix(3)(4)
in the
'controller
'Parameters: "MyMatrix - user variable name
'           Ch.ACSC_NONE - global variable
'           0 - From1
'           1 - To1
'           1 - From2
'           1 - To2
Var = Ch.ReadVariable("MyMatrix ", Ch.ACSC_NONE, 0,1,1,1)
'Now Var is a vector of size 2
'Reading sub-matrix from matrix
'Reading sub-matrix from with rows from 0 to 1 and columns from 0 to
2
'Assumed that MyMatrix variable is declared as global MyMatrix(3)(4)

```

```

in the
    'controller
    'Parameters: "MyMatrix - user variable name
    '              Ch.ACSC_NONE - global variable
    '              0 - From1
    '              1 - To1
    '              0 - From2
    '              2 - To2
    Var = Ch.ReadVariable ("MyMatrix ", Ch.ACSC_NONE, 0,1,0,2)
    'Now Var is a matrix 2x3
Exit Sub
except:
MyErrorMsg                                     'See Error Handling in Visual Basic
End Sub

```

4.5.2 ReadVariableAsScalar

Description

The method reads the variable from a controller and returns as a scalar.

Syntax

object.ReadVariableAsScalar (string Variable, long NBuf, [long Ind1], [long Ind2])

Arguments

Variable	Name of the controller variable
NBuf	Number of program buffer for local variable or ACSC_NONE for user global and ACSPL+ variables.
Ind1	Index of first dimension (row number) starting from zero. Default value: ACSC_NONE .
Ind2	Index of second dimension (column number) starting from zero. Default value: ACSC_NONE .

Return Value

VARIANT.

The return value is scalar, that is, the type of return value is real or integer, corresponding to the type of controller variable.

Remarks

Reads a scalar variable, one element of a vector or one element of a matrix from a controller and returns the value as scalar.

If you require a method return value as one vector element, use the [ReadVariableAsVector](#) method.

If you require a method return value as one matrix element, use the [ReadVariableAsMatrix](#) method.

The variable can be an ACSPL+ variable, a user-defined global variable, or a user-defined local variable.

ACSPL+ and user global variables have global scope; therefore **NBuf** must be **ACSC_NONE** (-1) for these classes of variables.

User-defined local variables exist only within a buffer. The buffer number must be specified for user-defined local variables.

The following explains how to read indexes in several typical cases.

1. Reading a scalar variable:

The **Ind1** and **Ind2** indexes should be omitted or specified as **ACSC_NONE**.

2. Reading one element from a vector (one-dimensional array):

Ind1 should specify index of the element in the vector. **Ind2** should be omitted or specified as **ACSC_NONE**.

3. Reading one element from a matrix (two-dimensional array):

Ind1 should specify a row number and **Ind2** should specify a column number of the element in the matrix.

If the method fails, the Error object is filled with the error description.

Example

```
Sub ReadVariableAsScalar_Sample()
    'Var in the next 3 examples is returned as scalar
    Dim Var
    Dim Str As String
    On Error GoTo except
    'Reading scalar variable
    'Parameters: "BAUD" - controller variable name.
    '           Ch.ACSC_NONE - ACSPL+ variable
    Var = Ch.ReadVariableAsScalar ("BAUD", Ch.ACSC_NONE)
    'Reading one element from vector (one-dimensional array)
    'Reading one element with index 2
    'Parameters: "VEL" - controller variable name.
    '           Ch.ACSC_NONE - ACSPL+ variable
    '           2 - Ind1
    Var = Ch.ReadVariableAsScalar ("VEL", Ch.ACSC_NONE, 2)
    'Reading one element from matrix (two-dimensional array)
    'Reading one element of matrix from row 1, column 2)
    'Assumed that MyMatrix variable is declared as global MyMatrix(3)(4)
    in the controller
    'Parameters: "MyMatrix" - user variable name
    '           Ch.ACSC_NONE - global variable
    '           1 - row number
    '           2 - column number
    Var = Ch.ReadVariableAsScalar ("MyMatrix", Ch.ACSC_NONE, 1, 2)
Exit Sub
except:
```

```
MyErrorMsg 'See Error Handling in Visual Basic
End Sub
```

4.5.3 ReadVariableAsVector

Description

The method reads a variable from a controller and returns it as vector.

Syntax

object.ReadVariable(string Variable, long NBuf, [long From1], [long To1], [long From2], [long To2])

Arguments

Variable	Name of the controller variable
NBuf	Number of program buffer for local variable or ACSC_NONE for global and ACSPL+ variables.
From1, To1	Index range (first dimension) starting from zero. Default value: ACSC_NONE .
From2, To2	Index range (second dimension) starting from zero. Default value: ACSC_NONE .

Return Value

VARIANT.

The return value is a one-dimensional array (vector). The type of return value is real or integer, corresponding to the type of controller variable.

Remarks

If you want a return value as scalar or matrix use [ReadVariableAsScalar](#) or [ReadVariableAsMatrix](#) methods correspondingly.

The variable can be an ACSPL+ variable, a user global variable, or a user local variable.

ACSPL+ and user global variables have global scope; therefore **NBuf** must be **ACSC_NONE** (–1) for these classes of variables.

User local variables exist only within a buffer. The buffer number must be specified for user local variable.

The index parameters can be specified in many different ways to read whole variable, one element of the variable or several elements of the variable. The following explains how to specify indexes in several typical cases.

1. Reading whole variable:

The **From1**, **To1**, **From2** and **To2** indexes should be omitted or specified as **ACSC_NONE**.

The method returns all elements of **Variable** as VARIANT initialized as:

Vector of size 1, if the **Variable** is scalar

Vector of size N, if the **Variable** is vector of size N

If the **Variable** is matrix, the method will return an error.

2. Reading sub-vector from vector:

The **From1** and **To1** should specify sub-vector index range, **From1** should be less than or equal to **To1**. The **From2** and **To2** should be omitted or specified as **ACSC_NONE**. The methods returns data as vector with number of elements equals to **To1-From1+1**.

3. Reading a row or part of a row from matrix:

The **From1** and **To1** should be equal and specify row number. The **From2** and **To2** should specify elements range within the specified row, **From2** should be less than or equal to **To2**. The methods returns data as vector with number of elements equals to **To2-From2+1**.

4. Reading a column or part of a column from matrix:

The **From1** and **To1** should specify rows range within the specified column, **From1** should be less than or equal to **To1**. The **From2** and **To2** should be equal and specify column number. The methods returns data as vector with number of elements equals to **To1-From1+1**.

If the method fails, the Error object is filled with the error description.

Example

```
'Reading column or part of column from matrix
'Reading a part of column 1 from element with index 0 till element
with
' index 1
'Assumed that MyMatrix variable is declared as global MyMatrix(3) (4)
'in the controller
'Parameters: "MyMatrix - user variable name
'            Ch.ACSC_NONE - global variable
'            0 - From1
'            1 - To1
'            1 - From2
'            1 - To2
Var = Ch.ReadVariableAsVector("MyMatrix ", Ch.ACSC_NONE, 0,1,1,1)
'Now Var is a vector of size 2
Exit Sub
except:
MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.5.4 ReadVariableAsMatrix

Description

The method reads the variable from a controller and returns it as matrix.

Syntax

object.ReadVariableAsMatrix(string Variable, long NBuf, [long From1], [long To1], [long From2], [long To2])

Arguments

Variable	Name of the controller variable
NBuf	Number of program buffer for local variable or ACSC_NONE for global and ACSPL+ variables.
From1, To1	Index range (first dimension) starting from zero. Default value: ACSC_NONE .
From2, To2	Index range (second dimension) starting from zero. Default value: ACSC_NONE .

Return Value

VARIANT.

The return value is a two-dimensional array (matrix). The type of return value is real or integer, corresponding to the type of controller variable.

Remarks

If you want a return value as scalar or vector use [ReadVariableAsScalar](#) or [ReadVariableAsVector](#) methods correspondingly.

The variable can be an ACSPL+ controller variable, a user global variable, or a user local variable.

ACSPL+ variables and user global variables have a global scope; therefore **NBuf** must be **ACSC_NONE** (-1) for these classes of variables.

User local variables exist only within a buffer. The buffer number must be specified for user local variable.

The index parameters can be specified in many different ways to read whole variable, one element of the variable or several elements of the variable. The following explains how to specify indexes in several typical cases.

- > Reading a whole variable

From1, To1, From2 and **To2** indexes should be omitted or specified as **ACSC_NONE**. The method returns all elements of **Variable** as VARIANT initialized as follows:

Matrix 1x1, if the **Variable** is scalar

Matrix 1xN, if the **Variable** is vector of size N

Matrix NxM, if the **Variable** matrix NxM

- > Reading a sub-vector from a vector

From1 and **To1** should specify the sub-vector index range, **From1** should be less than or equal to **To1**. **From2** and **To2** should be omitted or specified as **ACSC_NONE**. The method returns data as VARIANT, initialized as a matrix of size (**To1-From1+1**)x1.

- > Reading a sub-matrix from a matrix

From1 and **To1** should specify a range of rows, where **From1** should be less than or equal to **To1**. **From2** and **To2** should specify a range of columns, where **From2** should be less than or

equal to **To2**. The method returns data as VARIANT, initialized as matrix with the number of rows equal to **To1-From1+1** and the number of columns equals **To2-From2+1**.

If the method fails, the Error object is filled with the error description.

Example

```

Sub ReadVariableAsVector_Sample()
    'Var in the next 3 examples is returned as vector
Dim Var
Dim Str As String
On Error GoTo except
'Reading whole variable, if the variable is scalar
    'Parameters: "BAUD" - controller variable name.
    '                Ch.ACSC_NONE - ACSPL+ variable
    Var = Ch.ReadVariableAsVector("BAUD ", Ch.ACSC_NONE)
    'Now Var is a vector of size 1
'Reading whole variable, if the variable is one-dimensional array
    'Parameters: "VEL" - controller variable name.
    '                Ch.ACSC_NONE - ACSPL+ variable
    Var = Ch.ReadVariableAsVector ("VEL", Ch.ACSC_NONE)
    'Now Var is a vector of size 8
'Reading sub-vector from vector
    'Reading sub-vector with indexes from 1 to 5
    'Parameters: "VEL" - controller variable name
    '                Ch.ACSC_NONE - ACSPL+ variable
    '                1 - From1
    '                5 - To1
    Var = Ch.ReadVariableAsVector("VEL", Ch.ACSC_NONE,1,5)
    'Now Var is a vector of size 5
'Reading row or part of row from matrix
    'Reading a part of row 1 from element with index 0 till element with
    ' index 2
    'Assumed that MyMatrix variable is declared as global MyMatrix(3)(4)
    'in the controller
    'Parameters: "MyMatrix" - user variable name
    '                Ch.ACSC_NONE - global variable
    '                1 - From1
    '                1 - To1
    '                0 - From2
    '                2 - To2
    Var = Ch.ReadVariableAsVector("MyMatrix", Ch.ACSC_NONE, 1,1,0,2)
    'Now Var is a vector of size 3
Sub ReadVariableAsMatrix_Sample()
    'Var in the next 3 examples is returned as matrix
Dim Var
On Error GoTo except
'Reading whole variable, if the variable is scalar
    'Read the controller scalar variable BAUD to variable Var
    'Parameters: "BAUD " - controller variable name.
    '                Ch.ACSC_NONE - ACSPL+ variable
    Var = Ch.ReadVariableAsMatrix ("BAUD ", Ch.ACSC_NONE)
    'Now Var is a matrix of size 1x1
'Reading whole variable, if the variable is one-dimensional array
    'Parameters: "VEL" - controller variable name.
    '                Ch.ACSC_NONE - ACSPL+ variable

```

```

    Var = Ch.ReadVariableAsMatrix ("VEL", Ch.ACSC_NONE)
    'Now Var is a matrix of size 1x8
'Reading whole variable, if the variable is two-dimensional array
    'Assumed that MyMatrix variable is declared as global MyMatrix(3) (4)
in the
    'controller
    'Parameters: "MyMatrix - user variable name
    '              Ch.ACSC_NONE - global variable
    Var = Ch.ReadVariableAsMatrix("MyMatrix ", Ch.ACSC_NONE)
    'Now Var is a matrix of size 3x4
'Reading sub-vector (more then one element) from vector
    'Reading sub-vector with indexes from 1 to 5
    'Parameters: "VEL" - controller variable name
    '              Ch.ACSC_NONE - ACSPL+ variable
    '              1 - From1
    '              5 - To1
    Var = Ch.ReadVariableAsMatrix ("VEL", Ch.ACSC_NONE,1,5)
    'Now Var is a matrix of size 1x5
'Reading sub-matrix from matrix
    'Reading sub-matrix from with rows from 0 to 1 and colomns from 0 to
2
    'Assumed that MyMatrix variable is declared as global MyMatrix(3) (4)
in the
    'controller
    'Parameters: "MyMatrix - user variable name
    '              Ch.ACSC_NONE - global variable
    '              0 - From1
    '              1 - To1
    '              0 - From2
    '              2 - To2
    Var = Ch.ReadVariableAsMatrix("MyMatrix ", Ch.ACSC_NONE, 0,1,0,2)
    'Now Var is a matrix 2x3
Exit Sub
except:MyErrorMsg                                'See Error Handling in Visual Basic
End Sub

```

4.5.5 WriteVariable

Description

The method writes a value defined as VARIANT to the controller variable(s).

Syntax

object.WriteVariable(variant Value, string Variable, long NBuf, [long From1], [long To1], [long From2], [long To2])

Arguments

Value	Value to be assigned to the Variable . The Value can contain an integer or real number(s). The Value can be scalar, one-dimensional array (vector) or two-dimensional array (matrix).
Variable	Name of the controller variable
NBuf	Number of program buffer for local variable or ACSC_NONE for global and ACSPL+ variables.
From1, To1	Index range (first dimension) starting from zero of Variable (not of Value). Default value: ACSC_NONE .
From2, To2	Index range (second dimension) starting from zero of Variable (not of Value). Default value: ACSC_NONE .

Return Value

None.

Remarks

This method writes **Value** to the specified **Variable**. **Variable** can be scalar, vector (one-dimensional array) or matrix (two-dimensional array).

Variable can be either an ASCPL+ variable, user global variable or a user local variable.

ASCPL+ and user global variables have global scope; therefore **NBuf** must be **ACSC_NONE** (–1) for these classes of variables.

User local variable exist only within a buffer. The buffer number must be specified for user local variable.

The index parameters can be specified in many different ways. Write the entire variable, a single element of the variable or several elements of the variable as follows:

- > Write value to whole variable

From1, To1, From2 and **To2** indexes should be omitted or specified as **ACSC_NONE**.

The **Value** dimension should correspond to the **Variable** dimension as follows:

If **Variable** is scalar, **Value** should be scalar, or vector of size 1, or matrix of size 1x1

If **Variable** is vector of size N, **Value** should be vector of size N, or matrix of size 1xN, or matrix of size Nx1

If **Variable** is matrix of size NxM, **Value** should be matrix of size NxM

- > Writing a value to one element of vector (one-dimensional array)

The **From1** and **To1** should be equal and specify index of the element. **From2** and **To2** should be omitted or specified as **ACSC_NONE**.

- > Writing value to one element of matrix (two-dimensional array)

The **From1** and **To1** should be equal and specify row number. The **From2** and **To2** should be equal and specify column number.

- > Writing a value to sub-vector (more then one element) of vector

The **From1** and **To1** should specify sub-vector index range, **From1** should be less then **To1**. The **From2** and **To2** should be omitted or specified as **ACSC_NONE**.

- > Writing a value to row or part of row of matrix

The **From1** and **To1** should be equal and specify row number. The **From2** and **To2** should specify elements range within the specified row, **From2** should be less then **To2**.

- > Writing a value to column or part of column of matrix

The **From1** and **To1** should specify rows range within the specified column, **From1** should be less than **To1**. The **From2** and **To2** should be equal and specify column number.

- > Writing a value to sub-matrix of matrix

The **From1** and **To1** should specify rows range, **From1** should be less then **To1**. The **From2** and **To2** should specify columns range, **From2** should be less then **To2**.

If indexes (**From1**, **To1**, **From2**, **To2**) are specified, their values must correspond to **Value** dimensions described below.

Table 4-8. Value Dimension Indices

Value Dimension	Indices
Scalar	From1=To1 and From2=To2
Vector of size N	(To1-From1+1=N and To2=From2) or (To1=From1 and To2-From2+1=N)
Matrix of size NxM	To1-From1+1=N and To2-From2+1=M

Value type is not required to be the same as **Variable** type. The library provides automatic conversion from integer to real and from real to integer.

If the method fails, the Error object is filled with the error description.

Example

```
Sub WriteVariable_Sample()
Dim Scalar, Vector(7), Matrix(2, 3), SubVector(1), SubMatrix(1, 1)
On Error GoTo except
'Initialize Scalar
Scalar = 57600
'Initialize Vector
For Index = 0 To 7
    Vector(Index) = 1.1
Next Index
'Initialize Matrix
For Row = 0 To 2
    For Colomn = 0 To 3
```

```

        Matrix(Row, Column) = 2
    Next Column
Next Row

'Initialize SubVector
SubVector(0) = 3
SubVector(1) = 3
    'Initialize SubMatrix
For Row = 0 To 1
    For Column = 0 To 1
        SubMatrix(Row, Column) = 4
    Next Column
Next Row

'Writing scalar to whole variable
'Parameters:    Scalar - Value to be assigned to the Variable
'               "BAUD " - controller variable name
'               Ch.ACSC_NONE - ACSPL+ variable
Call Ch.WriteVariable(Scalar, "BAUD", Ch.ACSC_NONE)

'Writing vector(one-dimensional array)
'to the whole variable
'Parameters:    Vector - Value to be assigned to the Variable
'               "VEL" - controller variable name
'               Ch.ACSC_NONE - ACSPL+ variable
Call Ch.WriteVariable(Vector, "VEL", Ch.ACSC_NONE)

'Writing matrix to whole variable
'Assumed that MyMatrix variable is
'declared as global MyMatrix(3)(4) in
'the controller
'Parameters:    Matrix - Value to be assigned to the Variable
'               "MyMatrix" - user variable name
'               Ch.ACSC_NONE - global variable
Call Ch.WriteVariable(Matrix, "MyMatrix", Ch.ACSC_NONE)

'Writing value to one element of vector (one-
'dimensional array)
'Parameters:    Scalar - Value to be assigned to the Variable
'               "VEL" - controller variable name
'               Ch.ACSC_NONE - ACSPL+ variable
Call Ch.WriteVariable(Scalar, "VEL", Ch.ACSC_NONE, 2,2)

'Writing value to one element of matrix
'(two-dimensional array)
'Assumed that MyMatrix variable is
'declared as global MyMatrix(3)(4) in
'the controller
'Parameters:    Scalar - Value to be assigned to the Variable
'               "MyMatrix" - user variable name
'               Ch.ACSC_NONE - global variable
'               1 - From1
'               1 - To1
'               2 - From2
'               2 - To2
Call Ch.WriteVariable(Scalar, "MyMatrix", Ch.ACSC_NONE, 1, 1, 2, 2)

```

```

'Writing value to sub-vector (more then one element) of vector
'Reading sub-vector with indexes from 1 to 5
'Parameters: SubVector - Value to be assigned to the Variable
'           "VEL" - controller variable name
'           Ch.ACSC_NONE - ACSPL+ variable
'           2 - From1
'           3 - To1
Call Ch.WriteVariable(SubVector, "VEL", Ch.ACSC_NONE, 2, 3)
'Writing value to row or part of row of matrix
'Assumed that MyMatrix variable is declared as global MyMatrix(3) (4)
in
    'the controller
'Parameters: SubVector - Value to be assigned to the Variable
'           "MyMatrix" - user variable name
'           Ch.ACSC_NONE - global variable
'           1 - From1
'           1 - To1
'           2 - From2
'           3 - To2
Call Ch.WriteVariable(SubVector, "MyMatrix", Ch.ACSC_NONE, 1, 1, 2, 3)
'Writing value to column or part of column of matrix
'Assumed that MyMatrix variable is declared as global MyMatrix(3) (4)
in
    'the controller
'Parameters: SubVector - Value to be assigned to the Variable
'           "MyMatrix" - user variable name
'           Ch.ACSC_NONE - global variable
'           0 - From1
'           1 - To1
'           2 - From2
'           2 - To2
Call Ch.WriteVariable(SubVector, "MyMatrix", Ch.ACSC_NONE, 0, 1, 2, 2)
'Writing value to sub-matrix of matrix
'Assumed that MyMatrix variable is declared as global MyMatrix(3) (4)
in
    'the controller
'Parameters: SubMatrix - Value to be assigned to the Variable
'           "MyMatrix" - user variable name
'           Ch.ACSC_NONE - global variable
'           0 - From1
'           1 - To1
'           1 - From2
'           2 - To2
Call Ch.WriteVariable(SubMatrix, "MyMatrix", Ch.ACSC_NONE, 0, 1, 1, 2)
except:
MyErrorMsg      'See Error Handling in Visual Basic
End Sub
End Sub

```

4.6 History Buffer Management Methods

The History Buffer Management methods are:

Table 4-9. History Buffer Management Methods

OpenHistoryBuffer	Opens a history buffer.
CloseHistoryBuffer	Closes a history buffer.
GetHistory	Retrieves the contents of the history buffer.

4.6.1 OpenHistoryBuffer

Description

The method opens a history buffer.

Syntax

object.OpenHistoryBuffer(long Size)

Arguments

Size	Required size of the buffer in bytes
-------------	--------------------------------------

Return Value

None.

Remarks

The method allocates a history buffer that stores all commands sent to the controller and all responses and unsolicited messages received from the controller.

Only one history buffer can be open for each communication channel.

The buffer works as a cyclic buffer. When the amount of the stored data exceeds the buffer size, the newly stored data overwrites the earliest data in the buffer.

If the method fails, the Error object is filled with the error description.

Example

```
Sub OpenHistoryBuffer_Sample()
On Error GoTo except
    'The method opens a history buffer
    'size of the buffer is 5000
    Call Ch.OpenHistoryBuffer(5000)
Exit Sub
except:
    MyErrorMsg    'See Error Handling in Visual Basic
End Sub
```

4.6.2 CloseHistoryBuffer

Description

The method closes the history buffer and discards all stored history.

Syntax

object.CloseHistoryBuffer()

Arguments

None.

Return Value

None.

Remarks

The method closes the history buffer and releases the used memory. All information stored in the buffer is discarded.

If the method fails, the Error object is filled with the error description.

Example

```
Sub CloseHistoryBuffer_Sample()
On Error GoTo except
                                'The method closes the history buffer
                                'and discards all stored history
    Call Ch.CloseHistoryBuffer
Exit Sub
except:
    MyErrorMsg                    'See Error Handling in Visual Basic
End Sub
```

4.6.3 GetHistory**Description**

The method retrieves the contents of the history buffer.

Syntax

object.GetHistory(Boolean bClear)

Arguments

bClear

If TRUE, the method clears contents of the history buffer.
If FALSE, the history buffer content is not cleared.

Return Value

String

The method retrieves the communication history from the history buffer.

Remarks

The communication history includes all commands sent **to** the controller and all responses and unsolicited messages **from** the controller. The amount of history data is limited by the size of the history buffer. The history buffer works as a cyclic buffer: when the amount of the stored data exceeds the buffer size, the newly stored data overwrites the earliest data in the buffer.

Therefore, as a rule, the retrieved communication history includes only recent commands, responses and unsolicited messages. The depth of the retrieved history depends on the history buffer size.

The history data is retrieved in historical order, i.e. the earliest message is stored at the beginning of the returned string. The beginning of the return string might be incomplete, if it has been partially overwritten in the history buffer.

If the method fails, the Error object is filled with the error description.

Example

```
Sub GetHistory_Sample()
Dim Str As String
On Error GoTo except

        'The method opens a history buffer
        'size of the buffer is 5000
    Call Ch.OpenHistoryBuffer(5000)

        'Sending the command "enable 0"
    Call Ch.Send("enable 0")

        'The method retrieves the contents of the
        'history buffer.
    Str = Ch.GetHistory(False)
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.7 Unsolicited Messages Buffer Management Methods

The Unsolicited Messages Buffer Management methods are:

Table 4-10. Unsolicited Messages Buffer Management Methods

OpenMessageBuffer	Opens an unsolicited messages buffer.
CloseMessageBuffer	Closes an unsolicited messages buffer.
GetSingleMessage	Retrieves single unsolicited message from the buffer.

4.7.1 OpenMessageBuffer

Description

The method opens an unsolicited messages buffer.

Syntax

object.OpenMessageBuffer(long Size)

Arguments

Size	Required size of the buffer in bytes
-------------	--------------------------------------

Return Value

None.

Remarks

The method allocates a buffer that stores unsolicited messages from the controller.

Unsolicited messages are messages that the controller sends on its own initiative and not as a response to command. For example, the **disp** command in an ACSPL+ program forces the controller to send an unsolicited message.

Only one message buffer can be open for each communication channel.

The message buffer works as a FIFO buffer: **GetSingleMessage** extracts the earliest message stored in the buffer. If **GetSingleMessage** extracts the messages slower than the controller produces them, buffer overflow can occur, and some messages will be lost. Generally, the greater the buffer, the less likely is buffer overflow to occur.

If the method fails, the Error object is filled with the error description.

Example

```
Sub OpenMessageBuffer_Sample()
On Error GoTo except
                                'The method opens an unsolicited messages
                                'buffer size of the opened buffer is 5000
    Call Ch.OpenMessageBuffer(5000)
Exit Sub
except:
    MyErrorMsg                    'See Error Handling in Visual Basic
End Sub
```

4.7.2 CloseMessageBuffer**Description**

The method closes the messages buffer and discards all stored unsolicited messages.

Syntax

object.CloseMessageBuffer()

Arguments

None.

Return Value

None.

Remarks

The method closes the message buffer and releases the used memory. All unsolicited messages stored in the buffer are discarded.

If the method fails, the Error object is filled with the error description.

Example

```

Sub CloseMessageBuffer_Sample()
On Error GoTo except
                                'The method closes the messages buffer
                                'and discards all stored unsolicited
                                'messages
    Call Ch.CloseMessageBuffer
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub

```

4.7.3 GetSingleMessage**Description**

The method retrieves single unsolicited message from the buffer. This method only works if you setup a buffer using [OpenMessageBuffer](#). If there is no message in the buffer the method waits until the message arrives or time out expires.

Syntax

object.GetSingleMessage(long Timeout)

Arguments

Timeout	Maximum waiting time in milliseconds.

Return Value

String.

Remarks

The method retrieves single unsolicited message from the message buffer.

If no, unsolicited message is received the method waits until a message arrives.

If the timeout expires, the method exits with **ACSC_TIMEOUT** error.

If the method fails, the Error object is filled with the error description.

Example

```

Sub GetSingleMessage_Sample()
Dim Str As String
Dim Cmd As String
Cmd = "?FPOS0"
On Error GoTo except
                                'The method opens an unsolicited messages
                                'buffer, size of the opened buffer is 5000
    Call Ch.OpenMessageBuffer(5000)
                                'Sending the command "?FPOS0"
    Call Ch.Send(Cmd)
                                'The method retrieves unsolicited message

```

```

        'from the buffer and wait 1 second till a
        'message arrives
    Str = Ch.GetSingleMessage(1000)
Exit Sub
except:
    MyErrorMsg          'See Error Handling in Visual Basic
End Sub

```

4.8 Log File Management Methods

The Log File Management methods are:

Table 4-11. Log File Management Methods

OpenLogFile	Opens a log file.
CloseLogFile	Closes a log file.
WriteLogFile	Writes to a log file.

4.8.1 OpenLogFile

Description

The method opens a log file.

Syntax

object.OpenLogFile(long FileName)

Arguments

FileName	Pointer to the null-terminated string containing the name or path and name of the log file.
-----------------	---

Return Value

None.

Remarks

The method opens a binary file that stores all communication history.

Only one log file can be open for each communication channel.

If the log file has been open, the library writes all incoming and outgoing messages to the specified file. The messages are written to the file in binary format, i.e., exactly as they are received and sent, including all service bytes.

Unlike the history buffer, the log file cannot be read within the library. The main usage of the log file is for debug purposes.

If the method fails, the Error object is filled with the error description.

Example

```
Sub OpenLogFile_Sample()  
On Error GoTo except  
    'Opens log file  
    Call Ch.OpenLogFile("C:\COMLogFile.log")  
Exit Sub  
except:  
    MyErrorMsg          'See Error Handling in Visual Basic  
End Sub
```

4.8.2 CloseLogFile

Description

The method closes the log file.

Syntax

object.CloseLogFile()

Arguments

None.

Return Value

None.

Remarks

An application must always call [CloseLogFile](#) before it exits. Otherwise, the data written to the file might be lost.

If the method fails, the Error object is filled with the error description.

Example

```
Sub CloseLogFile_Sample()  
On Error GoTo except  
    'The method opens the log file  
    'COMLogFile.log"  
    Call Ch.OpenLogFile("C:\COMLogFile.log")  
    'The method closes the log file  
    'COMLogFile.log"  
    Call Ch.CloseLogFile  
Exit Sub  
except:  
    MyErrorMsg          'See Error Handling in Visual Basic  
End Sub
```

4.8.3 WriteLogFile

Description

The method writes to log file.

Syntax

object.WriteLogFile(string Buf, long Count)

Arguments

Buf	Pointer to the buffer that contains the string to be written to log file.
Count	Number of characters to be written.

Return Value

None.

Remarks

The method writes data from a buffer to log file. The log file should be previously opened by the [OpenLogFile](#) method.

The method adds its arguments to the internal UMD binary buffer. In previous C Library versions, the log file had to be explicitly opened by the [OpenLogFile](#) method, otherwise the function would return an error. Starting with COM Library version 5.0, this is no longer the case. See, [FlushLogFile](#).

If the method fails, the Error object is populated with the error description.

Example

```
Sub WriteLogFile_Sample()
On Error GoTo except
    'Opens a log file
    Call Ch.OpenLogFile("C:\COMLogFile.log")
    'Write to log file
    Call Ch.WriteLogFile("Writing to log file")
    'Close the log file
    Call Ch.CloseLogFile
Exit Sub
except:
    MyErrorMsg      'See Error Handling in Visual Basic
End Sub
```

4.8.4 GetLogData**Description**

The method is used to retrieve the data of firmware log.

Syntax

object.GetLogData (BSTR* Buf, int Count, int* Recevied, ULONGLONG CallType, VARIANT *pWait);

Arguments

Buf	Pointer to the buffer that receives the log data.
Count	Size of the buffer in bytes.
Receivied	Number of characters that were actually received.
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None.

Example

```

string Buff = "";
int Value=0;
try
{
    Ch.GetLogData(out Buff, 300000, out Value);
}
catch (COMException Ex)
{
    ErorMsg(Ex);
}

```

4.9 SPiiPlusSC Log File Management Methods

These methods can only be used with the SPiiPlusSC Motion Controller.

The SPiiPlusSC Log File Management methods are:

Table 4-12. SPiiPlusSC Log File Management Methods

OpenSCLogFile	Used for opening the SPiiPlusSC log file.
CloseSCLogFile	Used for closing the SPiiPlusSC log file.
WriteSCLogFile	Used for writing to the SPiiPlusSC log file.
FlushSCLogFile	Enables flushing the SPiiPlusSC internal buffer to a specified text file from the COM Library application.

4.9.1 *OpenSCLogFile*

Description

The method opens the SPiiPlusSC log file.

Syntax

object.OpenSCLogFile(BSTR FileName);

Arguments

FileName

Pointer to a buffer containing the name or path of the log file.

Return Value

None.

Remarks

The method opens a binary file that stores all SPiiPlusSC log history. The messages are written to the file in binary format, i.e., exactly as they are received and sent, including all service bytes.

The main use of the SPiiPlusSC log file is for debugging purposes.

Example

```

try
{
    Ch.OpenSCLogFile("C:\\SCLogFile2.txt");
}
catch (COMException Ex)
{
    ErrorMsg(Ex);
}

```

4.9.2 *CloseSCLogFile*

Description

The method closes the SPiiPlusSC log file.

Syntax

object.CloseSCLogFile();

Arguments

None.

Return Value

None.

Remarks

An application must always call **CloseSCLogFile** before it exits; otherwise, the data written to the file might be lost.

Example

```
try
{
    Ch.CloseSCLogFile();
}
catch (COMException Ex)
{
    ErorMsg(Ex);
}
```

4.9.3 WriteSCLogFile

Description

The method writes to the SPiiPlusSC log file.

Syntax

object. WriteSCLogFile(BSTR Buf, int Count);

Arguments

Buf	Pointer to a buffer containing the string to be written to the log file.
Count	Number of characters in the buffer.

Return Value

None.

Remarks

The method writes data from a buffer to the SPiiPlusSC log file.

Example

```
try
{
    Ch.WriteSCLogFile("Hello World!", ("Hello World!").Length);
}
catch (COMException Ex)
{
}
```

```

    ErorMsg (Ex) ;
}

```

4.9.4 FlushSCLogFile

Description

The method enables flushing the SPiiPlusSC internal buffer to a specified text file from the COM Library application.

Syntax

object. FlushSCLogFile (BSTR FileName, BOOL bClear);

Arguments

FileName	String that specifies the file name.
bClear	Can be TRUE or FALSE: TRUE – the method clears contents of the log buffer. FALSE – the log buffer content is not cleared.

Return Value

None.

Remarks

If Continuous Log is active, the function will fail.

Example

```

try
{
    Ch.FlushSCLogFile ("C:\\SCLogFile3.txt", 1);
}
catch (COMException Ex)
{
    ErorMsg (Ex) ;
}

```

4.10 Shared Memory Methods



The Shared Memory methods have been added in support of SPiiPlus SC to enable accessing memory addresses. They cannot be used with any other SPiiPlus family product.

The Shared Memory methods are:

Table 4-13. Shared Memory Methods

GetSharedMemoryAddress	Reads the address of shared memory variable.
ReadSharedMemoryInteger	Reads value(s) from an integer shared memory variable.
WriteSharedMemoryInteger	Writes value(s) to an integer shared memory variable.
ReadSharedMemoryReal	Reads value(s) from a real shared memory variable.
WriteSharedMemoryReal	Writes value(s) to a real shared memory variable.

4.10.1 *GetSharedMemoryAddress*

Description

The method reads the address of shared memory variable.

Syntax

object.[GetSharedMemoryAddress](#) (long NBuf, BSTR Var, long* Address, ULONGLONG CallType, VARIANT *pWait);

Arguments

NBuf	Number of program buffer for local variable or ACSC_NONE for global and ACSPL+ variable.
Var	Pointer to a buffer that contains a name of the variable.
Address	Pointer to the variable that receives the address of specified variable.
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None.

4.10.2 ReadSharedMemoryInteger

Description

The method reads value(s) from an integer shared memory variable.

Syntax

object. ReadSharedMemoryInteger (long Address, VARIANT* Values, long From1, long To1, long From2, long To2,);

Arguments

Address	Shared memory address of the variable that should be read.
Values	Pointer to the buffer that receives requested values.
From1, To1	Index range (first dimension) for one dimensional array variables.
From2, To2	Index range (second dimension) for two dimensional matrix variables.

Return Value

None.

Example

See [Shared Memory Program Example](#).

4.10.3 WriteSharedMemoryInteger

Description

The method writes value(s) to the integer shared memory variable.

Syntax

object. WriteSharedMemoryInteger (long Address, VARIANT Values, long From1, long To1, long From2, long To2);

Arguments

Address	Shared memory address of the variable is to written to.
Values	Pointer to the buffer containing values that should be written.
From1, To1	Index range (first dimension) for one dimensional array variables.
From2, To2	Index range (second dimension) for two dimensional matrix variables.

Return Value

None.

Example

See [Shared Memory Program Example](#).

4.10.4 ReadSharedMemoryReal

Description

The method reads value(s) from a real shared memory variable.

Syntax

object. ReadSharedMemoryReal (long Address, VARIANT* Values ,long From1, long To1, long From2, long To2,);

Arguments

Address	Shared memory address of the variable that should be read.
Values	Pointer to the buffer that receives requested values.
From1, To1	Index range (first dimension) for one dimensional array variables.
From2, To2	Index range (second dimension) for two dimensional matrix variables.

Return Value

None.

Example

See [Shared Memory Program Example](#).

4.10.5 WriteSharedMemoryReal

Description

The method writes value(s) to the real shared memory variable.

Syntax

object. WriteSharedMemoryInteger (long Address, VARIANT Values ,long From1, long To1, long From2, long To2,);

Arguments

Address	Shared memory address of the variable is to written to.
Values	Pointer to the buffer containing values that should be written.
From1, To1	Index range (first dimension) for one dimensional array variables.
From2, To2	Index range (second dimension) for two dimensional matrix variables.

Return Value

None.

Example

See [Shared Memory Program Example](#).

4.10.6 Shared Memory Program Example

```
try
{
    int address1 = new int();
    int address2 = new int();
    double[,] Values1 = new double[2,2];
    int[,] Values2 = new int[2,2];
    object Result1, Result2;
    Values1[0,0] = 666.66;
        Values1[0,1] = 777.77;
        Values1[1,0] = 888.88;
        Values1[1,1] = 999.99;
        Values2[0,0] = 1;
        Values2[0,1] = 2;
        Values2[1,0] = 3;
        Values2[1, 1] = 4;
    Ch.GetSharedMemoryAddress(Ch.ACSC_NONE, "realVariable", out address1);
    Ch.WriteSharedMemoryReal(address1, Values1, 0, 1, 0, 1);
    Ch.ReadSharedMemoryReal(address1, out Result1, 0, 1, 0, 1);
    Ch.GetSharedMemoryAddress(Ch.ACSC_NONE, "intVariable", out address2);
    Ch.WriteSharedMemoryInteger(address2, Values2, 0, 1, 0, 1);
    Ch.ReadSharedMemoryReal(address2, out Result2, 0, 1, 0, 1);
}
catch (COMException Ex)
{
    ErrorMsg(Ex);
}
```

4.11 System Configuration Methods

The System Configuration methods are:

Table 4-14. System Configuration Methods

SetConf	The method writes system configuration data.
GetConf	The method reads system configuration data.
GetVolatileMemoryUsage	The function retrieves the volatile memory load in percentage.
GetVolatileMemoryTotal	The function retrieves the amount of total volatile memory.

GetVolatileMemoryFree	The function retrieves the amount of free volatile memory.
GetNonVolatileMemoryUsage	The function retrieves the non-volatile memory load in percentage.
GetNonVolatileMemoryUsage	The function retrieves the amount of total non-volatile memory.
GetNonVolatileMemoryFree	The function retrieves the amount of free non-volatile memory.

4.11.1 SetConf

Description

The method writes system configuration data.

Syntax

object.SetConf(long Key, long Index, double Value)

Arguments

Key	Configuration Key (see Configuration Keys) that specifies the configured feature. Assigns value of key argument in ACSPL+ SetConf command (see <i>SPiiPlus Command & Variable Reference Guide</i>).
Index	Specifies corresponding axis or buffer number. Assigns value of index argument in ACSPL+ SetConf command.
Value	Value to write to specified key. Assigns value of value argument in ACSPL+ SetConf command.

Return Value

None.

Remarks

The method writes system configuration data. **Key** specifies the feature number and **Index** defines the axis or buffer to which it should be applied. Use ACSC_CONF_XXX properties in the value field. For detailed description of system configuration see the description of the **SetConf** method in the *SPiiPlus Command and Variable Guide*.

If the method fails, the Error object is filled with the error description.

Example

```
Sub SetConf_Sample()
On Error GoTo except
    'Writes system configuration data of axis 0
    'to 1 with key 205
    Call Ch.SetConf(Ch.ACSC_CONF_DIGITAL_SOURCE_KEY, Ch.ACSC_AXIS_0, 1)
```

```
Exit Sub
except:
    MyErrorMsg                                     'See Error Handling in Visual Basic
End Sub
```

4.11.2 GetConf

Description

The method reads system configuration data.

Syntax

object.GetConf(long Key, long Index)

Arguments

Key	Configuration Key (see Configuration Keys) that specifies the configured feature. Assigns value of key argument in ACSPL+ GetConf command (see <i>SPiiPlus Command & Variable Reference Guide</i>).
Index	Specifies corresponding axis or buffer number. Assigns value of index argument in ACSPL+ GetConf command.

Return Value

Double.

The method reads system configuration data.

Remarks

Key specifies the feature number and **Index** defines axis or buffer to which it should be applied. For detailed description of system configuration see *SPiiPlus Command & Variable Reference Guide*.

If the method fails, the Error object is filled with the error description.

Example

```
Sub GetConf_Sample()
    Dim GetC As Double
    On Error GoTo except
                                'Reads system configuration data of axis 0
                                'with key 205
    GetC = Ch.GetConf(Ch.ACSC_CONF_DIGITAL_SOURCE_KEY, Ch.ACSC_AXIS_0)
    Exit Sub
except:
    MyErrorMsg                                     'See Error Handling in Visual Basic
End Sub
```

4.11.3 GetVolatileMemoryUsage

Description

The function retrieves the volatile memory load in percentage.

Syntax

object.GetVolatileMemory(double* Value, ULONGLONG CallType, VARIANT* pWait)

Arguments

Value[out]	The variable that receives the volatile memory load in percentage.
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None

Example

```
object waitObject = 0;
Double Value = new Double();
try
{
    Ch. GetVolatileMemoryUsage(0, out Value, Ch.ACSC_SYNCHRONOUS, ref
waitObject);
}
catch (COMException Ex)
{
    ErorMsg(Ex);
}
```

4.11.4 GetVolatileMemoryTotal**Description**

The function retrieves the amount of total volatile memory.

Syntax

object.GetVolatileMemoryTotal(double* Value, ULONGLONG CallType, VARIANT* pWait)

Arguments

Value[out]	The variable that receives the amount of total volatile memory in bytes.
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None

Example

```

object waitObject = 0;
Double Value = new Double();
try
{
    Ch. GetVolatileMemoryTotal(0, out Value, Ch.ACSC_SYNCHRONOUS, ref
waitObject);
}
catch (COMException Ex)
{
    ErorMsg(Ex);
}

```

4.11.5 GetVolatileMemoryFree**Description**

The function retrieves the amount of free volatile memory.

Syntax

object.GetVolatileMemoryFree(double* Value, ULONGLONG CallType, VARIANT* pWait)

Arguments

Value[out]	The variable that receives the amount of free volatile memory in bytes.
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None

Example

```

object waitObject = 0;
Double Value = new Double();
try
{
    Ch. GetVolatileMemoryFree (0, out Value, Ch.ACSC_SYNCHRONOUS, ref
waitObject);
}
catch (COMException Ex)
{
    ErorMsg(Ex);
}

```

4.11.6 GetNonVolatileMemoryUsage**Description**

The function retrieves the non-volatile memory load in percentage.

Syntax

object.GetNonVolatileMemoryUsage(double* Value, ULONGLONG CallType, VARIANT* pWait)

Arguments

Value[out]	The variable that receives the non-volatile memory load in percentage.
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None

Example

```

object waitObject = 0;
Double Value = new Double();
try
{
    Ch. GetNonVolatileMemoryUsage (0, out Value, Ch.ACSC_SYNCHRONOUS, ref
waitObject);
}
catch (COMException Ex)
{
    ErorMsg(Ex);
}

```

4.11.7 GetNonVolatileMemoryTotal**Description**

The function retrieves the amount of total non-volatile memory.

Syntax

object.GetNonVolatileMemoryTotal(double* Value, ULONGLONG CallType, VARIANT* pWait)

Arguments

Value[out]	The variable that receives the amount of total non-volatile memory in bytes.
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None

Example

```

object waitObject = 0;
Double Value = new Double();
try
{
    Ch. GetNonVolatileMemoryTotal (0, out Value, Ch.ACSC_SYNCHRONOUS, ref
waitObject);
}
catch (COMException Ex)
{
    ErrorMsg(Ex);
}

```

4.11.8 GetNonVolatileMemoryFree**Description**

The function retrieves the amount of free non-volatile memory.

Syntax

object.GetNonVolatileMemoryFree(double* Value, ULONGLONG CallType, VARIANT* pWait)

Arguments

Value[out]	The variable that receives the amount of free non-volatile memory in bytes.
ulonglong CallType	If CallType is:

	ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None

Example

```

object waitObject = 0;
Double Value = new Double();
try
{
    Ch. GetNonVolatileMemoryFree (0, out Value, Ch.ACSC_SYNCHRONOUS, ref
waitObject);
}
catch (COMException Ex)
{
    ErorMsg (Ex);
}

```

4.12 Setting and Reading Motion Parameters Methods

The Setting and Reading Motion Parameters methods are:

Table 4-15. Setting and Reading Motion Parameters Methods

SetVelocity	Defines a value of motion velocity.
SetVelocityImm	Defines a value of motion velocity having an immediate effect on any executed as well as impending motion.
GetVelocity	Retrieves a value of motion velocity.
SetAcceleration	Defines a value of motion acceleration.
SetAccelerationImm	Defines a value of motion acceleration having an immediate effect on any executed as well as impending motion.
GetAcceleration	Retrieves a value of motion acceleration.

SetDeceleration	Defines a value of motion deceleration.
SetDecelerationImm	Defines a value of motion deceleration having an immediate effect on any executed as well as impending motion.
GetDeceleration	Retrieves a value of motion deceleration.
SetJerk	Defines a value of motion jerk.
SetJerkImm	Defines a value of motion jerk having an immediate effect on any executed as well as impending motion.
GetJerk	Retrieves a value of motion jerk.
SetKillDeceleration	Defines a value of kill deceleration.
SetKillDecelerationImm	Defines a value of kill deceleration having an immediate effect on any executed as well as impending motion.
GetKillDeceleration	Retrieves a value of kill deceleration.
SetFPosition	Assigns a current value of feedback position.
GetFPosition	Retrieves a current value of motor feedback position.
SetRPosition	Assigns a current value of reference position.
GetRPosition	Retrieves a current value of reference position.
GetFVelocity	Retrieves a current value of motor feedback velocity.
GetRVelocity	Retrieves a current value of reference velocity.

4.12.1 *SetVelocity*

Description

The method defines a value of motion velocity.

Syntax

object.SetVelocity(long Axis, double Velocity)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Velocity	The value specifies required motion velocity. The value will be used in the subsequent motions except for the master-slave motions and the motions activated with the ACSC_AMF_VELOCITY flag.

Return Value

None.

Remarks

The method writes the specified value to the controller.

One value can be specified for each axis.

A single-axis motion uses the value of the corresponding axis. A multi-axis motion uses the value of the leading axis. The leading axis is an axis specified first in the motion command.

The method affects the motions initiated after the method call. The method has no effect on any motion that was started or planned before the method call. To change velocity of an executed or planned motion, use the [SetVelocityImm](#) method.

The method has no effect on the master-slave motions and the motions activated with the **ACSC_AMF_VELOCITY** flag.

If the method fails, the Error object is filled with the error description.

Example

```
Sub SetVelocity_Sample()
Dim Index As Long
Dim Axes(8)
Axes(0) = Ch.ACSC_AXIS_0
Axes(1) = Ch.ACSC_AXIS_1
Axes(2) = Ch.ACSC_AXIS_2
Axes(3) = Ch.ACSC_AXIS_3
Axes(4) = Ch.ACSC_AXIS_4
Axes(5) = Ch.ACSC_AXIS_5
Axes(6) = Ch.ACSC_AXIS_6
Axes(7) = Ch.ACSC_AXIS_7
Axes(8) = -1
On Error GoTo except
                                'Sets the velocity to 10000 to all axes
    For Index = 0 To 7
        Call Ch.SetVelocity(Axes(Index), 10000)
    Next Index
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.12.2 SetVelocityImm**Description**

The method defines a value of motion velocity. Unlike [SetVelocity](#), the method has immediate effect on any executed or planned motion.

Syntax

object.SetVelocityImm(long Axis, doubleVelocity)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Velocity	The value specifies required motion velocity.

Return Value

None.

Remarks

The method writes the specified value to the controller.

One value can be specified for each axis.

A single-axis motion uses the value of the corresponding axis. A multi-axis motion uses the value of the leading axis. The leading axis is an axis specified first in the motion command.

The method affects:

- > The currently executed motion. The controller provides a smooth transition from the instant current velocity to the specified new value.
- > The waiting motions that were planned before the method call.
- > The motions that will be commanded after the method call.

The method has no effect on the master-slave motions.

If the method fails, the Error object is filled with the error description.

Example

```
Sub SetVelocityImm_Sample()
On Error GoTo except
    'Writes the specified value of velocity to
    'the controller
    Call Ch.SetVelocityImm(Ch.ACSC_AXIS_0, 80000)
Exit Sub
except:
    MyErrorMsg          'See Error Handling in Visual Basic
End Sub
```

4.12.3 GetVelocity**Description**

The method retrieves a value of motion velocity.

Syntax

object.GetVelocity(long Axis)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
-------------	---

Return Value

Double.

The method retrieves the value of the motion velocity.

Remarks

The retrieved value is the value defined by a previous call of [SetVelocity](#) or the default value if the method was not previously called.

If the method fails, the Error object is filled with the error description.

Example

```
Sub GetVelocity_Sample()
Dim Velocity As Double
On Error GoTo except
    'Retrieves a value of motion velocity of
    'the 0 axis
    Velocity = Ch.GetVelocity(Ch.ACSC_AXIS_0)
Exit Sub
except:
    MyErrorMsg    'See Error Handling in Visual Basic
End Sub
```

4.12.4 SetAcceleration**Description**

The method defines a value of motion acceleration.

Syntax

object.SetAcceleration(long Axis, double Acceleration)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Acceleration	The value specifies required motion acceleration. The value will be used in the subsequent motions except the master-slave motions.

Return Value

None.

Remarks

The method writes the specified value to the controller.

One value can be specified for each axis.

A single-axis motion uses the value of the corresponding axis. A multi-axis motion uses the value of the leading axis. The leading axis is an axis specified first in the motion command.

The method affects the motions initiated after the method call. The method has no effect on any motion that was started or planned before the method call. To change acceleration of an executed or planned motion, use the [SetAccelerationImm](#) method.

The method has no effect on the master-slave motions.

If the method fails, the Error object is filled with the error description.

Example

```
Sub SetAcceleration_Sample()
On Error GoTo except
    'Defines a value of motion acceleration to
    '200000
    Call Ch.SetAcceleration(Ch.ACSC_AXIS_0, 200000)
Exit Sub
except:
    MyErrorMsg    'See Error Handling in Visual Basic
End Sub
```

4.12.5 SetAccelerationImm**Description**

The method defines a value of motion acceleration. Unlike [SetAcceleration](#), the method has immediate effect on any executed and planned motion.

Syntax

object.SetAccelerationImm(long Axis, double Acceleration)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Acceleration	The value specifies required motion acceleration.

Return Value

None.

Remarks

The method writes the specified value to the controller.

One value can be specified for each axis.

A single-axis motion uses the value of the corresponding axis. A multi-axis motion uses the value of the leading axis. The leading axis is an axis specified first in the motion command.

The method affects:

- > The currently executed motion.
- > The waiting motions that were planned before the method call.
- > The motions that will be commanded after the method call.

The method has no effect on the master-slave motions.

If the method fails, the Error object is filled with the error description.

Example

```
Sub SetAccelerationImm_Sample()
On Error GoTo except
    'Writes the specified acceleration value to
    'the controller
    Call Ch.SetAccelerationImm(Ch.ACSC_AXIS_0, 300000)
Exit Sub
except:
    MyErrorMsg          'See Error Handling in Visual Basic
End Sub
```

4.12.6 GetAcceleration

Description

The method retrieves a value of motion acceleration.

Syntax

object.GetAcceleration(long Axis)

Arguments

Axis

Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see [Axis Definitions](#).

Return Value

Long.

The method retrieves the value of the motion acceleration.

Remarks

The retrieved value is either the value defined by a previous call of the [SetAcceleration](#) method or the default value if there was no previous call of [SetAcceleration](#).

If the method fails, the Error object is filled with the error description.

Example

```
Sub GetAcceleration_Sample()  
Dim Acceleration As Double  
On Error GoTo except  
    'Get acceleration of axis 0  
    Acceleration = Ch.GetAcceleration(Ch.ACSC_AXIS_0)  
Exit Sub  
except:  
    MyErrorMsg 'See Error Handling in Visual Basic  
End Sub
```

4.12.7 SetDeceleration

Description

The method defines a value of motion deceleration.

Syntax

object.SetDeceleration(long Axis, double Deceleration)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Deceleration	The value specifies a required motion deceleration. The value will be used in the subsequent motions except the master-slave motions.

Return Value

None.

Remarks

The method writes the specified value to the controller.

One value can be specified for each axis.

A single-axis motion uses the value of the corresponding axis. A multi-axis motion uses the value of the leading axis. The leading axis is an axis specified first in the motion command.

The method affects the motions initiated after the method call. The method has no effect on any motion that was started or planned before the method call. To change deceleration of an executed or planned motion, use the [SetDecelerationImm](#) method.

The method has no effect on the master-slave motions.

If the method fails, the Error object is filled with the error description.

Example

```
Sub SetDeceleration_Sample()  
On Error GoTo except
```

```

        'Sets deceleration value of 200000 to the 0 axis
    Call Ch.SetDeceleration(Ch.ACSC_AXIS_0, 200000)
Exit Sub
except:
    MyErrorMsg                                     'See Error Handling in Visual Basic
End Sub

```

4.12.8 SetDecelerationImm

Description

The method defines a value of motion deceleration. Unlike [SetDeceleration](#), the method immediately affects any executed or intended motion.

Syntax

object.SetDecelerationImm(long Axis, double Deceleration)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Deceleration	The value specifies a required motion deceleration. The value will be applied to all motions whether in progress or to be started.

Return Value

None.

Remarks

The method writes the specified value to the controller.

One value can be specified for each axis.

A single-axis motion uses the value of the corresponding axis. A multi-axis motion uses the value of the leading axis. The leading axis is an axis specified first in the motion command.

The method affects:

- > The currently executed motion.
- > The waiting motions that were planned before the method call.
- > The motions that will be commanded after the method call.

The method has no effect on the master-slave motions.

If the method fails, the Error object is filled with the error description.

Example

```

Sub SetDecelerationImm_Sample()
On Error GoTo except
                                     'Writes the specified deceleration value

```

```

        'to the controller
    Call Ch.SetDecelerationImm(Ch.ACSC_AXIS_0, 300000)
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub

```

4.12.9 GetDeceleration

Description

The method retrieves a value of motion deceleration.

Syntax

object.GetDeceleration(long Axis)

Arguments

Axis

Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see [Axis Definitions](#).

Return Value

Double.

The method retrieves the value of the motion deceleration.

Remarks

The retrieved value is either the value defined by a previous call of the [SetDeceleration](#) method or the default value if [SetDeceleration](#) was not previously called.

If the method fails, the Error object is filled with the error description.

Example

```

Sub GetDeceleration_Sample()
Dim Deceleration As Double
On Error GoTo except
        'Retrieves a value of motion deceleration
    Deceleration = Ch.GetDeceleration(Ch.ACSC_AXIS_0)
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub

```

4.12.10 SetJerk

Description

The method defines a value of motion jerk.

Syntax

object.SetJerk(long Axis, double Jerk)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Jerk	The value specifies a required motion jerk. The value will be used in the subsequent motions except for the master-slave motions.

Return Value

None.

Remarks

The method writes the specified value to the controller.

One value can be specified for each axis.

A single-axis motion uses the value of the corresponding axis. A multi-axis motion uses the value of the leading axis. The leading axis is an axis specified first in the motion command.

The method affects the motions initiated after the method call. The method has no effect on any motion that was started or planned before the method call. To change the jerk of an executed or planned motion, use the [SetJerkImm](#) method.

The method has no effect on the master-slave motions.

If the method fails, the Error object is filled with the error description.

Example

```
Sub SetJerk_Sample()
Dim Index As Long
On Error GoTo except
                                'Sets jerk value of 2e7 to all axes
    For Index = 0 To 7
        Call Ch.SetJerk(Index, 20000000)
    Next Index
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.12.11 SetJerkImm**Description**

The method defines a value of motion jerk. Unlike [SetJerk](#), the method has an immediate effect on any executed and pending motion.

Syntax

object.SetJerkImm(long Axis, double Jerk)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Jerk	The value specifies a required motion jerk.

Return Value

None.

Remarks

The method writes the specified value to the controller.

One value can be specified for each axis.

A single-axis motion uses the value of the corresponding axis. A multi-axis motion uses the value of the leading axis. The leading axis is an axis specified first in the motion command.

The method affects:

- > The currently executed motion.
- > The waiting motions that were planned before the method call.
- > The motions that will be commanded after the method call.

The method has no effect on the master-slave motions.

If the method fails, the Error object is filled with the error description.

Example

```
Sub SetJerkImm_Sample()  
On Error GoTo except  
    'Writes the specified jerk value to the  
    'controller  
    Call Ch.SetJerkImm(Ch.ACSC_AXIS_0, 2000000)  
Exit Sub  
except:  
    MyErrorMsg          'See Error Handling in Visual Basic  
End Sub
```

4.12.12 GetJerk

Description

The method retrieves a value of motion jerk.

Syntax

object.GetJerk(long Axis)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
-------------	---

Return Value

Double.

The method retrieves the value of the motion jerk.

Remarks

The retrieved value is a value defined by a previous call of [SetJerk](#), or the default value if the method was not called before.

If the method fails, the Error object is filled with the error description.

Example

```
Sub GetJerk_Sample()
Dim Index As Long
Dim Jerk(7) As Double
On Error GoTo except

                                'Retrieves the value of the motion jerk
    For Index = 0 To 7
        Jerk(Index) = Ch.GetJerk(Index)
    Next Index
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.12.13 SetKillDeceleration**Description**

The method defines a value of motion kill deceleration.

Syntax

object.SetKillDeceleration(long Axis, double KillDeceleration)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
KillDeceleration	The value specifies a required motion kill deceleration. The value will be used in the subsequent motions.

Return Value

None.

Remarks

The method writes the specified value to the controller.

One value can be specified for each axis.

A single-axis motion uses the value of the corresponding axis. A multi-axis motion uses the value of the leading axis. The leading axis is an axis specified first in the motion command.

The method affects the motions initiated after the method call. The method has no effect on any motion that was started or planned before the method call.

If the method fails, the Error object is filled with the error description.

Example

```
Sub SetKillDeceleration_Sample()
Dim Index As Long
Dim Axes(8)
Axes(0) = Ch.ACSC_AXIS_0
Axes(1) = Ch.ACSC_AXIS_1
Axes(2) = Ch.ACSC_AXIS_2
Axes(3) = Ch.ACSC_AXIS_3
Axes(4) = Ch.ACSC_AXIS_4
Axes(5) = Ch.ACSC_AXIS_5
Axes(6) = Ch.ACSC_AXIS_6
Axes(7) = Ch.ACSC_AXIS_7
Axes(8) = -1
On Error GoTo except

                                'Sets kill deceleration value of 100000
                                'to all axes

    For Index = 0 To 7
        Call Ch.SetKillDeceleration(Axes(Index), 100000)
    Next Index
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.12.14 SetKillDecelerationImm**Description**

The method defines a value of motion kill deceleration. Unlike [SetKillDeceleration](#), the method has an immediate effect on any executed and planned motion.

Syntax

object.SetKillDecelerationImm(long Axis, double KillDeceleration)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
KillDeceleration	The value specifies a required motion kill deceleration. The value will be used in the subsequent motions.

Return Value

None.

Remarks

The method writes the specified value to the controller.

One value can be specified for each axis.

A single-axis motion uses the value of the corresponding axis. A multi-axis motion uses the value of the leading axis. The leading axis is an axis specified first in the motion command.

The method affects:

- > The currently executed motion.
- > The waiting motions that were planned before the method call.
- > The motions that will be commanded after the method call.

The method has no effect on the master-slave motions.

If the method fails, the Error object is filled with the error description.

Example

```
Sub SetKillDecelerationImm_Sample()
On Error GoTo except
'Defines a value of motion kill deceleration of 200000
Call Ch.SetKillDecelerationImm(Ch.ACSC_AXIS_0, 200000)
Exit Sub
except:
MyErrorMsg 'See Error Handling in Visual Basic
End Sub
```

4.12.15 GetKillDeceleration**Description**

The method retrieves a value of motion kill deceleration.

Syntax

object.GetKillDeceleration(long Axis)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
-------------	---

Return Value

Double.

The method retrieves the value of the motion kill deceleration.

Remarks

The retrieved value is a value defined by a previous call of [SetKillDeceleration](#), or the default value if the method was not previously called.

If the method fails, the Error object is filled with the error description.

Example

```
Sub GetKillDeceleration_Sample()
Dim Index As Long
Dim KDec(7) As Double
On Error GoTo except

'Retrieves a value of motion kill
'deceleration

For Index = 0 To 7
KDec(Index) = Ch.GetKillDeceleration(Index)
Next Index
Exit Sub
except:
MyErrorMsg 'See Error Handling in Visual Basic
End Sub
```

4.12.16 SetFPosition**Description**

The method assigns a current value of feedback position.

Syntax

object.SetFPosition(long Axis, double FPosition)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
FPosition	The value specifies the current value of feedback position.

Return Value

None.

Remarks

The method assigns a current value to the feedback position. No physical motion occurs. The motor remains in the same position; only the internal controller offsets are adjusted so that the periodic calculation of the feedback position will provide the required results.

For more information see the explanation of the **SET** command in the *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```
Sub SetFPosition_Sample()
Dim Index As Long
On Error GoTo except

                                'Sets feedback position value of 0 to all
                                'axes
    For Index = 0 To 7
        Call Ch.SetFPosition(Ch.ACSC_AXIS_0, 0)
    Next Index
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.12.17 GetFPosition**Description**

The method retrieves the current value of the motor feedback position.

Syntax

object.GetFPosition(long Axis)

Arguments

Axis

Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see [Axis Definitions](#).

Return Value

Double.

The method retrieves an instant value of the motor feedback position.

Remarks

The feedback position is a measured position of the motor transferred to user units.

If the method fails, the Error object is filled with the error description.

Example

```

Sub GetFPosition_Sample()
Dim FPosition As Double
On Error GoTo except
                                'Retrieves an instant value of the motor
                                'feedback position
    FPosition = Ch.GetFPosition(Ch.ACSC_AXIS_0)
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub

```

4.12.18 SetRPosition**Description**

The method assigns a current value of reference position.

Syntax

object.SetRPosition(long Axis, long RPosition)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
RPosition	The value specifies the current value of reference position.

Return Value

None.

Remarks

The method assigns a current value to the reference position. No physical motion occurs. The motor remains in the same position; only the internal controller offsets are adjusted so that the periodic calculation of the reference position will provide the required results.

For more information see explanation of the **SET** command in the *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```

Sub SetRPosition_Sample()
Dim Index As Long
Dim Axes(8)
Axes(0) = Ch.ACSC_AXIS_0
Axes(1) = Ch.ACSC_AXIS_1
Axes(2) = Ch.ACSC_AXIS_2
Axes(3) = Ch.ACSC_AXIS_3
Axes(4) = Ch.ACSC_AXIS_4
Axes(5) = Ch.ACSC_AXIS_5
Axes(6) = Ch.ACSC_AXIS_6
Axes(7) = Ch.ACSC_AXIS_7
Axes(8) = -1
On Error GoTo except

                                'Assigns a current value of 0 to reference
                                'position to all axes.

    For Index = 0 To 7
        Call Ch.SetRPosition(Axes(Index), 0)
    Next Index
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub

```

4.12.19 GetRPosition**Description**

The method retrieves an instant value of the motor reference position.

Syntax

object.GetRPosition(long Axis)

Arguments

Axis

Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see [Axis Definitions](#).

Return Value

Double.

The method retrieves the current value of the motor reference position.

Remarks

The method retrieves the current value of the motor reference position. The reference position is a value calculated by the controller as a reference for the motor.

If the method fails, the Error object is filled with the error description.

Example

```

Sub GetRPosition_Sample()
Dim RPosition As Double
On Error GoTo except
        'Retrieves an instant value of the motor
        'reference position
    RPosition = Ch.GetRPosition(Ch.ACSC_AXIS_0)
Exit Sub
except:
    MyErrorMsg        'See Error Handling in Visual Basic
End Sub

```

4.12.20 GetFVelocity**Description**

The method retrieves the instantaneous value of the motor feedback velocity. Unlike [GetVelocity](#), this method retrieves the actually measured velocity and not the required value.

Syntax

object.GetFVelocity(long Axis)

Arguments

Axis

Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see [Axis Definitions](#).

Return Value

Double.

Remarks

The feedback velocity is a measured velocity of the motor transferred to user units.

If the method fails, the Error object is filled with the error description.

Example

```

Sub GetFVelocity_Sample()
Dim FVelocity As Double
On Error GoTo except
        'Retrieves an instant value of the motor
        'feedback velocity
    FVelocity = Ch.GetFVelocity(Ch.ACSC_AXIS_0)
Exit Sub
except:
    MyErrorMsg        'See Error Handling in Visual Basic
End Sub

```

4.12.21 GetRVelocity

Description

The method retrieves an instant value of the motor reference velocity.

Syntax

object.GetRVelocity(long Axis)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
-------------	---

Return Value

Double.

The method retrieves the current value of the motor reference velocity.

Remarks

The reference velocity is a value calculated by the controller in the process of motion generation.

If the method fails, the Error object is filled with the error description.

Example

```
Sub GetRVelocity_Sample()
Dim RVelocity As Double
On Error GoTo except
        'Retrieves an instant value of the motor
        'reference velocity
    RVelocity = Ch.GetRVelocity(Ch.ACSC_AXIS_0)
Exit Sub
except:
    MyErrorMsg        'See Error Handling in Visual Basic
End Sub
```

4.13 Axis/Motor Management Methods

The Axis/Motor Management methods are:

Table 4-16. Axis/Motor Management Methods

CommutExt	Commutates a motor.
Enable	Activates an axis.
EnableM	Activates several axes.
Disable	Shuts off an axis.
DisableAll	Shuts off all axes.

DisableExt	Shuts off an axis and defines the disable reason.
DisableM	Shuts off several axes.
Group	Creates a coordinate system for a multi-axis motion.
Split	Breaks down a previously created axis group.
SplitAll	Breaks down all previously created axis groups.

4.13.1 CommutExt

Description

The function initiates motor commutation.

Syntax

object.CommutExt(long Axis, float Current, int Settle, int Slope, ULONGLONG CallType, VARIANT *pWait)

Arguments

Axis	The axis to perform commutation on. ACSC_AXIS_0 corresponds to axis0, ACSC_AXIS_1 –to axis1, etc. For axis constants see Axis Definitions
Current	Desired excitation current in percentage 0-100, ACSC_NONE for default value.
Settle	Specifies the time it takes for auto commutation to settle, in milliseconds. ACSC_NONE for the default value of 500ms.
Slope	Specifies the time it takes for the current to rise to the desired value, ACSC_NONE for default value.
ulonglong CallType	- If CALLTYPE is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary ()function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to the GetString or GetBinary functions.

Return Value

None

Comments

The function activates a motor. After the activation, the motor begins to follow the reference and physical motion is available.

Settle can only be set if Current is set. Similarly Slope can only be set if Settle is set.

This function is supported in Versions 2.70 and higher.

Example

```
Ch.CommutExt(Ch.ACSC_AXIS_0, // Commuting axis 0
43, // Commutation current
Ch.ACSC_NONE, // Use default settle
Ch.ACSC_NONE // Use default slope
);
```

4.13.2 Enable

Description

The method activates an axis.

Syntax

object.Enable(long Axis)

Arguments

Axis

Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see [Axis Definitions](#).

Return Value

None.

Remarks

The method activates an axis. After activation, the axis begins to follow the reference and physical motion is available.

If the method fails, the Error object is filled with the error description.

Example

```
Sub Enable_Sample()
On Error GoTo except
    'Enable axis 0
    Call Ch.Enable(Ch.ACSC_AXIS_0)
    'Wait till axis 0 is enabled during 5 sec
    Call Ch.WaitMotorEnabled(Ch.ACSC_AXIS_0, 1, 5000)
Exit Sub
except:
MyErrorMsg    'See Error Handling in Visual Basic
End Sub
```

4.13.3 EnableM

Description

The method activates several motors.

Syntax

object.EnableM(variant Axes)

Arguments

Axes

Array of axis constants. Each element specifies one involved axis: ACSC_AXIS_0 corresponds to the 0axis, ACSC_AXIS_1 to the 1axis, etc. After the last axis, one additional element must be included that contains -1 and marks the end of the array.

For the axis constants see [Axis Definitions](#)..

Return Value

None.

Remarks

The method activates several motors. After the activation, the motors begin to follow the corresponding reference and physical motions for the specified motors are available.

If the method fails, the Error object is filled with the error description.

Example

```
Sub EnableM_Sample()
    Dim Axes(2)
    Axes(0) = Ch.ACSC_AXIS_0
    Axes(1) = Ch.ACSC_AXIS_1
    Axes(2) = -1
    On Error GoTo except
        Call Ch.EnableM(Axes)
    Exit Sub
except:
    MyErrorMsg
End Sub
```

'Enable of axes 0 and Y1

'See [Error Handling in Visual Basic](#)

4.13.4 Disable

Description

The method shuts off an axis.

Syntax

object.Disable(long Axis)

Arguments**Axis**

Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see [Axis Definitions](#).

Return Value

None.

Remarks

The method shuts off a motor. After shutting off the motor cannot follow the reference and remains at idle.

If the method fails, the Error object is filled with the error description.

Example

```
Sub Disable_Sample()
On Error GoTo except
    'Disable of axis 0
    Call Ch.Disable(Ch.ACSC_AXIS_0)
    'Wait for motor 0 to disable for 5 sec
    Call Ch.WaitMotorEnabled(Ch.ACSC_AXIS_0, 0, 5000)
Exit Sub
except:
    MyErrorMsg    'See Error Handling in Visual Basic
End Sub
```

4.13.5 DisableAll**Description**

The method shuts off all axes.

Syntax

object.DisableAll()

Arguments

None.

Return Value

None.

Remarks

The method shuts off all motors. After the shutting off none of motors can follow the corresponding, reference and all motors remain idle.

If no motors are currently enabled, the method has no effect.

If the method fails, the Error object is filled with the error description.

Example

```

Sub DisableAll_Sample()
On Error GoTo except
                                'Disable all motors
    Call Ch.DisableAll
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub

```

4.13.6 DisableExt**Description**

The method shuts off an axis and defines the disable reason.

Syntax

object.DisableExt(long Axis, long Reason)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Reason	Integer number that defines the reason of disabling the axis. The specified value is stored in the MERR variable in the controller and so modifies the state of the disabled motor.

Return Value

None.

Remarks

The method shuts off a motor. After shutting off the motor cannot follow the reference and remains at idle.

If **Reason** specifies one of the available motor termination codes, the state of the disabled motor will be identical to the state of the motor disabled for the corresponding fault. This provides an enhanced implementation of user-defined fault response.

If the second parameter specifies an arbitrary number, the motor state will be displayed as "Kill/disable reason: <number> - customer code". This provides the ability to separate different **DISABLE** commands in the application.

If the method fails, the Error object is filled with the error description.

Example

```

Sub DisableExt_Sample()
Dim MyCode As Long
My Code = 10
On Error GoTo except

```

```

        'Disable axis 0 with internal customer
        'code
    Call Ch.DisableExt(Ch.ACSC_AXIS_0, MyCode)
        'Wait for motor 0 to disable for 5 sec
    Call Ch.WaitMotorEnabled(Ch.ACSC_AXIS_0, 0, 5000)
Exit Sub
except:
    ErrorMessage
End Sub
        'See Error Handling in Visual Basic

```

4.13.7 DisableM

Description

The method shuts off several axes.

Syntax

object.DisableM(variant Axes)

Arguments

Axes

Array of axis constants. Each element specifies one involved axis: ACSC_AXIS_0 corresponds to the 0axis, ACSC_AXIS_1 to the 1axis, etc. After the last axis, one additional element must be included that contains -1 and marks the end of the array.

For the axis constants see [Axis Definitions](#).

Return Value

None.

Remarks

The method shuts off several axes. After the shutting off, the axes cannot follow the corresponding reference and remain idle.

If the method fails, the Error object is filled with the error description.

Example

```

Sub DisableM_Sample()
    Dim Axes(2)
    Axes(0) = Ch.ACSC_AXIS_00
    Axes(1) = Ch.ACSC_AXIS_1
    Axes(2) = -1
    On Error GoTo except
        'Disable axes 0 and 1
    Call Ch.DisableM(Axes)
Exit Sub
except:
    MyErrorMsg
End Sub
        'See Error Handling in Visual Basic

```


4.13.8 Group

Description

The method creates a coordinate system for a multi-axis motion.

Syntax

object.Group(variant Axes)

Arguments

Axes

Array of axis constants. Each element specifies one involved axis: ACSC_AXIS_0 corresponds to the 0axis, ACSC_AXIS_1 to the 1axis, etc. After the last axis, one additional element must be included that contains -1 and marks the end of the array.

For the axis constants see [Axis Definitions](#).

Return Value

None.

Remarks

The method creates a coordinate system for a multi-axis motion. The first element of the **Axes** array specifies the leading axis. The motion parameters of the leading axis become the default motion parameters for the group.

An axis can belong to only one group at a time. If the application requires restructuring the axes, it must split the existing group and only then create the new one. To split the existing group, use [Split](#). To split all existing groups, use [SplitAll](#).

If the method fails, the Error object is filled with the error description.

Example

```
Sub Group_Sample()
    Dim Axes(3)
    Axes(0) = Ch.ACSC_AXIS_0
    Axes(1) = Ch.ACSC_AXIS_1
    Axes(2) = Ch.ACSC_AXIS_2
    Axes(3) = -1
    On Error GoTo except
        'Create group of axes 0, 1 and 3
        Call Ch.Group(Axes)
    Exit Sub
except:
    MyErrorMsg 'See Error Handling in Visual Basic
End Sub
```

4.13.9 Split

Description

The method breaks down a previously created axis group.

Syntax

object.Split(variant Axes)

Arguments**Axes**

Array of axis constants. Each element specifies one involved axis: ACSC_AXIS_0 corresponds to the 0axis, ACSC_AXIS_1 to the 1axis, etc. After the last axis, one additional element must be included that contains -1 and marks the end of the array.

For the axis constants see [Axis Definitions](#).

Return Value

None.

Remarks

The method breaks down an axis group created before by the [Group](#) method. **Axes** must specify the same axes as for the **Group** method that created the group. After the splitting up, the group no longer exists.

If the method fails, the Error object is filled with the error description.

Example

```
Sub Split_Sample()
Dim Axes(4)
Axes(0) = Ch.ACSC_AXIS_0
Axes(1) = Ch.ACSC_AXIS_1
Axes(2) = Ch.ACSC_AXIS_2
Axes(3) = Ch.ACSC_AXIS_3
Axes(4) = -1
On Error GoTo except
    Call Ch.Group(Axes)           'Group axes 0, 1, 2, and 3
    Call Ch.Split(Axes)          'Split the group of axes 0, 1, 2, and 3
Exit Sub
except:
    MyErrorMsg    'See Error Handling in Visual Basic
End Sub
```

4.13.10 SplitAll**Description**

The method breaks down all axis groups created before.

Syntax

object.SplitAll()

Arguments

None.

Return Value

None.

Remarks

The method breaks down all axis groups created before by the [Group](#) method.

The application may call this method to ensure that no axes are grouped. If no groups currently exist, the method has no effect.

If the method fails, the Error object is filled with the error description.

Example

```
Sub SplitAll_Sample()
Dim Axes(4)
Axes(0) = Ch.ACSC_AXIS_0
Axes(1) = Ch.ACSC_AXIS_1
Axes(2) = Ch.ACSC_AXIS_2
Axes(3) = Ch.ACSC_AXIS_3
Axes(4) = -1
On Error GoTo except
    Call Ch.Group(Axes)                'Group of axes 0, 1, 2, and 3
    Call Ch.SplitAll                   'Split the group of axes 0, 1, 2, and 3
Exit Sub
except:
    MyErrorMsg    'See Error Handling in Visual Basic
End Sub
```

4.14 Motion Management Methods

The Motion Management methods are:

Table 4-17. Motion Management Methods

Go	Starts up a motion that is waiting in the specified motion queue.
GoM	Synchronously starts up several motions that are waiting in the specified motion queues.
Halt	Terminates a motion using the full deceleration profile.
HaltM	Terminates several motions using the full deceleration profile.
Kill	Terminates a motion using the reduced deceleration profile.
KillAll	Terminates all currently executed motions.
KillM	Terminates several motions using the reduced deceleration profile.

KillExt	Terminates a motion using reduced deceleration profile and defines the kill reason.
Break	Terminates a motion immediately and provides a smooth transition to the next motion.
BreakM	Terminates several motions immediately and provides a smooth transition to the next motions.

4.14.1 Go

Description

The method starts up a motion waiting in the specified motion queue.

Syntax

object.Go(long Axis)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
-------------	---

Return Value

None.

Remarks

A motion that was set with the **ACSC_AMF_WAIT** flag does not start until the **Go** method is executed. The motion waits in the appropriate motion queue.

Each axis has a separate motion queue. A single-axis motion waits in the motion queue of the corresponding axis. A multi-axis motion waits in the motion queue of the leading axis. The leading axis is an axis specified first in the motion command.

The **Go** method initiates the motion waiting in the motion queue of the specified axis. If no motion waits in the motion queue, the method has no effect.

If the method fails, the Error object is filled with the error description.

Example

```
Sub Go_Sample()
On Error GoTo except
    'Enable axis 0
    Call Ch.Enable(Ch.ACSC_AXIS_0)
    'Wait till axis 0 is enabled for 5 sec
    Call Ch.WaitMotorEnabled(Ch.ACSC_AXIS_0, 1, 5000)
    'Wait till GO command executed on axis 0,
    'target position of 10000
    Call Ch.ToPoint(Ch.ACSC_AMF_WAIT, Ch.ACSC_AXIS_0, 10000)
    'Motion start
    Call Ch.Go(Ch.ACSC_AXIS_0)
except
```

```

'Finish the motion
'End of the multi-point motion
Call Ch.EndSequence(Ch.ACSC_AXIS_0)
Exit Sub
except:
    MyErrorMsg 'See Error Handling in Visual Basic
End Sub

```

4.14.2 GoM

Description

The method synchronously starts up several motions that are waiting in the specified motion queues.

Syntax

object.GoM(variant Axes)

Arguments

Axes

Array of axis constants: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc.. After the last axis, one additional element must be included that contains -1 and marks the end of the array.

For the axis constants see [Axis Definitions](#).

Return Value

None.

Remarks

Axes motions that were set with the **ACSC_AMF_WAIT** flag do not start until the **GoM** method is executed. The motions wait in the appropriate motion queue.

Each axis has a separate motion queue. A single-axis motion waits in the motion queue of the corresponding axis. A multi-axis motion waits in the motion queue of the leading axis. The leading axis is an axis specified first in the motion command.

The **GoM** method initiates the motions waiting in the motion queues of the specified axes. If no motion waits in one or more motion queues, the corresponding axes are not affected.

If the method fails, the Error object is filled with the error description.

Example

```

Sub GoM_Sample()
Dim Points(1) As Double
Dim Axes(2)
Axes(0) = Ch.ACSC_AXIS_0
Axes(1) = Ch.ACSC_AXIS_1
Axes(2) = -1
Points(0) = 10000
Points(1) = 10000

```

```

On Error GoTo except
    'Enable axes 0 and 1
    Call Ch.EnableM(Axes)
    'Wait till GO command executed on axes
    '0 and 1 target position of 10000,10000
    Call Ch.ToPointM(Ch.ACSC_AMF_WAIT, Axes, Points)
    'Start the motion of 0 and 1
    Call Ch.GoM(Axes)
    'Finish the motion
    'End of the multi-point motion
    Call Ch.EndSequenceM(Axes)
Exit Sub
except:
    MyErrorMsg 'See Error Handling in Visual Basic
End Sub

```

4.14.3 Halt

Description

The method terminates a motion using the full deceleration profile.

Syntax

object.Halt(long Axis)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
------	---

Return Value

None.

Remarks

The method terminates the executed motion that involves the specified axis. The terminated motion can be either single-axis or multi-axis. Any other motion waiting in the corresponding motion queue is discarded and will not be executed.

If no executed motion involves the specified axis, the method has no effect.

The terminated motion finishes using the full third-order deceleration profile and the motion deceleration value.

If the method fails, the Error object is filled with the error description.

Example

```

Sub Halt_Sample()
On Error GoTo except

    'Enable axis 0
    Call Ch.Enable(Ch.ACSC_AXIS_0)
    'Wait till axis 0 is enabled during 5 sec
    Call Ch.WaitMotorEnabled(Ch.ACSC_AXIS_0, 1, 5000)
    'Relative motion of axis 0, target position
    'of 10000
    Call Ch.ToPoint(Ch.ACSC_AMF_RELATIVE, Ch.ACSC_AXIS_0, 10000)
    'Halt executed to axis 0
    Call Ch.Halt(Ch.ACSC_AXIS_0)
    'Finish the motion
    'End of the multi-point motion
    Call Ch.EndSequence(Ch.ACSC_AXIS_0)
Exit Sub
except:
    MyErrorMsg          'See Error Handling in Visual Basic
End Sub

```

4.14.4 HaltM**Description**

The method terminates several motions using the full deceleration profile.

Syntax

object.HaltM(variant Axes)

Arguments

Axes	Array of axis constants. Each element specifies one involved axis: ACSC_AXIS_0 corresponds to the 0axis, ACSC_AXIS_1 to the 1axis, etc. After the last axis, one additional element must be included that contains -1 and marks the end of the array. For the axis constants see Axis Definitions.
-------------	--

Return Value

None.

Remarks

The method terminates all executed motions that involve the specified axes. The terminated motions can be either single-axis or multi-axis. All other motions waiting in the corresponding motion queues are discarded and will not be executed.

If no executed motion involves a specified axis, the method has no effect on the corresponding axis.

The terminated motions finish using the full third-order deceleration profile and the motion deceleration values.

If the method fails, the Error object is filled with the error description.

Example

```

Sub HaltM_Sample()
Dim Points(1) As Double
Dim Index As Long
Dim Axes(2)
Axes(0) = Ch.ACSC_AXIS_0
Axes(1) = Ch.ACSC_AXIS_1
Axes(2) = -1
Points(0) = 10000
Points(1) = 10000
On Error GoTo except

    Call Ch.EnableM(Axes)                                'Enable axes 0 and 1

    Call Ch.ToPointM(Ch.ACSC_AMF_RELATIVE, Axes, Points) 'Relative motion of axes 0 and 1 with
                                                         'target position of 10000,10000

    Call Ch.HaltM(Axes)                                   'Halt executed on axes 0 and 1

    Call Ch.EndSequenceM(Axes)                            'Finish the motion
                                                         'End of the multi-point motion

Exit Sub
except:
    MyErrorMsg                                           'See Error Handling in Visual Basic
End Sub

```

4.14.5 Kill**Description**

The method terminates a motion using reduced deceleration profile.

Syntax

object.Kill(long Axis)

Arguments

Axis	
	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .

Return Value

None.

Remarks

The method terminates the executed motion that involves the specified axis. The terminated motion can be either single-axis or multi-axis. Any other motion waiting in the corresponding motion queue is discarded and will not be executed.

If no executed motion involves the specified axis, the method has no effect.

The terminated motion finishes with the reduced second-order deceleration profile and the kill deceleration value.

If the method fails, the Error object is filled with the error description.

Example

```
Sub Kill_Sample()
On Error GoTo except
    'Enable axis 0
    Call Ch.Enable(Ch.ACSC_AXIS_0)
    'Wait till 0 enabled during 5 sec
    Call Ch.WaitMotorEnabled(Ch.ACSC_AXIS_0, 1, 5000)
    'Relative motion of axis 0 with target
    'position of 10000
    Call Ch.ToPoint(Ch.ACSC_AMF_RELATIVE, Ch.ACSC_AXIS_0, 10000)
    'Kill axis 0
    Call Ch.Kill(Ch.ACSC_AXIS_0)
    'Finish the motion
    'End of the multi-point motion
    Call Ch.EndSequence(Ch.ACSC_AXIS_0)
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.14.6 KillAll

Description

The method terminates all currently executed motions.

Syntax

object.KillAll()

Arguments

None.

Return Value

None.

Remarks

The method terminates all currently executed motions. Any other motion waiting in any motion queue is discarded and will not be executed.

If no motion is currently executed, the method has no effect.

The terminated motions finish with the reduced second-order deceleration profile and the kill deceleration values.

If the method fails, the Error object is filled with the error description.

Example

```

Sub KillAll_Sample()
Dim Axes(2)
Dim Points(1) As Double
Axes(0) = Ch.ACSC_AXIS_0
Axes(1) = Ch.ACSC_AXIS_1
Axes(2) = -1
Points(0) = 10000
Points(1) = 10000
On Error GoTo except

    Call Ch.EnableM(Axes)                'Enable axes 0 and 1

    Call Ch.ToPointM(Ch.ACSC_AMF_RELATIVE, Axes, Points)
                                        'Relative motion of axes 0 and 1 with
                                        'target position of 10000,10000

    Call Ch.KillAll                      'Kill axes 0 and 1

    Call Ch.EndSequenceM(Axes)           'End of the multi-point motion
Exit Sub
except:
    MyErrorMsg                          'See Error Handling in Visual Basic
End Sub

```

4.14.7 KillM**Description**

The method terminates several motions using reduced deceleration profile.

Syntax

object.KillM(variant Axes)

Arguments**Axes**

Array of axis constants. Each element specifies one involved axis: ACSC_AXIS_0 corresponds to the 0axis, ACSC_AXIS_1 to the 1axis, etc. After the last axis, one additional element must be included that contains -1 and marks the end of the array.

For the axis constants see [Axis Definitions](#).

Return Value

None.

Remarks

The method terminates all executed motions that involve the specified axes. The terminated motions can be either single-axis or multi-axis. All other motions waiting in the corresponding motion queues are discarded and will not be executed.

If no executed motion involves a specified axis, the method has no effect on the corresponding axis.

The terminated motions finish with the reduced second-order deceleration profile and the kill deceleration values.

If the method fails, the Error object is filled with the error description.

Example

```
Sub KillM_Sample()
Dim Axes(2)
Dim Points(1) As Double
Axes(0) = Ch.ACSC_AXIS_0
Axes(1) = Ch.ACSC_AXIS_1
Axes(2) = -1
Points(0) = 1000000
Points(1) = 1000000
On Error GoTo except

    Call Ch.EnableM(Axes)                'Enable axes 0 and 1

    Call Ch.ToPointM(Ch.ACSC_AMF_RELATIVE, Axes, Points)
                                         'Relative motion of axes 0 and 1 with
                                         'target position of 10000,10000
    Call Ch.KillM(Axes)                  'Kill axes 0 and 1
                                         'Finish the motion
                                         'End of the multi-point motion

    Call Ch.EndSequenceM(Axes)
Exit Sub
except:
    MyErrorMsg                          'See Error Handling in Visual Basic
End Sub
```

4.14.8 KillExt

Description

The method terminates a motion using reduced deceleration profile and defines the kill reason.

Syntax

object.KillExt(long Axis, long Reason)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Reason	Integer number that defines the reason of disabling the axis. The specified value is stored in the MERR variable in the controller and so modifies the state of the disabled motor.

Return Value

None.

Remarks

The method terminates the executed motion that involves the specified axis. The terminated motion can be either single-axis or multi-axis. Any other motion waiting in the corresponding motion queue is discarded and will not be executed.

If no executed motion involves the specified axis, the method has no effect.

The terminated motion finishes with the reduced second-order deceleration profile and the kill deceleration value.

If **Reason** specifies one of the available motor termination codes, the state of the killed motor will be identical to the state of the motor killed for the corresponding fault. This provides an enhanced implementation of user-defined fault response.

If the second parameter specifies an arbitrary number, the motor state will be displayed as "Kill/disable reason: <number> - customer code". This provides ability to separate different [Kill](#) commands in the application.

If the method fails, the Error object is filled with the error description.

Example

```
Sub KillExt_Sample()
Dim MyCode As Long
MyCode = 10
On Error GoTo except

' Enable axis 0
Call Ch.Enable(Ch.ACSC_AXIS_0)

' Wait till 0 enabled during 5 sec
Call Ch.WaitMotorEnabled(Ch.ACSC_AXIS_0, 1, 5000)

' Relative motion of axis 0 with target
' position of 10000
Call Ch.ToPoint(Ch.ACSC_AMF_RELATIVE, Ch.ACSC_AXIS_0, 10000)

' Kill axis 0 with internal customer code
Call Ch.KillExt(Ch.ACSC_AXIS_0, MyCode)

' Finish the motion
' End of the multi-point motion
Call Ch.EndSequence(Ch.ACSC_AXIS_0)
Exit Sub
except:
    ErrorMessage 'See Error Handling in Visual Basic
End Sub
```

4.14.9 Break**Description**

Terminates a motion immediately and provides a smooth transition to the next motion.

Syntax

object.Break(long Axis)

Arguments

Axis
Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .

Return Value

None.

Remarks

The method terminates the executed motion that involves the specified axis only if the next motion is waiting in the corresponding motion queue. The terminated motion can be either single-axis or multi-axis.

If the motion queue contains no waiting motion, the break command is not executed immediately. The current motion continues instead until the next motion is planned to the same motion queue. Only then is the break command executed.

If no executed motion involves the specified axis, or the motion finishes before the next motion is planned, the method has no effect.

When executing the break command, the controller terminates the motion immediately without any deceleration profile. The controller builds instead a smooth third-order transition profile to the next motion.

Use caution when implementing the break command with a multi-axis motion, because the controller provides a smooth transition profile of the vector velocity. In a single-axis motion, this ensures a smooth axis velocity. However, in a multi-axis motion an axis velocity can change abruptly if the terminated and next motions are not tangent in the junction point. To avoid jerk, the terminated and next motion must be tangent or nearly tangent in the junction point.

If the method fails, the Error object is filled with the error description.

Example

```

Sub Break_Sample()
On Error GoTo except
    'Enable axis 0
    Call Ch.Enable(Ch.ACSC_AXIS_0)
    'Wait axis 0 enabled during 5 sec
    Call Ch.WaitMotorEnabled(Ch.ACSC_AXIS_0, 1, 5000)
    'Start up the motion of axis 0 to point
    '10000
    Call Ch.ToPoint(Ch.ACSC_AMF_RELATIVE, Ch.ACSC_AXIS_0, 10000)
    'Executing of break to axis 0
    Call Ch.Break(Ch.ACSC_AXIS_0)
    'Change the end of the point to point 0
    'on the fly
    Call Ch.ToPoint(Ch.ACSC_AMF_RELATIVE, Ch.ACSC_AXIS_0, 0)
    'Finish the motion
    'End of the multi-point motion
    Call Ch.EndSequence(Ch.ACSC_AXIS_0)
Exit Sub
except:
MyErrorMsg                                'See Error Handling in Visual Basic
End Sub

```

4.14.10 BreakM**Description**

Terminates several motions immediately and provides a smooth transition to the next motions.

Syntax

object.BreakM(variant Axes)

Arguments

Axes	Array of axis constants. Each element specifies one involved axis: ACSC_AXIS_0 corresponds to the 0axis, ACSC_AXIS_1 to the 1axis, etc. After the last axis, one additional element must be included that contains -1 and marks the end of the array. For the axis constants see Axis Definitions.
-------------	--

Return Value

None.

Remarks

The method terminates the executed motions that involve the specified axes. Only those motions are terminated that have the next motion waiting in the corresponding motion queue. The terminated motions can be either single-axis or multi-axis.

If a motion queue contains no waiting motion, the break command does not immediately affect the corresponding axis. The current motion continues instead until the next motion is planned to the same motion queue. Only then, the break command is executed.

If no executed motion involves the specified axis, or the corresponding motion finishes before the next motion is planned, the method does not affect the axis.

When executing the break command, the controller terminates the motion immediately without any deceleration profile. Instead, the controller builds a smooth third-order transition profile to the next motion.

Use caution when implementing the break command with a multi-axis motion, because the controller provides a smooth transition profile of the vector velocity. In a single-axis motion, this ensures a smooth axis velocity, but in a multi-axis motion, an axis velocity can change abruptly if the terminated and next motions are not tangent in the junction point. To avoid jerk, the terminated and next motion must be tangent or nearly tangent in the junction point.

If the method fails, the Error object is filled with the error description.

Example

```

Sub BreakM_Sample()
Dim Axes(8)
Dim Points(7) As Double
Dim Index As Long
Axes(0) = Ch.ACSC_AXIS_0
Axes(1) = Ch.ACSC_AXIS_1
Axes(2) = Ch.ACSC_AXIS_2
Axes(3) = Ch.ACSC_AXIS_3
Axes(4) = Ch.ACSC_AXIS_4
Axes(5) = Ch.ACSC_AXIS_5
Axes(6) = Ch.ACSC_AXIS_6
Axes(7) = Ch.ACSC_AXIS_7
Axes(8) = -1
For Index = 0 To 7
Points(Index) = 10000
Next Index
On Error GoTo except

    'Enable all axes
    Call Ch.EnableM(Axes)

    'Start up the motion of axes to point 10000
    Call Ch.ToPointM(Ch.ACSC_AMF_RELATIVE, Axes, Points)

    'Executing of break to all axes
    Call Ch.BreakM(Axes)
    For Index = 0 To 7
        Points(Index) = -10000
    Next Index

    'Change the end points to point -10000 on
    'thefly
    Call Ch.ToPointM(Ch.ACSC_AMF_RELATIVE, Axes, Points)
    'Finish the motion
    'End of the multi-point motion
    Call Ch.EndSequenceM(Axes)
Exit Sub
except:
MyErrorMsg 'See Error Handling in Visual Basic
End Sub

```

4.15 Point-to-Point Motion Methods

The Point-to-Point Motion methods are:

Table 4-18. Point-to-Point Motion Methods

ToPoint	Initiates a single-axis motion to the specified point.
ToPointM	Initiates a multi-axis motion to the specified point.
ExtToPoint	Initiates a single-axis motion to the specified point using the specified velocity or end velocity.

ExtToPointM

Initiates a multi-axis motion to the specified point using the specified velocity or end velocity.

4.15.1 ToPoint**Description**

The method initiates a single-axis motion to the specified point.

Syntax

object.ToPoint(long Flags, long Axis, double Point)

Arguments

Flags	Bit-mapped parameter that can include one or more of the following flags: ACSC_AMF_WAIT: plan the motion but do not start it until the Go method is executed. ACSC_AMF_RELATIVE: the Point value is relative to the end point of the previous motion. If the flag is not specified, the Point specifies an absolute coordinate.
Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions.
Point	Coordinate of the target point.

Return Value

None.

Remarks

The method initiates a single-axis point-to-point motion.

The motion is executed using the required motion velocity and finishes with zero end velocity. The required motion velocity is the velocity specified by the previous call of the [SetVelocity](#) method or the default velocity if the method was not called.

To execute multi-axis point-to-point motion, use [ToPointM](#). To execute motion with other motion velocity or non-zero end velocity, use [ExtToPoint](#) or [ExtToPointM](#).

The controller response indicates that the command was accepted and the motion was planned successfully. The method does not wait for the motion end. To wait for the motion end, use [WaitMotionEnd](#) method.

The motion builds the velocity profile using the required values of velocity, acceleration, deceleration, and jerk of the specified axis.

If the method fails, the Error object is filled with the error description.

```
Sub ToPoint_Sample()  
On Error GoTo except
```

```

                                'Enable axis 0
Call Ch.Enable(Ch.ACSC_AXIS_0)
                                'Wait axis 0 enabled during 5 sec
Call Ch.WaitMotorEnabled(Ch.ACSC_AXIS_0, 1, 5000)
                                'Relative motion of axis 0 to target point
                                'of 10000
Call Ch.ToPoint(Ch.ACSC_AMF_RELATIVE, Ch.ACSC_AXIS_0, 10000)
                                'Wait till motion ends
Call Ch.WaitMotionEnd(Ch.ACSC_AXIS_0, 10000)
                                'Finish the motion
                                'End of the multi-point motion
Call Ch.EndSequence(Ch.ACSC_AXIS_0)
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub

```

4.15.2 ToPointM

Description

The method initiates a multi-axis motion to the specified point.

Syntax

object.ToPointM(long Flags, variant Axes, variant Point)

Arguments

Flags	Bit-mapped parameter that can include one or more of the following flags: ACSC_AMF_WAIT: plan the motion but do not start it until the Go method is executed. ACSC_AMF_RELATIVE: the Point value is relative to the end point of the previous motion. If the flag is not specified, the Point specifies an absolute coordinate. ACSC_AMF_MAXIMUM: not to use the motion parameters from the leading axis but to calculate the maximum allowed motion velocity, acceleration, deceleration, and jerk of the involved axes.
Axes	Array of axis constants. Each element specifies one involved axis: ACSC_AXIS_0 corresponds to the 0axis, ACSC_AXIS_1 to the 1axis, etc. After the last axis, one additional element must be included that contains -1 and marks the end of the array. For the axis constants see Axis Definitions.

Point

Array of the target coordinates. The number and order of values must correspond to the **Axes** array. The **Point** must specify a value for each element of **Axes** except the last -1 element.

Return Value

None.

Remarks

The method initiates a multi-axis point-to-point motion.

The motion is executed using the required motion velocity and finishes with zero end velocity. The required motion velocity is the velocity specified by the previous call of the [SetVelocity](#) method, or the default velocity if the method was not called.

To execute single-axis point-to-point motion, use [ToPoint](#). To execute motion with other motion velocity or non-zero end velocity, use [ExtToPoint](#) or [ExtToPointM](#).

The controller response indicates that the command was accepted and the motion was planned successfully. The method does not wait for the motion end. To wait for the motion end, use [WaitMotionEnd](#) method.

The motion builds the velocity profile using the required values of velocity, acceleration, deceleration, and jerk of the leading axis. The leading axis is the first axis in the **Axes** array.

If the method fails, the Error object is filled with the error description.

Example

```
Sub ToPointM_Sample()
Dim Point(1) As Double
Dim Index As Long
Dim result(1) As Double
Dim Axes(2)
Axes(0) = Ch.ACSC_AXIS_0
Axes(1) = Ch.ACSC_AXIS_1
Axes(2) = -1
Point(0) = 10000
Point(1) = 10000
On Error GoTo except

    Call Ch.EnableM(Axes)
    'Enable axes 0 and 1

    Call Ch.ToPointM(0, Axes, Point)
    'Immediately start the motion of the axes XY
    'to the absolute target points 10000,10000

    Call Ch.EndSequenceM(Axes)
    'Finish the motion
    'End of the multi-point motion

Exit Sub
except:
    MyErrorMsg 'See Error Handling in Visual Basic
End Sub
```

4.15.3 ExtToPoint

Description

The method initiates a single-axis motion to the specified point using the specified velocity or end velocity.

Syntax

object.ExtToPoint(long Flags, long Axis, double Point, [double Velocity], [double EndVelocity])

Arguments

Flags	Bit-mapped parameter that can include one or more of the following flags: ACSC_AMF_WAIT: plan the motion but don't start it until the method Go is executed. ACSC_AMF_RELATIVE: the Point value is relative to the end point of the previous motion. If the flag is not specified, the Point specifies an absolute coordinate. ACSC_AMF_VELOCITY: the motion will use velocity specified by the Velocity argument instead of the default velocity. ACSC_AMF_ENDVELOCITY: the motion will come to the end point with the velocity specified by the EndVelocity argument.
Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions.
Point	Coordinate of the target point.
Velocity	Motion velocity. The argument is used only if the ACSC_AMF_VELOCITY flag is specified.
EndVelocity	Velocity in the target point. The argument is used only if the ACSC_AMF_ENDVELOCITY flag is specified. Otherwise, the motion finishes with zero velocity.

Return Value

None.

Remarks

The method initiates a single-axis point-to-point motion.

If the **ACSC_AMF_VELOCITY** flag is specified, the motion is executed using the velocity specified by **Velocity**. Otherwise, the required motion velocity is used. The required motion velocity is the velocity specified by the previous call of [SetVelocity](#), or the default velocity if the method was not called.

If the **ACSC_AMF_ENDVELOCITY** flag is specified, the motion velocity at the final point is specified by the **EndVelocity**. Otherwise, the motion velocity at the final point is zero.

To execute a multi-axis point-to-point motion with the specified velocity or end velocity, use [ExtAddPointM](#). To execute motion with default motion velocity and zero end velocity, use [ToPoint](#) or [ToPointM](#).

The controller response indicates that the command was accepted and the motion was planned successfully. The method does not wait for the motion end. To wait for the motion end, use [WaitMotionEnd](#).

The motion builds the velocity profile using the required values of acceleration, deceleration and jerk of the specified axis.

If the method fails, the Error object is filled with the error description.

Example

```
Sub ExtToPoint_Sample()
On Error GoTo except
    'Enable axis 0
    Call Ch.Enable(Ch.ACSC_AXIS_0)
    'Wait untill axis 0 is disabled for 5 sec
    Call Ch.WaitMotorEnabled(Ch.ACSC_AXIS_0, 1, 5000)
    'Parameters: Ch.ACSC_AMF_VELOCITY Or
    'Ch.ACSC_AMF_ENDVELOCITY -
    'Start up the motion with specified velocity
    '5000
    'Come to the end point with specified
    'velocity 1000
    'Ch.ACSC_AXIS_0 - axis 0
    '10000 - target point
    '5000 - motion velocity
    '1000 - velocity in the target point
    Call Ch.ExtToPoint(Ch.ACSC_AMF_VELOCITY Or Ch.ACSC_AMF_ENDVELOCITY,
Ch.ACSC_AXIS_0, 10000, 5000, 1000)
    'Wait motion ends during 20 sec
    Call Ch.WaitMotionEnd(Ch.ACSC_AXIS_0, 20000)
    'Finish the motion
    'End of the multi-point motion
    Call Ch.EndSequence(Ch.ACSC_AXIS_0)
Exit Sub
except:
    MyErrorMsg
End Sub
```

'See [Error Handling in Visual Basic](#)

4.15.4 ExtToPointM

Description

The method initiates a multi-axis motion to the specified point using the specified velocity or end velocity.

Syntax

object.ExtToPointM(long Flags, variant Axes, variant Point, [double Velocity], [double EndVelocity])

Arguments

Flags	Bit-mapped parameter that can include one or more of the following flags: ACSC_AMF_WAIT: plan the motion but don't start it until the method GoM is executed. ACSC_AMF_RELATIVE: the Point value is relative to the end point of the previous motion. If the flag is not specified, the Point specifies an absolute coordinate. ACSC_AMF_VELOCITY: the motion will use velocity specified by the Velocity argument instead of the default velocity. ACSC_AMF_ENDVELOCITY: the motion will come to the end point with the velocity specified by the EndVelocity argument.
Axes	Array of axis constants. Each element specifies one involved axis: ACSC_AXIS_0 corresponds to the 0axis, ACSC_AXIS_1 to the 1axis, etc. After the last axis, one additional element must be included that contains -1 and marks the end of the array. For the axis constants see Axis Definitions.
Point	Array of the target coordinates. The number and order of values must correspond to the Axes array. The Point must specify a value for each element of Axes except the last -1 element.
Velocity	Motion velocity. The argument is used only if the ACSC_AMF_VELOCITY flag is specified.
EndVelocity	Velocity in the target point. The argument is used only if the ACSC_AMF_ENDVELOCITY flag is specified. Otherwise, the motion finishes with zero velocity.

Return Value

None.

Remarks

The method initiates a multi-axis point-to-point motion.

If the **ACSC_AMF_VELOCITY** flag is specified, the motion is executed using the velocity specified by **Velocity**. Otherwise, the required motion velocity is used. The required motion velocity is the velocity specified by the previous call of [SetVelocity](#), or the default velocity if the method was not called.

If the **ACSC_AMF_ENDVELOCITY** flag is specified, the motion velocity at the final point is specified by the **EndVelocity**. Otherwise, the motion velocity at the final point is zero.

To execute a single-axis point-to-point motion with the specified velocity or end velocity, use [ExtToPoint](#). To execute a motion with default motion velocity and zero end velocity, use [ToPoint](#) or [ToPointM](#).

The controller response indicates that the command was accepted and the motion was planned successfully. The method does not wait for the motion end. To wait for the motion end, use the [WaitMotionEnd](#) method.

The motion builds the velocity profile using the required values of acceleration, deceleration and jerk of the leading axis. The leading axis is the first axis in the **Axes** array.

If the method fails, the Error object is filled with the error description.

Example

```
Sub ExtToPointM_Sample()
Dim Index As Long
Dim Points(1) As Double
Dim Axes(2)
Axes(0) = Ch.ACSC_AXIS_0
Axes(1) = Ch.ACSC_AXIS_1
Axes(2) = -1
Points(0) = 1000
Points(1) = 2000
On Error GoTo except
    'Enable axes 0 and 1
    Call Ch.EnableM(Axes)
    'Parameters: Ch.ACSC_AMF_VELOCITY Or
    'Ch.ACSC_AMF_ENDVELOCITY -
    'Start up the motion with specified velocity
    '5000
    'come to the end point with specified velocity
    '1000
    'Axes - axes 0 and 1
    'Points - target point
    '5000 - motion velocity
    '1000 - velocity in the target point
    Call Ch.ExtToPointM(Ch.ACSC_AMF_VELOCITY Or Ch.ACSC_AMF_ENDVELOCITY,
    Axes, Points, 5000, 1000)
    'Finish the motion
    'End of the multi-point motion
    Call Ch.EndSequenceM(Axes)
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.16 Track Motion Control Method

The Track Motion Control method is:

Table 4-19. Track Motion Control Method

Track	The method initiates a single-axis track motion.
--------------	--

4.16.1 Track

Description

The method initiates a single-axis track motion.

Syntax

object.Track(long Flags, long Axis)

Arguments

Flags	Bit-mapped parameter that can include one or more of the following flag: ACSC_AMF_WAIT : plan the motion but do not start it until the Go method is executed.
Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .

Return Value

None.

Remarks

The method initiates a single-axis track motion. After the motion is initialized, PTP motion will be generated with every change in the **TPOS** value.

The controller response indicates that the command was accepted and the motion was planned successfully.

If the method fails, the Error object is filled with the error description.

Example

```
Sub Track_Sample()
On Error GoTo except
    'Enable axis 0
    Call Ch.Enable(Ch.ACSC_AXIS_0)
    'Wait axis 0 enabled during 5 sec
    Call Ch.WaitMotorEnabled(Ch.ACSC_AXIS_0, 1, 5000)
    'Start up immediately the motion of the axis
    '0
    Call Ch.Track(0, Ch.ACSC_AXIS_0)
    'Target position to 5000
    Call Ch.Transaction("TPOS0=5000")
    End If
Exit Sub
except:
    MyErrorMsg    'See Error Handling in Visual Basic
End Sub
```


4.17 Jog Methods

The Jog methods are:

Table 4-20. Jog Methods

Jog	Initiates a single-axis jog motion.
JogM	Initiates a multi-axis jog motion.

4.17.1 Jog

Description

The method initiates a single-axis jog motion.

Syntax

object.Jog(long Flags, long Axis, [doubleVelocity])

Arguments

Flags	Bit-mapped parameter that can include one or more of the following flags: ACSC_AMF_WAIT : plan the motion but don't start it until the method Go is executed. ACSC_AMF_VELOCITY : the motion will use velocity specified by the Velocity argument instead of the default velocity.
Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Velocity	If the ACSC_AMF_VELOCITY flag is specified, the velocity profile is built using the value of Velocity . The sign of Velocity defines the direction of the motion. If the ACSC_AMF_VELOCITY flag is not specified, only the sign of Velocity is used in order to specify the direction of motion. In this case, the properties ACSC_POSITIVE_DIRECTION or ACSC_NEGATIVE_DIRECTION can be used.

Return Value

None.

Remarks

The method initiates a single-axis jog. To execute multi-axis jog, use [JogM](#).

The jog motion is a motion with constant velocity and no defined ending point. The jog motion continues until the next motion is planned, or the motion is killed for any reason.

The motion builds the velocity profile using the default values of acceleration, deceleration and jerk of the specified axis. If the **ACSC_AMF_VELOCITY** flag is not specified, the default value of velocity is used as well. In this case, only the sign of **Velocity** is used in order to specify the direction of motion.

The positive velocity defines a positive direction, the negative velocity defines the negative direction.

If the **ACSC_AMF_VELOCITY** flag is specified, the value of **Velocity** is used instead of the default velocity. The sign of **Velocity** defines the direction of the motion.

The method waits for the controller response.

The controller response indicates that the command was accepted and the motion was planned successfully. No waiting for the motion end is provided.

If the method fails, the Error object is filled with the error description.

Example

```
Sub Jog_Sample()
On Error GoTo except
    'Enable axis 0
    Call Ch.Enable(Ch.ACSC_AXIS_0)
    'Wait till axis 0 is enabled during 5 sec
    Call Ch.WaitMotorEnabled(Ch.ACSC_AXIS_0, 1, 5000)
    'Initiates a single-axis jog motion to axis
    '0 to positive direction with the specified
    'velocity 10000
    Call Ch.Jog(Ch.ACSC_POSITIVE_DIRECTION, Ch.ACSC_AXIS_0, 10000)
    'Finish the motion

Exit Sub
except:
    MyErrorMsg 'See Error Handling in Visual Basic
End Sub
```

4.17.2 JogM

Description

The method initiates a multi-axis jog motion.

Syntax

object.JogM(long Flags, variant Axes, variant Direction, [double Velocity])

Arguments

Flags	Bit-mapped parameter that can include one or more of the following flags: ACSC_AMF_WAIT: plan the motion but don't start it until the method GoM is executed. ACSC_AMF_VELOCITY: the motion will use velocity specified by the Velocity argument instead of the default velocity.
Axes	Array of axis constants. Each element specifies one involved axis: ACSC_AXIS_0 corresponds to the 0axis, ACSC_AXIS_1 to the 1axis, etc. After the last axis, one additional element must be included that contains -1 and marks the end of the array. For the axis constants see Axis Definitions.
Direction	Array of directions - The number and order of values must correspond to the Axes array. The Direction array must specify direction for each element of Axes except the last -1 element. The property ACSC_POSITIVE_DIRECTION in the Direction array specifies the correspondent axis to move in positive direction, the property ACSC_NEGATIVE_DIRECTION specifies the correspondent axis to move in the negative direction.
Velocity	If the ACSC_AMF_VELOCITY flag is specified, the velocity profile is built using the value of Velocity. If the ACSC_AMF_VELOCITY flag is not specified, Velocity is not used.

Return Value

None

Remarks

The method initiates a multi-axis jog motion. To execute single-axis jog motion, use [Jog](#).

The jog motion is a motion with constant velocity and no defined ending point. The jog motion continues until the next motion is planned, or the motion is killed for any reason.

The motion builds the vector velocity profile using the default values of velocity, acceleration, deceleration and jerk of the axis group. If the **ACSC_AMF_VELOCITY** flag is not specified, the default value of velocity is used as well. If the **ACSC_AMF_VELOCITY** flag is specified, the value of **Velocity** is used instead of the default velocity.

The method waits for the controller response.

The controller response indicates that the command was accepted and the motion was planned successfully. The method cannot wait or validate the end of the motion. To wait for the motion end, use [WaitMotionEnd](#).

If the method fails, the Error object is filled with the error description.

Example

```

Sub JogM_Sample()
Dim Directions(1)
Dim Axes(2)
Directions(0) = Ch.ACSC_POSITIVE_DIRECTION
Directions(1) = Ch.ACSC_POSITIVE_DIRECTION
Axes(0) = Ch.ACSC_AXIS_0
Axes(1) = Ch.ACSC_AXIS_1
Axes(2) = -1
On Error GoTo except

    Call Ch.EnableM(Axes)
                                'Enable axes 0 and 1

                                'Start up immediately the jog motion of axes
                                'XY to positive directions with the
                                'specified velocity 5000
    Call Ch.JogM(0, Axes, Directions, 5000)
                                'Finish the motion
                                'End of the multi-point motion

    Call Ch.EndSequenceM(Axes)
Exit Sub
except:
    MyErrorMsg
                                'See Error Handling in Visual Basic
End Sub

```

4.18 Slaved Motion Methods

The Slaved Motion methods are:

Table 4-21. Slaved Motion Methods

SetMaster	Initiates calculation of a master value for an axis.
Slave	Initiates a master-slave motion.
SlaveStalled	Initiates master-slave motion with limited following area.

4.18.1 SetMaster

Description

The method initiates calculating of master value for an axis.

Syntax

object.SetMaster(long Axis, string Formula)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Formula	ASCII string that specifies a rule for calculating master value.

Return Value

None.

Remarks

The method initiates calculating of master value for an axis.

The master value for each axis is presented in the controller as one element of the **MPOS** array. Once the **SetMaster** method is called, the controller calculates the master value for the specified axis each controller cycle.

The **SetMaster** method can be called again for the same axis at any time. At that moment, the controller discards the previous formula and accepts the newly specified formula for the master calculation.

The method waits for the controller response.

The controller response indicates that the command was accepted and the controller starts calculating the master value according to the formula.

The **Formula** string can specify any valid ACSPL+ expression that uses any ACSPL+ or user global variables as its operands.

If the method fails, the Error object is filled with the error description.

Example

```
Sub SetMaster_Sample()
Dim MFormula As String

'Master value is calculated as feedback
'position of the axis T with scale factor
'equal 2

MFormula = "2 * T_FPOS"
Dim Axes(2)
Axes(0) = Ch.ACSC_AXIS_Z
Axes(1) = Ch.ACSC_AXIS_T
Axes(2) = -1
On Error GoTo except

'Enable axes ZT
Call Ch.EnableM(Axes)

'Set master formula "Z_MPOS=2 * T_FPOS"
Call Ch.SetMaster(Ch.ACSC_AXIS_Z, MFormula)

'Set axis Z to slave
Call Ch.Slave(0, Ch.ACSC_AXIS_Z)

'Relative motion with target position of
'10000
Call Ch.ToPoint(Ch.ACSC_AMF_RELATIVE, Ch.ACSC_AXIS_T, 10000)

'Finish the motion
'End of the multi-point motion
Call Ch.EndSequenceM(Axes)
Exit Sub
except:
MyErrorMsg 'See Error Handling in Visual Basic
End Sub
```

4.18.2 Slave

Description

The method initiates a master-slave motion.

Syntax

object.Slave(long Flags, long Axis)

Arguments

Flags	Bit-mapped parameter that can include one or more of the following flags: ACSC_AMF_WAIT: plan the motion but don't start it until the method Go is executed. ACSC_AMF_POSITIONLOCK: the motion will use position lock. If the flag is not specified, velocity lock is used (see Remarks).
Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions.

Return Value

None.

Remarks

The method initiates a single-axis master-slave motion with an unlimited area of following. If the area of following must be limited, use [SlaveStalled](#).

The master-slave motion continues until the motion is killed or the motion fails for any reason.

The method waits for the controller response.

The controller response indicates that the command was accepted and the motion was planned successfully.

The master value for the specified axis must be defined before by the call to [SetMaster](#) method. The **SetMaster** method can be called again in order to change the formula of master calculation. If at this moment the master-slave motion is in progress, the slave can come out from synchronism. The controller then regains synchronism, probably with a different value of offset between the master and slave.

If the ACSC_AMF_POSITIONLOCK flag is not specified, the method activates a velocity-lock mode of slaved motion. When synchronized, the **APOS** axis reference follows the **MPOS** with a constant offset:

$$APOS = MPOS + C$$

The value of **C** is latched at the moment when the motion comes to synchronism, and then remains unchanged as long as the motion is synchronous. If at the moment of motion start the master velocity is zero, the motion starts synchronously and **C** is equal to the difference between initial values of **APOS** and **MPOS**.

If the **ACSC_AMF_POSITIONLOCK** flag is specified, the method activates a position-lock mode of slaved motion. When synchronized, the **APOS** axis reference strictly follows the **MPOS**:

$$APOS = MPOS$$

If the method fails, the Error object is filled with the error description.

Example

```
Sub Slave_Sample()
Dim MFormula As String

'Master value is calculated as feedback
'position of the axis T with scale factor
'equal 2

MFormula = "2 * T_FPOS"
Dim Axes(2)
Axes(0) = Ch.ACSC_AXIS_Z
Axes(1) = Ch.ACSC_AXIS_T
Axes(2) = -1
On Error GoTo except

'Enable axes ZT
Call Ch.EnableM(Axes)

'Set master formula "Z_MPOS=2 * T_FPOS"
Call Ch.SetMaster(Ch.ACSC_AXIS_Z, MFormula)

'Set axis Z to slave
Call Ch.Slave(0, Ch.ACSC_AXIS_Z)

'Relative motion with target position of
'10000
Call Ch.ToPoint(Ch.ACSC_AMF_RELATIVE, Ch.ACSC_AXIS_T, 10000)

'Finish the motion
'End of the multi-point motion

Call Ch.EndSequenceM(Axes)
Exit Sub
except:
MyErrorMsg 'See Error Handling in Visual Basic
End Sub
```

4.18.3 SlaveStalled

Description

The method initiates master-slave motion within predefined limits.

Syntax

object.SlaveStalled(long Flags, long Axis, double Left, double Right)

Arguments

Flags	Bit-mapped parameter that can include one or more of the following flags: ACSC_AMF_WAIT: plan the motion but don't start it until the method Go is executed. ACSC_AMF_POSITIONLOCK: the motion will use position lock. If the flag is not specified, velocity lock is used (see Remarks).
Axis	Axis constant of slaved axis: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions.
Left	Left (negative) limit of the axis.
Right	Right (positive) limit of the axis.

Return Value

None.

Remarks

The method initiates single-axis master-slave motion within predefined limits. Use [Slave](#) to initiate unlimited motion. For sophisticated forms of master-slave motion, use slaved variants of segmented and spline motions.

The method waits for the controller response.

The controller response indicates that the command was accepted and the motion was planned successfully.

The master-slave motion continues until the [Kill](#) command is executed, or the motion fails for any reason.

The master value for the specified axis must be defined before by the call to [SetMaster](#) method. The [SetMaster](#) method can be called again in order to change the formula of master calculation. If at this moment the master-slave motion is in progress, the slave can come out from synchronism. The controller then regains synchronism, probably with a different value of offset between the master and slave.

If the ACSC_AMF_POSITIONLOCK flag is not specified, the method activates a velocity-lock mode of slaved motion. When synchronized, the **APOS** axis reference follows the **MPOS** with a constant offset:

$$APOS = MPOS + C$$

The value of **C** is latched at the moment when the motion comes to synchronism, and then remains unchanged as long as the motion is synchronous. If at the moment of motion start the master velocity is zero, the motion starts synchronously and **C** is equal to the difference between initial values of **APOS** and **MPOS**.

If the ACSC_AMF_POSITIONLOCK flag is specified, the method activates a position-lock mode of slaved motion. When synchronized, the **APOS** axis reference strictly follows the **MPOS**:

$$APOS = MPOS$$

The **Left** and **Right** values define the allowed area of changing the **APOS** value. The **MPOS** value is not limited and can exceed the limits. In this case, the motion comes out from synchronism, and the **APOS** value remains (stalls) in one of the limits until the change of **MPOS** allows following again.

If the method fails, the Error object is filled with the error description.

Example

```
Sub SlaveStalled_Sample()
Dim MFormula As String

'Master value is calculated as
'feedback position of the axis T with
'scale factor equal 2

MFormula = "2 * T_FPOS"
Dim Axes(2)
Axes(0) = Ch.ACSC_AXIS_Z
Axes(1) = Ch.ACSC_AXIS_T
Axes(2) = -1
On Error GoTo except

'Enable axes ZT
Call Ch.EnableM(Axes)

'Set master formula "Z_MPOS=2 * T_FPOS"
Call Ch.SetMaster(Ch.ACSC_AXIS_Z, MFormula)

'Left (negative) limit of the following
'area, right (positive) limit of the
'following area
Call Ch.SlaveStalled(0, Ch.ACSC_AXIS_Z, -5000, 5000)

'Relative motion with target position of
'10000
Call Ch.ToPoint(Ch.ACSC_AMF_RELATIVE, Ch.ACSC_AXIS_T, 10000)

'Finish the motion
'End of the multi-point motion

Call Ch.EndSequenceM(Axes)
Exit Sub
except:
MyErrorMsg 'See Error Handling in Visual Basic
End Sub
```

4.19 Multi-Point Motion Methods

The Multi-Point Motion methods are:

Table 4-22. Multi-Point Motion Methods

MultiPoint	Initiates a single-axis multi-point motion.
MultiPointM	Initiates a multi-axis multi-point motion.

4.19.1 MultiPoint

Description

The method initiates a single-axis multi-point motion.

Syntax

object.MultiPoint(long Flags, long Axis, double Dwell)

Arguments

Flags	Bit-mapped parameter that can include one or more of the following flags: ACSC_AMF_WAIT: plan the motion but don't start it until the method Go is executed. ACSC_AMF_RELATIVE: the coordinates of each point are relative. The first point is relative to the instant position when the motion starts; the second point is relative to the first, etc. If the flag is not specified, the coordinates of each point are absolute. ACSC_AMF_VELOCITY: the motion uses the velocity specified with each point instead of the default velocity. ACSC_AMF_CYCLIC: the motion uses the point sequence as a cyclic array. After positioning to the last point it does positioning to the first point and continues.
Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions.
Dwell	Dwell in each point in milliseconds.

Return Value

None.

Remarks

The method initiates a single-axis multi-point motion. To execute multi-axis multi-point motion, use [MultiPointM](#).

The motion executes sequential positioning to each of the specified points, optionally with dwell in each point.

The method itself does not specify any point, so that the created motion starts only after the first point is specified. The points of motion are specified by using the [AddPoint](#) or [AddPoint](#) methods that follow this method.

The motion finishes when the [EndSequence](#) method is executed. If the **EndSequence** call is omitted, the motion will stop at the last point of the sequence and wait for the next point. No transition to the next motion in the motion queue will occur until the method **EndSequence** executes.

The method waits for the controller response.

The controller response indicates that the command was accepted and the motion was planned successfully. The method cannot wait or validate the end of the motion. To wait for the motion end, use the [WaitMotionEnd](#).

During positioning to each point, a velocity profile is built using the default values of acceleration, deceleration, and jerk of the specified axis. If the ACSC_AMF_VELOCITY flag is not specified, the default value of velocity is used as well. If the ACSC_AMF_VELOCITY flag is specified, the value of velocity specified in subsequent [ExtAddPoint](#) methods is used.

If the method fails, the Error object is filled with the error description.

Example

```
Sub MultiPoint_Sample()
Dim Index As Long
On Error GoTo except
    'Enable axis 0
    Call Ch.Enable(Ch.ACSC_AXIS_0)
    'Wait till axis 0 enabled during 5 sec
    Call Ch.WaitMotorEnabled(Ch.ACSC_AXIS_0, 1, 5000)
    'Create multi-point motion of axis 0 with
    'default velocity with dwell 1 ms
    Call Ch.MultiPoint(0, Ch.ACSC_AXIS_0, 1)
    'Add some points
    For Index = 0 To 5
        Call Ch.AddPoint(Ch.ACSC_AXIS_0, 100 * Index)
    Next Index
    'Finish the motion
    'End of the multi-point motion
    Call Ch.EndSequence(Ch.ACSC_AXIS_0)
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.19.2 MultiPointM

Description

The method initiates a multi-axis multi-point motion.

Syntax

object.MultiPointM(long Flags, variant Axes, double Dwell)

Arguments

Flags	Bit-mapped parameter that can include one or more of the following flags: ACSC_AMF_WAIT: plan the motion but don't start it until the method GoM is executed. ACSC_AMF_RELATIVE: the coordinates of each point are relative. The first point is relative to the instant position when the motion starts; the second point is relative to the first, etc. If the flag is not specified, the coordinates of each point are absolute. ACSC_AMF_VELOCITY: the motion uses the velocity specified with each point instead of the default velocity. ACSC_AMF_CYCLIC: the motion uses the point sequence as a cyclic array. After positioning to the last point it does positioning to the first point and continues.
Axes	Array of axis constants. Each element specifies one involved axis: ACSC_AXIS_0 corresponds to the 0axis, ACSC_AXIS_1 to the 1axis, etc. After the last axis, one additional element must be included that contains -1 and marks the end of the array. For the axis constants see Axis Definitions.
Dwell	Dwell in each point in milliseconds.

Return Value

None.

Remarks

The method initiates a multi-axis multi-point motion. To execute single-axis multi-point motion, use [MultiPoint](#).

The motion executes sequential positioning to each of the specified points, optionally with dwell in each point.

The method itself does not specify any point, so the created motion starts only after the first point is specified. The points of motion are specified by using [AddPointM](#) or [ExtAddPointM](#), methods that follow this method.

The motion finishes when the [EndSequenceM](#) method is executed. If the call of **EndSequenceM** is omitted, the motion will stop at the last point of the sequence and wait for the next point. No transition to the next motion in the motion queue will occur until the method **EndSequenceM** executes.

The method waits for the controller response.

The controller response indicates that the command was accepted and the motion was planned successfully. The method cannot wait or validate the end of the motion. To wait for the motion end, use the [WaitMotionEnd](#) method.

During positioning to each point, a vector velocity profile is built using the default values of velocity, acceleration, deceleration, and jerk of the axis group. If the **AFM_VELOCITY** flag is not specified, the default value of velocity is used as well. If the **AFM_VELOCITY** flag is specified, the value of velocity specified in subsequent [ExtAddPointM](#) methods is used.

If the method fails, the Error object is filled with the error description.

Example

```
Sub MultiPointM_Sample()
Dim Index As Long
Dim Points(1)
Dim Axes(2)
Axes(0) = Ch.ACSC_AXIS_0
Axes(1) = Ch.ACSC_AXIS_1
Axes(2) = -1
On Error GoTo except

    Call Ch.EnableM(Axes)                'Enable axes 0 and 1

    Call Ch.MultiPointM(0, Axes, 0)      'Create multi-point motion of axes 0 and 1
                                        'with default velocity without dwell in the
                                        'points

    Call Ch.MultiPointM(0, Axes, 0)      'Add some points

    For Index = 0 To 5
        Points(0) = 100 * Index
    Points(1) = 100 * Index
        Call Ch.AddPointM(Axes, Points)
    Next Index

    Call Ch.EndSequenceM(Axes)           'End of the multi-point motion

Exit Sub
except:
    MyErrorMsg                          'See Error Handling in Visual Basic
End Sub
```

4.20 Arbitrary Path Motion Methods

The Arbitrary Path Motion methods are:

Table 4-23. Arbitrary Path Motion Methods

Spline	Initiates a single-axis spline motion. The motion follows an arbitrary path defined by a set of points.
SplineM	Initiates a multi-axis spline motion. The motion follows an arbitrary path defined by a set of points.

4.20.1 Spline

Description

The method initiates a single-axis spline motion. The motion follows an arbitrary path defined by a set of points.

Syntax

object.Spline(long Flags, long Axis, [double Period])

Arguments

Flags	Bit-mapped parameter that can include one or more of the following flags: ACSC_AMF_WAIT: plan the motion but don't start it until the method Go is executed. ACSC_AMF_RELATIVE: the coordinates of each point are relative. The first point is relative to the instant position when the motion starts; the second point is relative to the first, etc. If the flag is not specified, the coordinates of each point are absolute. ACSC_AMF_VARTIME: the time interval between adjacent points is non-uniform and is specified along with each added point. If the flag is not specified, the interval is uniform and is specified in the Period argument. ACSC_AMF_CYCLIC: use the point sequence as a cyclic array: after the last point come to the first point and continue. ACSC_AMF_CUBIC: use a cubic interpolation between the specified points (third-order spline).
Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions.
Period	Time interval between adjacent points. The parameter is used only if the ACSC_AMF_VARTIME flag is not specified.

Return Value

None.

Remarks

The method initiates a single-axis spline motion. To execute multi-axis spline motion, use [SplineM](#).

The method waits for the controller response.

The controller response indicates that the command was accepted and the motion was planned successfully. The method cannot wait or validate the end of the motion. To wait for the motion end, use the [WaitProgramEnd](#) method.

The motion does not use the default values of velocity, acceleration, deceleration, and jerk. The points and the time intervals between the points completely define the motion profile.

Points for arbitrary path motion are defined by the consequent calls of [AddPoint](#) or [ExtAddPoint](#) methods. The [EndSequence](#) method terminates the point sequence. After execution of the [EndSequence](#) method, no [AddPoint](#) or [ExtAddPoint](#) methods for this motion are allowed.

If **Flag** is not specified, linear interpolation is used (first-order spline).

Time intervals between the points are uniform, or non-uniform as defined by the **ACSC_AMF_VARTIME** flag. If **ACSC_AMF_VARTIME** is not specified, the controller builds PV spline motion.

If **ACSC_AMF_VARTIME** is specified, the controller builds PVT spline motion. The trajectory of the motion follows through the defined points. Each point presents the instant desired position at a specific moment.

This motion does not use a motion profile generation. The time intervals between the points are typically short, so that the array of the points implicitly specifies the desired velocity in each point.

If the time interval does not coincide with the controller cycle, the controller provides interpolation of the points according to the **ACSC_AMF_CUBIC** flag.

If the method fails, the Error object is filled with the error description.

Example

```
Sub Spline_Sample()
Dim Index As Long
Dim Points(1) As Double
On Error GoTo except

    'Enable axis 0
    Call Ch.Enable(Ch.ACSC_AXIS_0)
    'Wait till axis 0 enabled during 5 sec
    Call Ch.WaitMotorEnabled(Ch.ACSC_AXIS_0, 1, 5000)
    'Create the arbitrary path motion to axis 0
    'use a cubic interpolation between the
    'specified Points with uniform interval 500
    'ms
    Call Ch.Spline(Ch.ACSC_AMF_CUBIC, Ch.ACSC_AXIS_0, 500)
    'Add some points
    For Index = 0 To 5
        Points(0) = 100 * Index
        Points(1) = 20000 * Index
        Call Ch.AddPVPoint(Ch.ACSC_AXIS_0, Points(0), Points(1))
    Next Index
    'Finish the motion
    'End of the multi-point motion
    Call Ch.EndSequence(Ch.ACSC_AXIS_0)
Exit Sub
except:
    MyErrorMsg 'See Error Handling in Visual Basic
End Sub
```

4.20.2 SplineM

Description

The method initiates a multi-axis spline motion. The motion follows an arbitrary path defined by a set of points.

Syntax

object.SplineM(long Flags, variant Axes, [double Period])

Arguments

Flags	Bit-mapped parameter that can include one or more of the following flags: ACSC_AMF_WAIT: plan the motion but don't start it until the method GoM is executed. ACSC_AMF_RELATIVE: the coordinates of each point are relative. The first point is relative to the instant position when the motion starts; the second point is relative to the first, etc. If the flag is not specified, the coordinates of each point are absolute. ACSC_AMF_VARTIME: the time interval between adjacent points is non-uniform and is specified along with each added point. If the flag is not specified, the interval is uniform and is specified in the Period argument. ACSC_AMF_CYCLIC: use the point sequence as a cyclic array: after the last point come to the first point and continue. ACSC_AMF_CUBIC: use a cubic interpolation between the specified points (third-order spline).
Axes	Array of axis constants. Each element specifies one involved axis: ACSC_AXIS_0 corresponds to the 0axis, ACSC_AXIS_1 to the 1axis, etc. After the last axis, one additional element must be included that contains -1 and marks the end of the array. For the axis constants see Axis Definitions.
Period	Time interval between adjacent points. The parameter is used only if the ACSC_AMF_VARTIME flag is not specified.

Return Value

None.

Remarks

The method initiates a multi-axis spline motion. To execute a single-axis spline motion, use [Spline](#).
The method waits for the controller response.

The controller response indicates that the command was accepted and the motion was planned successfully. The method cannot wait or validate the end of the motion. To wait for the motion end, use the [WaitMotionEnd](#) method.

The motion does not use the default values of velocity, acceleration, deceleration, and jerk. The points and the time intervals between the points define the motion profile completely.

Points for arbitrary path motion are defined by the consequent calls of the [AddPVPointM](#) or [ExtAddPointM](#) methods. The [EndSequenceM](#) method terminates the point sequence. After execution of the EndSequenceM method, no AddPointM or ExtAddPointM methods for this motion are allowed.

The trajectory of the motion follows through the defined points. Each point presents the instant desired position at a specific moment. Time intervals between the points are uniform, or non-uniform as defined by the **ACSC_AMF_VARTIME** flag.

This motion does not use motion profile generation. Typically, the time intervals between the points are short, so that the array of the points implicitly specifies the desired velocity in each point.

If the time interval does not coincide with the controller cycle, the controller provides interpolation of the points according to the **ACSC_AMF_CUBIC** flag.

If the method fails, the Error object is filled with the error description.

Example

```

Sub SplineM_Sample()
Dim Index As Long
Dim Points(1) As Double
Dim Axes(2)
Axes(0) = Ch.ACSC_AXIS_0
Axes(1) = Ch.ACSC_AXIS_1
Axes(2) = -1
On Error GoTo except

    Call Ch.EnableM(Axes)                                'Enable axes 0 and 1

    'Create the arbitrary path motion to axes
    '0 and 1 with uniform interval 10 ms use a
    'cubic interpolation between the specified
    'Points
    Call Ch.SplineM(Ch.ACSC_AMF_CUBIC, Axes, 10)
    'Add some points

For Index = 0 To 5
    Points(0) = 100 * Index
    Points(1) = 200 * Index
    Call Ch.AddPVPointM(Axes, Points, Points)
Next Index

    'Finish the motion
    'End of the multi-point motion

    Call Ch.EndSequenceM(Axes)
Exit Sub
except:
    MyErrorMsg      'See Error Handling in Visual Basic
End Sub

```

4.21 PVT Methods

The PVT methods are:

Table 4-24. PVT Methods

AddPVPoint	Adds a point to a single-axis multi-point or spline motion.
AddPVPointM	Adds a point to a multi-axis multi-point or spline motion.
AddPVTPoint	Adds a point to a single-axis multi-point or spline motion.
AddPVTPointM	Adds a point to a multi-axis multi-point or spline motion.

4.21.1 AddPVPoint**Description**

The method adds a point to a single-axis PV spline motion and specifies velocity.

Syntax

object.AddPVPoint(long Axis, double Point, double Velocity)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Point	Coordinate of the added point.
Velocity	Desired velocity at the point

Return Value

None.

Remarks

Before this method can be used, initiate PV spline motion by calling [Spline](#) with the appropriate flags.

The method adds a point to a single-axis PV spline motion with a uniform time and specified velocity at that point

To add a point to a multi-axis PV motion, use [AddPVPointM](#) and [ExtAddPointM](#).

To add a point to a PVT motion with non-uniform time interval, use the [AddPVPoint](#) and [AddPVPointM](#) methods. The method waits for the controller response.

The controller response indicates that the command was accepted and the point is added to the motion buffer. The point can be rejected if the motion buffer is full. In this case, call this method periodically until the method returns a non-zero value.

If the method fails, the Error object is filled with the error description.

Example

```
Sub AddPVPoint_Sample()
Dim Index As Long
Points(1) As Double
On Error GoTo except
                                'Create PV motion with uniform time
                                'interval with uniform interval 500 ms
Call Ch.Spline(Ch.ACSC_AMF_CUBIC, Ch.ACSC_AXIS_0, 500)
                                'Add some points
For Index = 0 To 5
Points(0) = 100 * Index
Points(1) = 20000 * Index                                'Position and velocity for each point
                                'of axis 0
Call Ch.AddPVPoint(Ch.ACSC_AXIS_0, Points(0), Points(1))
Next Index
For Index = 0 To 10000000000: Next Index
                                'Finish the motion
                                'The end of the arbitrary path motion
Call Ch.EndSequence(Ch.ACSC_AXIS_0)
Exit Sub
except:
```

```
MyErrorMsg 'See Error Handling in Visual Basic
End Sub
```

4.21.2 AddPVPointM

Description

The method adds a point to a multiple-axis PV spline motion and specifies velocity.

Syntax

object.AddPVPointM(variant Axes, variant Point, variant Velocity)

Arguments

Axes	Array of axis constants. Each element specifies one involved axis: ACSC_AXIS_0 corresponds to the 0axis, ACSC_AXIS_1 to the 1axis, etc. After the last axis, one additional element must be included that contains -1 and marks the end of the array. For the axis constants see Axis Definitions .
Point	Array of the coordinates of added point. The number and order of values must correspond to the Axes array. Point must specify a value for each element of Axes except the last -1 element.
Velocity	Array of the velocities of added point. The number and order of values must correspond to the Axes array. The Velocity must specify a value for each element of Axes except the last -1 element.

Return Value

None.

Remarks

Before this method can be used, PVT spline motion must be initiated by calling [SplineM](#) with the appropriate flags.

The method adds a point to a multiple-axis PV spline motion with a uniform time and specified velocity at that point.

To add a point to a single-axis PV motion, use [AddPVPoint](#). To add a point to a PVT motion with non-uniform time interval, use the [AddPVTPoint](#) and [AddPVTPointM](#) methods.

The method waits for the controller response.

The controller response indicates that the command was accepted and the point is added to the motion buffer. The point can be rejected if the motion buffer is full. In this case, you can call this method periodically until the method returns non-zero value.

All axes specified in the **Axes** array must be specified before calling the [MultiPointM](#) or the [SplineM](#) method. The number and order of the axes in the **Axes** array must correspond exactly to the number and order of the axes of [MultiPointM](#) or the [SplineM](#) methods.

If the method fails, the Error object is filled with the error description.

Example

```
Sub AddPVPointM_Sample()
Dim Index As Long
Dim Points(1) As Double
Dim Axes(2) As String
Axes(0) = Ch.ACSC_AXIS_0
Axes(1) = Ch.ACSC_AXIS_1
Axes(2) = -1
On Error GoTo except                                'Enable axes 0 and 1
    Call Ch.EnableM(Axes)                            'Create PV motion with uniform time
                                                    'interval with uniform interval 1000 ms
    Call Ch.SplineM(Ch.ACSC_AMF_CUBIC, Axes, 1000)
                                                    'Add some points

    For Index = 0 To 5
        Points(0) = 100 * Index
        Points(1) = 200 * Index                    'Position and velocity for each point
        Call Ch.AddPVPointM(Axes, Points, Points)
        Next Index                                'Finish the motion
                                                    'End of the arbitrary path motion

    Call Ch.EndSequenceM(Axes)
Exit Sub
except:
    MyErrorMsg                                     'See Error Handling in Visual Basic
End Sub
```

4.21.3 AddPVTPoint

Description

The method adds a point to a single-axis PVT spline motion and specifies velocity and motion time.

Syntax

object.AddPVTPoint(long Axis, double Point, double Velocity, [double TimeInterval])

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Point	Coordinate of the added point.
Velocity	Desired velocity at the point
TimeInterval	If the motion was activated by the Spline method with the ACSC_AMF_VARTIME flag, this parameter defines the time interval between the previous point and the present one.

Return Value

None.

Remarks

Before this method can be used, PV spline motion must be initiated by calling [Spline](#) with the appropriate flags.

The method adds a point to a single-axis PVT spline motion with a non-uniform time and specified velocity at that point.

To add a point to a multi-axis PVT motion, use [AddPVTPointM](#). To add a point to a PV motion with uniform time interval, use the [AddPVPoint](#) and [AddPVPointM](#) methods.

The method waits for the controller response.

The controller response indicates that the command was accepted and the point is added to the motion buffer. The point can be rejected if the motion buffer is full. In this case, you can call this method periodically until the method returns non-zero value.

If the method fails, the Error object is filled with the error description.

Example

```
Sub AddPVTPoint_Sample()
Dim Index As Long
Dim Points(1) As Double
On Error GoTo except          'Enable axis 0
    Call Ch.Enable(Ch.ACSC_AXIS_0) 'Wait axis 0 enabled during 5 sec
Call Ch.WaitMotorEnabled(Ch.ACSC_AXIS_0, 1, 5000)
                                'PVT motion and uniform interval is
                                'not used
    Call Ch.Spline(Ch.ACSC_AMF_CUBIC Or Ch.ACSC_AMF_VARTIME,
Ch.ACSC_AXIS_0, 0)              'Add some points
    For Index = 0 To 5
        Points(0) = 100 * Index
        Points(1) = 200 * Index          'Position, velocity and time
                                        'interval of 500 ms for each point
    Call Ch.AddPVTPoint(Ch.ACSC_AXIS_0, Points(0), Points(1), 500)
    Next Index                        'Finish the motion
                                        'End of the arbitrary path motion
    Call Ch.EndSequence(Ch.ACSC_AXIS_0)
Exit Sub
except:
    MyErrorMsg                    'See Error Handling in Visual Basic
End Sub
```

4.21.4 AddPVTPointM**Description**

The method adds a point to a multiple-axis PVT spline motion and specifies velocity and motion time.

Syntax

object.AddPVTPointM(variant Axes, variant Point, variant Velocity, [double TimeInterval])

Arguments

Axes	Array of axis constants. Each element specifies one involved axis: ACSC_AXIS_0 corresponds to the 0axis, ACSC_AXIS_1 to the 1axis, etc. After the last axis, one additional element must be included that contains -1 and marks the end of the array. For the axis constants see Axis Definitions .
Point	Array of the coordinates of added point. The number and order of values must correspond to the Axes array. The Point must specify a value for each element of Axes except the last -1 element.
Velocity	Array of the velocities of added point. The number and order of values must correspond to the Axes array. The Velocity must specify a value for each element of Axes except the last -1 element.
TimeInterval	If the motion was activated by the Spline method with the ACSC_AMF_VARTIME flag, this parameter defines the time interval between the previous point and the present one.

Return Value

None.

Remarks

Before this method can be used, PVT spline motion must be initiated by calling [SplineM](#) with the appropriate flags.

The method adds a point to a multiple-axis PVT spline motion with a non-uniform time and specified velocity at that point.

To add a point to a single-axis PVT motion, use [AddPVTPoint](#). To add a point to a PV motion with a uniform time interval, use the [AddPVPoint](#) and [AddPVPointM](#) methods.

The method waits for the controller response.

The controller response indicates that the command was accepted and the point is added to the motion buffer. The point can be rejected if the motion buffer is full. In this case, you can call this method periodically until the method returns non-zero value.

If the method fails, the Error object is filled with the error description.

Example

```
Sub AddPVTPointM_Sample ()
Dim Index As Long
Dim Points(1)
Dim Axes(2)
```

```

Axes(0) = Ch.ACSC_AXIS_0
Axes(1) = Ch.ACSC_AXIS_1
Axes(2) = -1
On Error GoTo except
    'Enable axes 0 and 1
    Call Ch.EnableM(Axes)
    'PVT motion and uniform interval is not
    'used
    Call Ch.SplineM(Ch.ACSC_AMF_CUBIC Or Ch.ACSC_AMF_VARTIME, Axes, 0)
    'Add some points
    For Index = 0 To 5
        Points(0) = 100 * Index
        Points(1) = 200 * Index
        'Position, velocity and time interval of
        '1000 ms for each point
        Call Ch.AddPVTPointM(Axes, Points, Points, 1000)
    Next Index
    'End of the multi-point motion
    Call Ch.EndSequenceM(Axes)
Exit Sub
except:
    MyErrorMsg
End Sub
'See Error Handling in Visual Basic

```

4.22 Segmented Motion Methods

The Segmented Motion methods are:

Table 4-25. Segmented Motion Methods

ExtendedSegmentedMotionExt	Initiates a multi-axis extended segmented motion.
SegmentLineExt	The function adds a linear segment that starts at the current point and ends at the destination point of segmented motion.

ExtendedSegmentArc1	Adds an arc segment to a segmented motion and specifies the coordinates of center point, coordinates of the final point, and the direction of rotation.
ExtendedSegmentArc2	Adds an arc segment to a segmented motion and specifies the coordinates of center point and rotation angle.
BlendedSegmentMotion	The function initiates a multi-axis blended segmented motion. Extended segmented motion provides new features.
BlendedLine	The function adds a linear segment that starts at the current point and ends at the destination point of segmented motion.

BlendedArc1	The function adds to the motion path an arc segment that starts at the current point and ends at the destination point with the specified center point.
BlendedArc2	The function adds an arc segment to a segmented motion and specifies the coordinates of the center point and the rotation angle.
Stopper	Provides a smooth transition between two segments of segmented motion.
Projection	Sets a projection matrix for a segmented motion.

4.22.1 *ExtendedSegmentedMotionExt*

Description

The method initiates a multi-axis extended segmented motion.

Syntax

```
object.ExtendedSegmentedMotionExt(long Flags, VARIANT Axes, VARIANT Point, double Velocity,
double EndVelocity, double JunctionVelocity, double Angle, double CurveVelocity, double Deviation,
double Radius, double MaxLength, double StarvationMargin, BSTR Segments, ULONGLONG CallType,
VARIANT *pWait);
```

Arguments

Flags	Bit-mapped argument that can include one or more of the flags in the Flags Table
Axes	Array of axis constants. Each element specifies one involved axis: ACSC_AXIS_0 corresponds to axis 0, ACSC_AXIS_1 to axis 1, etc. After the last axis, one additional element must be located that contains -1 which marks the end of the array. For axis constants see Axis Definitions
Point	Array of the starting point coordinates. The number and order of values must correspond to the Axes array. The Point must specify a value for each element of Axes except the last -1 element.
Velocity	If ACSC_AMF_VELOCITY flag has been specified, this argument specifies a motion velocity for current segment. Set this argument to ACSC_NONE if not used.
EndVelocity	If ACSC_AMF_ENDVELOCITY flag has been specified, this argument defines the required velocity at the end of the current segment. Set this argument to ACSC_NONE if not used.
JunctionVelocity	If ACSC_AMF_JUNCTIONVELOCITY flag has been specified, this argument defines the required velocity at the junction. Set this argument to ACSC_NONE if not used.
Angle	if ACSC_AMF_ANGLE flag has been specified, this argument specifies the threshold above which a junction will be treated as a corner. Set this argument to ACSC_NONE if not used.
CurveVelocity	If ACSC_AMF_CURVEVELOCITY flag has been specified, this argument defines the required velocity at curvature discontinuity points. Set this argument to ACSC_NONE if not used.
Deviation	– If ACSC_AMF_CORNERDEVIATION flag has been specified, this argument defines the maximal allowed trajectory deviation from the corner point. Set this argument to ACSC_NONE if not used.
Radius	If ACSC_AMF_CORNERRADIUS flag has been specified, this argument defines the maximal allowed rounding radius of the additional segment. Set this argument to ACSC_NONE if not used.

MaxLength	If ACSC_AMF_CORNERLENGTH flag has been specified, this argument defines the maximum length of the segment for processing automatic corner rounding. If a segment's corner attempts to exceed the maximum length, the corner will not be rounded. Set this argument to ACSC_NONE if not used.
StarvationMargin	Starvation margin in milliseconds. The controller sets the AST.#NEWSEGM bit to the StarvationMargin millisecond before the starvation condition occurs. Set this argument to ACSC_NONE if not used. By default, if this argument is not specified, the starvation margin is 500 milliseconds.
 If the Segments parameter is used, then the starvation margin must also be defined.	
Segments	- Pointer to the null-terminated character string that contains the name of a one dimensional user-defined array used to store added segments. Set this argument to NULL if not used. By default, if this argument is not specified, the controller allocates internal buffer for storing 50 segments only. The argument allows the user application to reallocate the buffer for storing a larger number of segments. The larger number of segments may be required if the application needs to add many very small segments in advanced. For most applications, the internal buffer size is enough and should not be enlarged. The buffer is for the controller internal use only and should not be used by the user application. The buffer size calculation rule: each segment requires about 600 bytes, so if it is necessary to allocate the buffer for 200 segments, it should be at least $600 * 200 = 120,000$ bytes. The following declaration defines a 120,000 bytes buffer: <code>real buf(15000)</code> See XARRSIZE explanation in the ACSPL+ Command and Variable Reference Guide for details on how to declare a buffer with more than 100,000 elements.

ulonglong CallType	If CALLTYPE is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary ()function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to the GetString or GetBinary functions..

Flags

General Flags

ACSC_AMF_WAIT	Plan the motion but do not start it until the function GoM is executed.
ACSC_AMF_VELOCITY	The motion will use velocity specified for each segment instead of the default velocity.
ACSC_AMF_MAXIMUM	Use maximum velocity under axis limits. With this suffix, no required velocity should be specified. The required velocity is calculated for each segment individually on the base of segment geometry and axis velocities (VEL values) of the involved axes.
ACSC_AMF_AXISLIMIT	Enable velocity limitations under axis limits. With this flag set, setting the ACSC_AMF_VELOCITY flag will result in the requested velocity being restrained by the velocity limits of all involved axes.

Velocity Look-ahead Flags

ACSC_AMF_ENDVELOCITY	This flag requires additional an parameter that specifies the end velocity. The controller decelerates to the specified velocity at the end of the segment. The specified value should be less than the required velocity, otherwise the parameter is ignored. This flag affects only one segment. This flag also disables corner detection and processing at the end of segment. If this flag is not specified, deceleration is not required. However, in special case the deceleration might occur due to corner processing or other velocity control conditions.
ACSC_AMF_JUNCTIONVELOCITY	Decelerate to corner. This flag requires an additional parameter that specifies corner velocity. The controller detects corners on the path and decelerates to the specified velocity before the corner. The specified value should be less than the required velocity; otherwise the parameter is ignored. If ACSC_AMF_JUNCTIONVELOCITY flag is not specified while ACSC_AMF_ANGLE flag is specified, a value of zero is assumed for the corner velocity. If none of the ACSC_AMF_JUNCTIONVELOCITY and ACSC_AMF_ANGLE flags are specified, the controller provides automatic calculation as described in Automatic corner processing.
ACSC_AMF_CURVEVELOCITY	Decelerate to curvature discontinuity point. This flag requires an additional parameter that specifies the velocity at curvature discontinuity points. Curvature discontinuity occurs in linear-to-arc or arc-to-arc smooth junctions. If the flag is not set, the controller does not decelerate to a smooth junction, disregarding curvature discontinuity in the junction. If the flag is set, the controller detects curvature discontinuity points on the path and provides deceleration to the specified velocity. The specified value should be less than the required velocity; otherwise the parameter is ignored. The flag can be set together with flags ACSC_AMF_JUNCTIONVELOCITY and/or ACS_AMF_ANGLE. If neither of ACSC_AMF_JUNCTIONVELOCITY, ACS_AMF_ANGLE or ACSC_AMF_CURVEVELOCITY is set, the controller provides automatic calculation of the corner processing.

ACSC_AMF_ANGLE	Do not treat junction as a corner, if junction angle is less than or equal to the specified value in radians. This flag requires additional parameter that specifies negligible angle in radians. If ACSC_AMF_ANGLE flag is not specified while ACSC_AMF_JUNCTIONVELOCITY flag is specified, the controller accepts default value of 0.01 radians that is about 0.57 degrees. If neither the ACSC_AMF_JUNCTIONVELOCITY nor the ACSC_AMF_ANGLE flags are specified the controller follows the automatic calculation described in Automatic corner processing.
ACSC_AMF_CURVEAUTO	If this flag is specified the controller provides automatic calculations as described in enhanced automatic corner and curvature discontinuity points processing.

Corner Rounding Flags

ACSC_AMF_CORNERDEVIATION	Use a corner rounding option with the specified permitted deviation. This flag requires an additional parameter that specifies maximal allowed deviation of motion trajectory from the corner point. This flag cannot be set together with flags ACSC_AMF_CORNERRADIUS and ACSC_AMF_CORNERLENGTH.
ACSC_AMF_CORNERRADIUS	Use a corner rounding option with the specified permitted curvature. This flag requires an additional parameter that specifies maximal allowed rounding radius of the additional segment. This flag cannot be specified together with flags ACSC_AMF_CORNERLENGTH or ACSC_AMF_CORNERDEVIATION.
ACSC_AMF_CORNERLENGTH	Use automatic corner rounding option. This flag requires an additional parameter that specifies the maximum length of the segment for automatic corner rounding. If the length of one of the segments constituting a corner exceeds the specified maximal length, the corner will not be rounded. The automatic corner rounding is only applied pairs of linear segments. If one of the segments in a pair is an arc, the rounding is not applied for this corner. This flag cannot be set together with flags ACSC_AMF_CORNERDEVIATION or ACSC_AMF_CORNERRADIUS.

Return Value

None

Remarks

The function itself does not specify any segments, so the created motion starts only after the first segment is specified.

The segments of motion are specified by using **SegmentLineExt**, **ExtendedSegmentArc1**, **ExtendedSegmentArc2** functions that follow this function.

The motion finishes when the **EndSequenceM** function is executed. If the call of **EndSequenceM** is omitted, the motion will stop at the last segment of the sequence and wait for the next segment. No transition to the next motion in the motion queue will occur until the function **EndSequenceM** is executed.

During motion to each point, a vector velocity profile is built using the default values of velocity, acceleration, deceleration, and jerk of the axis group. If the **ACSC_AFM_VELOCITY** flag is not specified, the default value of velocity is used as well. If the **ACSC_AFM_VELOCITY** flag is specified, the value of velocity specified in subsequent **SegmentLineExt**, **SegmentArc1Ext**, or **SegmentArc2Ext** functions is used.

This function is supported in Versions 2.70 and higher.

Example

```
int[] Axes = new int[3];
double[] Point = new double[2];
double[] Center = new double[2];
Axes[0] = Ch.ACSC_AXIS_0;
Axes[1] = Ch.ACSC_AXIS_1;
Axes[2] = -1;
Point[0] = 0;
Point[1] = 0;
Ch.ExtendedSegmentedMotionExt(Ch.ACSC_AMF_VELOCITY | Ch.ACSC_AMF_
CORNERRADIUS, // Velocity and corner radius flags are set, parameters now
required.
Axes, //Axes 0,1 active
Point, // Starting point
1000, // Velocity is set to 1000
Ch.ACSC_NONE, //EndVelocity is the default value
Ch.ACSC_NONE, //JunctionVelocity is the default value
Ch.ACSC_NONE, // Angle is the default value
Ch.ACSC_NONE, // CurveVELOCITY is the default value
Ch.ACSC_NONE, // Deviation
10, // Radius is set
Ch.ACSC_NONE, // Maximum corner length is default
Ch.ACSC_NONE, // Starvation margin is the default value
null); //Segments are set to NULL
Center[0] = 500;
Center[1] = 500;
Ch.ExtendedSegmentArc1(Ch.ACSC_AMF_VARTIME | Ch.ACSC_AMF_ENDVELOCITY,
Axes, Center, Point, Ch.ACSC_CLOCKWISE, Ch.ACSC_NONE, 10, 1000, null,
null, Ch.ACSC_NONE, null);
Ch.EndSequenceM(Axes);
```


4.22.2 SegmentLineExt

Description

The function adds a linear segment that starts at the current point and ends at the destination point of segmented motion.

Syntax

object.SegmentLineExt(long Flags, VARIANT Axes, VARIANT Point, double Velocity, double EndVelocity, double Time, BSTR Values, BSTR Variables, int Index, BSTR Masks, ULONGLONG CallType, VARIANT* pWait);

Arguments

Flags	Bit-mapped argument that can include one or more of the flags defined in the flags table below.
Axes	Array of axis constants. Each element specifies one involved axis: ACSC_AXIS_0 corresponds to axis 0, ACSC_AXIS_1 to axis 1, etc. After the last axis, one additional element must be located that contains -1 which marks the end of the array. For axis constants see Axis Definitions
Point	Array of the final point coordinates. The number and order of values must correspond to the Axes array. The Point must specify a value for each element of Axes except the last -1 element.
Velocity	If ACSC_AMF_VELOCITY flag has been specified, this argument specifies a motion velocity for current segment. Set this argument to ACSC_NONE if not used.
EndVelocity	If ACSC_AMF_ENDVELOCITY flag has been specified, this argument defines the required velocity at the end of the current segment. Set this argument to ACSC_NONE if not used.
Time	If ACSC_AMF_VARTIME flag has been specified, this argument defines the segment processing time in milliseconds, for the current segment only and has no effect on subsequent segments. Set this argument to ACSC_NONE if not used.
Values	Pointer to the null-terminated character string that contains the name of a one-dimensional user-defined array of integer or real type with a maximum size of 10 elements. If ACSC_AMF_USERVARIABLES flag has been specified, this argument defines the values to be written to the Variables array at the beginning of the current segment execution. Set this argument to NULL if not used.
Index	If ACSC_AMF_USERVARIABLES has not been specified, this argument defines the first element (starting from zero) of the Variables array, to

	which Values data will be written to. Set this argument to ACSC_NONE if not used.
Masks	Pointer to the null-terminated character string that contains the name of a one-dimensional user defined array of integer type and same size as the Values array. If ACSC_AMF_USERVARIABLES flag has been specified, this argument defines the masks that are applied to Values before the Values are written to variables array at the beginning of the current segment execution. The masks are only applied for integer values: $\text{variables}(n) = \text{values}(n) \text{ AND mask}(n)$. If Values is a real array, the masks argument should be NULL. Set this argument to NULL if not used.
ulonglong CallType	If CALLTYPE is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary ()function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to the GetString or GetBinary functions..

Flags

ACSC_AMF_VELOCITY	The motion will use velocity specified for each segment instead of the default velocity.
ACSC_AMF_ENDVELOCITY	This flag requires an additional parameter that specifies end velocity. The controller decelerates to the specified velocity at the end of segment. The specified value should be less than the required velocity; otherwise the parameter is ignored. This flag affects only one segment.

ACSC_AMF_VARTIME	This flag requires an addition parameter that specifies the segment processing time in milliseconds. The segment processing time defines velocity at the current segment only and has no effect on subsequent segments. This flag also disables corner detection and processing at the end of segment. If this flag is not specified, deceleration is not required. However, in special cases the deceleration might occur due to corner processing or other velocity control conditions.
ACSC_AMF_USERVARIABLES	Synchronize user variables with segment execution. This flag requires additional parameters that specify values, user variable and mask. See details in the Values, Variables, and Masks arguments.

Return Value

None

Comments

The function adds a linear segment that starts at the current point and ends at the destination point to segmented motion.

All axes specified in the **Axes** array must be specified before the call of the **SegmentedMotion** or **SegmentLineExt** function. The number and order of the axes in the **Axes** array must correspond exactly to the number and order of the axes of the **SegmentedMotion** or **ExtendedSegmentedMotionExt** functions.

The **Point** argument specifies the coordinates of the final point. The coordinates are absolute in the plane.

ACSC_AMF_VELOCITY and ACSC_AMF_VARTIME are mutually exclusive, meaning they cannot be used together.

The method can wait for the controller response or can return immediately as specified by the Wait argument.

The controller response indicates that the command was accepted and the segment is added to the motion buffer. The segment can be rejected if the motion buffer is full. In this case, you can call this function periodically until the function returns a non-zero value.

This function is supported in Versions 2.70 and higher.

Example

```
int[] Axes = new int[3];
    double[] Point = new double[2];
    double[] Center = new double[2];
    Axes[0] = Ch.ACSC_AXIS_0;
    Axes[1] = Ch.ACSC_AXIS_1;
    Axes[2] = -1;
// create segmented motion, coordinates of the initial point are
// (1000, 1000)
Point[0] = 1000; Point[1] = 1000;
Ch.ExtendedSegmentedMotionExt(Ch.ACSC_AMF_VELOCITY, // Create the
```

```

segmented motion with specified velocity
Axes, // Axes 0 and 1
Point, // Initial point
1000, // Vector velocity is specified
Ch.ACSC_NONE, // End velocity is not specified
Ch.ACSC_NONE, // Junction velocity is not specified
Ch.ACSC_NONE, // Angle is not specified
Ch.ACSC_NONE, // Curve velocity is not specified
Ch.ACSC_NONE, // Deviation is not specified
Ch.ACSC_NONE, // curve radius is not specified
Ch.ACSC_NONE, // maximal curve length is not specified
Ch.ACSC_NONE, // Default starvation margin will be used
null // Internal buffer will be used
);
// add line segment with final point (1000, -1000), vector velocity 25000
Point[0] = 1000; Point[1] = -1000;
Ch.SegmentLineExt(
Ch.ACSC_AMF_VELOCITY, // Velocity is specified
Axes, // Axes 0 and 1
Point, // Final point
25000, // Vector velocity
Ch.ACSC_NONE, // End velocity is not specified
Ch.ACSC_NONE, // Time is not specified
null, // Values array is not specified
null, // Variables array is not specified
Ch.ACSC_NONE, // Index is not specified
null // Masks array is not specified
);
// finish the motion
Ch.EndSequenceM(Axes);

```

4.22.3 *ExtendedSegmentArc1*

Description

The function adds to the motion path an arc segment that starts at the current point and ends at the destination point with the specified center point.

Syntax

object. ExtendedSegmentArc1(long Flags, VARIANT Axes, VARIANT Center, VARIANT FinalPoint, int Rotation, double Velocity, double EndVelocity, double Time, BSTR Values, BSTR Variables, int Index, BSTR Masks, ULONGLONG CallType, VARIANT* pWait);

Arguments

Flags	Bit-mapped argument that can include one or more of the flags defined in the flags table below.
Axes	Array of axis constants. Each element specifies one involved axis: ACSC_AXIS_0 corresponds to axis 0, ACSC_AXIS_1 to axis 1, etc. After the last axis, one additional element must be located that contains -1 which marks the

	end of the array. For axis constants see Axis Definitions
Center	Array of the center coordinates. The number and order of values must correspond to the Axes array. The Center must specify a value for each element of the Axes except the last –1 element.
FinalPoint	Array of the final point coordinates. The number and order of values must correspond to the Axes array. The FinalPoint must specify a value for each element of Axes except the last –1 element.
Rotation	This argument defines the direction of rotation. If Rotation is set to ACSC_COUNTERCLOCKWISE, then the rotation is counterclockwise. If Rotation is set to ACSC_CLOCKWISE, then rotation is clockwise.
Velocity	If ACSC_AMF_VELOCITY flag has been specified, this argument specifies a motion velocity for current segment. Set this argument to ACSC_NONE if not used.
EndVelocity	If ACSC_AMF_VARTIME flag has been specified, this argument defines the segment processing time in milliseconds, for the current segment only and has no effect on subsequent segments. Set this argument to ACSC_NONE if not used.
Time	If ACSC_AMF_VARTIME flag has been specified, this argument defines the segment processing time in milliseconds, for the current segment only and has no effect on subsequent segments. Set this argument to ACSC_NONE if not used.
Values	Pointer to the null-terminated character string that contains the name of a one-dimensional user-defined array of integer or real type with a size of 10 elements maximum. If ACSC_AMF_USERVARIABLES flag has been specified, this argument defines the values to be written to the Variables array at the beginning of the current segment execution. Set this argument to NULL if not used.
Variables	Pointer to the null-terminated character string that contains the name of a one-dimensional user-defined array of the same type and size as Values array. If ACSC_AMF_USERVARIABLES flag has been specified, this argument defines the user defined array, which will be written with Values data at the beginning of the current segment execution. Set this argument to NULL if not used.

Index	If ACSC_AMF_USERVARIABLES has not been specified, this argument defines the first element (starting from zero) of the Variables array, to which Values data will be written to. Set this argument to ACSC_NONE if not used.
Masks	Pointer to the null-terminated character string that contains the name of a one-dimensional user defined array of integer type and same size as the Values array. If ACSC_AMF_USERVARIABLES flag has been specified, this argument defines the masks that are applied to Values before the Values are written to variables array at the beginning of the current segment execution. The masks are only applied for integer values: $\text{variables}(n) = \text{values}(n) \text{ AND mask}(n)$. If Values is a real array, the masks argument should be NULL. Set this argument to NULL if not used.
ulonglong CallType	If CALLTYPE is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary ()function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to the GetString or GetBinary functions..

Flags

ACSC_AMF_VELOCITY	The motion will use velocity specified for each segment instead of the default velocity.
ACSC_AMF_ENDVELOCITY	This flag requires additional parameter that specifies end velocity. The controller decelerates to the specified velocity in the end of segment. The specified value should be less than the required velocity; otherwise the parameter is ignored. This flag affects only one segment.

ACSC_AMF_VARTIME	This flag requires an additional parameter that specifies the segment processing time in milliseconds. The segment processing time defines velocity at the current segment only and has no effect on subsequent segments. This flag also disables corner detection and processing at the end of segment. If this flag is not specified, deceleration is not required. However, in special cases the deceleration might occur due to corner processing or other velocity control conditions.
ACSC_AMF_USERVARIABLES	Synchronize user variables with segment execution. This flag requires additional parameters that specify values, user variable and mask. See details in Values, Variables, and Masks below.

Return Value

None

Comments

All axes specified in the Axes array must be specified before the call of the **SegmentedMotion** or **ExtendedSegmentedMotion** function. The number and order of the axes in the Axes array must correspond exactly to the number and order of the axes of the **SegmentedMotion** or **ExtendedSegmentedMotionExt** function.

The Point argument specifies the coordinates of the final point. The coordinates are absolute in the plane.

ACSC_AMF_VELOCITY and ACSC_AMF_VARTIME are mutually exclusive, meaning they cannot be used together.

This function is supported in Versions 2.70 and higher.

Example

```
int[] Axes = new int[3];
double[] Point = new double[2];
double[] Center = new double[2];
Axes[0] = Ch.ACSC_AXIS_0;
Axes[1] = Ch.ACSC_AXIS_1;
Axes[2] = -1;
Point[0] = 0;
Point[1] = 0;
Ch.ExtendedSegmentedMotionExt(Ch.ACSC_AMF_VELOCITY | Ch.ACSC_AMF_
CORNERRADIUS, // Velocity and corner radius flags are set, parameters now
required.
Axes, //Axes 0,1 active
Point, // Starting point
1000, // Velocity is set to 1000
Ch.ACSC_NONE, //EndVelocity is the default value
Ch.ACSC_NONE, //JunctionVelocity is the default value
```

```

Ch.ACSC_NONE, // Angle is the default value
Ch.ACSC_NONE, // CurveVELOCITY is the default value
Ch.ACSC_NONE, // Deviation
10, // Radius is set
Ch.ACSC_NONE, // Maximum corner length is default
Ch.ACSC_NONE, // Starvation margin is the default value
null); //Segments are set to NULL
    Center[0] = 500;
    Center[1] = 500;
Ch.ExtendedSegmentArc2(Ch.ACSC_AMF_VELOCITY, // Velocity is specified
Axes, // Axes 0 and 1
Center, //Center point
6.28319, //Positive movement angle
Ch.ACSC_NONE, // FinalPoints not used
5000, // Vector velocity
Ch.ACSC_NONE, // End velocity is not specified
Ch.ACSC_NONE, // Time is not specified
null, // Values array is not specified
null, // Variables array is not specified
Ch.ACSC_NONE, // Index is not specified
null // Masks array is not specified
);
Ch.EndSequenceM(Axes);

```

4.22.4 ExtendedSegmentArc2

Description

The function adds an arc segment to a segmented motion and specifies the coordinates of the center point and the rotation angle.

Syntax

object.ExtendedSegmentArc2 (long Flags, VARIANT Axes, VARIANT Center, double Angle, VARIANT FinalPoints, double Velocity, double EndVelocity, double Time, BSTR Values, BSTR Variables, int Index, BSTR Masks, ULONGLONG CallType, VARIANT* pWait);

Arguments

Flags	Bit-mapped argument that can include one or more of the flags defined in the flags table below.
Axes	Array of axis constants. Each element specifies one involved axis: ACSC_AXIS_0 corresponds to axis 0, ACSC_AXIS_1 to axis 1, etc. After the last axis, one additional element must be located that contains -1 which marks the end of the array. For axis constants see Axis Definitions

Center	Array of the center coordinates. The number and order of values must correspond to the Axes array. The Center must specify a value for each element of the Axes except the last -1 element.
Angle	Rotation angle in radians. Positive angle for counterclockwise rotation, negative for clockwise rotation.
FinalPoints	Array indicating the final points of the secondary axes, array size must be number of secondary axes (size of Axes - 2). Set this argument to NULL if not used.
Velocity	If ACSC_AMF_VELOCITY flag has been specified, this argument specifies a motion velocity for current segment. Set this argument to ACSC_NONE if not used.
EndVelocity	If ACSC_AMF_VARTIME flag has been specified, this argument defines the segment processing time in milliseconds, for the current segment only and has no effect on subsequent segments. Set this argument to ACSC_NONE if not used.
Time	If ACSC_AMF_VARTIME flag has been specified, this argument defines the segment processing time in milliseconds, for the current segment only and has no effect on subsequent segments. Set this argument to ACSC_NONE if not used.
Values	Pointer to the null-terminated character string that contains the name of a one-dimensional user-defined array of integer or real type with a size of 10 elements maximum. If ACSC_AMF_USERVARIABLES flag has been specified, this argument defines the values to be written to the Variables array at the beginning of the current segment execution. Set this argument to NULL if not used.
Variables	Pointer to the null-terminated character string that contains the name of a one-dimensional user-defined array of the same type and size as Values array. If ACSC_AMF_USERVARIABLES flag has been specified, this argument defines the user defined array, which will be written with Values data at the beginning of the current segment execution. Set this argument to NULL if not used.

Index	If ACSC_AMF_USERVARIABLES has not been specified, this argument defines the first element (starting from zero) of the Variables array, to which Values data will be written to. Set this argument to ACSC_NONE if not used.
Masks	Pointer to the null-terminated character string that contains the name of a one-dimensional user defined array of integer type and same size as the Values array. If ACSC_AMF_USERVARIABLES flag has been specified, this argument defines the masks that are applied to Values before the Values are written to variables array at the beginning of the current segment execution. The masks are only applied for integer values: $\text{variables}(n) = \text{values}(n) \text{ AND mask}(n)$. If Values is a real array, the masks argument should be NULL. Set this argument to NULL if not used.
ulonglong CallType	If CALLTYPE is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary ()function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to the GetString or GetBinary functions..

Flags

ACSC_AMF_VELOCITY	The motion will use velocity specified for each segment instead of the default velocity.
ACSC_AMF_ENDVELOCITY	This flag requires additional parameter that specifies end velocity. The controller decelerates to the specified velocity in the end of segment. The specified value should be less than the required velocity; otherwise the parameter is ignored. This flag affects only one segment.

ACSC_AMF_VARTIME	This flag requires an additional parameter that specifies the segment processing time in milliseconds. The segment processing time defines velocity at the current segment only and has no effect on subsequent segments. This flag also disables corner detection and processing at the end of segment. If this flag is not specified, deceleration is not required. However, in special cases the deceleration might occur due to corner processing or other velocity control conditions.
ACSC_AMF_USERVARIABLES	Synchronize user variables with segment execution. This flag requires additional parameters that specify values, user variable and mask. See details in Values, Variables, and Masks below.

Return Value

None

Comments

All axes specified in the **ExtendedSegmentedMotion** function. The number and order of the axes in the Axes array must correspond exactly to the number and order of the axes of the **SegmentedMotion** or **ExtendedSegmentedMotionExt** function.

The Point argument specifies the coordinates of the final point. The coordinates are absolute in the plane.

ACSC_AMF_VELOCITY and ACSC_AMF_VARTIME are mutually exclusive, meaning they cannot be used together.

This function is supported in Versions 2.70 and higher.

Example

```
int[] Axes = new int[3];
double[] Point = new double[2];
double[] Center = new double[2];
Axes[0] = Ch.ACSC_AXIS_0;
Axes[1] = Ch.ACSC_AXIS_1;
Axes[2] = -1;
Point[0] = 0;
Point[1] = 0;
Ch.ExtendedSegmentedMotionExt(Ch.ACSC_AMF_VELOCITY | Ch.ACSC_AMF_
CORNERRADIUS, // Velocity and corner radius flags are set, parameters now
required.
Axes, //Axes 0,1 active
Point, // Starting point
1000, // Velocity is set to 1000
Ch.ACSC_NONE, //EndVelocity is the default value
Ch.ACSC_NONE, //JunctionVelocity is the default value
Ch.ACSC_NONE, // Angle is the default value
```

```

Ch.ACSC_NONE, // CurveVelocity is the default value
Ch.ACSC_NONE, // Deviation
10, // Radius is set
Ch.ACSC_NONE, // Maximum corner length is default
Ch.ACSC_NONE, // Starvation margin is the default value
null); //Segments are set to NULL
    Center[0] = 500;
    Center[1] = 500;
Ch.ExtendedSegmentArc1(Ch.ACSC_AMF_VARTIME | Ch.ACSC_AMF_ENDVELOCITY,
// Velocity and end velocity is specified
Axes, // Axes 0 and 1
Center, //Center point
Point, // Final point
Ch.ACSC_CLOCKWISE, // CLOCKWISE rotation
Ch.ACSC_NONE, // Vector velocity is not specified
10, // End velocity
1000, // Time 1000 ms
null, // Values array is not specified
null, // Variables array is not specified
Ch.ACSC_NONE, // Index is not specified
null // Masks array is not specified
);
Ch.EndSequenceM(Axes);

```

4.22.5 *BlendedSegmentMotion*

Description

The function initiates a multi-axis blended segmented motion. Extended segmented motion provides new features.

Syntax

object.BlendedSegmentMotion(long Flags, VARIANT Axes, VARIANT Point, double SegmentTime, double AccelerationTime, double JerkTime, double DwellTime, ULONGLONG CallType, VARIANT *pWait)

Arguments

Flags	Bit-mapped argument that can include one or more of the flags defined in the flags table below.
Axes	Array of axis constants. Each element specifies one involved axis: ACSC_AXIS_0 corresponds to axis 0, ACSC_AXIS_1 to axis 1, etc. After the last axis, one additional element must be located that contains -1 which marks the end of the array. For axis constants see Axis Definitions

Position	Array of the starting coordinates. The number and order of values must correspond to the Axes array. The Center must specify a value for each element of the Axes except the last -1 element.
SegmentTime	This parameter will set the default initial segment time in milliseconds.
AccelerationTime	This parameter will set the default acceleration time in milliseconds.
JerkTime	This parameter will set the default Jerk time in milliseconds.
DwellTime	If ACSC_AMF_DWELLTIME is set, this parameter will set the initial dwell time between segments in milliseconds. If this argument is specified, no blending will be done for all segments of the motion. That means that the motion will be stopped at the end of each segment for the specified DwellTime milliseconds.
ulonglong CallType	If CALLTYPE is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString / GetBinary function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to the GetString or GetBinary functions..

Flags

ACSC_AMF_WAIT	Plan the motion but do not start it until the function acsc_GoM is executed.
ACSC_AMF_DWELLTIME	Dwell time between segments. This flag requires an additional parameter that specifies the dwell time, in milliseconds, at the final point of the segment. If this argument is specified, no blending will be done for all segments of the motion. That means that the motion will be stopped at the end of each segment for the specified DwellTime milliseconds.

Return Value

None

Comments

Blended segmented motion is a type of segmented motion that doesn't provide look-ahead capabilities, unlike extended segmented motion. Both type of motions are intended for processing a complex multi-axis trajectory and smoothing corners between segments, but do it in different ways. The extended segmented motion (XSEG) maximizes throughput within the defined axis limitations

and the defined accuracy. The blended segmented motion (BSEG) allows traversal of the trajectory with the defined timing constraints.

The function itself does not specify any movement, so the created motion starts only after the first segment is specified.

The segments of motion are specified by using **BlendedLine**, **BlendedArc1**, **BlendedArc2** functions that follow this function. The motion finishes when the **ENDSEQUENCEM** function is executed. If the call to **ENDSEQUENCEM** is omitted, the motion will stop at the last segment of the sequence and wait for the next segment. No transition to the next motion in the motion queue will occur until the function **ENDSEQUENCEM** is executed.

This function is supported in Versions 2.70 and higher.

Example

```
int[] Axes = new int[3];
    double[] Point = new double[2];
    double[] Center = new double[2];
    Axes[0] = Ch.ACSC_AXIS_0;
    Axes[1] = Ch.ACSC_AXIS_1;
    Axes[2] = -1;
    Point[0] = 0;
    Point[1] = 0;

    Ch.BlendedSegmentMotion(0, // No flags are set
    Axes, // Axes 0 and 1
    Point, // Starting point of motion
    1000, // Segment time
    100, // Segment Acceleration time
    50, // Segment jerk time
    Ch.ACSC_NONE // Segment Dwell time is default
    );
    Point[0] = 1000;
    Point[1] = -1000;
    Ch.BlendedLine(0, // Flags are not specified, default parameters
    // will be used
    Axes, // Axes 0 and 1
    Point, // Final point
    Ch.ACSC_NONE, // Segment time is not specified
    Ch.ACSC_NONE, // Acceleration time is not specified
    Ch.ACSC_NONE, // Jerk time is not specified
    Ch.ACSC_NONE // Dwell time is not specified
    );
    Ch.EndSequenceM(Axes);
```

4.22.6 *BlendedLine*

Description

The function adds a linear segment that starts at the current point and ends at the destination point of segmented motion.

Syntax

Int acsc_BlendedLine (long Flags, VARIANT Axes, VARIANT Point, double SegmentTime, double AccelerationTime, double JerkTime, double DwellTime, ULONGLONG CallType, VARIANT* pWait);

Arguments

Flags	Bit-mapped argument that can include one or more of the flags defined in the flags table below.
Axes	Array of axis constants. Each element specifies one involved axis: ACSC_AXIS_0 corresponds to axis 0, ACSC_AXIS_1 to axis 1, etc. After the last axis, one additional element must be located that contains -1 which marks the end of the array. For axis constants see Axis Definitions
Point	Array of the final point coordinates. The number and order of values must correspond to the Axes array. The Point must specify a value for each element of Axes except the last -1 element.
SegmentTime	If ACSC_AMF_BSEGTIME is set, this parameter will set the segment time, in milliseconds, for the current and all following segments – until the parameter is redefined.
AccelerationTime	If ACSC_AMF_BSEGACC is set, this parameter will set the Acceleration time, in milliseconds, for the current and all following segments – until the parameter is redefined.
JerkTime	If ACSC_AMF_BSEGJERK is set, this parameter will set the default Jerk time, in milliseconds, for the current and all following segments – until the parameter is redefined.
DwellTime	If ACSC_AMF_DWELLTIME is set, this parameter will set the dwell time between segments in milliseconds. If this argument is specified, no blending will be done for all segments of the motion. That means that the motion will be stopped at the end of each segment for the specified DwellTime milliseconds.
ulonglong CallType	If CALLTYPE is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary ()function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and

	cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to the GetString or GetBinary functions..

Flags

ACSC_AMF_BSEGTIME	This flag requires an additional parameter that defines the required segment time in milliseconds.
ACSC_AMF_BSEGACC	This flag requires an additional parameter that defines the required segment acceleration time in milliseconds.
ACSC_AMF_BSEGJERK	This flag requires an additional parameter that defines the required jerk time in milliseconds.
ACSC_AMF_DWELLTIME	This flag requires an additional parameter that specifies the dwell time, in milliseconds, at the final point of the segment.

Return Value

None

Comments

The function adds a linear segment that starts at the current point and ends at the destination point to segmented motion.

All axes specified in the Axes array must be specified in a previous call to the **BlendedSegmentMotion** function. The number and order of the axes in the Axes array must correspond exactly to the number and order of the axes of the call to the **BlendedSegmentMotion** function.

The **Point** argument specifies the coordinates of the final point. The coordinates are absolute in the plane. The function can wait for the controller response or can return immediately as specified by the **WAIT** argument.

This function is supported in Versions 2.70 and higher.

Example

```
int[] Axes = new int[3];
    double[] Point = new double[2];
    double[] Center = new double[2];
    Axes[0] = Ch.ACSC_AXIS_0;
    Axes[1] = Ch.ACSC_AXIS_1;
    Axes[2] = -1;
    Point[0] = 0;
    Point[1] = 0;

    Ch.BlendedSegmentMotion(0, // No flags are set
    Axes, // Axes 0 and 1
```



```

Point, // Starting point of motion
1000, // Segment time
100, // Segment Acceleration time
50, // Segment jerk time
Ch.ACSC_NONE // Segment Dwell time is default
);
Point[0] = 1000;
Point[1] = -1000;
Ch.BlendedLine(Ch.ACSC_AMF_BSEGTIME,
// BSEGTIME flag is set, BSEGTIME parameter is required.
Axes, // Axes 0 and 1
Point, // Final point
2000, // Segment time
Ch.ACSC_NONE, // Acceleration time is not specified
Ch.ACSC_NONE, // Jerk time is not specified
Ch.ACSC_NONE // Dwell time is not specified
);
Ch.EndSequenceM(Axes);

```

4.22.7 *BlendedArc1*

Description

The function adds to the motion path an arc segment that starts at the current point and ends at the destination point with the specified center point.

Syntax

object.BlendedArc1(long Flags, VARIANT Axes, VARIANT Center, VARIANT FinalPoint, int Rotation, double SegmentTime, double AccelerationTime, double JerkTime, double DwellTime, ULONGLONG CallType, VARIANT* pWait);

Arguments

Flags	Bit-mapped argument that can include one or more of the flags defined in the flags table below.
Axes	Array of axis constants. Each element specifies one involved axis: ACSC_AXIS_0 corresponds to axis 0, ACSC_AXIS_1 to axis 1, etc. After the last axis, one additional element must be located that contains -1 which marks the end of the array. For axis constants see Axis Definitions .
Center	Array of the center coordinates. The number and order of values must correspond to the Axes array. The Center must specify a value for each element of the Axes except the last -1 element .
FinalPoint	Array of the final point coordinates. The number and order of values must correspond to the Axes array. The Point must specify a value for each element of Axes except the last -1 element.

Rotation	This argument defines the direction of rotation. If Rotation is set to ACSC_COUNTERCLOCKWISE, then the rotation is counterclockwise. If Rotation is set to ACSC_CLOCKWISE, then rotation is clockwise.
SegmentTime	If ACSC_AMF_BSEGTIME is set, this parameter will set the segment time, in milliseconds, for the current and all following segments – until the parameter is redefined.
AccelerationTime	If ACSC_AMF_BSEGACC is set, this parameter will set the Acceleration time, in milliseconds, for the current and all following segments – until the parameter is redefined.
JerkTime	If ACSC_AMF_BSEGJERK is set, this parameter will set the default Jerk time, in milliseconds, for the current and all following segments – until the parameter is redefined.
DwellTime	If ACSC_AMF_DWELLTIME is set, this parameter will set the dwell time between segments in milliseconds. If this argument is specified, no blending will be done for all segments of the motion. That means that the motion will be stopped at the end of each segment for the specified DwellTime milliseconds.
ulonglong CallType	- If CALLTYPE is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary ()function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Flags

ACSC_AMF_BSEGTIME	This flag requires an additional parameter that defines the required segment time in milliseconds.
ACSC_AMF_BSEGACC	This flag requires an additional parameter that defines the required segment acceleration time in milliseconds.
ACSC_AMF_BSEGJERK	This flag requires an additional parameter that defines the required jerk time in milliseconds.
ACSC_AMF_DWELLTIME	This flag requires an additional parameter that specifies the dwell time, in milliseconds, at the final point of the segment.

Return Value

None

Comments

All axes specified in the **Axes** array must be specified in a previous call to the **BlendedSegmentMotion** function. The number and order of the axes in the **Axes** array must correspond exactly to the number and order of the axes of the call to the **BlendedSegmentMotion** function.

The **FinalPoint** argument specifies the coordinates of the final point. The coordinates are absolute in the plane. The function can wait for the controller response or can return immediately as specified by the **Wait** argument.

This function is supported in Versions 2.70 and higher.

Example

```
int[] Axes = new int[3];
    double[] Point = new double[2];
    double[] Center = new double[2];
    Axes[0] = Ch.ACSC_AXIS_0;
    Axes[1] = Ch.ACSC_AXIS_1;
    Axes[2] = -1;
    Point[0] = 0;
    Point[1] = 0;

    Ch.BlendedSegmentMotion(0, // No flags are set
    Axes, // Axes 0 and 1
    Point, // Starting point of motion
    1000, // Segment time
    100, // Segment Acceleration time
    50, // Segment jerk time
    Ch.ACSC_NONE // Segment Dwell time is default
    );
    Center[0] = 500;
    Center[1] = 500;
    Ch.BlendedArc1(Ch.ACSC_AMF_DWELLTIME,
    // DwellTime parameter is now required.
    Axes, // Axes 0 and 1
    Center, // Center point
    Point, // Final point
    1, // Positive rotation angle
    Ch.ACSC_NONE, // Segment time is not specified
    Ch.ACSC_NONE, // Acceleration time is not specified
    Ch.ACSC_NONE, // Jerk time is not specified.
    10 // Dwell time at the end of segment is set.
    );
    Ch.EndSequenceM(Axes);
```

4.22.8 *BlendedArc2*

Description

The function adds an arc segment to a segmented motion and specifies the coordinates of the center point and the rotation angle.

Syntax

object.BlendedArc2 (long Flags, VARIANT Axes, VARIANT Center, double Angle, double SegmentTime, double AccelerationTime, double JerkTime, double DwellTime, ULONGLONG CallType, VARIANT* pWait);

Arguments

Flags	Bit-mapped argument that can include one or more of the flags defined in the flags table below.
Axes	Array of axis constants. Each element specifies one involved axis: ACSC_AXIS_0 corresponds to axis 0, ACSC_AXIS_1 to axis 1, etc. After the last axis, one additional element must be located that contains -1 which marks the end of the array. For axis constants see Axis Definitions
Center	Array of the center coordinates. The number and order of values must correspond to the Axes array. The Center must specify a value for each element of the Axes except the last -1 element .
Angle	Rotation angle in radians. Positive angle for counterclockwise rotation, negative for clockwise rotation.
SegmentTime	If ACSC_AMF_BSEGTIME is set, this parameter will set the segment time, in milliseconds, for the current and all following segments – until the parameter is redefined.
AccelerationTime	If ACSC_AMF_BSEGACC is set, this parameter will set the Acceleration time, in milliseconds, for the current and all following segments – until the parameter is redefined.
JerkTime	– If ACSC_AMF_BSEGJERK is set, this parameter will set the default Jerk time, in milliseconds, for the current and all following segments – until the parameter is redefined.
DwellTime	If ACSC_AMF_DWELLTIME is set, this parameter will set the dwell time between segments in milliseconds. If this argument is specified, no blending will be done for all segments of the motion. That means that the motion will be stopped at the end of each segment for the specified DwellTime milliseconds.

ulonglong CallType	- If CALLTYPE is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary ()function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to the GetString or GetBinary functions.

Flags

ACSC_AMF_BSEGTIME	This flag requires an additional parameter that defines the required segment time in milliseconds.
ACSC_AMF_BSEGACC	This flag requires an additional parameter that defines the required segment acceleration time in milliseconds.
ACSC_AMF_BSEGJERK	This flag requires an additional parameter that defines the required jerk time in milliseconds.
ACSC_AMF_DWELLTIME	This flag requires an additional parameter that specifies the dwell time, in milliseconds, at the final point of the segment.

Return Value

None

Comments

All axes specified in the Axes array must be specified in a previous call to the **BlendedSegmentMotion** function. The number and order of the axes in the Axes array must correspond exactly to the number and order of the axes of the call to the **BlendedSegmentMotion** function.

The **Center** argument specifies the coordinates of the Center point. The coordinates are absolute in the plane.

The function can wait for the controller response or can return immediately as specified by the **Wait** argument.

This function is supported in Versions 2.70 and higher.

Example

```
int[] Axes = new int[3];
double[] Point = new double[2];
double[] Center = new double[2];
Axes[0] = Ch.ACSC_AXIS_0;
Axes[1] = Ch.ACSC_AXIS_1;
```

```

        Axes[2] = -1;
        Point[0] = 0;
        Point[1] = 0;

    Ch.BlendedSegmentMotion(0, // No flags are set
    Axes, // Axes 0 and 1
    Point, // Starting point of motion
    1000, // Segment time
    100, // Segment Acceleration time
    50, // Segment jerk time
    Ch.ACSC_NONE // Segment Dwell time is default
    );
    Center[0] = 500;
    Center[1] = 500;
    Ch.BlendedArc2(Ch.ACSC_AMF_DWELLTIME,
    // DwellTime parameter is now required.
    Axes, // Axes 0 and 1
    Center, // Center point
    -3.14, // Rotation angle, counter clockwise
    Ch.ACSC_NONE, // Segment time is not specified
    Ch.ACSC_NONE, // Acceleration time is not specified
    Ch.ACSC_NONE, // Jerk time is not specified.
    10 // Dwell time at the end of segment is set.
    );
    Ch.EndSequenceM(Axes);

```

4.22.9 Stopper

Description

The method provides a smooth transition between two segments of segmented motion.

Syntax

object.Stopper(variant Axes)

Arguments

Axes

Array of axis constants. Each element specifies one involved axis: ACSC_AXIS_0 corresponds to the 0axis, ACSC_AXIS_1 to the 1axis, etc. After the last axis, one additional element must be included that contains -1 and marks the end of the array. For the axis constants see [Axis Definitions](#).

Return Value

None.

Remarks

The controller builds the motion so that the vector velocity follows a smooth velocity diagram. The segments define the projection of the vector velocity to axis velocities. If all segments are connected smoothly, axis velocity is also smooth. However, if the user defined a path with an

inflection point, axis velocity has a jump in this point. The jump can cause a motion failure due to the acceleration limit.

The method is used to avoid velocity jump in the inflection points. If the method is specified between two segments, the controller provides smooth deceleration to zero in the end of first segment and smooth acceleration to specified velocity in the beginning of second segment.

If the method fails, the Error object is filled with the error description.

Example

```
Sub Stopper_Sample()
Dim Points(1) As Double
Dim Axes(2)
Axes(0) = Ch.ACSC_AXIS_0
Axes(1) = Ch.ACSC_AXIS_1
Axes(2) = -1
On Error GoTo except

                                'Create segmented motion, coordinates of
                                'the initial point are(1000,1000)
Points(0) = 1000
    Points(1) = 1000

                                'Enable axes 0 and 1
Call Ch.EnableM(Axes)
Call Ch.Segment(0, Axes, Points)

                                'Add line segment with final point
                                '(1000, -1000), vector velocity 25000
    Points(0) = 1000
    Points(1) = -1000
    Call Ch.ExtLine(Axes, Points, 25000)
    Call Ch.Stopper(Axes)

                                'Add line segment with final point (-1000,
                                '-1000), vector velocity 25000
    Points(0) = -1000
    Points(1) = -1000
    Call Ch.ExtLine(Axes, Points, 25000)
    Call Ch.Stopper(Axes)

                                'Add line segment with final point (-1000,
                                '1000), vector velocity 25000
    Points(0) = -1000
    Points(1) = 1000
    Call Ch.ExtLine(Axes, Points, 25000)
    Call Ch.Stopper(Axes)

                                'Add line segment with final point (1000,
                                '1000), vector velocity 25000
    Points(0) = 1000
    Points(1) = 1000
    Call Ch.ExtLine(Axes, Points, 25000)

                                'Finish the motion
                                'End of the multi-point motion
    Call Ch.EndSequenceM(Axes)
Exit Sub
except
```

```

except:
    MyErrorMsg      'See Error Handling in Visual Basic
End Sub
Call Ch.EnableM(Axes)
Call Ch.Segment(0, Axes, Points)
                                'Add line segment with final point (1000,
                                '-1000), vector velocity 25000

    Points(0) = 1000
    Points(1) = -1000
    Call Ch.ExtLine(Axes, Points, 25000)
    Call Ch.Stopper(Axes)
                                'Add line segment with final point (-1000,
                                '-1000), vector velocity 25000

    Points(0) = -1000
    Points(1) = -1000
    Call Ch.ExtLine(Axes, Points, 25000)
    Call Ch.Stopper(Axes)
                                'Add line segment with final point (-1000,
                                '1000), vector velocity 25000

    Points(0) = -1000
    Points(1) = 1000
    Call Ch.ExtLine(Axes, Points, 25000)
    Call Ch.Stopper(Axes)
                                'Add line segment with final point (1000,
                                '1000), vector velocity 25000

    Points(0) = 1000
    Points(1) = 1000
    Call Ch.ExtLine(Axes, Points, 25000)
                                'Finish the motion
                                'End of the multi-point motion

    Call Ch.EndSequenceM(Axes)
Exit Sub
except:
    MyErrorMsg      'See Error Handling in Visual Basic
End Sub

```

4.22.10 Projection

Description

The method sets a projection matrix for segmented motion.

Syntax

object.Projection(variant Axes, string Matrix)

Arguments

Axes	Array of axis constants. Each element specifies one involved axis: ACSC_AXIS_0 corresponds to the 0axis, ACSC_AXIS_1 to the 1axis, etc. After the last axis, one additional element must be included that contains -1 and marks the end of the array. For the axis constants see Axis Definitions .
Matrix	Pointer to the null-terminated string containing the name of the matrix that provides the specified projection.

Return Value

None.

Remarks

The method sets a projection matrix for segmented motion.

The projection matrix connects the plane coordinates and the axis values in the axis group. The projection can provide any transformation as rotation or scaling. The number of the matrix rows must be equal to the number of the specified axes. The number of the matrix columns must equal two.

The matrix must be declared before as a global variable by an ACSPL+ program or by the [DeclareVariable](#) method and must be initialized by an ACSPL+ program or by the method.

For more information about projection, see the *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```
Sub Projection_Sample()
Dim Axes(3)
Dim Point(2) As Double
Dim Center(2) As Double
Dim Matrix(0 To 2, 0 To 1) As Double
                                'Prepare the projection matrix

Matrix(0, 0) = 1
Matrix(0, 1) = 0
Matrix(1, 0) = 0
Matrix(1, 1) = 1.41421
Matrix(2, 0) = 0
Matrix(2, 1) = 1.41421
Axes(0) = Ch.ACSC_AXIS_0
Axes(1) = Ch.ACSC_AXIS_1
Axes(2) = Ch.ACSC_AXIS_2
Axes(3) = -1
Point(0) = 10000
Point(1) = 10000
On Error GoTo except
                                'Declare the matrix that will contain the
                                'projection
```

```

Call Ch.DeclareVariable(Ch.ACSC_REAL_TYPE, "ProjectionMatrix(3)(2)")
'Initialize the projection matrix
Call Ch.WriteVariable(Matrix, "ProjectionMatrix", Ch.ACSC_NONE)
Call Ch.EnableM(Axes)
'Create a group of the involved axes
Call Ch.Group(Axes)
'Create segmented motion, coordinates of
'the initial point are(10000,10000)

Axes(0) = Ch.ACSC_AXIS_0
Axes(1) = Ch.ACSC_AXIS_1
Axes(2) = -1
Call Ch.Segment(0, Axes, Point)
'Incline the working plane XY by 45°

Axes(0) = Ch.ACSC_AXIS_0
Axes(1) = Ch.ACSC_AXIS_1
Axes(2) = Ch.ACSC_AXIS_2
Axes(3) = -1
Call Ch.Projection(Axes, "ProjectionMatrix")
'Describe circle with center (1000, 0),
'clockwise rotation
'Although the circle was defined, really
'on the plane XY we will get the Ellipse
'stretched along the Y axis

Axes(0) = Ch.ACSC_AXIS_0
Axes(1) = Ch.ACSC_AXIS_1
Axes(2) = -1
Center(0) = 1000
Center(1) = 0
Call Ch.Arc2(Axes, Center, -2 * 3.141529)
'Finish the motion
'End of the multi-point motion

Call Ch.EndSequenceM(Axes)
Exit Sub
except:
MyErrorMsg 'See Error Handling in Visual Basic
End Sub

```

4.23 Points and Segments Manipulation Methods

The Points and Segments Manipulation methods are:

Table 4-26. Points and Segments Manipulation Methods

AddPoint	Adds a point to a single-axis multi-point or spline motion.
AddPointM	Adds a point to a multi-axis multi-point or spline motion.
ExtAddPoint	Adds a point to a single-axis multi-point or spline motion and specifies a specific velocity or motion time.

ExtAddPointM	Adds a point to a multi-axis multi-point or spline motion and specifies a specific velocity or motion time.
EndSequence	Informs the controller that no more points will be specified for the current single-axis motion.
EndSequenceM	Informs the controller that no more points or segments will be specified for the current multi-axis motion.

4.23.1 AddPoint

Description

The method adds a point to a single axis multi-point or spline motion.

Syntax

object.AddPoint(long Axis, double Point)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Point	Coordinate of the added point.

Return Value

None.

Remarks

The method adds a point to a single-axis multi-point or spline motion. To add a point to a multi-axis motion, use [AddPVPPointM](#). To add a point with a specified non-default velocity or time interval use [ExtAddPoint](#) or [ExtAddPointM](#).

The method waits for the controller response.

The controller response indicates that the command was accepted and the point is added to the motion buffer. The point can be rejected if the motion buffer is full. In this case, you can call this method periodically until the method returns a non-zero value.

If the method fails, the Error object is filled with the error description.

Example

```
Sub AddPoint_Sample()
Dim Str(1) As String
    On Error GoTo except          'Enable axis 0
    Call Ch.Enable(Ch.ACSC_AXIS_0)
                                'Wait till axis 0 enabled during 5 secCall Ch.WaitMotion
    (Ch.ACSC_AXIS_0, 1, 5000)
                                'Create multi-point motion with
                                'default velocity and dwell 1 ms
    Call Ch.MultiPoint(0, Ch.ACSC_AXIS_0, 1)
```

```

        For Index = 0 To 5                                'Add points to axis 0
        Call Ch.AddPoint(Ch.ACSC_AXIS_0, 100 * Index)
        Next Index                                       'Finish the motion
                                                'End of the multi-point motion
        Call Ch.EndSequence(Ch.ACSC_AXIS_0)
Exit Sub
except:
    MyErrorMsg                                           'See Error Handling in Visual Basic
End Sub

```

4.23.2 AddPointM

Description

The method adds a point to a multi-axis multi-point or spline motion.

Syntax

object.AddPointM(variant Axes, variant Point)

Arguments

Axes	Array of axis constants. Each element specifies one involved axis: ACSC_AXIS_0 corresponds to the 0axis, ACSC_AXIS_1 to the 1axis, etc. After the last axis, one additional element must be included that contains -1 and marks the end of the array. For the axis constants see Axis Definitions .
Point	Array of the coordinates of added point. The number and order of values must correspond to the Axes array. Point must specify a value for each element of Axes except the last -1 element.

Return Value

None.

Remarks

The method adds a point to a multi-axis multi-point or spline motion. To add a point to a single-axis motion, use [AddPoint](#). To add a point with a specified non-default velocity or time interval use [ExtAddPoint](#) or [ExtAddPointM](#).

The method waits for the controller response.

The controller response indicates that the command was accepted and the point is added to the motion buffer. The point can be rejected if the motion buffer is full. In this case, you can call this method periodically until the method returns non-zero value.

All axes specified in the **Axes** array must be specified before the call of the [MultiPointM](#) or [SplineM](#) method. The number and order of the axes in the **Axes** array must correspond exactly to the number and order of the axes of [MultiPointM](#) or [SplineM](#) methods.

If the method fails, the Error object is filled with the error description.

Example

```

Sub AddPointM_Sample()
Dim Index As Long
Dim Points(1) As Double
Dim Axes(2) As String
Axes(0) = Ch.ACSC_AXIS_0
Axes(1) = Ch.ACSC_AXIS_1
Axes(2) = -1
On Error GoTo except                                'Enable axes 0 and 1
    Call Ch.EnableM(Axes)                            'Create multi-point motion with
                                                    'default velocity without dwell in
                                                    'the points
    Call Ch.MultiPointM(0, Axes, 0)
                                                    'Add some points to axes 0 and 1
        For Index = 0 To 5
            Points(0) = 100 * Index
            Points(1) = 100 * Index
        Call Ch.AddPointM(Axes, Points)
        Next Index                                'Finish the motion
                                                    'End of the multi-point motion
    Call Ch.EndSequenceM(Axes)
Exit Sub
except:
MyErrorMsg                                'See Error Handling in Visual Basic
End Sub

```

4.23.3 ExtAddPoint**Description**

The method adds a point to a single-axis multi-point or spline motion and specifies a specific velocity or motion time.

Syntax

object.ExtAddPoint(long Axis, double Point, double Rate)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Point	Coordinate of the added point.
Rate	If the motion was activated by the MultiPoint method with the ACSC_AMF_VELOCITY flag, this parameter defines the motion velocity. If the motion was activated by the Spline method with the ACSC_AMF_VARTIME flag, this parameter defines the time interval between the previous point and the present one.

Return Value

None.

Remarks

The method adds a point to a single-axis multi-point motion with specific velocity or to single-axis spline motion with a non-uniform time.

To add a point to a multi-axis motion, use [ExtAddPointM](#). To add a point to a motion with default velocity or uniform time interval, the [AddPoint](#) and [AddPointM](#) methods are more convenient.

The method waits for the controller response.

The controller response indicates that the command was accepted and the point is added to the motion buffer. The point can be rejected if the motion buffer is full. In this case, you can call this method periodically until the method returns a non-zero value.

If the method fails, the Error object is filled with the error description.

Example

```
Sub ExtAddPoint_Sample()
On Error GoTo except
    'Enable axis 0
    Call Ch.Enable(Ch.ACSC_AXIS_0)
    'Wait till axis 0 is enabled during 5 sec
    Call Ch.WaitMotorEnabled(Ch.ACSC_AXIS_0, 1, 5000)
    'Create multi-point motion uses the
    'velocity specified with each point with
    'dwell 1 ms
    Call Ch.MultiPoint(Ch.ACSC_AMF_VELOCITY, Ch.ACSC_AXIS_0, 1)
    'Add some points
    For Index = 0 To 5
        'Adds points to axis 0, defines a motion
        'velocity of 5000
        Call Ch.ExtAddPoint(Ch.ACSC_AXIS_0, 100 * Index, 5000)
    Next Index
    'Finish the motion
    'End of the multi-point motion
    Call Ch.EndSequence(Ch.ACSC_AXIS_0)
Exit Sub
except:
    MyErrorMsg    'See Error Handling in Visual Basic
End Sub
```

4.23.4 ExtAddPointM**Description**

The method adds a point to a multi-axis multi-point or spline motion and specifies a specific velocity or motion time.

Syntax

object.ExtAddPointM(variant Axes, variant Point, double Rate)

Arguments

Axes	Array of axis constants. Each element specifies one involved axis: ACSC_AXIS_0 corresponds to the 0axis, ACSC_AXIS_1 to the 1axis, etc. After the last axis, one additional element must be included that contains -1 and marks the end of the array. For the axis constants see Axis Definitions .
Point	Array of the coordinates of added point. The number and order of values must correspond to the Axes array. The Point must specify a value for each element of Axes except the last -1 element.
Rate	If the motion was activated by the MultiPoint method with the ACSC_AMF_VELOCITY flag, this parameter defines as motion velocity. If the motion was activated by the Spline method with the ACSC_AMF_VARTIME flag, this parameter defines as time interval between the previous point and the present one.

Return Value

None.

Remarks

The method adds a point to a multi-axis multi-point or spline motion. To add a point to a single-axis motion, use [ExtAddPoint](#). To add a point with to a motion with default velocity or uniform time interval the [ExtAddPoint](#) and [ExtAddPointM](#) methods are more convenient.

The method waits for the controller response.

The controller response indicates that the command was accepted and the point is added to the motion buffer. The point can be rejected if the motion buffer is full. In this case, you can call this method periodically until the method returns a non-zero value.

All axes specified in the **Axes** array must be specified before calling the [MultiPointM](#) or [SplineM](#) methods. The number and order of the axes in the **Axes** array must correspond exactly to the number and order of the axes of **MultiPointM** or **SplineM** methods.

If the method fails, the Error object is filled with the error description.

Example

```
Sub ExtAddPointM_Sample()
Dim Index As Long
Dim Points(1)
Dim Axes(2)
Axes(0) = Ch.ACSC_AXIS_0
Axes(1) = Ch.ACSC_AXIS_1
Axes(2) = -1
On Error GoTo except
'Enable axes 0 and 1
```

```

    Call Ch.EnableM(Axes)
                                'Create multi-point motion uses the
                                'velocity specified with each point without
                                'dwell in the points
    Call Ch.MultiPointM(Ch.ACSC_AMF_VELOCITY, Axes, 0)
                                'Add some points
    For Index = 0 To 5
        Points(0) = 100 * Index
        Points(1) = 100 * Index
                                'Adds points to axes 0 and 1, defines a
                                'motion velocity of 5000
    Call Ch.ExtAddPointM(Axes, Points, 5000)
    Next Index
                                'Finish the motion
                                'End of the multi-point motion
    Call Ch.EndSequenceM(Axes)
Exit Sub
except:
MyErrorMsg                                'See Error Handling in Visual Basic
End Sub

```

4.23.5 EndSequence

Description

The method informs the controller that no more points will be specified for the current single-axis motion.

Syntax

object.EndSequence(long Axis)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
------	---

Return Value

None.

Remarks

The motion finishes when the **EndSequence** method is executed. If the call of **EndSequence** is omitted, the motion will stop at the last point of the sequence and wait for the next point. No transition to the next motion in the motion queue will occur until the **EndSequence** method executes.

The method waits for the controller response.

This method applies to the single-axis multi-point or spline (arbitrary path) motions.

If the method fails, the Error object is filled with the error description.

Example

```

Sub EndSequence_Sample()
Dim Index As Long
On Error GoTo except

        'Enable axis 0
    Call Ch.Enable(Ch.ACSC_AXIS_0)
        'Wait till axis 0 is enabled during 5 sec
    Call Ch.WaitMotorEnabled(Ch.ACSC_AXIS_0, 1, 5000)
        'Create multi-point motion with default
        'velocity and dwell 1 ms
    Call Ch.MultiPoint(0, Ch.ACSC_AXIS_0, 1)
        'Add some points

    For Index = 0 To 5
    Call Ch.AddPoint(Ch.ACSC_AXIS_0, 100 * Index)
    Next Index

        'Finish the motion
        'End of the multi-point motion of axis 0
    Call Ch.EndSequence(Ch.ACSC_AXIS_0)
Exit Sub
except:
    MyErrorMsg        'See Error Handling in Visual Basic
End Sub

```

4.23.6 EndSequenceM**Description**

The method informs the controller that no more points or segments will be specified for the current multi-axis motion.

Syntax

object.EndSequence(variant Axes)

Arguments

Axes	Array of axis constants. Each element specifies one involved axis: ACSC_AXIS_0 corresponds to the 0axis, ACSC_AXIS_1 to the 1axis, etc. After the last axis, one additional element must be included that contains -1 and marks the end of the array. For the axis constants see Axis Definitions.
-------------	--

Return Value

None.

Remarks

The motion finishes when the **EndSequenceM** method is executed. If the call of **EndSequenceM** is omitted, the motion will stop at the last point or segment of the sequence and wait for the next point. No transition to the next motion in the motion queue will occur until the **EndSequenceM** method executes.

The method waits for the controller response.

This method applies to the multi-axis multi-point, spline (arbitrary path) and segmented motions.

If the method fails, the Error object is filled with the error description.

Example

```
Sub EndSequenceM_Sample()
    Dim Index As Long
    Dim Points(1)
    Dim Axes(2)
    Axes(0) = Ch.ACSC_AXIS_0
    Axes(1) = Ch.ACSC_AXIS_1
    Axes(2) = -1
    On Error GoTo except
        'Enable axes 0 and 1
        Call Ch.EnableM(Axes)
        'Create multi-point motion with default
        'velocity without dwell in the points
        Call Ch.MultiPointM(0, Axes, 0)
        'Add some points
        For Index = 0 To 5
            Points(0) = 100 * Index
            Points(1) = 100 * Index
            Call Ch.AddPointM(Axes, Points)
        Next Index
        'Finish the motion
        'End of the multi-point motion of axes 0
        'and 1
        Call Ch.EndSequenceM(Axes)
    Exit Sub
except:
    MyErrorMsg 'See Error Handling in Visual Basic
End Sub
```

4.24 Data Collection Methods

The Data Collection methods are:

Table 4-27. Data Collection Methods

DataCollectionExt	Initiates data collection.
StopCollect	Terminates data collection.

4.24.1 DataCollectionExt

Description

The function initiates data collection.

Syntax

object.DataCollectionExt (long Flags, [long Axis,] string Array, long Nsample, double Period, string vars, ULONGLONG CallType, VARIANT *pWait)

Arguments

Flags	Bit-mapped argument that can include one or more of the flags defined in the flags table below.
Axis	Axis constant of the axis to which the data collection must be synchronized: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For axis constants see Axis Definitions . This argument is required only for axis data collection (ACSC_DCF_SYNC flag).
Array	Name of the array that stores the collected samples. The array must be declared as a global variable by an ACSPL+ program or by the DECLAREVARIABLE method.
Nsample	Number of samples to be collected.
Period	Sampling period in milliseconds. If the ACSC_DCF_TEMPORAL flag is specified, this parameter defines a minimal period.
Vars	The string contains chained names of the variables, separated by '\r'(13) character. The values of these variables will be collected in the Array. If variable name specifies an array, the name must be supplemented with indexes in order to specify one element of the array.
ulonglong CallType	- If CALLTYPE is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString or GetBinary to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Flags

ACSC_DCF_SYNC	Start data collection synchronously with motion.
ACSC_DCF_WAIT	Create the synchronous data collection, but do not start until the Go method is called. This flag can only be used with the ACSC_DCF_SYNC flag.

ACSC_DCF_TEMPORAL	Temporal data collection. The sampling period is calculated automatically according to the collection time.
ACSC_DCF_CYCLIC	Cyclic data collection uses the collection array as a cyclic buffer and continues infinitely. When the array is full, each new sample overwrites the oldest sample in the array.

Return Value

None

Comments

Data collection started by this method without the ACSC_DCF_SYNC flag is called system data collection.

Data collection started with the ACSC_DCF_SYNC flag, is called axis data collection.

Data collection started with ACSC_DCF_CYCLIC flag, called cyclic data collection. Unlike the standard data collection that finishes when the collection array is full, cyclic data collection does not self-terminate. Cyclic data collection uses the collection array as a cyclic buffer and can continue to collect data indefinitely. When the array is full, each new sample overwrites the oldest sample in the array. Cyclic data collection can only be terminated by calling the **StopCollect** method. The array that stores the samples can be one or two-dimensional. A one-dimensional array is allowed only if the variable list contains one variable name.

The number of the array rows must be equal to or more than the number of variables in the variable list. The number of the array columns must be equal to or more than the number of samples specified by the NSample argument.

If the method fails, the Error object is filled with the error description.

This function is supported in Versions 2.70 and higher.

Example

```
try
{
channel.DataCollectionExt (Flags, Axis, "MyArrayForDC", 100, 100, "FPOS
(0)\rFVEL(0)\rFACC(0)");
}
catch (Exception ex)
{
ShowException(ex);
}
```

4.24.2 StopCollect**Description**

The method terminates data collection.

Syntax

object.StopCollect()

Arguments

None.

Return Value

None.

Remarks

The usual system data collection finishes when the required number of samples is collected or the **StopCollect** method is executed. The application can wait for data collection end with the method.

The temporal data collection runs until the **StopCollect** method is executed.

The method terminates the data collection prematurely. The application can determine the number of actually collected samples from the **S_DCN** variable and the actual sampling period from the **S_DCP** variable.

If the method fails, the Error object is filled with the error description.

Example

```
Sub StopCollect_Sample()
    'Matrix consisting of two rows with 1000
    'columns each

    Dim ArrayName As String
    ArrayName = "DCA(2)(1000)"

    'Positions of axes 0 and 1 will be
    'collected

    Dim Vars As String
    Vars = "FPOS(0)" & Chr$(13) & FPOS(1)
    On Error GoTo except

    'Enable axis 0
    Call Ch.Enable(Ch.ACSC_AXIS_0)
    'Wait till axis 0 enabled during 5 sec
    Call Ch.WaitMotorEnabled(Ch.ACSC_AXIS_0, 1, 5000)
    Call Ch.ExtToPoint(Ch.ACSC_AMF_VELOCITY Or Ch.ACSC_AMF_ENDVELOCITY,
    Ch.ACSC_AXIS_0, 10000, 5000, 1000)

    'Array to collect
    Call Ch.DeclareVariable(Ch.ACSC_REAL_TYPE, ArrayName)
    Call Ch.CollectB(0, ArrayName, 1000, 1, Vars)
    'Stop to collect

    Call Ch.StopCollect

    'End of the multi-point motion
    Call Ch.EndSequence(Ch.ACSC_AXIS_0)
Exit Sub
except:
    MyErrorMsg      'See Error Handling in Visual Basic
End Sub
```

4.25 Status Report Methods

The Status Report methods are:

Table 4-28. Status Report Methods

GetMotorState	Retrieves the current motor state.
GetAxisState	Retrieves the current axis state.
GetIndexState	Retrieves the current state of the index and mark variables.
ResetIndexState	Resets the specified bit of the index/mark state.
GetProgramState	Retrieves the current state of the program buffer.

4.25.1 *GetMotorState*

Description

The method retrieves the current motor state.

Syntax

object.GetMotorState(long Axis)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
-------------	---

Return Value

Long.

The method retrieves the current motor state.

Remarks

The retrieved value can include one or more of the following flags:

- > ACSC_MST_ENABLE - a motor is enabled
- > ACSC_MST_INPOS - a motor has reached a target position
- > ACSC_MST_MOVE - a motor is moving
- > ACSC_MST_ACC - a motor is accelerating

If the method fails, the Error object is filled with the error description.

Example

```

Sub GetMotorState_Sample()
Dim State As Long
On Error GoTo except
    'Retrieves the current motor state of 0
    State = Ch.GetMotorState(Ch.ACSC_AXIS_0)
Exit Sub
except:
    MyErrorMsg 'See Error Handling in Visual Basic
End Sub

```

4.25.2 GetAxisState

Description

The method retrieves the current axis state.

Syntax

object.GetAxisState(long Axis)

Arguments

Axis

Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see [Axis Definitions](#).

Return Value

Long.

The method retrieves the current axis state.

Remarks

The return value can comprise one or more flags. For example, **ACSC_AST_LEAD** (leading axis), **ACSC_AST_DC** (data collection in progress for the axis), etc. For the complete list of flags, see [Axis State Flags](#), [Axis State Flags](#).

If the method fails, the Error object is filled with the error description.

Example

```
Sub GetAxisState_Sample()
    Dim State As Long
    On Error GoTo except
        'Retrieves the current axis state
        State = Ch.GetAxisState(Ch.ACSC_AXIS_0)
    Exit Sub
except:
    MyErrorMsg 'See Error Handling in Visual Basic
End Sub
```

4.25.3 GetIndexState

Description

The method retrieves the current set of bits that indicate the index and mark state.

Syntax

object.GetIndexState(long Axis)

Arguments

Axis

Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see [Axis Definitions](#).

Return Value

Long.

The method retrieves the current set of bits that indicate the index and mark state.

Remarks

The return value can include one or more of the following flags:

- > **ACSC_IST_IND** – a primary encoder index of the specified axis is latched
- > **ACSC_IST_IND2** – a secondary encoder index of the specified axis is latched
- > **ACSC_IST_MARK** – a MARK1 signal has been generated and position of the specified axis was latched
- > **ACSC_IST_MARK2** – a MARK2 signal has been generated and position of the specified axis was latched

The controller processes index/mark signals as follows:

When an index/mark signal is encountered for the first time, the controller latches feedback positions and raises the corresponding bit. As long as a bit is raised, the controller does not latch feedback position even if the signal occurs again. To resume latching logic, the application must call [ResetIndexState](#) to explicitly reset the corresponding bit.

If the method fails, the Error object is filled with the error description.

Example

```
Sub GetIndexState_Sample()
    'Retrieves the current set of bits that
    'indicate the index and mark state

    Dim IState As Long
    On Error GoTo except
        IState = Ch.GetIndexState(Ch.ACSC_AXIS_0)
    Exit Sub
except:
    MyErrorMsg 'See Error Handling in Visual Basic
End Sub
```

4.25.4 ResetIndexState**Description**

The method resets the specified bit of the index/mark state.

Syntax

object.ResetIndexState(long Axis, long Mask)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Mask	The parameter contains bit to be cleared. Only one of the following flags can be specified: ACSC_IST_IND – a primary encoder index of the specified axis is latched ACSC_IST_IND2 – a secondary encoder index of the specified axis is latched ACSC_IST_MARK – a MARK1 signal has been generated and position of the specified axis was latched ACSC_IST_MARK2 – a MARK2 signal has been generated and position of the specified axis was latched

Return Value

None.

Remarks

The method resets the specified bit of the index/mark state. **Mask** contains a bit that must be cleared, i.e., the method resets only that bit of the index/mark state, which corresponds to non-zero bit of **Mask**. To get the current index/mark state, use the [GetIndexState](#) method.

If the method fails, the Error object is filled with the error description.

Example

```
Sub ResetIndexState_Sample()
Dim IState As Long
On Error GoTo except

'Resets the specified bit of the
'index/mark state
Call Ch.ResetIndexState(Ch.ACSC_AXIS_0, Ch.ACSC_IST_IND)
Call Ch.ResetIndexState(Ch.ACSC_AXIS_0, Ch.ACSC_IST_IND2)
Call Ch.ResetIndexState(Ch.ACSC_AXIS_0, Ch.ACSC_IST_MARK)
Call Ch.ResetIndexState(Ch.ACSC_AXIS_0, Ch.ACSC_IST_MARK2)
IState = Ch.GetIndexState(Ch.ACSC_AXIS_0)
Exit Sub
except:
MyErrorMsg 'See Error Handling in Visual Basic
End Sub
```

4.25.5 GetProgramState**Description**

The method retrieves the current state of the program buffer.

Syntax

object.GetProgramState(long Buffer)

Arguments

Buffer

Number of the buffer in which the program resides.

Return Value

Long.

The method retrieves the current state of the program buffer.

Remarks

The retrieved value can include one or more of the following flags:

- > **ACSC_PST_COMPILED** – a program in the specified buffer is compiled
- > **ACSC_PST_RUN** – a program in the specified buffer is running
- > **ACSC_PST_AUTO** – an auto routine in the specified buffer is running
- > **ACSC_PST_DEBUG** – a program in the specified buffer is executed in debug mode, i.e. breakpoints are active
- > **ACSC_PST_SUSPEND** – a program in the specified buffer is suspended after the step execution or due to breakpoint in debug mode

If the method fails, the Error object is filled with the error description.

Example

```
Sub GetProgramState_Sample()
Dim PState As Long
On Error GoTo except
    'Appends 1 line to buffer 0
    Call Ch.AppendBuffer(0, "!Empty buffer;stop")
    'Run buffer 0
    Call Ch.RunBuffer(0)
    'Retrieves the current state of the
    'program buffer
    PState = Ch.GetProgramState(0)
Exit Sub
except:
    MyErrorMsg 'See Error Handling in Visual Basic
End Sub
```

4.26 Inputs/Outputs Access Methods

The Inputs/Outputs Access methods are:

Table 4-29. Inputs/Outputs Access Methods

GetInput	Retrieves the current state of the specified digital input.
GetInputPort	Retrieves the current state of the specified digital input port.
GetOutput	Retrieves the current state of the specified digital output.
GetOutputPort	Retrieves the current state of the specified digital output port.
SetOutput	Sets the specified digital output to the specified value.
SetOutputPort	Sets the specified digital output port to the specified value.
GetAnalogInputNT	Retrieves the current value of the specified analog input.
GetAnalogOutputNT	Retrieves the current value of the specified analog output.
SetAnalogOutputNT	Writes the specified value to the specified analog output.
GetExtInput	Retrieves the current state of the specified extended input.
GetExtInputPort	Retrieves the current state of the specified extended input port.
GetExtOutput	Retrieves the current state of the specified extended output.
GetExtOutputPort	Retrieves the current state of the specified extended output port.
SetExtOutput	Sets the specified extended output to the specified value.
SetExtOutputPort	Sets the specified extended output port to the specified value.

4.26.1 *GetInput*

Description

The method retrieves the current state of the specified digital input.

Syntax

object.GetInput(long Port, long Bit)

Arguments

Port	Number of the input port.
Bit	Number of the specific bit.

Return Value

Long.

The method retrieves the current state of the specified digital input.

Remarks

To get values of all inputs of the specific port, use the [GetInputPort](#) method.

Digital inputs are represented in the controller variable **IN**. For more information about digital inputs, see the *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```
Sub GetInput_Sample()
    'The method reads input 0 of port 0 ( IN(0).0 )
    Dim Port As Long
    On Error GoTo except
        Port = Ch.GetInput(0, 0)
    Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.26.2 GetInputPort

Description

The method retrieves the current state of the specified digital input port.

Syntax

object.GetInputPort(long Port)

Arguments

Port	Number of the input port.
------	---------------------------

Return Value

Long.

The method retrieves the current state of the specified digital input port.

Remarks

To get the value of the specific input of the specific port, use the [GetInput](#) method.

Digital inputs are represented in the controller variable **IN**. For more information about digital inputs, see the *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```
Sub GetInputPort_Sample()
    'The method reads input port 0 to 15( IN(0) to IN(15) )
    Dim Index As Long
    Dim Port(15) As Long
    On Error GoTo except
        For Index = 0 To 15
            Port(Index) = Ch.GetInputPort(Index)
        Next Index
    Exit Sub
except:
```

```

except:
    MyErrorMsg                                     'See Error Handling in Visual Basic
End Sub

```

4.26.3 GetOutput

Description

The method retrieves the current state of the specified digital output.

Syntax

object.GetOutput(long Port, long Bit)

Arguments

Port	Pointer to a variable that receives the current state of the specific output. The value will be populated by 0 or 1.
Bit	Number of the specific bit.

Return Value

Long.

The method retrieves the current state (0 or 1) of the specified digital output.

Remarks

To get values of all outputs of the specific port, use [GetExtOutputPort](#).

Digital outputs are represented in the controller variable **OUT**. For more information about digital outputs, see the *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```

Sub GetOutput_Sample()
    'The method reads output 0 of port 0 ( OUT(0).0 )
    Dim Port As Long
    On Error GoTo except
        Port = Ch.GetOutput(0, 0)
    Exit Sub
except:
    MyErrorMsg                                     'See Error Handling in Visual Basic
End Sub

```

4.26.4 GetOutputPort

Description

The method retrieves the current state of the specified digital output port.

Syntax

object.GetOutputPort(long Port)

Arguments

Port	Number of the output port.
------	----------------------------

Return Value

Long.

The method retrieves the current state (0 or 1) of the specified digital output port.

Remarks

To get the value of the specific output of the specific port, use [GetOutput](#).

Digital outputs are represented in the controller variable **OUT**. For more information about digital outputs, see *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```
Sub GetOutputPort_Sample()  
    'The method reads output port 0 to 15  
    '( OUT(0) to OUT(15) )  
  
    Dim Index As Long  
    Dim Port(15) As Double  
    On Error GoTo except  
        For Index = 0 To 15  
            Port(Index) = Ch.GetOutputPort(Index)  
        Exit Sub  
    except:  
        MyErrorMsg 'See Error Handling in Visual Basic  
    End Sub
```

4.26.5 SetOutput

Description

The method sets the specified digital output to the specified value.

Syntax

object.SetOutput(long Port, long Bit, long Value)

Arguments

Port	Number of the output port.
Bit	Number of the specific bit.
Value	The value to be writes to the specified output. Any non-zero value is interpreted as 1.

Return Value

None.

Remarks

The method sets the specified digital output to the specified value. To set values of all outputs of a specific port, use the [SetOutputPort](#) method.

Digital outputs are represented in the controller variable **OUT**. For more information about digital outputs, see the *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```
Sub SetOutput_Sample()
    'The method sets output 0 of port 0 to 1
    '( ACSPL+ equivalent: OUT(0).0 = 1 )

    On Error GoTo except
        Call Ch.SetOutput(0, 0, 1)
    Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.26.6 SetOutputPort**Description**

The method sets the specified digital output port to the specified value.

Syntax

object.SetOutputPort(long Port, long Value)

Arguments

Port	Number of the digital output port.
Value	The value to be written to the specified port.

Return Value

None.

Remarks

The method sets the specified digital output port to the specified value. To set the value of the specific output of the specific port, use the [SetOutput](#) method.

Digital outputs are represented in the controller variable **OUT**. For more information about digital outputs, see the *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```
Sub SetOutputPort_Sample()
    'The method sets outputs 0 to 15 to 100

    Dim Index As Long
    On Error GoTo except
```

```

    For Index = 0 To 15
        Call Ch.SetOutputPort(Index, 100)
    Exit Sub
except:
    MyErrorMsg                                     'See Error Handling in Visual Basic
End Sub

```

4.26.7 GetAnalogInputNT

Description

The function retrieves the current value of the specified analog input.

Syntax

object.GetAnalogInputNT(int Port, double* Value, ULONGLONG CallType, VARIANT* pWait)

Arguments

Port[in]	Index of analog input port - a number between 0 and up to the maximum number of analog input signals minus one.
Value[out]	Variable that receives the current value of a specific analog input. Units: in scaling, by percent, of the signal and ranges from -100 to +100 of the maximum level.
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None

Remarks

The function retrieves the current value of specified analog input.

Analog inputs are represented in the controller variable **AIN**. For more information about analog inputs, see *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```

object waitObject = 0;
Double Value = new Double();
try
{
    Ch.GetAnalogInputNT(0, out Value, Ch.ACSC_SYNCHRONOUS, ref
waitObject);
}
catch (COMException Ex)
{
    ErorMsg(Ex);
}

```

4.26.8 GetAnalogOutputNT**Description**

The function retrieves the current value of the specified analog output.

Syntax

object.GetAnalogOutputNT(int Port, double* Value, ULONGLONG CallType, VARIANT* pWait)

Arguments

Port[in]	Index of analog output port - a number between 0 and up to the maximum number of analog output signals minus one.
Value[out]	Variable that receives the current value of a specific analog output. Units: in scaling, by percent, of the signal and ranges from -100 to +100 of the maximum level.
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None.

Remarks

The method retrieves the current value of the specified analog output. To write a value to the specific analog outputs use the [SetAnalogOutputNT](#) method.

Analog outputs are represented in the controller variable **AOUT**. For more information about analog outputs, see *ACSPL+ Programmer's Guide*.

Example

```
object waitObject = 0;
Double Value = new Double();
try
{
    Ch.GetAnalogOutputNT(0, out Value, Ch.ACSC_SYNCHRONOUS, ref
waitObject);
}
catch (COMException Ex)
{
    ErrorMsg(Ex);
}
```

4.26.9 SetAnalogOutputNT**Description**

The function writes the specified value to the specified analog output.

Syntax

object.SetAnalogOutputNT(int Port, double Value, ULONGLONG CallType, VARIANT* pWait)

Arguments

Port[in]	Index of analog output port - a number between 0 and up to the maximum number of analog output signals minus one.
Value[in]	The value to be written to the specified analog output. Units: in scaling, by percent, of the signal and ranges from -100 to +100 of the maximum level.
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None.

Remarks

The function writes the specified value to the specified analog outputs. To get a value of the specific analog output, use the [GetAnalogOutputNT](#) method.

Analog outputs are represented in the controller variable **AOUT**. For more information about analog outputs, see the *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```
Sub SetAnalogOutput_Sample()
'The method writes the value 100 to the analog outputs of port 0 to 15
Dim Index As Long
On Error GoTo except
    For Index = 0 To 15
        Call Ch.SetAnalogOutput(Index, 100)
    Next Index
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.26.10 GetExtInput**Description**

The method retrieves the current state of the specified extended input.

Syntax

object.GetExtInput(long Port, long Bit)

Arguments

Port	Number of the extended input port.
Bit	Number of the specific bit.

Return Value

Long.

The method retrieves the current state (0 or 1) of the extended input specified by **Port** and **Bit**.

Remarks

To get that states for all the inputs of the specific extended **Port**, use the [GetExtInputPort](#) method.

Extended inputs are represented in the controller variable **EXTIN**. For more information about extended inputs, see *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```

Sub GetExtInput_Sample()
    'The method reads extended input 0 of port 0
    '( EXTIN(0).0 )
Dim Port As Long
On Error GoTo except
    Port = Ch.GetExtInput(0, 0)
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub

```

4.26.11 GetExtInputPort**Description**

The method retrieves the current state of the specified extended input port.

Syntax

object.GetExtInputPort(long Port)

Arguments

Port	Number of the extended input port.

Return Value

Long.

The method retrieves the current state of the specified extended input port.

Remarks

To get the value of a specific input of the specific extended port, use the [GetExtInput](#) method.

Extended inputs are represented in the controller variable **EXTIN**. For more information about extended inputs, see *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```

Sub GetExtInputPort_Sample()
    'The method reads extended input port 0 to 15
    '( EXTIN(0) to EXTIN(15) )
Dim Index As Long
Dim Port(15) As Long
On Error GoTo except
    For Index = 0 To 15
        Port(Index) = Ch.GetExtInputPort(Index)
    Next Index
Exit Sub
except:

```

```
MyErrorMsg                                     'See Error Handling in Visual Basic
End Sub
```

4.26.12 GetExtOutput

Description

The method retrieves the current state of the specified extended output.

Syntax

object.GetExtOutput(long Port, long Bit)

Arguments

Port	Number of the extended output port.
Bit	Number of the specific bit.

Return Value

Long.

The method retrieves the current state (0 or 1) of the specified extended output.

Remarks

To get values of all outputs of the specific extended port, use the [GetExtOutputPort](#) method.

Extended outputs are represented in the controller variable **EXTOUT**. For more information about extended outputs, see *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```
Sub GetExtOutput_Sample()
    'The method reads extended output 0 of port 0 ( EXTOUT(0).0 )
    Dim Port As Long
    On Error GoTo except
        Port = Ch.GetExtOutput(0, 0)
    Exit Sub
except:
    MyErrorMsg                                     'See Error Handling in Visual Basic
End Sub
```

4.26.13 GetExtOutputPort

Description

The method retrieves the current state of the specified extended output port.

Syntax

object.GetExtOutputPort(long Port)

Arguments

Port	Number of the extended output port.
-------------	-------------------------------------

Return Value

Long.

The method retrieves the current state (0 or 1) of the specified extended output port.

Remarks

To get the value of the specific output of the specific extended port, use the [GetExtOutput](#) method.

Extended outputs are represented in the controller variable **EXTOUT**. For more information about extended outputs, see *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```
Sub GetExtOutputPort_Sample()
    'The method reads extended output port 0 ( EXTOUT(0) )
    Dim Index As Long
    Dim Port(15) As Double
    On Error GoTo except
        For Index = 0 To 15
            Port(Index) = Ch.GetExtOutputPort(Index)
        Next Index
    Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.26.14 SetExtOutput**Description**

The method sets the specified extended output to the specified value.

Syntax

object.SetExtOutput(long Port, long Bit, long Value)

Arguments

Port	Number of the extended output port.
Bit	Number of the specific bit.
Value	The value to be written to the specified output. Any non-zero value is interpreted as 1.

Return Value

None.

Remarks

The method sets the specified extended output to the specified value. To set values of all outputs of the specific extended port, use the [SetExtOutputPort](#) method.

Extended outputs are represented in the controller **EXTOUT** variable. For more information about extended outputs, see the *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```
Sub SetExtOutput_Sample()
    'The method sets output 0 of extended
    'port 0 to 1
    '(ACSPL+ equivalent: EXTOUT(0).0 = 1)

    On Error GoTo except
    Call Ch.SetExtOutput(0, 0, 1)
    Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.26.15 SetExtOutputPort**Description**

The method sets the specified extended output port to the specified value.

Syntax

object.SetExtOutputPort(long Port, long Value)

Arguments

Port	Number of the extended output port.
Value	The value to be written to the specified output port.

Return Value

None.

Remarks

The method sets the specified extended output port to the specified value. To set the value of the specific output of the specific extended port, use the [SetExtOutput](#) method.

Extended outputs are represented in the controller variable **EXTOUT**. For more information about extended outputs, see the *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```
Sub SetExtOutputPort_Sample()
    'The method sets outputs of extended
    'port 0 to 15 to 100
```

```

Dim Index As Double
On Error GoTo except
    For Index = 0 To 15
        Call Ch.SetExtOutputPort(Index, 100)
    Next Index
Exit Sub
except:
    MyErrorMsg                                     'See Error Handling in Visual Basic
End Sub

```

4.27 Safety Control Methods

The Safety Control methods are:

Table 4-30. Safety Control Methods

GetFault	Retrieves the set of bits that indicate the motor or system faults.
SetFaultMask	Sets the mask, that enables/disables the examination and processing of the controller faults.
GetFaultMask	Retrieves the mask that defines which controller faults are examined and processed.
EnableFault	Enables the specified axis or system fault.
DisableFault	Disables the specified axis or system fault.
SetResponseMask	Sets the mask that defines for which axis or system faults the controller provides default response.
GetResponseMask	Retrieves the mask that defines for which axis or system faults the controller provides default response.
EnableResponse	Enables the default response to the specified axis or system fault.
DisableResponse	Disables the default response to the specified axis or system fault.
GetSafetyInput	Retrieves the current state of the specified safety input.
GetSafetyInputPort	Retrieves the current state of the specified safety input port.
GetSafetyInputPortInv	Retrieves the set of bits that define inversion for the specified safety input port.
SetSafetyInputPortInv	Sets the set of bits that define inversion for the specified safety input port.

FaultClear	The method clears the current faults and results of previous faults stored in the MERR variable.
FaultClearM	The method clears the current faults and results of previous faults stored in the MERR variable for multiple axis.

4.27.1 GetFault

Description

The method retrieves the set of bits that indicate the motor or system faults.

Syntax

object.GetFault(long Axis)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
-------------	---

Return Value

Long.

The method retrieves the set of bits that indicate motor or system faults.

Remarks

The motor faults are related to a specific motor, the power amplifier, and the Servo processor. For example: Position Error, Encoder Error, or Driver Alarm.

The system faults are not related to any specific motor. For example: Emergency Stop or Memory Fault.

The parameter **Fault** receives the set of bits that indicates the controller faults. To recognize the specific fault, properties ACSC_SAFETY_*** can be used. See [Safety Control Masks](#) for a detailed description of these properties.

For more information about the controller faults, see the *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```
Sub GetFault_Sample()
Dim Fault As Long
On Error GoTo except
    'Retrieves the set of bits that indicate the
    'motor or system faults
    Fault = Ch.GetFault(Ch.ACSC_AXIS_0)
Exit Sub
except:
    MyErrorMsg 'See Error Handling in Visual Basic
End Sub
```

4.27.2 SetFaultMask

Description

The method sets the mask that enables or disables the examination and processing of the controller faults.



Certain controller faults provide protection against potential serious bodily injury and damage to the equipment. Be aware of the implications before disabling any alarm, limit, or error

Syntax

object.SetFaultMask(long Axis, long Mask)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Mask	The mask to be set: If a bit of the Mask is zero, the corresponding fault is disabled. To set/reset a specified bit, use ACSC_SAFETY_*** properties. See Safety Control Masks for a detailed description of these properties. If the Mask is ACSC_NONE, then all the faults for the specified axis are enabled. If the Mask is zero, then all the faults for the specified axis are disabled.

Return Value

None.

Remarks

The method sets the mask, that enables/disables the examination and processing of the controller faults. The two types of controller faults are motor faults and system faults.

The motor faults are related to a specific motor, the power amplifier or the Servo processor. For example: Position Error, Encoder Error or Driver Alarm.

The system faults are not related to any specific motor. For example: Emergency Stop or Memory Fault.

For more information about the controller faults, see the *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```

Sub SetFaultMask_Sample()
On Error GoTo except
    'Enable all faults of axis 0
    Call Ch.SetFaultMask(Ch.ACSC_AXIS_0, ACSC_NONE)
Exit Sub
except:
    MyErrorMsg 'See Error Handling in Visual Basic
End Sub

```

4.27.3 GetFaultMask**Description**

The method retrieves the mask that defines which controller faults are examined and processed.

Syntax

object.GetFaultMask(long Axis)

Arguments**Axis**

Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see [Axis Definitions](#).

Return Value

Long.

The method retrieves the mask that defines which controller faults are examined and processed.

Remarks

If a bit of the mask is zero, the corresponding fault is disabled.

Use the ACSC_SAFETY_*** properties to examine a specific bit. See [Safety Control Masks](#) for a detailed description of these properties.

Controller faults are of two types: motor faults and system faults.

The motor faults are related to a specific motor, the power amplifier or the Servo processor. For example: Position Error, Encoder Error or Driver Alarm.

The system faults are not related to any specific motor, for example: Emergency Stop or Memory Fault.

For more information about the controller faults, see the *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```

Sub GetFaultMask_Sample()
Dim Mask As Long
On Error GoTo except
    'Retrieves the mask that defines which

```

```

        'controller Faults are examined
        'and processed of axis 0
Mask = Ch.GetFaultMask(Ch.ACSC_AXIS_0)
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub

```

4.27.4 EnableFault

Description

The method enables the specified axis or system fault.

Syntax

object.EnableFault(long Axis, long Fault)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Fault	The fault to be enabled. Only one fault can be enabled at a time. To specify the fault, one of the properties ACSC_SAFETY_*** can be used. See Safety Control Masks for a detailed description of these properties.

Return Value

None.

Remarks

The method enables the examination and processing of the specified motor or system fault by setting the specified bit of the fault mask to one.

The motor faults are related to a specific motor, the power amplifier, and the Servo processor. For example: Position Error, Encoder Error, and Driver Alarm.

The system faults are not related to any specific motor. For example: Emergency Stop, Memory Fault.

For more information about the controller faults, see *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```

Sub EnableFault_Sample()
On Error GoTo except
    'Enable fault Right Limit of axis 0
    Call Ch.EnableFault(Ch.ACSC_AXIS_0, Ch.ACSC_SAFETY_RL)
Exit Sub
except:
    MyErrorMsg    'See Error Handling in Visual Basic
End Sub

```

4.27.5 DisableFault**Description**

The method disables the specified axis or system fault.

Syntax

object.DisableFault(long Axis, long Fault)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Fault	The fault to be disabled. Only one fault can be enabled at a time. To specify the fault, one of the properties ACSC_SAFETY_*** can be used. See Safety Control Masks for a detailed description of these properties.

Return Value

None.

Remarks

The method disables the examination and processing of the specified motor or system fault by setting the specified bit of the fault mask to zero.

The motor faults are related to a specific motor, the power amplifier, and the Servo processor. For example: Position Error, Encoder Error, and Driver Alarm.

The system faults are not related to any specific motor, for example: Emergency Stop, Memory Fault.

For more information about the controller faults, see *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```

Sub DisableFault_Sample()
On Error GoTo except
    'Disable fault Software Right Limit in axis 0

```

```

        Call Ch.DisableFault(Ch.ACSC_AXIS_0, Ch.ACSC_SAFETY_RL)
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub

```

4.27.6 SetResponseMask

Description

The method retrieves the mask that defines the motor or the system faults for which the controller provides the default response.



Certain controller faults provide protection against potential serious bodily injury and damage to the equipment. Be aware of the implications before disabling any alarm, limit, or error

Syntax

object.SetResponseMask(long Axis, long Mask)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Mask	The mask to be set: If a bit of the Mask is zero, the corresponding fault is disabled. To set/reset a specified bit, use ACSC_SAFETY_*** properties. See Safety Control Masks for a detailed description of these properties. If the Mask is ACSC_NONE, then all the faults for the specified axis are enabled. If the Mask is zero, then all the faults for the specified axis are disabled.

Return Value

None.

Remarks

The method retrieves the mask that defines the motor or the system faults for which the controller provides the default response.

The default response is a controller-predefined action for the corresponding fault. For more information about the controller faults and default responses, see the *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```

Sub SetResponseMask_Sample()
On Error GoTo except
    'Enable all default responses
    Call Ch.SetResponseMask(Ch.ACSC_AXIS_0, ACSC_NONE)
Exit Sub
except:
    MyErrorMsg 'See Error Handling in Visual Basic
End Sub

```

4.27.7 GetResponseMask

Description

The method retrieves the mask that defines the motor or the system faults for which the controller provides the default response.

Syntax

object.GetResponseMask(long Axis)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
-------------	---

Return Value

Long.

Remarks

The method retrieves the mask that determines whether the controller will respond to a motor or system fault with the fault's default response.

If a bit of the parameter **Mask** is zero, it deactivates the default controller response for the corresponding fault.

For more information about the controller faults and default, responses see the *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```

Sub GetResponseMask_Sample()
Dim Mask As Long
On Error GoTo except
    'Retrieves the mask of axis 0
    Mask = Ch.GetResponseMask(Ch.ACSC_AXIS_0)
Exit Sub
except:
    MyErrorMsg 'See Error Handling in Visual Basic
End Sub

```

4.27.8 EnableResponse

Description

The method enables the response to the specified axis or system fault.

Syntax

object.EnableResponse(long Axis, long Response)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Response	The default response to be enabled. Only one default response can be enabled at a time. To specify the default response, one of the properties ACSC_SAFETY_*** can be used. See Safety Control Masks for a detailed description of these properties.

Return Value

None.

Remarks

The method enables the default response to the specified axis or system fault by setting the specified bit of the response mask to one.

The default response is a controller-predefined action for the corresponding fault. For more information about the controller faults and default responses, see *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```
Sub EnableResponse_Sample()
On Error GoTo except
    'Enable the default response to the
    'Position Error fault of axis 0
    Call Ch.EnableResponse(Ch.ACSC_AXIS_0, Ch.ACSC_SAFETY_PE)
Exit Sub
except:
MyErrorMsg    'See Error Handling in Visual Basic
End Sub
```

4.27.9 DisableResponse

Description

The method disables the default response to the specified axis or system fault.

Syntax

object.DisableResponse(long Axis, long Response)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Response	The default response to be disabled. Only one default response can be enabled at a time. To specify the default response, one of the properties ACSC_SAFETY_*** can be used. See Safety Control Masks for a detailed description of these properties.

Return Value

None.

Remarks

The method disables the default response to the specified motor or system fault by setting the specified bit of the response mask to zero.

The default response is a controller-predefined action for the corresponding fault. For more information about the controller faults and default responses, see *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```
Sub DisableResponse_Sample()
On Error GoTo except
                                'Disable the default response to the
                                'Right Limit fault
    Call Ch.DisableResponse(Ch.ACSC_AXIS_0, Ch.ACSC_SAFETY_RL)
Exit Sub
except:
    MyErrorMsg                    'See Error Handling in Visual Basic
End Sub
```

4.27.10 GetSafetyInput**Description**

The method retrieves the current state of the specified safety input.

Syntax

object.GetSafetyInput(long Axis, long Input)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Input	The specific safety input. The safety input can be one or any combination of the following: ACSC_SAFETY_RL ACSC_SAFETY_LL ACSC_SAFETY_NETWORK ACSC_SAFETY_HOT ACSC_SAFETY_DRIVE ACSC_SAFETY_ES See Safety Control Masks for a detailed description of these properties.

Return Value

Long.

The method retrieves the current state of the specified safety input.

Remarks

To get values of all safety inputs of the specific axis, use [GetSafetyInputPort](#).

Safety inputs are represented in the controller variables **SAFIN** and **S_SAFIN**. For more information about safety inputs, see *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```
Sub GetSafetyInput_Sample()
    'The method reads Emergency Stop system
    'safety input

    Dim SInput As Long
    On Error GoTo except

    'Reads right Limit Safety input of axis 0
    SInput = Ch.GetSafetyInput(Ch.ACSC_AXIS_0, Ch.ACSC_SAFETY_RL)
Exit Sub
except:
    MyErrorMsg          'See Error Handling in Visual Basic
End Sub
```

4.27.11 GetSafetyInputPort

Description

The method retrieves the current state of the specified safety input port.

Syntax

object.GetSafetyInputPort(long Axis)

Arguments

Axis

Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see [Axis Definitions](#).

Return Value

Long.

The method retrieves the current state of the specified safety input port.

To recognize a specific motor safety input, only one of the following properties can be used:

- > ACSC_SAFETY_RL
- > ACSC_SAFETY_LL
- > ACSC_SAFETY_NETWORK
- > ACSC_SAFETY_HOT
- > ACSC_SAFETY_DRIVE

To recognize a specific system safety input, only the ACSC_SAFETY_ES property can be used.

See [Safety Control Masks](#), [Safety Control Masks](#) for a detailed description of these properties.

Remarks

To get the state of the specific safety input of a specific axis, use [GetSafetyInput](#).

Safety inputs are represented in the controller variables **SAFIN** and **S_SAFIN**. For more information about safety inputs, see *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```
Sub GetSafetyInputPort_Sample()
    Dim SInput As Long
    On Error GoTo except
        'The method reads safety input port of the
        'axis 0
        SInput = Ch.GetSafetyInputPort(Ch.ACSC_AXIS_0)
    Exit Sub
except:
    MyErrorMsg          'See Error Handling in Visual Basic
End Sub
```

4.27.12 GetSafetyInputPortInv

Description

The method retrieves the set of bits that define inversion for the specified safety input port.

Syntax

object.GetSafetyInputPortInv(long Axis)

Arguments

Axis

Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see [Axis Definitions](#).

Return Value

Long.

The method retrieves the set of bits that define inversion for the specified safety input port.

To recognize a specific bit, use the following properties:

- > ACSC_SAFETY_RL
- > ACSC_SAFETY_LL
- > ACSC_SAFETY_NETWORK
- > ACSC_SAFETY_HOT
- > ACSC_SAFETY_DRIVE

Use the ACSC_SAFETY_ES property to recognize an inversion for the specific system safety input port.

See [Safety Control Masks](#) for a detailed description of these properties.

Remarks

To set the specific inversion for the specific safety input port, use [GetSafetyInputPortInv](#).

If a bit of the retrieved set is zero, the corresponding signal is not inverted and therefore high voltage is considered an active state. If a bit is raised, the signal is inverted and low voltage is considered an active state.

Inversions of safety inputs are represented in the controller variables **SAFIN** and **S_SAFIN**. For more information about safety inputs, see *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```
Sub GetSafetyInputPortInv_Sample()
    'The method reads an inversion of safety
    'input port of the axis 0

    Dim State As Long
    On Error GoTo except
        State = Ch.GetSafetyInputPortInv(Ch.ACSC_AXIS_0)
    Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.27.13 SetSafetyInputPortInv

Description

The method sets the set of bits that define inversion for the specified safety input port.

Syntax

object.SetSafetyInputPortInv(long Axis, long Value)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Value	The specific inversion. To set a specific bit, use the following properties: ACSC_SAFETY_RL ACSC_SAFETY_LL ACSC_SAFETY_NETWORK ACSC_SAFETY_HOT ACSC_SAFETY_DRIVE. To set an inversion for the specific system safety input port, use only the ACSC_SAFETY_ES property. See Safety Control Masks for a detailed description of these properties.

Return Value

Long.

The method retrieves the current state of the specified safety input.

Remarks

The method sets the bits that define inversion for the specified safety input port. To retrieve an inversion for the specific safety input port, use the [GetSafetyInputPortInv](#) method.

If a bit of the set is zero, the corresponding signal will not be inverted and therefore high voltage is considered an active state. If a bit is raised, the signal will be inverted and low voltage is considered an active state.

The inversions of safety inputs are represented in the controller variables **SAFIN** and **S_SAFIN**. For more information about safety inputs, see *ACSPL+ Programmer's Guide*.

If the method fails, the Error object is filled with the error description.

Example

```
Sub SetSafetyInputPortInv_Sample()
    'The method sets the inversion for safety
    'input port of the axis 0 to 100
On Error GoTo except
    Call Ch.SetSafetyInputPortInv(Ch.ACSC_AXIS_0, 100)
```

```
Exit Sub
except:
    MyErrorMsg                                     'See Error Handling in Visual Basic
End Sub
```

4.27.14 FaultClear

Description

The method clears the current faults and the result of the previous fault stored in the **MERR** variable.

Syntax

object.FaultClear(long Axis)

Arguments

Axis

Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see [Axis Definitions](#).

Return Value

None.

Remarks

The method clears the current faults of the specified axis and the result of the previous fault stored in the **MERR** variable.

If the method fails, the Error object is filled with the error description.

Example

```
Sub FaultClear_Sample()
Dim result As Long
On Error GoTo except

                                'Clears the current faults and results of
                                'previous faults stored in the MERR
                                'variable of axis 0
    Call Ch.FaultClear(Ch.ACSC_AXIS_0)
Exit Sub
except:
    MyErrorMsg                                     'See Error Handling in Visual Basic
End Sub
```

4.27.15 FaultClearM

Description

The method clears the current faults and results of previous faults stored in the **MERR** variable for multiple axis.

Syntax

object.FaultClearM(variant Axes)

Arguments**Axes**

Array of axis constants. Each element specifies one involved axis: ACSC_AXIS_0 corresponds to the 0axis, ACSC_AXIS_1 to the 1axis, etc. After the last axis, one additional element must be included that contains -1 and marks the end of the array.

For the axis constants see [Axis Definitions](#).

Return Value

None.

Remarks

If the reason for the fault is still active, the controller will set the fault immediately after this command is performed. If cleared fault is Encoder Error, the feedback position is reset to zero.

If the method fails, the Error object is filled with the error description.

Example

```
Sub FaultClearM_Sample()
Dim Axes(2)
Axes(0) = Ch.ACSC_AXIS_0
Axes(1) = Ch.ACSC_AXIS_1
Axes(2) = -1
On Error GoTo except

    Call Ch.FaultClearM(Axes)
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.28 Wait-for-Condition Methods

The Wait-for-Condition methods are:

Table 4-31. Wait-for-Condition Methods

WaitMotionEnd	Waits for the end of a motion.
WaitLogicalMotionEnd	Waits for the logical end of a motion.
WaitCollectEndExt	Waits for the end of data collection.
WaitProgramEnd	Waits for the program termination in the specified buffer.
WaitMotorCommutated	Waits for the specified state of the specified motor.
WaitMotorEnabled	Waits for the specified state of the specified motor.

WaitInput

Waits for the specified state of the specified digital input.

4.28.1 WaitMotionEnd**Description**

The method waits for the end of a motion.

Syntax

object.WaitMotionEnd (long Axis, long Timeout)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Timeout	Maximum waiting time in milliseconds. If Timeout is INFINITE, the method's time-out interval never elapses.

Return Value

None.

Remarks

The method does not return while the specified axis is involved in a motion, the motor has not settled in the final position and the specified time-out interval has not elapsed.

The method differs from the [WaitLogicalMotionEnd](#) method. Examining the same motion, the [WaitMotionEnd](#) method will return latter. The [WaitLogicalMotionEnd](#) method returns when the generation of the motion finishes. On the other hand, the [WaitMotionEnd](#) method returns when the generation of the motion finishes and the motor has settled in the final position.

If the method fails, the Error object is filled with the error description.

Example

```
Sub WaitMotionEnd_Sample()
On Error GoTo except
                                'Wait for the end of motion of axis 0
                                'during 10 sec
    Call Ch.WaitMotionEnd(Ch.ACSC_AXIS_0, 10000)
Exit Sub
except:
    MyErrorMsg    'See Error Handling in Visual Basic
End Sub
```

4.28.2 WaitLogicalMotionEnd**Description**

The method waits for the logical end of a motion.

Syntax

object.WaitLogicalMotionEnd (long Axis, long Timeout)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Timeout	Maximum waiting time in milliseconds. If Timeout is INFINITE, the method's time-out interval never elapses.

Return Value

None.

Remarks

The method does not return while the specified axis is involved in a motion and the specified time-out interval has not elapsed.

The method differs from the [WaitMotionEnd](#) method. Examining the same motion, the [WaitMotionEnd](#) method will return later. The [WaitLogicalMotionEnd](#) method returns when the generation of the motion finishes. On the other hand, the [WaitMotionEnd](#) method returns when the generation of the motion finishes and the motor has settled in the final position.

If the method fails, the Error object is filled with the error description.

Example

```
Sub WaitLogicalMotionEnd_Sample()
On Error GoTo except
    'Wait for the logical end of motion of axis
    '0 during 10 sec
    Call Ch.WaitLogicalMotionEnd(Ch.ACSC_AXIS_0, 10000)
Exit Sub
except:
    MyErrorMsg    'See Error Handling in Visual Basic
End Sub
```

4.28.3 WaitCollectEndExt**Description**

The function waits for the end of data collection.

Syntax

object.WaitCollectEndExt (long Timeout, long Axis)

Arguments

Timeout	Maximum wait time in milliseconds, If INFINITE - timeout interval never elapses.
Axis	If using axis data collection – the axis number, otherwise ACSC_NONE. ACSC_AXIS_0 corresponds to axis 0, ACSC_AXIS_1 to axis 1 etc.

Return Value

None

Comments

If using Axis data collection (data collection is synced to axis) set Axis to be the axis number you are collecting data from, otherwise set Axis to be ACSC_NONE.

The function does not return while data collection is in progress and the correct parameters have been passed.

Verifies AST<Axis#>.#DC flag for Synced data collection, otherwise verifies S_ST.#DC system flag. Using the function with the wrong parameters (i.e. calling the function with an axis number while performing a system data collection) will result in undefined behavior.

This function is supported in Versions 2.70 and higher.

Example

```
Ch.DataCollection(Ch.ACSC_DCF_SYNC, Ch.ACSC_AXIS_2, ArrayName, 1000,
Ch.ACSC_DCF_TEMPORAL, "FPOS(2)");

Ch.WaitCollectEndExt(2000, // timeout
                    Ch.ACSC_AXIS_2 // Axis number
                    );
```

4.28.4 WaitProgramEnd**Description**

The method waits for the program termination in the specified buffer.

Syntax

object.WaitProgramEnd (long Buffer, long Timeout)

Arguments

Buffer	Buffer number, from 0 to 9.
Timeout	Maximum waiting time in milliseconds. If Timeout is INFINITE, the method's time-out interval never elapses.

Return Value

None.

Remarks

The method does not return while the ACSPL+ program in the specified buffer is in progress and the specified time-out interval has not elapsed.

If the method fails, the Error object is filled with the error description.

Example

```
Sub WaitProgramEnd_Sample()
Dim Index As Long
On Error GoTo except
  For Index = 0 To 8
      'Append a line to all buffers
      Call Ch.AppendBuffer(Index, "WAIT 5000;STOP")
      'Run all buffers
      Call Ch.RunBuffer(Index)
      'Wait program ends during 10 sec
      Call Ch.WaitProgramEnd(Index, 10000)
  Next Index
Exit Sub
except:
  MyErrorMsg      'See Error Handling in Visual Basic
End Sub
```

4.28.5 WaitMotorCommutated**Description**

The method waits for the specified state of the specified motor.

Syntax

object.WaitMotorCommutated (long Axis, long State, long Timeout)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
State	1 - The method waits for the motor to be commutated. Maximum waiting time in milliseconds.
Timeout	If Timeout is INFINITE, the method's time-out interval never elapses.

Return Value

None.

Remarks

The method does not return while the specified motor is not in the desired state and the specified time-out interval has not elapsed. The method examines the **MFLAGS.#BRUSHOK** flag.

If the method fails, the Error object is populated with the error description

Example

```

Sub WaitMotorCommutated_Sample()
On Error GoTo except
    'Commut axis 0
    Call Ch.Commut(Ch.ACSC_AXIS_0)
    'Wait motor 0 commutated during 5 sec
    Call Ch.WaitMotorCommutated(Ch.ACSC_AXIS_0, 1, 5000)
Exit Sub
except:
    MyErrorMsg      'See Error Handling in Visual Basic
End Sub

```

4.28.6 WaitMotorEnabled

Description

The method waits until the specified motor is commutated.

Syntax

object.WaitMotorEnabled (long Axis, long State, long Timeout)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
State	1 – the method waits for the motor to be enabled, 0 – the method waits for the motor to be disabled.
Timeout	Maximum waiting time in milliseconds. If Timeout is INFINITE, the method's time-out interval never elapses.

Return Value

None.

Remarks

The method does not return while the specified motor is not in the desired state and the specified time-out interval has not elapsed. The method examines the **MST.#ENABLED** motor flag.

If the method fails, the Error object is filled with the error description.

Example

```

Sub WaitMotorEnabled_Sample()
On Error GoTo except
    'Enable axis 0
    Call Ch.Enable(Ch.ACSC_AXIS_0)
    'Wait motor 0 enabled during 5 sec
    Call Ch.WaitMotorEnabled(Ch.ACSC_AXIS_0, 1, 5000)
Exit Sub
except:
    MyErrorMsg      'See Error Handling in Visual Basic
End Sub

```

4.28.7 WaitInput**Description**

The method waits for the specified state of digital input.

Syntax

object.WaitInput (long Port, long Bit, long State, long Timeout)

Arguments

Port	Number of input port: 0 corresponds to IN0 , 1 – to IN1 , etc.
Bit	Selects one bit from the port, from 0 to 31.
State	Specifies a desired state of the input, 0 or 1.
Timeout	Maximum waiting time in milliseconds. If Timeout is INFINITE, the method's time-out interval never elapses.

Return Value

None.

Remarks

The basic configuration of the SPiiPlus PCI model provides only 16 inputs. Therefore, **Port** must be 0, and **Bit** can be specified only from 0 to 15.

If the method fails, the Error object is filled with the error description.

Example

```
Sub WaitInput_Sample()  
On Error GoTo except  
    'Wait for IN0.0 = 1 during 5 sec  
    'Parameters: 0 - IN0  
    '              0 - IN0.0  
    '              1 - wait for IN0.0 = 1  
    '              5000 - during 5 sec  
    Call Ch.WaitInput(0, 0, 1, 5000)  
Exit Sub  
except:  
    MyErrorMsg      'See Error Handling in Visual Basic  
End Sub
```

4.29 Event and Interrupt Handling Methods

The Event and Interrupt Handling methods are:

Table 4-32. Event and Interrupt Handling Methods

EnableEvent	Enables ActiveX event generation for the specified interrupt condition.
DisableEvent	Disables ActiveX event generation for the specified interrupt condition.
SetEventMask	Sets the mask for the specified interrupt.
GetEventMask	Retrieves the mask for the specified interrupt.

4.29.1 EnableEvent

Description

The method enables ActiveX event generation for the specified interrupt condition.

Syntax

object.EnableEvent (long Interrupt)

Arguments

Interrupt	Specifies one of the interrupts such as ACSC_INTR_COMMAND, ACSC_INTR_EMERGENCY, etc. For the full list of the interrupts, see Interrupt Types .
------------------	---

Return Value

None.

Remarks

The library generates an event when the specified **Interrupt** occurs.

SPiiPlus COM Library has events for all types of interrupts:

PEG ([in] long Param), MARK1 ([in] long Param) etc. For full list of SPiiPlus COM Library events see [Events, Events](#).

The bit-mapped event parameter **Param** identifies which axis/buffer/input the interrupt was generated for. See [Callback Interrupt Masks](#) for a detailed description of the **Param** argument for each interrupt.

One event can be associated with each controller interrupt. All events are handled in the main application thread, therefore the application execution is stopped until the event is handled.

To disable a specific event, call the [DisableEvent](#) method with the **Interrupt** argument equal to the specified interrupt type.



Before the PEG interrupts can be detected, the [AssignPins](#) method must be called.

If the method fails, the Error object is filled with the error description.

Example

```
'Sub EnableEvent_Sample()
Private Sub ChWithEvents_ACSPLPROGRAM(ByVal Param As Long)
MsgBox "Parameter=" & Param, vbExclamation, "ACSPLPROGRAM"
End Sub
Private Sub ChWithEvents_PROGRAMEND(ByVal Param As Long)
MsgBox "Parameter=" & Param, vbExclamation, "PROGRAMEND"
End Sub
Sub EnableEvent_Sample()
On Error GoTo except
        'Enable event ACSC_INTR_ACSPL_PROGRAM
        Call ChWithEvents.EnableEvent(ChWithEvents.ACSC_INTR_ACSPL_PROGRAM)
        'Enable event ACSC_INTR_PROGRAM_END
        Call ChWithEvents.EnableEvent(ChWithEvents.ACSC_INTR_PROGRAM_END)
        'Delete all lines in buffer 5
        Call ChWithEvents.Transaction("#5D1,100000")
        'Appends a line to buffer 5
        Call ChWithEvents.AppendBuffer(5, "interrupt stop")
        'Execute buffer 5
        ChWithEvents.Transaction("#5X")
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.29.2 DisableEvent

Description

The method disables ActiveX event generation for the specified interrupt condition.

Syntax

object.[DisableEvent](#) (long Interrupt)

Arguments

Interrupt
Specifies one of the interrupts such as ACSC_INTR_COMMAND, ACSC_INTR_EMERGENCY, etc. For the full list of the interrupts, see Interrupt Types .

Return Value

None.

Remarks

The method disables ActiveX event generation, that was enabled with method [EnableEvent](#)

If the method fails, the Error object is filled with the error description.

Example

```
Sub DisableEvent_Sample()  
On Error GoTo except  
                'Disable event PROGRAM_END  
    Ch. DisableEvent (Ch.ACSC_INTR_PROGRAM_END)  
Exit Sub  
except:  
    MyErrorMsg                'See Error Handling in Visual Basic  
End Sub
```

4.29.3 SetEventMask

Description

The method sets the mask for specified interrupt.

Syntax

object.SetEventMask(long Interrupt, long Mask)

Arguments

Interrupt	Specifies one of the following interrupts: ACSC_INTR_ACSPL_PROGRAM – an ACSPL+ program has generated the interrupt by INTERRUPT command ACSC_INTR_ACSPL_PROGRAM_EX – an ACSPL+ program has generated the interrupt by INTERRUPT command ACSC_INTR_COMM_CHANNEL_CLOSED - a communication channel has been closed. ACSC_INTR_COMMAND – a line of ACSPL+ commands has been executed in a dynamic buffer ACSC_INTR_EMERGENCY - an EMERGENCY STOP signal has been generated. ACSC_INTR_ETHERCAT_ERROR - an EtherCAT error occurred ACSC_INTR_LOGICAL_MOTION_END – a logical motion has finished ACSC_INTR_MOTION_FAILURE – a motion has been interrupted due to a fault ACSC_INTR_MOTION_PHASE_CHANGE – motion profile changes the phase ACSC_INTR_MOTION_START – motion starts ACSC_INTR_MOTOR_FAILURE – a motor has been disabled due to a fault ACSC_INTR_NEWSEGM - an AST.#NEWSEGM bit is high. ACSC_INTR_PHYSICAL_MOTION_END – a physical motion has finished ACSC_INTR_PROGRAM_END – an ACSPL+ program has finished ACSC_INTR_SOFTWARE_ESTOP - the EStop button was clicked. ACSC_INTR_SYSTEM_ERROR - a system error occurred. ACSC_INTR_TRIGGER – AST.#TRIGGER bit goes high
Mask	The mask to be set. If some bit = 0 then the interrupt for the corresponding axis/buffer/input does not occur – interrupt is disabled. Use ACSC_MASK_*** properties to set/reset a specified bit. See GetEventMask for a detailed description of these properties. If Mask is ACSC_NONE, the interrupts for all axes/buffers/inputs are enabled.

If **Mask** is 0, the interrupts for all axes/buffers/inputs are disabled.

As default all bits for each interrupts are set to one.

Return Value

None.

Remarks

The method sets the bit mask for specified interrupt. To get current mask for specified interrupt, call [GetEventMask](#).

Using a mask, you can reduce the number of generated events. The event will be generated only if the interrupt is caused by an axis/buffer/input that corresponds to non-zero bit in the related mask.

.If the method fails, the Error object is filled with the error description.

Example

```
Sub SetEventMask_Sample()
On Error GoTo except
                                'Sets the mask for specified interrupt only
                                'for axis 0
    Call Ch.SetEventMask(Ch.ACSC_INTR_PROGRAM_END, Ch.ACSC_MASK_AXIS_0)
Exit Sub
except:
    MyErrorMsg                    'See Error Handling in Visual Basic
End Sub
```

4.29.4 *GetEventMask*

Description

The method retrieves the mask for specified interrupt.

Syntax

object.GetEventMask(long Interrupt)

Arguments

Interrupt	Specifies one of the following interrupts: ACSC_INTR_ACSPL_PROGRAM – an ACSPL+ program has generated the interrupt by INTERRUPT command ACSC_INTR_ACSPL_PROGRAM_EX – an ACSPL+ program has generated the interrupt by INTERRUPT command ACSC_INTR_COMM_CHANNEL_CLOSED - a communication channel has been closed. ACSC_INTR_COMMAND – a line of ACSPL+ commands has been executed in a dynamic buffer ACSC_INTR_EMERGENCY - an EMERGENCY STOP signal has been generated. ACSC_INTR_ETHERCAT_ERROR - an EtherCAT error occurred. ACSC_INTR_LOGICAL_MOTION_END – a logical motion has finished ACSC_INTR_MOTION_FAILURE – a motion has been interrupted due to a fault ACSC_INTR_MOTION_PHASE_CHANGE – motion profile changes the phase ACSC_INTR_MOTION_START – motion starts ACSC_INTR_MOTOR_FAILURE – a motor has been disabled due to a fault ACSC_INTR_NEWSEGM - an AST.#NEWSEGM bit is high. ACSC_INTR_PHYSICAL_MOTION_END – a physical motion has finished ACSC_INTR_PROGRAM_END – an ACSPL+ program has finished ACSC_INTR_SOFTWARE_ESTOP - the EStop button was clicked. ACSC_INTR_SYSTEM_ERROR - a system error occurred. ACSC_INTR_TRIGGER – AST.#TRIGGER bit goes high.
-------------------------	--

Return Value

Long.

The method retrieves the bit mask for the specified interrupt.

Remarks

To set the mask for a specified interrupt, call [SetEventMask](#).

If a bit in the return value equals 0, the interrupt for the corresponding axis/buffer/input is disabled.

Use the ACSC_MASK_*** properties to find the values of specific bits. See [Callback Interrupt Masks](#), [Callback Interrupt Masks](#) for a detailed description of these properties.

By default all the interrupt bits are set to one.

If the method fails, the Error object is filled with the error description.

Example

```
Sub GetEventMask_Sample()  
'The example shows how to get the mask for specific interrupt  
Dim Mask As Long  
On Error GoTo except  
  
                                'An ACSPL+ program has finished  
Mask = Ch.GetEventMask(Ch.ACSC_INTR_PROGRAM_END)  
Exit Sub  
except:  
    MyErrorMsg                                'See Error Handling in Visual Basic  
End Sub
```

4.30 Variables Management Methods

The Variables Management methods are:

Table 4-33. Variables Management Methods

DeclareVariable	Creates the persistent global variable.
ClearVariables	Deletes all persistent global variables.

4.30.1 DeclareVariable

Description

The method creates a persistent global variable.

Syntax

object.DeclareVariable (long Type, string Name)

Arguments

Type	Type of the variable. For an integer variable Type must be ACSC_INT_TYPE . For a real variable, Type must be ACSC_REAL_TYPE .
Name	Name of the variable.

Return Value

None.

Remarks

The method creates the persistent global variable specified by **Name** of type specified by **Type**. The variable can be used as any other ACSPL+ or global variable.

If it is necessary to declare one or two-dimensional array, **Name** should also contains the dimensional size in brackets.

The lifetime of a persistent global variable is not connected with any program buffer. The persistent variable survives any change in the program buffers and can be erased only by the [ClearVariables](#) method.

The method waits for the controller response.

If the method fails, the Error object is filled with the error description.

Example

```
Sub DeclareVariable_Sample()
On Error GoTo except
    'The method creates the persistent global
    'variable "MyVar" as integer type.
    Call Ch.DeclareVariable(Ch.ACSC_INT_TYPE, "MyVar")
Exit Sub
Sexcept:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.30.2 ClearVariables

Description

Deletes all persistent global variables.

Syntax

object.ClearVariables

Arguments

None.

Return Value

None.

Remarks

The method deletes all persistent global variables created by the [DeclareVariable](#) method.

The method waits for the controller response.

If the method fails, the Error object is filled with the error description.

Example

```
Sub ClearVariables_Sample()
On Error GoTo except
    'The method declare global variable "MyVar"
    'as integer type.
    Call Ch.DeclareVariable(Ch.ACSC_INT_TYPE, "MyVar")
    'The method deletes all persistent global
    'variables.
    Call Ch.ClearVariables
Exit Sub
except:
```

```
MyErrorMsg                                     'See Error Handling in Visual Basic  
End Sub
```

4.31 Service Methods

The Service methods are:

Table 4-34. Service Methods

GetFirmwareVersion	Retrieves the firmware version of the controller
GetSerialNumber	Retrieves the controller serial number
GetSysInfo	Retrieves certain system information
GetBuffersCount	Returns the number of available ACSPL+ programming buffers
GetAxesCount	Returns the number of available axes
GetDBufferIndex	Retrieves the index of the D-Buffer
GetUMDVersion	Retrieves the UMD version number

4.31.1 [GetFirmwareVersion](#)

Description

The method retrieves the firmware version of the controller.

Syntax

```
object.GetFirmwareVersion()
```

Arguments

None.

Return Value

String.

The method retrieves the controller firmware version.

Remarks

If the method fails, the Error object is filled with the error description.

Example

```

Sub GetFirmwareVersion_Sample()
Dim Firm As String
On Error GoTo except
    'Retrieves the firmware version of the
    'controller
    Firm = Ch.GetFirmwareVersion()
Exit Sub
except:
    MyErrorMsg          'See Error Handling in Visual Basic
End Sub

```

4.31.2 GetSerialNumber**Description**

The method retrieves the controller serial number.

Syntax

object.GetSerialNumber()

Arguments

None.

Return Value

String.

The method retrieves the character string that contains the controller serial number.

Remarks

If the method fails, the Error object is filled with the error description.

Example

```

Sub GetSerialNumber_Sample()
    'Retrieves the character string that
    'contains the controller serial number

Dim SerN As String
On Error GoTo except
    SerN = Ch.GetSerialNumber
Exit Sub
except:
    MyErrorMsg          'See Error Handling in Visual Basic
End Sub

```

4.31.3 GetSysInfo**Description**

The method returns certain system information based on the argument that is specified (see [Table 4-35](#)).

Syntax

object.GetSysInfo(int Key, double *Value, ULONGLONG CallType, VARIANT* pWait)

Arguments

Key	Configuration key, specifies the configured feature. Assigns value of Key argument in the ACSPL+ SYSINFO function.
Value	Receives the configuration data..
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None.

Example

```
Double Value = new Double();
try
{
    Ch.GetSysInfo(Ch.ACSC_SYS_MODEL_KEY, out Value); //gets SPiiPlus
    Model                                           // Number
}
catch (COMException Ex)
{
    ErrorMsg(Ex);
}
```

Table 4-35. System Information Keys

Key Name	Key	Description
ACSC_SYS_MODEL_KEY	1	The SPiiPlus model number.
ACSC_SYS_VERSION_KEY	2	The SPiiPlus version number.

ACSC_SYS_NBUFFERS_KEY	10	Number of regular ACSPL+ program buffers.
ACSC_SYS_DBUF_INDEX_KEY	11	D-buffer index.
ACSC_SYS_NAXES_KEY	13	Total number of axes (in current configuration).
ACSC_SYS_NNODES_KEY	14	Number of EtherCAT nodes.
ACSC_SYS_NDCCH_KEY	15	Number of data collection channels per DSP.
ACSC_SYS_ECAT_KEY	16	EtherCAT support: 1 - Yes 0 - No

4.31.4 *GetBuffersCount*

Description

The method returns the number of available ACSPL+ programming buffers.

Syntax

object.GetBuffersCount(double* Value, ULONGLONG CallType, VARIANT *pWait)

Arguments

Value [out]	Receives the number of available ACSPL+ programming buffers.
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None.

Example

```
Double Value = new Double();
try
{
    Ch.GetBuffersCount(out Value);
}
catch (COMException Ex)
{
    ErorMsg(Ex);
}
```

4.31.5 GetAxesCount**Description**

The method returns the number of available axes.

Syntax

object.GetAxesCount(double* Value, ULONGLONG CallType,VARIANT *pWait)

Arguments

Value	Receives the number of available axes.
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None.

Example

```
Double Value = new Double();
try
{
    Ch.GetAxesCount(out Value);
}
catch (COMException Ex)
{
}
```

```

        ErorMsg (Ex) ;
    }

```

4.31.6 GetDBufferIndex

Description

The method returns the index of the D-Buffer.

Syntax

object.GetDBufferIndex(double* Value, ULONGLONG CallType,VARIANT *pWait);

Arguments

Value [out]	Receives the D-Buffer Index.
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None.

Example

```

Double Value = new Double();
try
{
    Ch.GetDBufferIndex(out Value);
}
catch (COMException Ex)
{
    ErorMsg (Ex) ;
}

```

4.31.7 GetUMDVersion

Description

The method retrieves the SPiiPlus User Mode Driver version number.

Syntax

object.GetUMDVersion()

Arguments

None.

Return Value

Long.

The method retrieves the SPiiPlus User Mode Driver version number.

Remarks

The User Mode Driver version consists of four (or less) numbers separated by points: #.#.#.#. The binary version number is represented by 32-bit unsigned integer value. Each byte of this value specifies one number in the following order: high byte of high word – first number, low byte of high word – second number, high byte of low word – third number and low byte of low word – fourth number. For example the version "2.10" has the following binary representation (hexadecimal format): 0x020A0000.

The first two numbers in the string form are obligatory. Any release version of the library consists of two numbers. The third and fourth numbers specify an alpha or beta version, special or private build, or other version.

If the method fails, the Error object is filled with the error description.

Example

```
Sub GetLibraryVersion_Sample()
    Dim LVersion As Double
    On Error GoTo except
        'Retrieves the UMD version number.
        LVersion = Ch.GetUMDVersion
    Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.32 Error Diagnosis Methods

The Error Diagnosis methods are:

Table 4-36. Error Diagnosis Methods

GetMotorError	Retrieves the reason why the motor was disabled.
GetMotionError	Retrieves the termination code of the last executed motion of the specified axis.
GetProgramError	Retrieves the error code of the last program error encountered in the specified buffer.
ParseCOMErrorCode	Parses the SPiiPlus error code and components of HRESULT.

4.32.1 GetMotorError

Description

The method retrieves the reason for motor disabling.

Syntax

object.GetMotorError(long Axis)

Arguments

Axis

Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see [Axis Definitions](#).

Return Value

Long.

The method retrieves the reason for the motor becoming disabled.

Remarks

If the motor is enabled the method returns zero. If the motor was disabled, the method returns the reason for the disabling. To get the error explanation, use the [GetErrorString](#) method.

See *ACSPL+ Programmer's Guide* for all available motor error code descriptions.

If the method fails, the Error object is filled with the error description.

Example

```
Sub GetMotorError_Sample()
    Dim MError As Long
    On Error GoTo except
        'Retrieves the reason for motor disabling
        MError = Ch.GetMotorError(Ch.ACSC_AXIS_0)
    Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.32.2 GetMotionError

Description

The method retrieves the termination code of the last executed motion of the specified axis.

Syntax

object.GetMotionError(long Axis)

Arguments

Axis

Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see [Axis Definitions](#).

Return Value

Long.

The method retrieves the termination code of the last executed motion of the specified axis.

Remarks

If the motion is in progress, the method returns zero. If the motion terminates for any reason, the method returns the termination code. To get the error explanation, use the method [GetErrorString](#).

See the *ACSPL+ Programmer's Guide* for all available motion termination codes description.

If the method fails, the Error object is filled with the error description.

Example

```
Sub GetMotionError_Sample()
Dim MError As Long
On Error GoTo except

'Retrieves the termination code of the last
'executed motion of axis 0
MError = Ch.GetMotionError(Ch.ACSC_AXIS_0)
Exit Sub
except:
MyErrorMsg 'See Error Handling in Visual Basic
End Sub
```

4.32.3 GetProgramError**Description**

The method retrieves the error code of the last program error encountered in the specified buffer.

Syntax

object.GetProgramError(long Buffer)

Arguments

Buffer	Number of the program buffer.

Return Value

Long.

The method retrieves the error code of the last program error encountered in the specified buffer.

Remarks

If the program is running, the method returns zero. If the program terminates for any reason, the method returns the termination code. To get the error explanation, use [GetErrorString](#).

If the method fails, the Error object is filled with the error description.

Example

```
Sub GetProgramError_Sample()
Dim PError As Long
```

```

On Error GoTo except
    'Appends buffer 0 with 1 line
    Call Ch.AppendBuffer(0, "!The program finished without STOP command")
    'Run buffer 0
    Call Ch.RunBuffer(0)
    'Retrieves error 3114
    PError = Ch.GetProgramError(0)
except:
    MyErrorMsg 'See Error Handling in Visual Basic
End Sub

```

4.32.4 ParseCOMErrorCode

Description

The method is used to parse the SPiiPlus error code and components of **HRESULT**.

Syntax

object.ParseCOMErrorCode (long COMErrorCode, [long Code,] [long Facility,] [long Severity])

Arguments

COMErrorCode	HRESULT : a 32-bit standard COM value representing the method return status. It consists of a severity code, a facility code, and an error code. HRESULT can be retrieved from the Error object (see Error Handling).
Code	HRESULT error code. The SPiiPlus error code is a component of the HRESULT error code.
Facility	HRESULT facility code.
Severity	HRESULT severity code.

Return Value

Long.

The function returns the SPiiPlus error codes (see [Error Codes](#)) parsed from **COMErrorCode** (which is **HRESULT**).

Remarks

In Visual Basic Script the method can only be used to retrieve the SPiiPlus error code. It cannot be used to retrieve values for **Code**, **Facility**, or **Severity**.

Example

```

Sub ParseCOMErrorCode _Sample()
On Error GoTo except
Dim Code As Long
Dim Facility As Long

```

```
Dim Severity As Long
                                'Parse COM Error Code
code = Ch.ParseCOMErrorCode(ErrorNum)
                                'code = Ch.ParseCOMErrorCode(ErrorNum, Code, Facility,
                                'Severity)
If (code <> 0) Then MsgBox Ch.GetErrorString(code)
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.33 Dual Port RAM (DPRAM) Access Methods



The COM Library DPRAM methods are not supported in SPiiPlus products.

The Dual Port RAM (DPRAM) Access methods are:

Table 4-37. Dual Port RAM (DPRAM) Access Methods

ReadDPRAMInteger	Reads 32-bit integer from DPRAM
WriteDPRAMInteger	Writes 32-bit integer to DPRAM
ReadDPRAMReal	Reads 64 real from DPRAM
WriteDPRAMReal	Writes 64-bit real to DPRAM

4.33.1 ReadDPRAMInteger

Description

The method reads 32-bit integer from the DPRAM.

Syntax

object. ReadDPRAMInteger(long Address)

Arguments

Address	Must be even number from 128 to 508.
----------------	--------------------------------------

Return Value

Long.

The method reads 32-bit integer from the DPRAM.

Remarks

Address has to be even number in the range of 128 to 508, because of 16-bit alignment when working with DPRAM. Addresses less than 128 are used for internal purposes.



The method can be used only with the PCI and Simulator communication channels.

If the method fails, the Error object is filled with the error description.

Example

```
Sub ReadDPRAMInteger_Sample()
Dim DPrint As Long
On Error GoTo except
        'Reads 32-bit integer from the DPRAM from
        'address 0xA0
    DPrint = Ch.ReadDPRAMInteger(160)
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.33.2 WriteDPRAMInteger

Description

The method writes 32-bit integer to the DPRAM.

Syntax

object.WriteDPRAMInteger(long address, long Value)

Arguments

Address	Must be even number from 128 to 508.
Value	Value to be written.

Return Value

The method returns non-zero on success.

Remarks

Address has to be even number in the range of 128 to 508, because we use 16-bit alignment when working with DPRAM. Addresses less than 128 are used for internal purposes.

If the method fails, the Error object is filled with the error description.

Example

```
Sub WriteDPRAMInteger_Sample()
On Error GoTo except
        'Write to DPRAM address 0xA0 the value 100
    Call Ch.WriteDPRAMInteger(160, 100)
Exit Sub
except:
```

```
MyErrorMsg      'See Error Handling in Visual Basic
End Sub
```

4.33.3 ReadDPRAMReal

Description

The method reads 64-bit Real from the DPRAM.

Syntax

object.ReadDPRAMReal(long Address)

Arguments

Address	Must be even number from 128 to 508.
----------------	--------------------------------------

Return Value

Double.

The method reads 64-bit Real from the DPRAM.

Remarks

Address has to be even number in the range of 128 to 504, because of 16-bit alignment when working with DPRAM. Addresses less than 128 are used for internal purposes.

If the method fails, the Error object is filled with the error description.

Example

```
Sub ReadDPRAMReal_Sample()
Dim DPRreal As Double
On Error GoTo except
        'Reads 32-bit real from the DPRAM from
        'address 0xA0
    DPRreal = Ch.ReadDPRAMReal(160)
Exit Sub
except:
    MyErrorMsg      'See Error Handling in Visual Basic
End Sub
```

4.33.4 WriteDPRAMReal

Description

The method writes 64-bit Real to the DPRAM.

Syntax

object.WriteDPRAMReal(long address, long Value)

Arguments

Address	Must be even number from 128 to 508.
Value	Value to be written.

Return Value

The method returns non-zero on success.

If the method fails, the Error object is filled with the error description.

Remarks

Address has to be even number in the range of 128 to 504, because we use 16-bit alignment when working with DPRAM. Addresses less than 128 are used for internal purposes.

If the method fails, the Error object is filled with the error description.

Example

```
Sub WriteDPRAMReal_Sample()
On Error GoTo except
    'Write to DPRAM address 0xA0 the value 100
    Call Ch.WriteDPRAMReal(160, 100)
Exit Sub
except:
    MyErrorMsg      'See Error Handling in Visual Basic
End Sub
```

4.34 Position Event Generation (PEG) Methods

There are two groups of Position Event Generation (PEG) methods, one specifically for non-SPiiPlus controllers, and one specifically for SPiiPlus controllers.

Table 4-38. Non-NT Position Event Generation (PEG) Methods

PegInc	Sets incremental PEG
PegRandom	Sets random PEG
AssignPins	Defines whether a digital output is allocated to the corresponding bit of the OUT array (for general purpose use) or allocated for PEG method use.
StopPeg	Stops PEG

Table 4-39. NT Position Event Generation (PEG) Methods

AssignPegNT	Assigns engine-to-encoder as well as additional digital outputs for use as PEG State and PEG Pulse outputs.
AssignPegOutputsNT	Sets output pins assignment and mapping between FGP_OUT signals to the bits of the ACSPL+ OUT(x) variable.
AssignFastInputsNT	Used for switching MARK_1 physical inputs to ACSPL+ variables as Fast General Purpose inputs.
PegIncNT	Sets the parameters for the Incremental PEG mode.
PegRandomNT	Sets the parameters for the Random PEG mode.

WaitPegReadyNT	Waits for the all values to be loaded and the PEG engine to be ready to respond to movement.
StartPegNT	Initiates the PEG process on the specified axis.
StopPegNT	Terminates the PEG process immediately on the specified axis.

4.34.1 PegInc

Description

The method initiates incremental PEG. The Incremental PEG method is defined by the first point, the last point and the interval.

Syntax

object.PegInc(long Flags, long Axis, double Width, long FirstPoint, long Interval, long LastPoint, long TbNumber, double TbPeriod)

Arguments

Flags	Bit-mapped parameter that can include the following flag: ACSC_AMF_SYNCHRONOUS . PEG starts synchronously with the motion sequence.
Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Width	Width of desired pulse in milliseconds.
FirstPoint	Position where the first pulse is generated.
Interval	Distance between the pulse-generating points.
LastPoint	Position where the last pulse is generated.
TbNumber	Number of time-based pulses generated after each encoder-based pulse, the range is from 0 to 511. If time-based pulses are not desired, assign ACSC_NONE .
TbPeriod	Period of time-based pulses, period from 0.000025 to 0.75 (milliseconds). If time-based pulses are not desired, assign ACSC_NONE .

Return Value

None.

Remarks

The method initiates incremental PEG generation. See details in the *ACSPL+ Programmer's Guide*. The time-based pulse generation is optional, if it is not used, **TbPeriod** and **TbNumber** should be **ACSC_NONE**.

If the method fails, the Error object is filled with the error description.

Example

```
Sub PegInc_Sample()
On Error GoTo except
    'Enable axis Z
    Call Ch.Enable(Ch.ACSC_AXIS_Z)
    'Wait till Z enabled during 5 sec
    Call Ch.WaitMotorEnabled(Ch.ACSC_AXIS_Z, 1, 5000)
    'Relative motion with target position of
    '10000
    Call Ch.ToPoint(Ch.ACSC_AMF_RELATIVE, Ch.ACSC_AXIS_Z, 10000)
    'Parameters: Ch.ACSC_AMF_SYNCHRONOUS -
    'synchronous PEG
    'Ch.ACSC_AXIS_Z - axis Z
    '0.01 - 10 microseconds pulse width
    '0 - start from 0
    '50 - generate pulse every 50 units
    '10000 - stop generating at 10000
    'Ch.ACSC_NONE, Ch.ACSC_NONE - with No
    'time-based pulse generation
    Call Ch.PegInc(Ch.ACSC_AMF_SYNCHRONOUS, Ch.ACSC_AXIS_Z, 0.01, 0, 50,
10000, Ch.ACSC_NONE, Ch.ACSC_NONE)
    'Finish the motion
    'End of the multi-point motion
    Call Ch.EndSequence(Ch.ACSC_AXIS_Z)
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.34.2 PegRandom

Description

The method initiates random PEG. Random PEG method specifies an array of points where position-based events should be generated.

Syntax

object.PegRandom(long Flags, long Axis, double Width, string PointArray, string StateArray, long TbNumber, double TbPeriod)

Arguments

Flags	Bit-mapped parameter that can include the following flag: ACSC_AMF_SYNCHRONOUS. PEG starts synchronously with the motion sequence.
Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions.
Width	Width of desired pulse in milliseconds.
PointArray	Null-terminated string contained the name of the real array that stores positions at which PEG pulse should be generated The array must be declared as a global variable by an ACSPL+ program or by the DeclareVariable method.
StateArray	Null-terminated string contained the name of the integer array that stores desired output state at each position. The array must be declared as a global variable by an ACSPL+ program or by the DeclareVariable method. If output state change is not desired, this parameter should be NULL.
TbNumber	Number of time-based pulses generated after each encoder-based pulse, the range is from 0 to 511. If time-based pulses are not desired, assign ACSC_NONE.
TbPeriod	Period of time-based pulses, period from 0.000025 to 0.75 (milliseconds). If time-based pulses are not desired, assign ACSC_NONE.

Return Value

None.

Remarks

The method initiates random PEG generation (see *ACSPL+ Programmer's Guide*).

StateArray should be NULL if output state won't change because of PEG.

The time-based pulse generation is optional, if it is not used, **TbPeriod** and **TbNumber** should be **ACSC_NONE**.

If the method fails, the Error object is filled with the error description.

Example

```
Sub PegRandom_Sample()
On Error GoTo except
'Declare variable "PointArr(10)"
```

```

Call Ch.DeclareVariable(Ch.ACSC_REAL_TYPE, "PointArr(10)")
                        'Declare variable "StateArr(10)"
Call Ch.DeclareVariable(Ch.ACSC_INT_TYPE, "StateArr(10)")
                        'Enable axis Z
Call Ch.Enable(Ch.ACSC_AXIS_Z)
                        'Wait till axis Z enabled during 5 sec
Call Ch.WaitMotorEnabled(Ch.ACSC_AXIS_Z, 1, 5000)
                        'Relative motion of axis Z with target
                        'position of 10000
Call Ch.ToPoint(Ch.ACSC_AMF_RELATIVE, Ch.ACSC_AXIS_Z, 10000)
                        'Parameters:
                        '0 - Not-Synchronous PEG
                        'Ch.ACSC_AXIS_Z - axis Z
                        '0.01 - 10 microseconds pulse width
                        '"PointArr" - point array
                        '"StateArr" - State array
                        'Ch.ACSC_NONE, Ch.ACSC_NONE - with No
                        'time-based pulse generation

Call Ch.PegRandom(0, Ch.ACSC_AXIS_Z, 0.01, "PointArr", "StateArr",
Ch.ACSC_NONE, Ch.ACSC_NONE)
                        'Finish the motion
                        'End of the multi-point motion

Call Ch.EndSequence(Ch.ACSC_AXIS_Z)
Exit Sub
except:
MyErrorMsg                                     'See Error Handling in Visual Basic
End Sub

```

4.34.3 AssignPins

Description

Defines whether a digital output is allocated to the corresponding bit of the OUT array (for general purpose use) or allocated for PEG method use.

Syntax

object.AssignPins(long Axis, short Mask)

Arguments

Axis	Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Mask	16-bit mask defines pins state.

Return Value

None.

Remarks

The method calls the ACSPL command: **SetConf** (205, axis, Mask), where Mask is the output mask. For a description of the output mask, see the description of **SetConf** in the *ACSPL+ programmers guide*.

If the method fails, the Error object is filled with the error description.

Example

```
Sub AssignPins_Sample()
On Error GoTo except
                                'Bit 8 is 1 means OUT3 is assigned to the
                                'pulse output of the 0 PEG
    Call Ch.AssignPins(Ch.ACSC_AXIS_0, 256)
Exit Sub
except:
MyErrorMsg                        'See Error Handling in Visual Basic
End Sub
```

4.34.4 StopPeg**Description**

The method stops PEG.

Syntax

object.StopPeg(long Axis)

Arguments

Axis

Axis constant: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see [Axis Definitions](#).

Return Value

None.

Remarks

If the method fails, the Error object is filled with the error description.

Example

```
Sub StopPeg_Sample()
On Error GoTo except
                                'Enable axis Z
    Call Ch.Enable(Ch.ACSC_AXIS_Z)
                                'Wait till axis Z enabled during 5 sec
    Call Ch.WaitMotorEnabled(Ch.ACSC_AXIS_Z, 1, 5000)
    Call Ch.ToPoint(Ch.ACSC_AMF_RELATIVE, Ch.ACSC_AXIS_Z, 1000000)
    Call Ch.PegInc(0, Ch.ACSC_AXIS_Z, 1, 0, 50, 10000, Ch.ACSC_NONE,
Ch.ACSC_NONE)
                                'Stop the peg in axis Z
```



```

    Call Ch.StopPeg(Ch.ACSC_AXIS_Z)
                                'End of the multi-point motion
    Call Ch.EndSequence(Ch.ACSC_AXIS_Z)
Exit Sub
except:
    MyErrorMsg      'See Error Handling in Visual Basic
End Sub

```

4.34.5 AssignPegNT

Description

The method is used for engine-to-encoder assignment as well as for assigning additional digital outputs assignment for use as PEG State and PEG Pulse outputs specifically for the SPiiPlus family controllers.

Syntax

object.AssignPegNT(int Axis, int EngToEncBitCode, int GpOutsBitCode, ULONGLONG CallType, VARIANT *pWait);

Arguments

Axis	PEG axis: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
EngToEncBitCode	Bit code for engines-to-encoders mapping according to the ASSIGNPEG chapter in <i>PEG and MARK Operations Application Notes</i>
GpOutsBitCode	General Purpose outputs assignment to use as PEG state and PEG pulse outputs according to the ASSIGNPEG chapter in <i>PEG and MARK Operations Application Notes</i> .
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None.

Example

```

int Axis = 1;
int EngToEncBitCode = 0x0;
int GpOutsBitCode = 0x0b11;
try
{
    Ch.AssignPegNT(Axis, EngToEncBitCode, GpOutsBitCode);
}
catch (COMException Ex)
{
    ErorMsg(Ex);
}

```

4.34.6 AssignPegOutputsNT**Description**

The method is used for setting output pins assignment and mapping between **FGP_OUT** signals to the bits of the ACSPL+ **OUT(x)** variable, where x is the index that has been assigned to the controller in the network during System Configuration, specifically for the SPiiPlus family controllers.

OUT is an integer array that can be used for reading or writing the current state of the General Purpose outputs - see the *SPiiPlus Command & Variable Reference Guide*.

Syntax

object.AssignPegOutputsNT(int Axis, int OutputIndex, int BitCode, ULONGLONG CallType, VARIANT *pWait);

Arguments

Axis	PEG axis: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
OutputIndex	0 for OUT_0 , 1 for OUT_1 , ..., 9 for OUT_9
BitCode	Bit code for engine outputs to physical outputs mapping according to the ASSIGNPEG chapter in the <i>PEG and MARK Operations Application Notes</i> .
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.

variant *pWait

A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None.

Example

```

int Axis = 1;
int OutputBitCode = 0x0b0;
int OutputIndex = 0;
try
{
    Ch.AssignPegOutputsNT(Axis, OutputIndex, OutputBitCode);
}
catch (COMException Ex)
{
    ErrorMsg(Ex);
}

```

4.34.7 AssignFastInputsNT**Description**

The method is used to switch MARK_1 physical inputs to ACSPL+ variables as fast General Purpose inputs for SPiiPlus family controllers.



While this method is not related to PEG activity, it is included with the PEG methods for the sake of completeness, since many times fast inputs are used in applications that use PEG functionality.

The method is used for setting input pins assignment and mapping between **FGP_IN** signals to the bits of the ACSPL+ **IN(x)** variable, where x is the index that has been assigned to the controller in the network during System Configuration.

IN is an integer array that can be used for reading the current state of the General Purpose inputs - see the *SPiiPlus Command & Variable Reference Guide*.

Syntax

object.AssignFastInputsNT(int Axis, int InputIndex, int BitCode, ULONGLONG CallType, VARIANT *pWait)

Arguments

Axis	PEG axis: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
InputIndex	0 for IN_0 , 1 for IN_1 , ..., 9 for IN_9

BitCode	Bit code for engine outputs to physical outputs mapping according to the ASSIGNPEG chapter in <i>PEG and MARK Operations Application Notes</i> .
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None.

Example

```
try
{
    Ch.AssignFastInputsNT(Ch.ACSC_AXIS_1, 1, 7);
}
catch (COMException Ex)
{
    ErrorMsg(Ex);
}
```

4.34.8 PegIncNT

The method is used for setting the parameters for the Incremental PEG mode for SPiiPlus family controllers. Incremental PEG is defined by first point, last point and the interval

object. PegIncNT(int Flags, int Axis, double Width, double FirstPoint, double Interval, double LastPoint, int TbNumber, double TbPeriod, ULONGLONG CallType, VARIANT *pWait)

Flags	Bit-mapped parameter that can include following flag: ACSC_AMF_WAIT - the execution of the PEG is delayed until the StartPegNT function is executed. ACSC_AMF_INVERT_OUTPUT - the PEG pulse output is inverted. ACSC_AMF_SYNCHRONOUS - PEG starts synchronously with the motion sequence.
Axis	PEG axis: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Width	Width of desired pulse in milliseconds.
FirstPoint	Position where the first pulse is generated.
Interval	Distance between the pulse-generating points.
LastPoint	Position where the last pulse is generated.
TbNumber	Number of time-based pulses generated after each encoder-based pulse. If time-based pulses are not desired, assign ACSC_NONE to this argument.
TbPeriod	Period of time-based pulses. If time-based pulses are not desired, assign ACSC_NONE to this parameter.
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

None.

```
try
{
    Ch.PegIncNT(0,
                Ch.ACSC_AXIS_1, 0.5, 0, 200, 10000, Ch.ACSC_NONE,
                Ch.ACSC_NONE);
}
```

```

catch (COMException Ex)
{
    ErorMsg(Ex);
}

```

4.34.9 PegRandomNT

Description

The method is used for setting the parameters for the Random PEG mode for SPiiPlus family controllers. Random PEG function specifies an array of points where position-based events should be generated.

Syntax

object.PegRandomNT(int Flags, int Axis, double Width, int Mode, int FirstIndex, int LastIndex, BSTR PointArray, BSTR StateArray, int TbNumber, double TbPeriod, ULONGLONG CallType, VARIANT *pWait)

Argument

Flags	Bit-mapped parameter that can include the following flag: ACSC_AMF_WAIT - the execution of the PEG is delayed until the StartPegNT function is executed. ACSC_AMF_INVERT_OUTPUT - the PEG pulse output is inverted. ACSC_AMF_SYNCHRONOUS - PEG starts synchronously with the motion sequence.
Axis	PEG axis: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Width	Width of desired pulse in milliseconds.
Mode	Output signal configuration according to the ASSIGNPEG chapter in <i>PEG and MARK Operations Application Notes</i> .
FirstIndex	Index of position in PointArray where the first pulse is generated.
LastIndex	Index of position in PointArray where the last pulse is generated.
PointArray	Null-terminated string contained the name of the real array that stores positions at which PEG pulse should be generated. The array must be declared as a global variable by an ACSPL+ program or by the DeclareVariable method.
StateArray	Null-terminated string containing the name of the integer array that stores the desired output state at each position.

	The array must be declared as a global variable by an ACSPL+ program or by the DeclareVariable method. If output state change is not desired, this parameter should be NULL.
TbNumber	Number of time-based pulses generated after each encoder-based pulse. If time-based pulses are not desired, assign ACSC_NONE to this argument.
TbPeriod	Period of time-based pulses. If time-based pulses are not desired, assign ACSC_NONE to this parameter.
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

None.

```
try
{
    Ch.PegRandomNT(0, Test.Ch.ACSC_AXIS_1, 1.745, 0x444, 0, 4, "Points",
    "States", Test.Ch.ACSC_NONE, Test.Ch.ACSC_NONE);
}
catch (COMException Ex)
{
    ErrorMsg(Ex);
}
```

4.34.10 WaitPegReadyNT

Description

The method waits for the all values to be loaded and the PEG engine to be ready to respond to movement on the specified axis for SPiiPlus family controllers.

The method can be used in both the Incremental and Random PEG modes.

Syntax

object.WaitPegReadyNT(int Axis, long Timeout);

Arguments

Axis	PEG axis: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
Timeout	Maximum waiting time in milliseconds. If Timeout is INFINITE, the method's time-out interval never elapses.

Return Value

None.

Example

```
try
{
    Ch.WaitPegReadyNT(Ch.ACSC_AXIS_1, 2000);
}
catch (COMException Ex)
{
    ErorMsg(Ex);
}
```

4.34.11 StartPegNT**Description**

The method is used to initiate the PEG process on the specified axis for SPiiPlus family controllers.

Syntax

object.StartPegNT(int Axis, ULONGLONG CallType, VARIANT *pWait);

Arguments

Axis	PEG axis: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None.

Example

```
try
{
    Ch.StartPegNT(Ch.ACSC_AXIS_1);
}
catch (COMException Ex)
{
    ErorMsg(Ex);
}
```

4.34.12 StopPegNT**Description**

The method is used to terminate the PEG process immediately on the specified axis for SPiiPlus family controllers.

The method can be used in both the Incremental and Random PEG modes.

Syntax

object.StopPegNT(int Axis, ULONGLONG CallType, VARIANT *pWait);

Arguments

Axis	PEG axis: ACSC_AXIS_0 corresponds to the axis 0, ACSC_AXIS_1 to the axis 1, etc. For the axis constants see Axis Definitions .
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None.

Example

```
try
{
    Ch.StopPegNT(Ch.ACSC_AXIS_1);
}
catch (COMException Ex)
{
    ErorMsg(Ex);
}
```

4.35 Application Save/Load Methods

The Application Save/Load methods are:

Table 4-40. Application Save/Load Methods

AnalyzeApplication	Analyzes the type of application
LoadApplication	Loads the application
SaveApplication	Saves the application
FreeApplication	Frees memory after application is taken off

4.35.1 Structures

The Application Loader methods employ the following structures:

4.35.1.1 ApplicationFileInfo

Description

This structure describes all the application file data.

Syntax

```
struct ApplicationFileInfo {BSTR filename; BSTR description; int isNewFile; SAFEARRAY(Attribute)
*attributes; SAFEARRAY (Section) *sections; __int64 reserved}
```

Arguments

filename	File name
description	File description
isNewFile	1 - If writing new file 0 - If adding to existing file
attributes	Attributes array
section	Sections array
reserved	Pointer to C structure for saving

4.35.1.2 Attribute

Description

This structure describes any attribute, which can include application file.

Syntax

```
struct Attribute {BSTR key; BSTR value}
```

Arguments

key	Attribute's key (name)
value	Key's value

4.35.1.3 Section

Description

This structure describes any sections (or controller files).

Syntax

```
struct Section{FileType type; BSTR filename; BSTR description; unsigned int offset; unsigned int CRC;
int inuse; _int64 error; BSTR *data}
```

Arguments

type	Section (controller file) type
filename	Section (controller file) name
description	Section (controller file) description
offset	Offset in the file data section
CRC	Data CRC
inuse	1 - In use 0 - Not in use
error	Error code
data	Pointer to start of data

4.35.2 Enum

The Application Loader methods employ the following enums:

4.35.2.1 FileType

Description

Describes possible file types.

Syntax

```
enum FileType {ADJ, SP, ACSPL, PAR, USER}
```

Arguments

ADJ	Value 0: File type is an Adjuster
SP	Value 1: File type is an SP application
ACSPL	Value 2: File type is a Program Buffer
PAR	Value 3: File type is a Parameters file.
USER	Value 4: File type is a User file

4.35.3 AnalyzeApplication**Description**

The method analyzes application file and returns information about the file components, such as, saved ACSPL+ programs, configuration parameters, user files, etc.

Syntax

```
object.AnalyzeApplication(string fileName)
```

Arguments

fileName	Variable that specifies path to host application file. If analyzing controller application, this parameter should be NULL.
-----------------	--

Return Value

Return value is **struct ApplicationFileInfo*** described in [ApplicationFileInfo](#). If the method fails, there is exception thrown.

Example

```
ApplicationFileInfo info;
string fileName = "neededFile";
info = channel.AnalyzeApplication(fileName);
```

4.35.4 LoadApplication**Description**

The method loads user application file from a host PC to the controller flash.

Syntax

```
object.LoadApplication (string fileName, ApplicationFileInfo info, Boolean isPreview);
```

Arguments

fileName	Variable that specifies path to host application file. If analyzing controller application, this parameter should be NULL.
info	Structure, that describes all the application file data.
isPreview	For internal usage only. It must always be 0.

Return Value

None

Example

```
ApplicationFileInfo info;
string fileName = "neededFile";
info = channel.AnalyzeApplication(fileName);
channel.LoadApplication(fileName, info, 0);
```

4.35.5 SaveApplication**Description**

The method saves user application from the controller to a file on host PC.

Syntax:

object.SaveApplication(string fileName, ApplicationFileInfo info, Boolean isPreview);

Arguments

fileName	Variable that specifies path to host application file. If analyzing controller application, this parameter should be NULL.
info	Structure, that describes all the application file data.

Return Value

None

Example

```
ApplicationFileInfo info;
ErrorCode res = ErrorCode.OK;
string fileName = "neededFile";
info = channel.AnalyzeApplication(NULL);
channel.SaveApplication(fileName, info);
```

4.35.6 FreeApplication**Description**

The method frees memory, previously allocated by the [AnalyzeApplication](#) method.

Syntax:

```
object.FreeApplication(ApplicationFileInfo info);
```

Arguments**info**

Structure, that describes all the application file data.

Return Value

None

Example

```
ApplicationFileInfo info;
info = channel.AnalyzeApplication (NULL);
...
channel.FreeApplication (info);
```

4.36 Load/Upload Data To/From Controller Methods

The Load/Upload Data To/From Controller methods are:

Table 4-41. Load/Upload Data To/From Controller Methods

LoadDataToController	Writes value(s) from text file to SPiiPlus controller (variable or file).
UploadDataFromController	Writes value(s) from SPiiPlus controller (variable or file) to text file.

4.36.1 LoadDataToController

Description

This method writes value(s) from text file to SPiiPlus controller (variable or file).

Syntax:

```
object.LoadDataToController(long Dest, string DestName, long From1, long To1, long From2, long To2,
string SrcFilename, long SrcNumFormat, int bTranspose)
```

Arguments

Dest	Number of program buffer for local variable ACSC_NONE for global and ACSPL+ variable ACSC_FILE for loading directly to file on flash memory (only arrays can be written directly into controller files)
DestName	String that contains name of the Variable or File
From1/To1	Index range (first dimension)

From2/To2	Index range (second dimension)
SrcFilename	Filename (including path) of the source text data file
SrcNumFormat	Format of number(s) in source file. Use: ACSC_INT_BINARY for integers, and ACSC_REAL_BINARY for real
bTranspose	If TRUE (1), then the array will be transposed before being loaded. Otherwise, this parameter has no affect

Return Value

None

Remarks

The method writes to a specified variable (scalar/array) or straight to binary file on controller's flash memory. The variable can be a ACSPL+ controller variable, user global or user local. The input file must be ANSI format, otherwise an error 168 (invalid file format) is returned.

ACSPL+ and user global variables have global scope. Therefore, **Dest** must be **ACSC_NONE** (-1) for these classes of variables. User local variable exists only within a buffer. The buffer number must be specified for user local variable.

If **Dest** is **ACSC_NONE** (-1) and there is no global variable with the name specified by **DestName**, it will be defined. Arrays will be defined with dimensions (**To1**+1, **To2**+1).

If performing loading straight to file, **From1**, **To1**, **From2** and **To2** are meaningless.

If the variable is scalar, all indexes **From1**, **To1**, **From2**, and **To2** must be **ACSC_NONE** (-1). The method writes the value from file specified by **SrcFileName**, to the variable specified by **Name**. In this case if the variable is a one-dimensional array, **From1**, **To1** must specify the index range and **From2**, **To2** must be **ACSC_NONE** (-1). The text file, pointed to by **SrcFileName**, must contain **To1-From1**+1 values at least. The method writes the values to the specified variable from index **From1** to index **To1** inclusively.

If the variable is a two-dimensional array, **From1**, **To1** must specify the index range of the first dimension and **From2**, **To2** must specify the index range of the second dimension. The text file, pointed to by **SrcFileName**, must contain $((\text{To1}-\text{From1}+1) \times (\text{To2}-\text{From2}+1))$ values at least. Otherwise, error will occur.

The method uses the text file as follows: first, the method retrieves the **To2-From2**+1 values and writes them to row **From1** of the specified controller variable, then retrieves next **To2-From2**+1 values and writes them to row **From1**+1 of the specified controller variable, and so forth. If **bTranspose** is TRUE, the method actions are inverted. It takes **To1-From1**+1 values and writes them to column **From2** of the specified controller variable, then retrieves next **To1-From1**+1 values and writes them to column **From2**+1 of the specified controller variable, and so forth.

The text file is processed line-by-line; any characters except numbers, dots, commas and exponent 'e' are translated as separators between the numbers. Line that starts with no digits is considered as comment and ignored.

Example

```
try
{
    channel.LoadDataToController(ACSC_FILE, "MyArrayInFile", ACSC_NONE, ACSC_
    NONE, ACSC_NONE, ACSC_NONE, "C:\\UserArray.txt", ACSC_INT_TYPE, false);
}
catch (Exception ex)
{
    ShowException(ex);
}
```

4.36.2 UploadDataFromController**Description**

This method writes value(s) from SPiiPlus controller (variable or file) to text file.

Syntax

object.UploadDataFromController(long Src, string SrcName, long SrcNumFormat, long From1, long To1, long From2, long To2, string DestFilename, string DestNumFormat, Boolean bTranspose)

Arguments

Src	Number of program buffer for local variable, ACSC_NONE for global and ACSPL+ variable and ACSC_FILE for loading directly from file on flash memory.
SrcName	String that contains name of the Variable or File
SrcNumFormat	Format of number(s) in Controller. Use ACSC_INT_BINARY for integer, and ACSC_REAL_BINARY for real
From1/To1	Index range (first dimension)
From2/To2	Index range (second dimension)
DestFilename	Filename (including path) of the source text data file
DestNumFormat	Formatting string that will be used for printing into file ("1:%d\n" for example). Use string with %d for integer, and %lf for real type
bTranspose	If TRUE (1), then the array will be transposed before being loaded. Otherwise, this parameter has no affect

Return Value

None

Remarks

The method writes data to file from a specified variable (scalar/array) or straight from a binary file in the controller's flash memory. The variable can be a ACSPL+ controller variable, user global or user local.

ACSPL+ and user global variables have global scope. Therefore, **Src** must have the value of ACSC_NONE (-1) for these classes of variables. User local variable exists only within a buffer. The buffer number must be specified for user local variable.

If there is no variable (or file) with the name specified by **SrcName**, there would be error.

If performing loading straight to file, **From1**, **To1**, **From2** and **To2** are meaningless.

If the variable is scalar, all indexes **From1**, **To1**, **From2** and **To2** must be ACSC_NONE (-1). The method writes the value from variable specified by **SrcName**, to the file specified by **DestFileName**.

If the variable is a one-dimensional array, **From1**, **To1** must specify the index range and **From2**, **To2** must be ACSC_NONE (-1). The method writes the values from the specified variable from index **From1** to index **To1** inclusively, to the file specified by **DestFileName**.

If the variable is a two-dimensional array, **From1**, **To1** must specify the index range of the first dimension and **From2**, **To2** must specify the index range of the second dimension.

The method uses the variable as follows: first, the method retrieves the **To2-From2+1** values from row **From1** and writes them to the file specified by **DestFileName**, then retrieves **To2-From2+1** values from row **From1+1** and writes them, and so forth.

If **bTranspose** is TRUE, the method actions are inverted. It takes **To1-From1+1** values from row **From2** and writes them to first column of the specified controller variable, then retrieves the next **To1-From1** values from row **From2+1** and writes them to the next column of the specified destination file.

The destination file's format will be determined by string specified by **DestNumFormat**. This string will be used as argument in the ***printf** function.

Example

```
try
{
    channel.UploadDataFromController(ACSC_NONE, "MyGlobalArray", ACSC_INT_
    BINARY, 0, 3, 2, 4, "C:\\MyTransposedArrayInFile.txt", "%d , ", true);
}
catch (Exception ex)
{
    ShowException(ex);
}
```

4.37 Emergency Stop Methods

The Emergency Stop methods are:

Table 4-42. Emergency Stop Methods

RegisterEmergencyStop	Enables Emergency Stop function.
UnregisterEmergencyStop	Disables Emergency Stop function.

4.37.1 RegisterEmergencyStop

Description

This method initiates the "Emergency Stop" functionality for the calling application.

Syntax

```
object.RegisterEmergencyStop();
```

Return Value

None

Remarks

SPiiPlus UMD and Library provide the user application with the ability to open/close the Emergency Stop button. Clicking the Emergency Stop button sends a **stop** to all motions and motors command to all channels, which used in calling application, thereby stopping all motions and disabling all motors.



Figure 4-1. Emergency Stop Button

Calling **RegisterEmergencyStop** causes the Emergency Stop button icon to appear in the right bottom corner of the screen. If there is already such a button that is in use by other applications, a new button does not appear; but all functionality is available for the new application.

Calling **RegisterEmergencyStop** requires having the local host SPiiPlus UMD running, even if it is used through a remote connection because the Emergency Stop button is part of the local SPiiPlus UMD. If there is no local SPiiPlus UMD running, the method fails.

Only a single call is required per application. It can be placed anywhere in code, even before the opening of communication with controllers.

The application can remove the Emergency Stop button by calling [UnregisterEmergencyStop](#). The Emergency Stop button disappears if there are no additional registered applications that use it. The termination of SPiiPlus UMD also removes the Emergency Stop button; therefore, if SPiiPlus UMD is restarted, **RegisterEmergencyStop** has to be called again.

Calling **RegisterEmergencyStop** more than once per application is meaningless, but the method succeeds anyway. In order to ensure that Emergency Stop button is active, it is recommended placing a call of **RegisterEmergencyStop** after each call of any of **OpenComm***** functions.

Example

```
try
{
channel.RegisterEmergencyStop();
}
catch (Exception ex)
{
ShowException(ex);
}
```

4.37.2 UnregisterEmergencyStop

Description

This method terminates the “Emergency Stop” functionality for the calling application.

Syntax

object.UnregisterEmergencyStop();

Return Value

None

Remarks

Calling **UnregisterEmergencyStop** causes an application not to respond if the Emergency Stop button is clicked. If there are no other applications that registered the Emergency Stop functionality, the button will disappear.

Calling **UnregisterEmergencyStop** more than once per application is meaningless, but method will succeed anyway.

Example

```
try
{
channel.UnregisterEmergencyStop();
}
catch (Exception ex)
{
ShowException(ex);
}
```

4.38 Reboot Methods

The Reboot methods are:

Table 4-43. Reboot Methods

ControllerReboot	Reboots controller and waits for process completion.
ControllerFactoryDefault	Reboots controller, restores factory default settings and waits for process completion.

4.38.1 ControllerReboot

Description

This method reboots controller and waits for process completion.

Syntax

object.ControllerReboot(int Timeout)

Arguments

Timeout	Maximum waiting time in milliseconds.

Return Value

If the method succeeds, the return value is non-zero.

If the method fails, the return value is zero.

Example

```
Sub ControllerReboot_Sample()
On Error GoTo except
                                'Open Ethernet with TCP protocol
                                'communication with the controller
                                'TCP/IP address: 10.0.0.100
Call Ch.OpenCommEthernetTCP("10.0.0.100", Ch.ACSC_SOCKET_STREAM_PORT)
Call Ch.ControllerReboot(30000)
Call Ch.CloseComm()
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub
```

4.38.2 ControllerFactoryDefault

Description

The method reboots the controller, restores factory default settings and waits for process completion.

Syntax

object.ControllerFactoryDefault(int Timeout)

Arguments

Timeout	Maximum waiting time in milliseconds.

Return Value

If the method succeeds, the return value is non-zero.

If the method fails, the return value is zero.

Example

```

Sub ControllerFactoryDefault_Sample()
On Error GoTo except
        'Open Ethernet with TCP protocol
        'communication with the controller
        'TCP/IP address: 10.0.0.100
Call Ch.OpenCommEthernetTCP("10.0.0.100", Ch.ACSC_SOCKET_STREAM_PORT)
Call Ch.ControllerFactoryDefault(30000)
Call Ch.CloseComm()
Exit Sub
except:
    MyErrorMsg                                'See Error Handling in Visual Basic
End Sub

```

4.39 Host-Controller File Operations

Host PC files can be copied to controller's non-volatile memory and user files can be deleted from the controller's non-volatile memory as described in this section.

Table 4-44. Host-Controller File Copying Methods

CopyFileToController	The function copies files from the host PC to the controller's non-volatile memory
DeleteFileFromController	The function deletes user files from the controller's non-volatile memory.

4.39.1 CopyFileToController

Description

The function copies file from host PC to controller's non-volatile memory.

Syntax

object.CopyFileToController(BSTR SourceFileName, BSTR DestinationFileName, ULONGLONG CallType, VARIANT *pWait);

Arguments

SourceFileName	Pointer to a buffer that contains the name of the source file on host PC
DestinationFileName	Pointer to a buffer that contains name of destination file on the controller.

ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result. ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None.

Example

```
try
{
Ch.CopyFileToController("C:\\tmp.txt", "C:\\tmp.txt");
}
catch (COMException Ex)
{
    ErrorMsg(Ex);
}
```

4.39.2 DeleteFileFromController**Description**

The function deletes user files from controller's non-volatile memory.

Syntax

object.DeleteFileFromController(BSTR FileName, ULONGLONG CallType, VARIANT* pWait)

Arguments

FileName [in]	Pointer to a buffer that contains name of the user file on controller's non-volatile memory.
ulonglong CallType	If CallType is: ACSC_SYNCHRONOUS - The function returns when the controller response is received. ACSC_ASYNCHRONOUS - The function returns immediately. The calling thread must then call the GetString() / GetBinary() function to retrieve the operation result.

	ACSC_IGNORE - The function returns immediately. In this case, the operation result is ignored by the library and cannot be retrieved to the calling thread.
variant *pWait	A pointer that is initialized by an asynchronous function call and should be passed to GetString() / GetBinary() function.

Return Value

None.

Example

```
object waitObject = 0;
try
{
    Ch.DeleteFileFromController("tmp.txt", Ch.ACSC_SYNCHRONOUS, ref
waitObject);
}
catch (COMException Ex)
{
    ErorMsg(Ex);
}
```

4.40 Save to Flash Functions

The Save to Flash method is:

Table 4-45. Host-Controller File Copying Methods

ControllerSaveToFlash	The function saves user application to the controller's non-volatile memory.
------------------------------	--

4.40.1 ControllerSaveToFlash

Description

The function saves user application to the controller's non-volatile memory.

Syntax

object. ControllerSaveToFlash(VARIANT Parameters, VARIANT Buffers, VARIANT SPPPrograms, BSTR UserArrays);

Arguments

Parameters [in]	Array of parameters constants. Each element specifies system parameters or one involved axis: ACSC_SYSTEM corresponds to system parameters; ACSC_AXIS_0 corresponds to axis 0, ACSC_AXIS_1 to axis 1, etc.
Buffers[in]	Array of buffer constants. Each element specifies one involved buffer: ACSC_BUFFER_0 corresponds to buffer 0, ACSC_BUFFER_1 to buffer 1, etc. If all buffers need to be specified, ACSC_BUFFER_ALL should be used. After the last buffer, one additional element must be located that contains -1 which marks the end of the array.
SPPPrograms[in]	Array of Servo Processor (SP) constants. Each element specifies one involved SP: ACSC_SP_0 corresponds to SP 0, ACSC_SP_1 to SP 1, etc. If all SPs need to be specified, ACSC_SP_ALL should be used. After the last SP, one additional element must be located that contains -1 which marks the end of the array.
UserArrays[in]	User Arrays list - Pointer to the null-terminated string. The string contains chained names of user arrays, separated by '\r'(13) character. If there is no need to save user arrays to controller's non-volatile memory, this parameter should be NULL.

Return Value

None.

Example

The following code sample saves all axes parameters and buffers to flash, with two user arrays: "MyArray" and "MyArray2".

```
int[] Parameters = new int[2];
int[] Buffers = new int[2];
int[] SPPPrograms = new int[2];
Parameters[0] = Ch.ACSC_PAR_ALL;
Parameters[1] = -1;
Buffers[0]=Ch.ACSC_BUFFER_ALL;
Buffers[1] = -1;
SPPPrograms[0]=-1;
try
{
    Ch.ControllerSaveToFlash(Parameters, Buffers, null,
    "MyArray\rMyArray2");
}
catch (COMException Ex)
{
}
```



```

    ErorMsg (Ex) ;
}

```

4.41 SPiiPlusSC Management

The SPiiPlusSC Management methods are:

Table 4-46. Host-Controller File Copying Methods

StartSPiiPlusSC	The function starts the SPiiPlusSC controller.
StopSPiiPlusSC	The function stops the SPiiPlusSC controller.

4.41.1 StartSPiiPlusSC

Description

The function starts the SPiiPlusSC controller.

Syntax

```
object.StartSPiiPlusSC()
```

Arguments

None

Return Value

None

Example

```

try
{
    Ch. StartSPiiPlusSC();
}
catch (COMException Ex)
{
    ErorMsg (Ex) ;
}

```

4.41.2 StopSPiiPlusSC

Description

The function stops the SPiiPlusSC controller.

Syntax

```
object.StopSPiiPlusSC()
```

Arguments

None

Return Value

None

Example

```
try
{
    Ch. StopSPiiPlusSC();
}
catch (COMException Ex)
{
    ErorMsg(Ex);
}
```

5. Properties

This chapter presents the built-in variables that are used for establishing various properties during program runtime. For each property, the name of the variable, its value and a description are given.

5.1 General Properties

Table 5-1. General Properties

Name	Value	Description
ACSC_NONE	-1	Placeholder for redundant values, like the second index in a one-dimensional array
ACSC_INT_TYPE	1	Integer type of the variable
ACSC_REAL_TYPE	2	Real type of the variable
ACSC_COUNTERCLOCKWISE	1	Counterclockwise rotation
ACSC_CLOCKWISE	-1	Clockwise rotation
ACSC_POSITIVE_DIRECTION	1	A move in positive direction
ACSC_NEGATIVE_DIRECTION	-1	A move in negative direction

5.2 General Communication Options

Table 5-2. General Communication Options

Name	Value	Description
ACSC_COMM_USE_CHECKSUM	0x00000001	The communication mode when each command is sent to the controller with checksum and the controller also responds with checksum.
ACSC_COMM_AUTORECOVER_HW_ERROR	0x00000002	When a hardware error is detected in the communication channel and this bit is set, the library automatically repeats the transaction, without counting iterations. By default, this flag is not set.

5.3 Ethernet Communication Options

Table 5-3. Ethernet Communication Options

Name	Value	Description
ACSC_SOCKET_DGRAM_PORT	700	The library opens Ethernet communication using the connectionless socket and UDP communication protocol.
ACSC_SOCKET_STREAM_PORT	701	The library opens Ethernet communication using the connection-oriented socket and TCP communication protocol.

5.4 Axis Definitions

The general format for any axis definition is:

ACSC_AXIS_index

Where **index** is a number that ranges between 0 and 63, such as **ACSC_AXIS_0**, **ACSC_AXIS_1**, **ACSC_AXIS_30**, etc. The axis constant contains the value associated with the index, that is, **ACSC_AXIS_0** has a value of 0, **ACSC_AXIS_1** has a value of 1, and so forth. **ACSC_PAR_ALL** stands for all parameters (system and all axes parameters) and **ACSC_SYSTEM** stands for the system parameters.

5.5 Buffer Definitions

The general format for any buffer definition is:

ACSC_BUFFER_index

Where **index** is a number that ranges between 0 and 64, such as **ACSC_BUFFER_0**, **ACSC_BUFFER_1**, **ACSC_BUFFER_64**, etc. The buffer constant contains the value associated with the index, that is, **ACSC_BUFFER_0** has a value of 0, **ACSC_BUFFER_1** has a value of 1, and so forth. **ACSC_BUFFER_ALL** stands for all buffers.

5.6 Servo Processor (SP) Definitions

The general format for any SP definition is:

ACSC_SP_index

Where **index** is a number that ranges between 0 and 63, such as **ACSC_SP_0**, **ACSC_SP_1**, **ACSC_SP_63**, etc. The SP constant contains the value associated with the index, that is, **ACSC_SP_0** has a value of 0, **ACSC_SP_1** has a value of 1, and so forth. **ACSC_SP_ALL** stands for all SPs.

5.7 Motion Flags

Table 5-4. Motion Flags

Name	Value	Description
ACSC_AMF_WAIT	0x00000001	The controller plans the motion but doesn't start it until the Go method is executed.
ACSC_AMF_RELATIVE	0x00000002	The value of the point coordinate is relative to the end point coordinate of the previous motion.
ACSC_AMF_VELOCITY	0x00000004	The motion uses the specified velocity instead of the default velocity.
ACSC_AMF_ENDVELOCITY	0x00000008	The motion comes to the end point with the specified velocity
ACSC_AMF_POSITIONLOCK	0x00000010	The slaved motion uses position lock. If the flag is not specified, velocity lock is used.
ACSC_AMF_VELOCITYLOCK	0x00000020	The slaved motion uses velocity lock.
ACSC_AMF_CYCLIC	0x00000100	The motion uses the point sequence as a cyclic array: after positioning to the last point it does positioning to the first point and continues.
ACSC_AMF_VARTIME	0x00000200	The time interval between adjacent points of the spline (arbitrary path) motion is non-uniform and is specified along with an each added point. If the flag is not specified, the interval is uniform.
ACSC_AMF_CUBIC	0x00000400	Use a cubic interpolation between the specified points (third-order spline) for the spline (arbitrary path) motion. If the flag is not specified, linear interpolation is used (first-order spline). [In the present version third-order spline is not supported].
ACSC_AMF_EXTRAPOLATED	0x00001000	Segmented slaved motion: if a master value travels beyond the specified path, the last or the first segment is extrapolated.
ACSC_AMF_STALLED	0x00002000	Segmented slaved motion: if a master value travels beyond the specified path, the motion stalls at the last or first point.

Name	Value	Description
ACSC_AMF_MAXIMUM	0x00004000	Multi-axis motion does not use the motion parameters from the leading axis but calculates the maximum allowed motion velocity, acceleration, deceleration and jerk of the involved axes.
ACSC_AMF_SYNCHRONOUS	0x00008000	Position Event Generation (PEG): Start PEG synchronously with the motion sequence.
ACSC_AMF_JUNCTIONVELOCITY	0x00010000	Decelerate to corner.
ACSC_AMF_ANGLE	0x00020000	Do not treat junction as a corner, if junction angle is less than or equal to the specified value in radians.
ACSC_AMF_USERVARIABLES	0x00040000	Synchronize user variables with segment execution.
ACSC_AMF_INVERT_OUTPUT	0x00080000	The PEG pulse output is inverted.
ACSC_AMF_CURVEVELOCITY	0x00100000	Decelerate to curvature discontinuity point.
ACSC_AMF_CORNERDEVIATION	0x00200000	Use a corner rounding option with the specified permitted deviation.
ACSC_AMF_CORNERRADIUS	0x00400000	Use a corner rounding option with the specified permitted curvature.
ACSC_AMF_CORNERLENGTH	0x00800000	Use automatic corner rounding option.
ACSC_AMF_DWELLTIME	0x00100000	Dwell time between segments
ACSC_AMF_BSEGTIME	0x00004000	Segment time
ACSC_AMF_BSEGACC	0x00020000	Segment acceleration time
ACSC_AMF_BSEGJERK	0x00008000	Segment jerk time
ACSC_AMF_CURVEAUTO	0x01000000	Automatic curve calculations
ACSC_AMF_AXISLIMIT	0x00002000	Axis velocity limitation enforcement
ACSC_AMF_ACCURATE	0x00020000	Rounding of the distance between incremental PEG events

5.8 Data Collection Flags

Table 5-5. Data Collection Flags

Name	Value	Description
ACSC_DCF_TEMPORAL	0x00000001	Temporal data collection. The sampling period is calculated automatically according to the collection time.
ACSC_DCF_CYCLIC	0x00000002	Cyclic data collection uses the collection array as a cyclic buffer and continues infinitely. When the array is full, each new sample overwrites the oldest sample in the array.
Name	Value	Description
ACSC_DCF_SYNC	0x00000004	Starts data collection synchronously to a motion. Data collection started with the ACSC_DCF_SYNC flag is called <i>axis data collection</i> .
ACSC_DCF_WAIT	0x00000008	Creates synchronous data collection, but does not start until the Go method is called. This flag can only be used with the ACSC_DCF_SYNC flag.

5.9 Motor State Flags

Table 5-6. Motor State Flags

Name	Value	Description
ACSC_MST_ENABLE	0x00000001	Motor is enabled
ACSC_MST_INPOS	0x00000010	Motor has reached a target position.
ACSC_MST_MOVE	0x00000020	Motor is moving.
ACSC_MST_ACC	0x00000040	Motor is accelerating.

5.10 Axis State Flags

Table 5-7. Axis State Flags

Name	Value	Description
ACSC_AST_LEAD	0x00000001	Axis is leading in a group.
ACSC_AST_DC	0x00000002	Axis data collection is in progress.

Name	Value	Description
ACSC_AST_PEG	0x00000004	PEG for the specified axis is in progress.
ACSC_AST_MOVE	0x00000020	Axis is moving.
ACSC_AST_ACC	0x00000040	Axis is accelerating.
ACSC_AST_SEGMENT	0x00000080	Construction of segmented motion for the specified axis is in progress.
ACSC_AST_VELLOCK	0x00000100	Slave motion for the specified axis is synchronized to master in velocity lock mode.
ACSC_AST_POSLOCK	0x00000200	Slave motion for the specified axis is synchronized to master in position lock mode.

5.11 Index and Mark State Flags

Table 5-8. Index and Mark State Flags

Name	Value	Description
ACSC_IST_IND	0x00000001	Primary encoder index of the specified axis is latched.
ACSC_IST_IND2	0x00000002	Secondary encoder index of the specified axis is latched.
ACSC_IST_MARK	0x00000004	MARK1 signal has been generated and position of the specified axis was latched.
ACSC_IST_MARK2	0x00000008	MARK2 signal has been generated and position of the specified axis was latched.

5.12 Program State Flags

Table 5-9. Program State Flags

Name	Value	Description
ACSC_PST_COMPILED	0x00000001	Program in the specified buffer is compiled.
ACSC_PST_RUN	0x00000002	Program in the specified buffer is running.

Name	Value	Description
ACSC_PST_SUSPEND	0x00000004	Program in the specified buffer is suspended after the step execution or due to breakpoint in debug mode.
ACSC_PST_DEBUG	0x00000020	Program in the specified buffer is executed in debug mode, i.e. breakpoints are active.
ACSC_PST_AUTO	0x00000080	Auto routine in the specified buffer is running.

5.13 Safety Control Masks

Table 5-10. Safety Control Masks

Name	Value	Description	Type
ACSC_SAFETY_RL	0x00000001	Right Limit	Motor fault
ACSC_SAFETY_LL	0x00000002	Left Limit	Motor fault
ACSC_SAFETY_NETWORK	0x00000004	Network Error	Network fault
ACSC_SAFETY_HOT	0x00000010	Motor Overheat	Motor fault
ACSC_SAFETY_SRL	0x00000020	Software Right Limit	Motor fault
ACSC_SAFETY_SLL	0x00000040	Software Left Limit	Motor fault
ACSC_SAFETY_ENCNC	0x00000080	Primary Encoder Not Connected	Motor fault
ACSC_SAFETY_ENC2NC	0x00000100	Secondary Encoder Not Connected	Motor fault
ACSC_SAFETY_DRIVE	0x00000200	Driver Alarm	Motor fault
ACSC_SAFETY_ENC	0x00000400	Primary Encoder Error	Motor fault
ACSC_SAFETY_ENC2	0x00000800	Secondary Encoder Error	Motor fault
ACSC_SAFETY_PE	0x00001000	Position Error	Motor fault
ACSC_SAFETY_CPE	0x00002000	Critical Position Error	Motor fault
ACSC_SAFETY_VL	0x00004000	Velocity Limit	Motor fault
ACSC_SAFETY_AL	0x00008000	Acceleration Limit	Motor fault

Name	Value	Description	Type
ACSC_SAFETY_CL	0x00010000	Current Limit	Motor fault
ACSC_SAFETY_SP	0x00020000	Servo Processor Alarm	Motor fault
ACSC_SAFETY_STO	0x00040000	Safe Torque Off	
ACSC_SAFETY_HSSINC	0x00100000	Hssi Not Connected	
ACSC_SAFETY_EXTNT	0x00800000	External Network Error	
ACSC_SAFETY_TEMP	0x01000000	MPU Overheat Fault	
ACSC_SAFETY_PROG	0x02000000	Program Error	System fault
ACSC_SAFETY_MEM	0x04000000	Memory Overuse	System fault
ACSC_SAFETY_TIME	0x08000000	Time Overuse	System fault
ACSC_SAFETY_ES	0x10000000	Emergency Stop	System fault
ACSC_SAFETY_INT	0x20000000	Servo Interrupt	System fault
ACSC_SAFETY_INTGR	0x40000000	Integrity Violation	System fault
ACSC_SAFETY_FAILURE	0x80000000	Component Failure	



See the *ACSPL+ Programmer's Guide* for detailed explanations of faults.

5.14 Interrupt Types

Table 5-11. Interrupt Types

Name	Value	Description
ACSC_INTR_ACSPL_PROGRAM	22	ACSPL+ program has generated the interrupt by INTERRUPT command.
ACSC_INTR_ACSPL_PROGRAM_EX	21	A user has sent the INTERRUPTEX command.
ACSC_INTR_COMM_CHANNEL_CLOSED	32	Communication channel has been closed.
ACSC_INTR_COMMAND	21	A line of ACSPL+ commands has been executed in a dynamic buffer.

Name	Value	Description
ACSC_INTR_EMERGENCY	15	EMERGENCY STOP signal has been generated.
ACSC_INTR_ETHERCAT_ERROR	29	EtherCAT error occurred.
ACSC_INTR_LOGICAL_MOTION_END	17	Logical motion has finished.
ACSC_INTR_MOTION_FAILURE	18	Motion has been interrupted due to a fault.
ACSC_INTR_MOTION_PHASE_CHANGE	25	Motion profile changed the phase
ACSC_INTR_MOTION_START	24	Physical motion has started.
ACSC_INTR_MOTOR_FAILURE	19	Motor has been disabled due to a fault.
ACSC_INTR_NEWSEGM	27	AST.#NEWSEGM bit went high.
ACSC_INTR_PHYSICAL_MOTION_END	16	Physical motion has finished.
ACSC_INTR_PROGRAM_END	20	ACSPL+ program has finished.
ACSC_INTR_SOFTWARE_ESTOP	33	EStop button was clicked.
ACSC_INTR_SYSTEM_ERROR	28	System error occurred.
ACSC_INTR_TRIGGER	26	AST.#TRIGGER bit went high

5.15 Callback Interrupt Masks

Table 5-12. Callback Interrupt Masks

Bit Name	Bit	Desc.	Interrupt
ACSC_MASK_AXIS_63	63	Axis 63	ACSC_INTR_PHYSICAL_MOTION_END, ACSC_INTR_LOGICAL_MOTION_END, ACSC_INTR_MOTION_FAILURE, ACSC_INTR_MOTOR_FAILURE, ACSC_INTR_MOTION_START, ACSC_INTR_MOTION_PHASE_CHANGE, ACSC_INTR_TRIGGER
ACSC_MASK_BUFFER_0	0	Buffer 0	ACSC_INTR_PROGRAM_END,
...	...	...	ACSC_INTR_COMMAND,
ACSC_MASK_BUFFER_31	31	Buffer 31	ACSC_INTR_ACSPL_PROGRAM

5.16 Configuration Keys

Table 5-13. Configuration Keys

Key Name	Key	Description
ACSC_CONF_WORD1_KEY	1	Bit 6 defines HSSI route, bit 7 defines source for interrupt generation.
ACSC_CONF_INT_EDGE_KEY	3	Sets the interrupt edge to be positive or negative.
ACSC_CONF_ENCODER_KEY	4	Sets encoder type: A&B or analog.
ACSC_CONF_OUT_KEY	29	Sets the specified output pin to be one of the following: OUT0 PEG Brake
ACSC_CONF_MFLAGS9_KEY	204	Controls value of MFLAGS.9
ACSC_CONF_DIGITAL_SOURCE_KEY	205	Assigns use of OUT0 signal: general purpose output or PEG output.

Key Name	Key	Description
ACSC_CONF_SP_OUT_PINS_KEY	206	Reads SP output pins.
ACSC_CONF_BRAKE_OUT_KEY	229	Controls brake method.

6. Events

SPiiPlus COM Library has events for the following types of interrupts:



The bit-mapped event parameter: **Param** is an interrupt mask that determines which axis/buffer/input a given interrupt was generated for. See [Callback Interrupt Masks](#), [Callback Interrupt Masks](#) for a description of **Param** for each interrupt.

Table 6-1. Events and Interrupts

Event	Interrupt
ACSPLCOMMAND (long Param)	ACSC_INTR_COMMAND
ACSPLPROGRAM (long Param)	ACSC_INTR_ACSPL_PROGRAM
COMMCHANNELCLOSED (long Param)	ACSC_INTR_COMM_CHANNEL_CLOSED
EMERGENCY (long Param)	ACSC_INTR_EMERGENCY
ETHERCATERROR (long Param)	ACSC_INTR_ETHERCAT_ERROR
LOGICALMOTIONEND (long Param)	ACSC_INTR_LOGICAL_MOTION_END
MOTIONFAILURE (long Param)	ACSC_INTR_MOTION_FAILURE
MOTIONPHASECHANGE	ACSC_INTR_MOTION_PHASE_CHANGE
MOTIONSTART(long Param).	ACSC_INTR_MOTION_START
MOTORFAILURE (long Param)	ACSC_INTR_MOTOR_FAILURE
NEWSEGM (long Param)	ACSC_INTR_NEWSEGM
PHYSICALMOTIONEND (long Param)	ACSC_INTR_PHYSICAL_MOTION_END
PROGRAMEND (long Param)	ACSC_INTR_PROGRAM_END
SOFTWAREESTOP (long Param)	ACSC_INTR_SOFTWARE_ESTOP
SYSTEMERROR (long Param)	ACSC_INTR_SYSTEM_ERROR
SYSTEMERROR (long Param)	ACSC_INTR_ACSPL_PROGRAM_EX
TRIGGER	ACSC_INTR_TRIGGER

7. Error Handling

The SPiiPlus COM Library supports a standard COM error object to provide error information. Each COM error object contains:

- > The source reporting the error as follows: SpiiPlusCom660.Channel.
- > A string with a description of the error and the SPiiPlus [Error Codes](#) (see [Error Codes](#)). For example:

```
Communication Initialization failure. Error - 132
```

- > **HRESULT** - a 32-bit standard COM value representing the method return status. To get the SPiiPlus error code from **HRESULT**, use [ParseCOMErrorCode](#). For more information about **HRESULT**, refer to Microsoft documentation.

7.1 Error Handling in Visual Basic

The following code illustrates error handling in Visual Basic:

```
Sub OpenCommSerial ()  
    On Error GoTo except  
        'Open serial communication through port COM1  
        'with baud rate 115200  
        Call Ch.OpenCommSerial(1, 115200)  
    Exit Sub  
except:  
    MyErrorMsg  
End Sub  
Sub MyErrorMsg()  
    Dim Str  
    Str = "Error from " & Err.Source & Chr$(10) & Chr$(13)  
    Str = Str & Err.Description & Chr$(10) & Chr$(13)  
    Str = Str & "HRESULT: " & Hex$(Err.Number)  
    MsgBox Str  
End Sub
```

If communication is not established, the following error message appears:

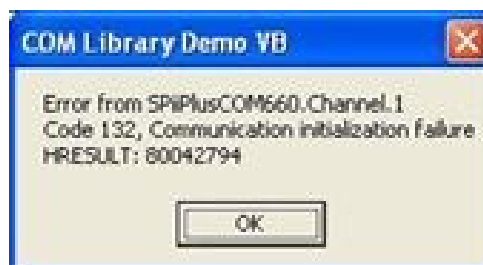


Figure 7-1. Communication not Established Message - Visual Basic

7.2 Error Handling Example in LabVIEW 7

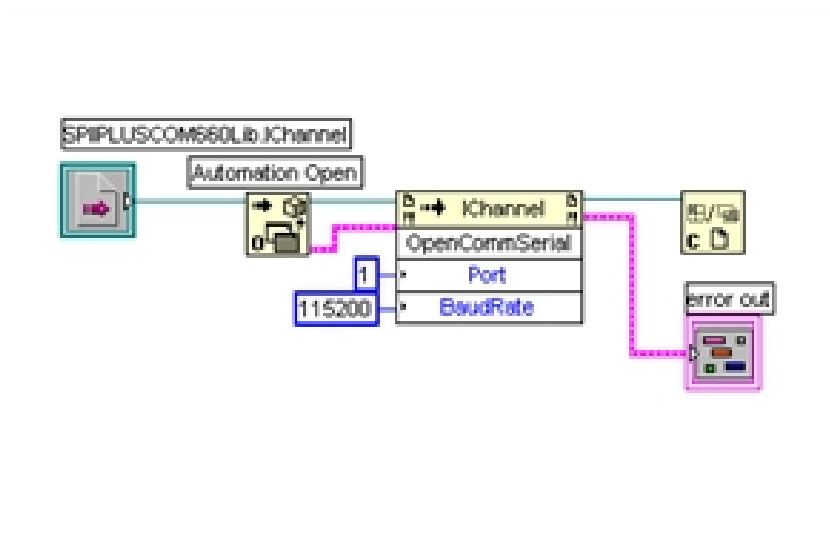


Figure 7-2. Error Handling Example in LabView 7

If communication is not established, the **error** cluster will be filled:

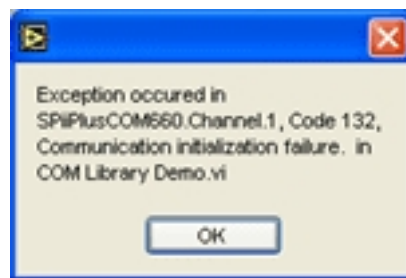


Figure 7-3. LabVIEW Error Message

7.3 Error Handling Example in C#

```
private void OpenCommSerial()  
{  
    try  
    {  
        Ch.OpenCommSerial(1, 115200);  
    }  
    catch (COMException Ex)  
    {  
        ErorMsg(Ex);  
    }  
}  
private void ErorMsg(COMException Ex)  
{  
    string Str = "Error from " + Ex.Source + "\n\r";  
    Str = Str + Ex.Message + "\n\r";  
    Str = Str + "HRESULT:" + String.Format("0x{0:X}", (Ex.ErrorCode));  
    MessageBox.Show(Str);  
}
```

If communication is not established, the message box will appear:



Figure 7-4. C# Error Message

7.4 Error Codes

This section provides a breakdown of the Error codes associated with the COM Library.



Any error code greater than 1000 is a controller error as defined in the *SPiiPlus Command & Variable Reference Guide*.

Table 7-1. COM Library Error Codes

Error Code	Error Message	Remarks
101	Asynchronous call is not supported.	Attempt was made to use a method asynchronously that is only support synchronously, for example, Send .
102	No such file or directory.	This error is returned by the OpenLogFile method if a component of a path does not specify an existing directory.
103	The FW version does not support the current COM Library version.	This error is returned by one of the OpenComm methods. Upgrade the FW of the controller.
104	Controllers reply is too long.	A timeout has occurred due to a lack of response of the controller.
109	Internal library error: Invalid file handle.	
122	Internal library error: Cannot open Log file.	
124	Too many open files.	This error is returned by the OpenLogFile method if no more file handles available.
128	No space left on device.	This error is returned by the WriteLogFile method if no more space for writing is available on the device (for example, when the disk is full).
130	Timeout expired.	A time out occurred while waiting for a controller response. This error indicates that during specified timeout the controller did not respond or response was invalid.
131	Attempt to stop Simulator while it was not running.	An attempt to stop simulator was made without it being run.

Error Code	Error Message	Remarks
132	Communication initialization failure.	This error is returned by one of the Open*** methods in the following cases: - the specified communication parameters are invalid - the corresponding physical connection is not established - the controller does not respond for specified communication channel.
133	SPiiPlus Simulator ports are in use by another application.	The ports set for simulator in UMD are taken by another application.
134	Invalid communication handle.	Specified communication handle must be a handle returned by one of the Open*** methods.
135	All channels are busy.	The maximum number of the concurrently opened communication channels is 10.
136	SPiiPlus Simulator execution path is not valid.	A simulator execution attempt was made without setting a simulator executable and default simulator executable was not found.
137	Received message is too long (more than size of user buffer).	This error cannot be returned and is present for compatibility with previous versions of the library.
138	The program string is long.	This error is returned by one of the an AppendBuffer or LoadBuffer method if ACSPL+ program contains a string longer than 2032 bytes.
139	Method parameters are invalid.	
140	History buffer is closed.	
141	Name of variable must be specified.	

Error Code	Error Message	Remarks
142	Error in index specification.	This error is returned by the ReadVariable , ReadVariableAsScalar , ReadVariableAsVector , ReadVariableAsMatrix , WriteVariable methods if the From1 , To1 , From2 , To2 arguments were specified incorrectly.
143	Controller reply contains less values than expected.	This error is returned by the ReadVariable , ReadVariableAsScalar , ReadVariableAsVector , ReadVariableAsMatrix , WriteVariable methods.
145	Function is not supported in current version	
147	Internal error: Error of the history buffer initialization.	
150	Unsolicited messages buffer is closed.	
151	Callback registration error.	This error is returned by the EnableEvent method for any of the communication channels. Only PCI Bus communication supports user callbacks.
152	Callback method has been already installed.	This error is returned by the EnableEvent method if the application tries to enable another event for the same interrupt that was already used. Only one event can be enabled for each interrupt.
153	Checksum of the controller response is incorrect.	
154	Internal library error: The controller replies sequence is invalid.	
155	Internal library error.	

Error Code	Error Message	Remarks
157	Internal library error: Error of the unsolicited messages buffer initialization.	
158	Non-waiting call has been aborted.	
159	Error of the non-waiting call cancellation.	
160	Internal error: Queue of transmitted commands is full.	
162	The library cannot send to the specified communication channel.	Check physical connection with the controller (or settings) and try to reconnect.
163	The library cannot receive from the specified communication channel.	Check physical connection with the controller (or settings) and try to reconnect.
164	Internal library error: Sending of the chain is failed.	
165	Specified IP address is duplicated.	
166	There is no Application with such Handle.	
167	Array name was expected.	
168	File is not Data File.	Input file is not in ANSI format.
171	Application Saver Loader CRC Error	

Error Code	Error Message	Remarks
172	Application Saver Loader Header CRC Error	
173	Application Saver Loader File Size Error	
174	Application Saver Loader File Open Error	
175	Application Saver Loader Unknown File Error	
176	Application Saver Loader Format Version Error	
177	Application Saver Loader Section Size is Zero	
179	Internal library error: Thread local storage error.	
180	PCI driver initialization error.	Returned by the GetPCICards method in the following cases: SpiiPlus PCI driver is not installed correctly The version of the SpiiPlus PCI Bus driver is incorrect - in this case, it is necessary to reinstall the SpiiPlus PCI driver (WINDRIVER) and the library.
185	Pointer to the buffer is invalid Null pointer received instead of user allocated object	
189	Specified priority for the callback thread cannot be set	

Error Code	Error Message	Remarks
190	Cannot access DPRAM directly through any channel but PCI and Direct.	Returned by DPRAM access methods, when attempting to call them with Serial or Ethernet channels.
192	Invalid DPRAM address was specified	Returned by DPRAM access methods, when attempting to access illegal address
193	This version of simulator does not support work with DPRAM.	Returned by DPRAM access methods, when attempting to access old version Simulator that does not support DPRAM.
195	File not found.	Returned by methods that work with host file system when a specified filename is not found. Check the path and filename.
196	Not enough data.	Returned by methods that analyze SPiiPlus application files when application file format is incorrect. Check application file and replace with a valid file.
197	The application cannot establish communication with the SPiiPlus UMD.	Returned by one of the OPENCOMM methods. Check the following: SPiiPlus UMD is loaded (that is, the UMD icon  appears in the Task tray). SPiiPlus UMD shows an error message. In case of remote connection, access from a remote application is enabled.
198	Not responding.	The controller does not reply for more than 20 seconds. Returned by any method that exchanges data with the controller. Check the following: Controller is powered on (MPU LED is green) Controller connected properly to host Controller executes a time consuming command like compilation of a large program, save to flash, load to flash, etc.

Error Code	Error Message	Remarks
199	The DLL and the UMD versions are not compatible.	Returned by one of the OpenComm methods. Verify that the files ACSCL.DLL and ACSCSRV.EXE are of the same version.
601	Error in array and/or index definition	
602	Communication channel is already open	
603	Argument Value has incorrect type	
604	Argument Value is not initialized	
605	Variable name must be specified	
606	Wrong call type was specified	
607	Wrong return type	

Smarter



Motion

5 HaTnufa St.
Yokne'am Illit 2066717
Israel
Tel: (+972) (4) 654 6440 Fax: (+972) (4) 654 6443

Contact us: sales@acsmotioncontrol.com | www.acsmotioncontrol.com

